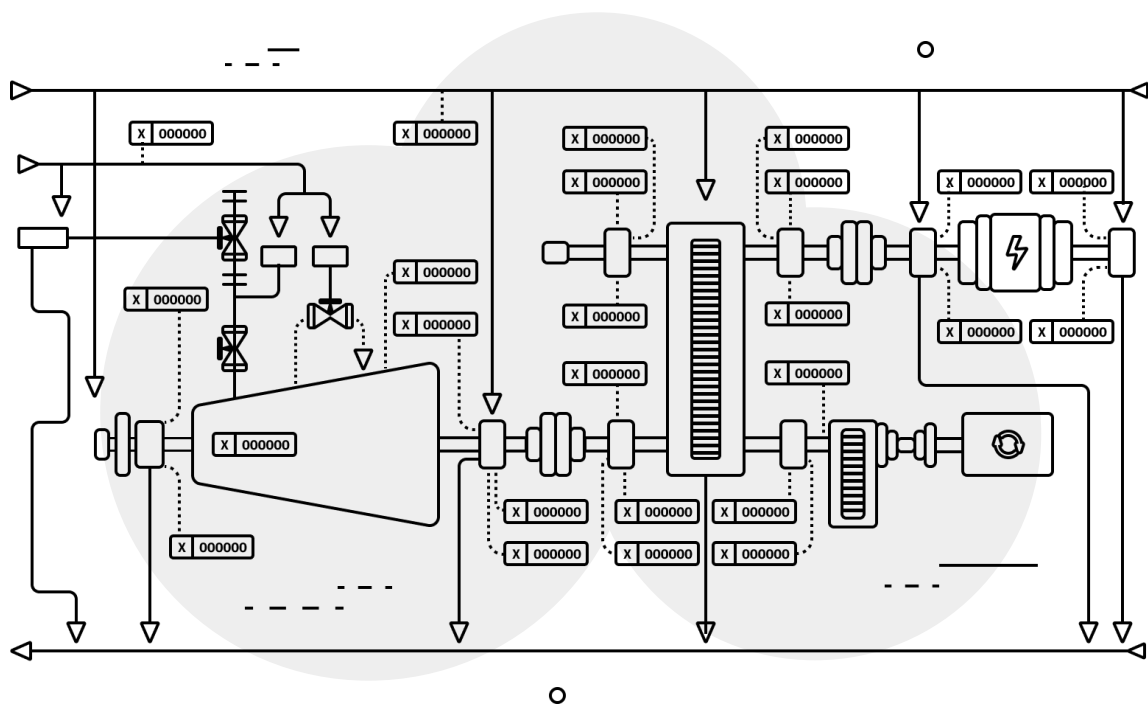


Архитектура высоконагруженных систем

Второе издание

Системы сбора информации
Распределенные системы управления
Системы реального времени



АРПП

Отечественный софт

Москва, 2024

Вадим Подольный

В. П. Подольный

**Архитектура
высоконагруженных систем**

Системы сбора информации

Распределенные системы управления

Системы реального времени

Второе издание

Москва

2024

УДК 004.6

ББК 16.35

П44 Подольный В.

Архитектура высоконагруженных систем

Издание второе. – М.: ООО «САМ Полиграфист» («OneBook»), 2024. –261¹ стр, ил.

Книга дает представление о том, как проектируют высоконагруженные распределенные системы управления. Книга адресована всем, кто хочет разобраться, как устроены и создаются такие системы.

УДК 004.6

ББК 16.35

ISBN 978-5-00227-164-1

© Подольный Вадим

¹ 246 стр. для электронной версии

Оглавление

Благодарности	9
Об авторе	10
О чем эта книга	11
Предисловие ко второму изданию	13
Обращение к читателю	14
Приветственное слово	15
Глава 1. Введение	16
Глава 2. Аппаратная архитектура	21
2.1. Выбор типа железа	21
2.2. Место размещения оборудования	22
2.3. Оценка конфигурации оборудования при нагрузке	22
2.4. Зависимость от вендора или технологии	23
2.5. Планирование масштабирования	24
Глава 3. Программная архитектура	25
3.1. Данные и метаданные	25
3.2. Способы обмена данными	26
3.3. Транзакции	28
3.4. Распределенность	31
3.5. Ограничения распределенных систем	32
3.6. Единый формат данных	36
3.7. Операции с данными	36
3.8. Распределенные операции с данными	37
Глава 4. Принципы распределения данных	39
4.1. Распределенная обработка данных	39
4.2. Распределенная разделяемая память	42
4.2.1. Алгоритм с центральным узлом или без	44
4.2.2. Миграционный алгоритм	45
4.2.3. Алгоритм размножения для чтения	46
4.2.4. Алгоритм полного размножения	47
4.3. Избыточность данных	48

4.4. Сегментирование данных	49
4.5. Консолидация данных	51
Глава 5. Распределенная обработка данных	52
5.1. Узловая модель	55
5.1.1. Корневые узлы	56
5.1.2. Клиентские узлы	56
5.2. Способы обмена данными между узлами	57
5.3. Способы координации узлов	59
5.4. Обработка сообщений	59
5.4.1. Обработка очередей	60
5.4.2. Брокер очередей	61
5.4.3. Приоритизированная обработка очередей	62
5.4.4. Многопоточная обработка очередей	62
5.4.5. Распределенная обработка очередей	65
5.4.6. Распределенный брокер очередей	67
Глава 6. Надежная распределенная архитектура	70
6.1. Резервирование: горячее, теплое и холодное	71
6.1.1. Резервирование корневых узлов	73
6.1.2. Режим мультиактивный	73
6.1.3. Режим мультиактивный с резервированием	74
6.2. Кэширование данных	75
6.2.1. Когерентность кэша	77
6.2.2. Архитектура NUMA	79
6.2.3. Алгоритмы кэширования	80
6.2.4. Вытеснение давно неиспользуемых данных	81
6.2.5. Вытеснение недавно использовавшихся данных	82
6.2.6. Алгоритм многопоточного кэширования	83
6.2.7. Статическое кэширование	83
6.2.8. Стратегическое кэширование	84
6.3. Резервирование данных	85
6.4. Репликация данных	87
6.5. Синхронизация данных	90
6.5.1. Стратегии синхронизации	94
6.6. Консистентность данных	95

Оглавление

6.6.1. Модели консистентности	97
6.6.2. Строгая консистентность	97
6.6.3. Слабая консистентность	98
6.6.4. Консистентность в конечном счете	99
6.6.5. Причинная консистентность	100
6.6.6. Последовательная консистентность	101
6.6.7. Консистентность по времени	102
6.6.8. Линеаризуемая консистентность	103
6.6.9. Проблемы оценки консистентности	105
6.6.10. Консистентность по выходу	105
6.6.11. Консистентность по входу	106
6.6.12. Строгая консистентность в конечном счете	106
6.7. Современная классификация PCY	107
6.7.1. Типы PCY в зависимости от консистентности	109
6.7.2. Византийская задача	110
6.7.3. Блокчейн	111
6.8. Персистентность данных	111
Глава 7. Отказоустойчивость и высокая доступность	112
7.1. Диагностика и качество сервиса (QoS)	112
7.1.1. Метрики качества сервиса	113
7.1.2. Метрики приоритета	113
7.1.3. Метрики состояния	114
7.1.4. Композитные метрики	115
7.1.5. Метрика латентности heartbeat	115
7.2. Отказоустойчивость и надежность	117
7.2.1. Отказоустойчивая архитектура	119
7.2.2. Время восстановления	120
7.2.3. Катастрофоустойчивость	121
7.2.4. Архитектура высокой доступности	123
7.2.5. Доступность сервиса SLA	123
7.3. Деградация	124
7.3.1. Неустойчивое состояние	126
7.3.2. Элегантная и изящная деградация	126
7.3.3. Управляемая деградация	127

7.3.4. Деградация отклика	127
7.3.5. Деградация очередей	128
7.3.6. Деградация при информационном шторме	128
Глава 8. Балансировка нагрузки	130
8.1. Задачи балансировки	131
8.2. Типы балансировки	132
8.2.1. Балансировщик на прокси-узле	132
8.2.2. Балансировщик на узле клиента	133
8.2.3. Балансировщик синхронизации с координатором	134
8.2.4. Балансировщик синхронизации распределенный	135
8.3. Логика балансировки	136
8.4. Балансировка без обратной связи	137
8.4.1. Циклическое распределение	137
8.4.2. Циклическое распределение взвешенное	137
8.4.3. Разновидности циклических алгоритмов	138
8.4.4. Минимум соединений	138
8.5. Балансировка с обратной связью	140
8.5.1. Циклическое распределение с ОС	141
8.5.2. Циклическое распределение взвешенное с ОС	142
8.6. Балансировка с приоритизацией	142
8.7. Теория массового обслуживания	143
Глава 9. Кластеризация	146
9.1. Принципы построения	146
9.1.1. Кластеры высокой доступности	147
9.1.2. Кластеры распределенной нагрузки	147
9.1.3. Вычислительные кластеры	148
9.1.4. Системы распределенных вычислений GRID	148
9.2. Кластерные вычисления	149
9.2.1. Параллельные вычисления	149
9.2.2. Катастрофоустойчивые кластеры	153
Глава 10. Масштабирование	154
10.1. Проблемы масштабирования	155
10.2. Линейное масштабирование	156
10.3. Функциональное масштабирование	156

Оглавление

10.4. Гипермасштабирование	157
10.5. Жизненный цикл целевой системы	160
Глава 11. Архитектурные шаблоны	162
11.1. Базовые архитектурные шаблоны	165
11.1.1. Каналы и фильтры	166
11.1.2. Многоуровневая архитектура, Клиент—сервер	167
11.1.3. Распределенная конвейерная архитектура	170
11.1.4. Брокер очередей	173
11.1.5. Многослойная архитектура	174
11.1.6. Выделенное ядро	176
11.1.7. Издатель-подписчик	177
11.2. Архитектурные шаблоны РСУ	178
11.2.1. Интерконнект	178
11.2.2. Событийно управляемая архитектура	181
11.2.3. Сервис-ориентированная архитектура	183
11.2.4. Микросервисная архитектура	185
11.2.5. Оркестратор и хореография	187
11.2.6. Распределенное ядро	190
11.2.7. Разделение ответственности команд и запросов	190
11.2.8. Двухфазная фиксация	193
11.2.9. Сага	196
Глава 12. Приемы реализации	199
12.1. Распределенные СУБД	199
12.1.1. Ключ-значение	199
12.1.2. Объектная надстройка	200
12.1.3. Связи и отношения между объектами	202
12.1.4. Индексирование	204
12.1.5. Временные ряды	205
12.2. Критические информационные системы	207
12.2.1. АСУ непрерывного производства	209
12.2.2. АСУ дискретного производства	213
12.2.3. Цифровой двойник	214
12.2.4. Предиктивная аналитика	214
12.2.5. Граничные и туманные вычисления	215

12.2.6. Интернет вещей	215
12.3. Обработка данных	216
12.3.1. Обработка временных рядов	216
12.3.2. Маркировка данных метками времени	219
Глава 13. Кибербезопасность РСУ	220
13.1. Угрозы в РСУ	221
13.1.1. Техногенные источники угроз	222
13.1.2. Антропогенные источники угроз	223
13.2. Уязвимости в РСУ	224
13.2.1. Методы исследования уязвимостей	224
13.3. Методы проведения атак РСУ	226
13.3.1. Виды атак	226
13.3.2. Жизненный цикл атак	229
13.3.3. Логические атаки	230
13.4. Методы защиты РСУ	230
13.4.1. Понятие доверия в РСУ	231
13.4.2. Повышение доверия	232
13.4.3. Инвентаризация	233
13.4.4. Доверенная загрузка	233
13.4.5. Распределенность как метод защиты	234
13.4.6. Защита управления	235
13.4.7. Типовые методы защиты	236
Приложение 1. Принятые сокращения	237
Приложение 2. Перечень шаблонов проектирования	241
Приложение 3. Список литературы	243
Приложение 4. Ссылки	246

Благодарности

Автор выражает искреннюю признательность коллегам за помощь в подготовке книги.

Прежде всего хотелось бы поблагодарить Клуб топ-менеджеров «Клуб директоров ИТ — 4 СИО.Ру», редакционную коллегию учебников по цифровой трансформации «4CIO» и «4CDTO», персонально Сергея Кирюшина, Феликса Карасева, Евгения Борисова, Владимира Определенова, Дмитрия Северова, Алексея Кравченко, Евгения Колесникова за полученный опыт разработки глав учебника 4CIO/4CDTO, мотивацию на создание собственной книги, конструктивную критику и отзывы.

Автор благодарит за активное участие в обсуждении книги, критику и ценные замечания профильных специалистов Антона Жбанкова, эксперта в области создания аппаратной инфраструктуры высоконагруженных систем, и Сергея Шелудько, эксперта в области разработки распределенных высоконагруженных программных систем.

Автор выражает благодарность опытному старшему товарищу, автору книг² по цифровизации АО «Концерн Росэнергоатом» «Цифровая трансформация», «Цифровой двойник», «Центры обработки данных» и «Искусственный интеллект» Александру Прохорову за замечания, которые позволили сделать книгу более качественной.

² Digital Atom — книги, [#A1](#)

Об авторе

Вадим Подольный с 2000 года работает в области разработки программного обеспечения промышленных распределенных высоконагруженных систем управления. За более чем 20 лет Вадим принял участие во многих крупных инфраструктурных проектах, связанных с обеспечением технологической независимости ключевых отраслей промышленности. В должности главного конструктора он руководил разработкой Российской программной платформы системы верхнего уровня АСУ ТП АЭС класса DCS/SCADA (ПО Распределенных Технологий Автоматизации Лицензированное, ПОРТАЛ) в ГК «Росатом». В настоящее время на базе ПОРТАЛ создано и успешно введено в эксплуатацию множество систем верхнего уровня АСУ ТП на современных энергоблоках АЭС с реакторами ВВЭР-1000, ВВЭР-1200 и БН-800. Вадим был главным конструктором и руководителем разработки специальной операционной системы «Заря», обеспечивающей выполнение задач сопряжения автоматизированных систем управления Вооруженными силами РФ и разрабатываемой в Центральном научно-исследовательском институте экономики, информатики и систем управления (ЦНИИ ЭИСУ). Как эксперт Вадим участвовал в разработке и обеспечении кибербезопасности крупных распределенных автоматизированных систем ГК «Ростех», ОАО «РЖД» и ряда других коммерческих структур. В настоящее время Вадим руководит несколькими проектами в области разработки промышленных распределенных высоконагруженных систем управления, обработки и хранения данных. Он является членом совета «Клуба Топ-менеджеров 4CIO», постоянным спикером многих отраслевых конференций, таких как «HighLoad++», «ИБ КВО», «4CIO» и др.

Вадим окончил факультет технической физики Национального исследовательского ядерного университета МИФИ (Московский инженерно-физический институт) по специальности «Ядерные реакторы и энергетические установки», аспирантуру Всероссийского научно-исследовательского института по эксплуатации АЭС (ВНИИАЭС), получил дополнительное профессиональное образование в области автоматизированных систем управления технологическими процессами, разработки программного обеспечения и индустриальной кибербезопасности.

Связаться с Вадимом: vadim@podolny.ru (email), [@vpp01](https://t.me/vpp01) (tg)

О чем эта книга

Идея создания книги появилась после долгого обсуждения с техническими заказчиками подходов к разработке высоконагруженных распределенных систем. Всегда возникал один и тот же вопрос: «Как будем делать эту систему?», — а за ним один и тот же ответ: «Да вот так и будем!» Заказчик уходил думать, пытаюсь «сколхозить» решение собственными силами, а когда в очередной раз не получалось, возвращался и говорил: «Вот еще требования добавились, как делать-то будем?». Да все так же! И это могло повторяться снова и снова достаточно долго. Бывало так: приходишь, а там уже другой человек, и все начинается сначала. Мне это надоело, и я решил, что вместо объяснений буду вручать эту книгу. Конечно, это шутка, но в каждой шутке есть доля правды.

Эта книга не претендует на звание универсального свода знаний о высоконагруженной обработке данных или создании систем реального времени. Она лишь отражает некоторый опыт в этой области. Мой опыт в основном касается создания распределенных систем управления промышленными критическими информационными системами. В таких системах присутствуют сотни тысяч источников изменений данных и их потребителей. Сценарии управления зависят от характера и интенсивности этих изменений. Возможно, экспертам в области корпоративных ИТ-систем используемая терминология покажется несколько непривычной, но она появилась из-за сильно отличающихся от ИТ и отчасти завышенных требований к промышленным системам.

Возникает вопрос: как обуздать этот хаос? Нужен *ordo ab chao*³. А что, если из хаоса достаточно создать порядок, но не полный? Как вы обычно просите своего ребенка, чтобы привел в порядок свою комнату? Каковы метрики требуемого результата? С какого момента беспорядок можно считать в большей степени порядком, чем беспорядком? Так происходит и с данными

³ Порядок из хаоса

в нагруженных системах. Чтобы с ними можно было работать, данные должны быть целостными. А какими должны быть метрики целостности (консистентности) данных, позволяющие однозначно сказать, готовы данные к обработке или нет? Ведь если управлять, например, опасным производством, основываясь на неконсистентных (устаревших) данных, может случиться авария.

Книга дает представление о том, как проектируют высоконагруженные распределенные системы управления и адресована всем, кто хочет разобраться, как устроены и создаются такие системы.

Вадим Подольный

Предисловие ко второму изданию

Первое издание успешно нашло своего читателя. Было получено множество положительных отзывов и критических замечаний, которые были учтены в настоящей редакции.

Второе издание книги существенно дополняет первое. Особое внимание уделено вопросу формирования кэша, консистентности, масштабирования распределенных высоконагруженных систем. Добавился раздел о применении разнообразных подходов и практик, используемых в архитектуре распределенных систем. Добавился раздел с некоторыми соображениями относительно кибербезопасности распределенных систем управления.

Таким образом, второе издание более целостно рассматривает проблематику архитектуры высоконагруженных, распределенных систем.

Обращение к читателю

Чаще всего в наше время понятие «высоконагруженные системы» используют применительно к компьютерной инфраструктуре массового предоставления услуг в Интернете 2.0. Причём массово используемые компьютерные системы — это и простейшие сим-карты размером с ноготь мизинца (да, сим-карты — это тоже компьютеры), и мощнейшие компьютерные установки-здания всем известных интернет-гигантов. Только энергопотребление в этих системах отличается в миллиард раз. При этом кажется, что нагрузить до предела возможностей можно любую компьютерную систему. Тем сложнее составить собственное представление о высоконагруженных компьютерных системах и связанных с ними понятиях.

Автор помогает читателю охватить широкий спектр понятий, связанных с высоконагруженными компьютерными системами, выступая как создатель и пользователь систем, которые успешно управляют работой энергоблоков атомных электростанций, обслуживают и защищают инфраструктуру крупнейших предприятий страны. Специалисты оценят емкость изложения тем, на которых автор предлагает сфокусировать внимание. Любознательные читатели в книге объемом всего лишь 250 страниц обнаружат большинство сочетаний терминов, связанных с понятием высоконагруженных компьютерных систем. Надеюсь, что книга подтолкнет читателя к более глубокому знакомству с высоконагруженными компьютерными системами, где и когда это будет ему наиболее интересно.

*Дмитрий Станиславович Северов,
экс-заместитель Министра связи и массовых коммуникаций РФ,
член совета Клуба топ-менеджеров
«Клуб директоров ИТ — 4 СИО.Ру»⁴*

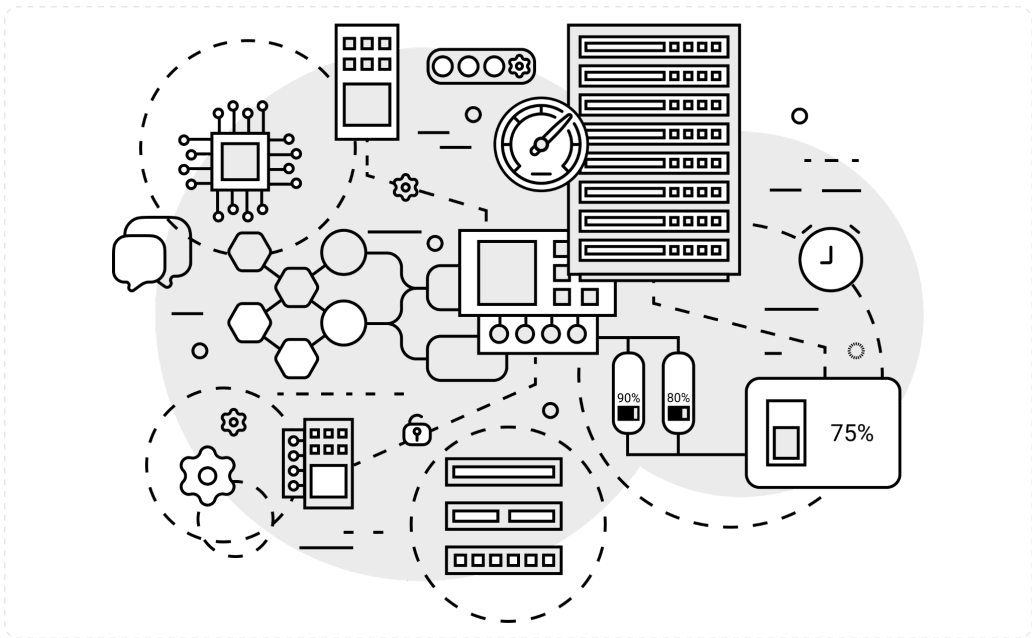
⁴ 4СИО, #A2

Приветственное слово

Современные тенденции развития программного обеспечения неразрывно связаны с необходимостью создания эффективной программной архитектуры. Наиболее чувствительной задачей, важной для экономики РФ, является разработка программных продуктов, направленных на решение задач построения распределенных систем управления объектами критической информационной инфраструктуры, таких как крупные промышленные предприятия, банки, сервисы массового обслуживания клиентов. В современных условиях необходимо уметь разрабатывать такие системы силами отечественных разработчиков. Именно перед российскими разработчиками стоит задача создания программных решений для обеспечения технологической независимости страны. Опыт, накопленный автором, в рамках решения указанных задач, консолидирует теоретические основы разработки распределенных высоконагруженных систем управления, которые будут полезны коллегам и начинающим разработчикам.

*Ренат Леонидович Лашин,
исполнительный директор, Ассоциации Разработчиков
Программных Продуктов (АРПП) «Отечественный софт»⁵*

⁵ АРПП, #A3



Глава 1. Введение

Что такое высоконагруженная система (ВНС)? Очевидно, ВНС — это система, которая обрабатывает высокие нагрузки. Как обрабатывает? На чем обрабатывает? Насколько эффективно обрабатывает? На все эти вопросы есть ответы. Любая ВНС должна обрабатывать данные эффективно. Эффективно, то есть задействуя все доступные ресурсы аппаратного обеспечения, такие как вычислительная мощность всех ядер процессоров, оперативная память, сетевые устройства. Не должно быть такого, чтобы из-за низкой эффективности (производительности) сетевых карт процессоры оставались загруженными не полностью. Такая ситуация означает, что аппаратное обеспечение подобрано неэффективно с точки зрения решения конкретной задачи обработки данных. Задачи обработки данных бывают самыми разными, их можно классифицировать. Для каждого класса задач обработки существует своя эффективная (оптимальная) аппаратная конфигурация. С экономической точки зрения

аппаратную конфигурацию рекомендуется подбирать таким образом, чтобы при максимальной загрузке все аппаратные компоненты использовались с максимальной эффективностью, а отдельные узлы и компоненты не простаивали.

Современные задачи требуют серьезных вычислительных ресурсов. Какова бы ни была конфигурация оборудования даже самого мощного сервера, который только можно собрать, одного такого устройства никогда не будет достаточно, чтобы решить задачу обработки высоких нагрузок, обусловленных наличием множества источников и потребителей данных. В ВНС обработку данных принято осуществлять на многочисленных объединенных в сеть устройствах, распределяя нагрузки между ними.

Оптимальное разделение нагрузки между распределенными устройствами — это сложная нетривиальная задача, которая напрямую определяет конечную стоимость аппаратного обеспечения. Она является ключевым показателем эффективности бизнеса — в особенности если таких устройств тысячи, десятки тысяч и более.

Существуют задачи, которые можно разделить (распределить) между совокупностью узлов: например, обслуживание сайтов в центрах обработки данных (ЦОД). За клиентом закрепляют определенный набор серверов (узлов), участвующих в его обслуживании. В рамках такого подхода в современных системах принято и рекомендовано использовать гиперконвергентные решения, которые представляют собой совокупность маломощных недорогих серверов. При этом вся работа, связанная с надежностью, выполняется за счет средств виртуализации и связанных средств резервирования. В таких случаях обслуживание клиента осуществляется на закрепленном за ним узле без особого взаимодействия с окружающими узлами. Естественно, средства виртуализации «съедают» часть ресурса оборудования, однако при этом они обеспечивают высокую бизнес-эффективность.

Существуют задачи обработки данных, в которых одни изменения влекут за собой созависимые каскадные изменения. В задачах автоматизированных систем управления технологическими процессами (АСУ ТП) изменения одних переменных оказывают влияние на другие переменные и требуют мгновенной реакции (отклика): например, перерасчета переменных. В таких задачах не всегда можно использовать гиперконвергентные подходы. Рекомендуется использовать принцип «быть ближе к железу», исключив все лишние прослойки, приводящие к потере производительности и эффективности использования аппаратных ресурсов.

ВНС, обеспечивающую выполнение проектных характеристик, таких как выполнение заданного числа операций в заданную единицу времени и время отклика операций, можно считать системой реального времени (CPV). CPV в заданный интервал времени обеспечивает выполнение запланированного числа операций. Любую ВНС можно проектировать как CPV или хотя бы как систему псевдореального времени. Как минимум, в ВНС стоит заложить некоторые полезные архитектурные свойства, присущие CPV.

CPV совсем не обязательно должна быть высоконагруженной, однако при решении современных задач большинство CPV представляют собой подмножество ВНС. Например, умный датчик (устройство IoT) вполне может работать в режиме реального времени (PV), но при этом не быть ВНС. Технологически такие компоненты обычно являются частью более крупных распределенных ВНС, о которых идет речь в этой книге.

Таковыми являются системы, в которых все компоненты должны работать максимально быстро и слаженно. Каналы связи должны обеспечивать высокую пропускную способность. Физические соединения и кабели должны быть произведены из лучших, эффективных по скорости прохождения импульсов и, следовательно, дорогостоящих материалов. В таких системах необходимо использовать быстродействующие микропроцессоры.

Должно быть обеспечено многократное резервирование компонентов. Использование таких ресурсов оправдано при решении ряда задач: например, таких как управление органами регулирования ядерного реактора и остальных систем управления атомной электростанцией, органами управления летательными аппаратами, системами управления вооружением, медицинскими системами. Такие системы принято относить к СРВ, а в некоторых случаях — к системам жесткого РВ, к компонентам которых предъявляют особенно высокие требования.

Для СРВ важнейшими понятиями являются дедлайн⁶, представляющий собой предельную длительность выполнения операции, и латентность⁷, представляющую собой время отклика (задержки) операции. Поэтому СРВ и некоторые ВНС принято называть низколатентными или ультра-низколатентными⁸ системами.

В области знаний о СРВ и их описаниях часто возникают путаница с терминами и споры об их использовании. С точки зрения РВ существуют системы как критичные по быстродействию, так и не очень. Некоторые эксперты пользуются связкой терминов «системы жесткого РВ» (критичные) и «системы РВ» (не очень критичные). Другие пользуются связкой «системы РВ» (критичные) и «системы псевдо-РВ» (не очень критичные).

Системы псевдореального времени наиболее широко применяют в условиях, при которых небольшое нарушение заданных дедлайнов является нормой, однако некоторое их превышение может привести к немедленному отказу системы. На практике такие системы создают иначе, нежели СРВ. Это связано с тем, что требования к ним несколько упрощают. Обычно проектируют некую систему, оценивают ее возможности, а затем масштабируют и оптимизируют в узких местах. Такой подход выходит дешевле создания канонической СРВ и тем более систем

⁶ Deadline, #1-01

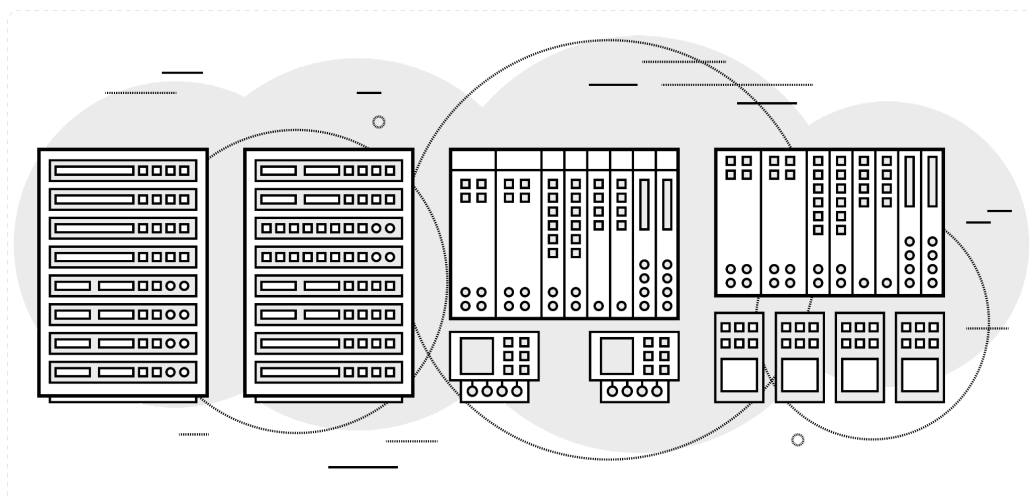
⁷ Latency, #1-02 ►

⁸ Low latency, ultra-low latency, #1-03

жесткого РВ, которые проектируют с большей ответственностью, предъявляя более жесткие требования к верификации и валидации.

Системы обычного времени (нормального времени) — это системы, в которые не закладывают требования к режимам работы РВ. Тем не менее такие системы могут быть эффективно и качественно разработаны, они могут обладать исключительными характеристиками производительности. Однако если в такую систему заранее не заложить требования режимов работы РВ, то при высоких нагрузках и перегрузках такая система с высокой вероятностью поведет себя непредсказуемо. Не всегда будет сразу понятно, что с ней не так, где именно возникли проблемы.

Самая большая проблема для лиц, принимающих решения, возникает в тот момент, когда появляется путаница в терминах. Разработчики навязывают систему, критичную к быстродействию, но реальная задача позволяет обойтись и не очень критичной. В таком случае возникает перерасход на разработку, закупку оборудования, пусконаладку, начинаются проблемы, связанные с обслуживанием серьезной, критичной к быстродействию системы. А такая дорогостоящая и «капризная» система нужна далеко не всегда.



Глава 2. Аппаратная архитектура

В ВНС и СРВ важнейшей составляющей является аппаратная инфраструктура. В этой книге не будут подробно описаны, но будут выделены несколько моментов, которые в конечном итоге влияют на надежность, эффективность и стоимость инфраструктуры целевой системы. При проектировании аппаратной инфраструктуры для ВНС РВ следует учитывать несколько факторов.

2.1. Выбор типа железа

При проектировании рассматриваемых систем всегда возникает вопрос выбора типа используемого железа: использовать меньше серверов, но крутых, мощных и дорогих с резервированием компонентов или больше, но дешевых и слабых без резервирования компонентов? На этот вопрос можно ответить, рассмотрев задачу как с точки зрения аппаратного обеспечения (АО), так и в совокупности с программным обеспечением (ПО), потому что именно оно влияет на итоговую эффективность работы железа.

При выборе между «круто и дорого» и «завалить кучей дешевого железа» есть один правильный алгоритм действий: необходимо тщательно оценить технические требования, географию, условия размещения оборудования, доступность

технического персонала на местном рынке труда и т. д. В абсолютном большинстве случаев этот выбор является надуманным. Если пройти все этапы проектирования системы, то либо многообразие вариантов сократится до одного безальтернативного, после чего останется только провести тендер на поставку, либо существенно сократится число вариантов выбора оборудования.

2.2. Место размещения оборудования

Место размещения оборудования почти всегда обусловлено правильно сформулированными при проектировании бизнес-требованиями и техническими требованиями. География распределенных кластеров напрямую зависит от показателей RPO/RTO (разд. 7.2.2) и требований по защите от катастроф (разд. 7.2.3). Выбор между коммерческим ЦОД и локальным размещением на площадке предприятия вытекает из требований к удаленности, безопасности, защищенности и резервированию каналов связи.

Проектирование обычно осуществляется сверху вниз путем декомпозиции от общего к частному. Важно своевременно учитывать возможности самого нижнего уровня, чтобы компенсировать его недостатки на верхнем. При отсутствии подходящих аппаратных решений и/или площадок для размещения оборудования можно снизить требования к нижнему уровню (площадке) за счет использования эффективной программной архитектуры и реализации общего уровня защищенности на более высоких уровнях. Важно помнить, что требования архитектуры целевой системы — это способ реализации основной бизнес-задачи, а не истина в последней инстанции. Всегда существует несколько способов реализации таких требований.

2.3. Оценка конфигурации оборудования при нагрузке

Необходима определенная аккуратность в оценке комплектации оборудования. Например, может так случиться, что при максимальной нагрузке на сетевые устройства процессоры

серверов не будут эффективно загружены. В таких случаях можно оптимизировать соответствующие аппаратные ресурсы, увеличить мощность сетевого оборудования или уменьшить мощность процессоров, при этом даже немного сэкономя на них.

Требования к конфигурации оборудования должны исходить из следующих целевых показателей:

- производительность и загрузка при установленной нагрузке;
- производительность и загрузка при пиковой нагрузке;
- уровень резервирования;
- размеры доменов единичного и двойного отказа;
- стандартизация конфигураций;
- доступность оборудования и конфигурации (комплектующих) на рынке, планы по поддержке производителем;
- доступность квалифицированного персонала на локальном рынке труда.

Зачастую имеет смысл разместить достаточно мощное оборудование с большим запасом по производительности и резервированию на удаленных площадках для максимизации времени безостановочной работы без обслуживания при возникшем сбое. При условии логистической доступности и наличия большого склада ЗИП⁹ в крупном центре можно обойтись меньшим уровнем резервирования, исходя из статистики отказов и своевременного пополнения ЗИП.

2.4. Зависимость от вендора или технологии

Необходимо оценить риски зависимости от одного вендора или конкретной технологии. Всегда лучше рассчитывать на то, что и вендора, и технологию в какой-то момент придется заменить. Или сам вендор может выбрать другую технологию для своего будущего оборудования, и ваши системы будут несовместимы. Например, бывает так, что программная инфраструктура

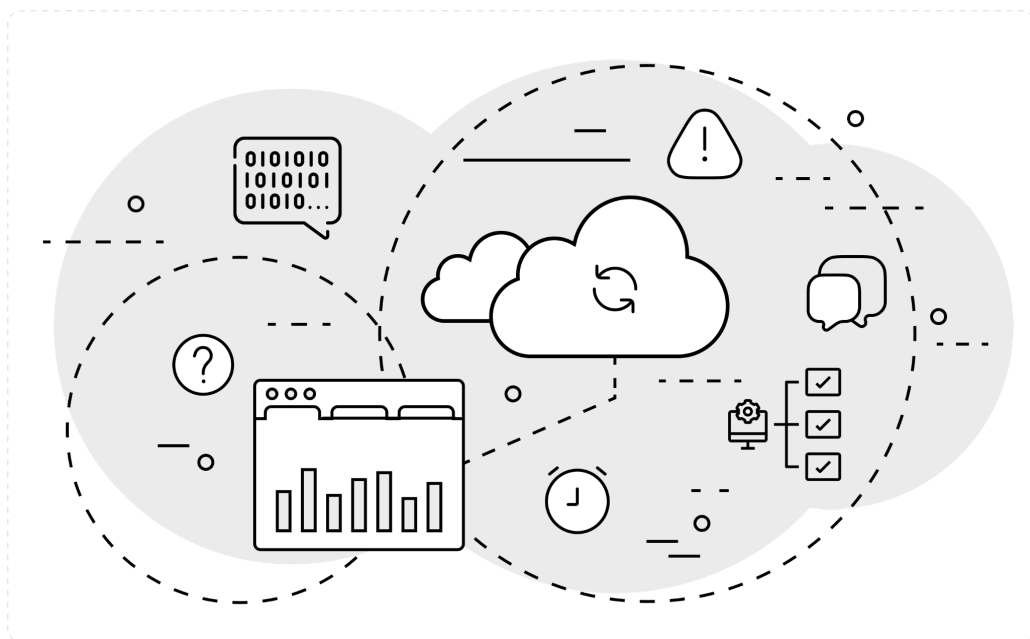
⁹ Запасные части, инструменты и принадлежности

использует особую аппаратную технологию. В этой связи можно вспомнить классический пример того, как компания HP перестала поддерживать серверы AlphaServer с архитектурой RISC в начале 2000-х годов. Такие решения продолжают использовать и сейчас на множестве предприятий, и у многих из них не осталось резервов (ЗИП). Системы такого рода продолжают эксплуатировать, потому что разработчики ПО, занимавшиеся уже устаревшей архитектурой, уже на пенсии или недоступны, поэтому портировать системы на современную архитектуру уже некому. Модернизировать такие системы можно только целиком. Представьте себе, например, что это АЭС с несколькими энергоблоками. Такие ситуации сплошь и рядом возникают во всех отраслях и странах мира. Возникает вопрос организации технической и гарантийной поддержки большого разнообразия оборудования и решениями на основе одного вендора.

Крупные сервисные провайдеры и интеграторы самостоятельно осуществляют сборку оптимальных конфигураций оборудования для конкретных инфраструктурных решений, что довольно эффективно. Например, Google, Amazon, Яндекс собирают свои серверные и инфраструктурные решения, самостоятельно контролируя архитектуру, весь процесс производства и жизненный цикл.

2.5. Планирование масштабирования

Необходимо заранее планировать сценарий масштабирования ресурсов ([Глава 10](#)). Следует планировать возможность увеличения мощности серверов за счет добавления в них дополнительных процессоров, оперативной памяти, сетевых карт. Кроме того, следует планировать увеличение числа серверов. Бывают ситуации, когда программная инфраструктура рассчитана на некое предельное число узлов и просто не может работать при большем их числе. О таких ограничениях лучше знать заранее. Может случиться так, что условия лицензии на дополнительные узлы не устроят потребителя.



Глава 3. Программная архитектура

3.1. Данные и метаданные

Основная нагрузка в ВНС и СРВ связана с обработкой большого объема (потока) сообщений (данных, событий) из разнообразных внешних источников и внутренних составных компонентов системы. Принято выделять следующие типы событий:

- асинхронные — полностью непредсказуемые события, такие как оповещения, тревоги, команды управления;
- синхронные — предсказуемые события, случающиеся с определенной частотой, такие как сообщения, связанные с обменом и синхронизацией данных;
- изохронные — регулярные события (разновидность асинхронных), случающиеся в определенный промежуток времени, такие как диагностические сообщения о состоянии компонентов системы (разд. [7.1.5](#)).

Не все данные следует обрабатывать в режиме РВ (данные РВ). В СРВ большая часть данных имеет временную привязку. Часть данных стоит отнести к вспомогательным — такие данные нужны для функционирования системы, но пользователю они незаметны. Их не всегда нужно обрабатывать в режиме РВ. Например, не следует обмениваться текстовыми переменными в режиме РВ, если переменные можно задать заранее и обмениваться их идентификаторами. Для облегчения работы с данными вводят дополнительные данные о содержании данных. Например, для фотоизображения вспомогательными данными могут служить дата, координаты, выдержка. Такие данные о данных принято называть метаданными.

Может быть и обратная ситуация, когда часть метаданных разумнее отнести к данным РВ в связи с высокой частотой их использования. Следовательно, такие данные необходимо обрабатывать соответствующим образом, при этом важно соблюдать баланс между объемом обрабатываемых данных РВ и метаданными. Если не оптимизировать формирование объема данных РВ, это может привести к излишним нагрузкам, которые могут повлечь за собой снижение эффективности системы. При этом производительность системы может и не снизиться. Системе не важно какие данные она обрабатывает: важные, вспомогательные или бесполезные, если только она не спроектирована с учетом соответствующих требований.

3.2. Способы обмена данными

В работе современных ВНС принимает участие множество узлов, выполняющих разнообразные задачи. Такие системы называют распределенными вычислительными системами (РВС), распределенными системами обслуживания (РСО), распределенными системами управления (РСУ¹⁰). В общем случае РВС и РСО чем-то управляют — хотя бы собственными вычислениями, обслуживанием потребителей.

¹⁰ DCS, distributed control system, #3-01

Такое допущение позволяет классифицировать их как РСУ для удобства описания. Основными объектами внимания настоящей книги являются высоконагруженные РСУ (ВРСУ).

Узлы РСУ могут быть загружены по-разному. Они могут представлять собой серверы, рабочие станции, коммутаторы, умные устройства интернета вещей. Узлы с разной степенью активности (непрерывности работы и доступности) могут обмениваться сообщениями между собой в рамках определенной событийной модели.

Чтобы обеспечить эффективное взаимодействие систем, следует выбрать выгодный способ взаимодействия (обмена данными) между узлами. В СРВ не принято терять время на установку TCP-соединений: если таких соединений требуется много, то суммарное время, затраченное на установку множества соединений, может повлиять на эффективность работы всей системы. TCP преимущественно является *unicast*-протоколом, поэтому каждое TCP-соединение будет обработано отдельно.

В РСУ возникает необходимость доставки одинаковых сообщений более чем на один узел одновременно — следовательно, множество (пул) TCP-соединений оказывается неэффективным. В таких случаях рекомендуется использовать протокол UDP в режиме широковещательных сообщений типа *multicast* или режиме *broadcast*. TCP можно использовать в режиме *broadcast*, однако при этом могут возникать некоторые задержки в связи с циркулирующими подтверждающими пакетами, которые создают лишнюю нагрузку. Бывают ситуации, когда выгодно установить пул устойчивых TCP-соединений и осуществить имитацию широковещательных сообщений поверх совокупности таких соединений. Обработка каждого следующего соединения может происходить с некоторой задержкой. В случае использования пула TCP-соединений латентность системы будет выше, чем при использовании протокола UDP (рис. 3-01).

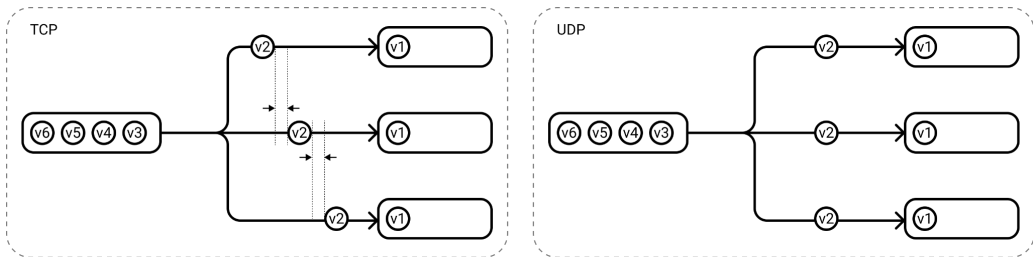


Рис. 3-01. Разница в задержках: UDP, TCP

Иногда возникают неприятные ситуации: новые сообщения могут устаревать в процессе получения адресатами. Следовательно, у части адресатов будут неактуальные данные, что может быть весьма неприятно в контексте задач управления. В СРВ невыгодно применять TCP-соединения в связи с тем, что на базе TCP сложнее реализовать пул асинхронных сообщений. В реальном мире события чаще происходят в асинхронном режиме. Кроме того, использовать UDP-соединения в асинхронном режиме выгоднее. Разработчик может самостоятельно контролировать обмен диагностическими сообщениями, которые могут содержать диагностические метрики. Такие метрики могут описывать показатели качества обмена сообщениями: например, метрики подтверждения доставки сообщений согласно требованиям разработчика, состояние отдельных компонентов и процессов системы (разд. 7.1).

3.3. Транзакции

В ВНС использование транзакций допустимо, а в СРВ — нет. Следует вспомнить принцип ACID¹¹, описывающий требования к транзакциям:

- *Atomicity* — атомарность гарантирует, что каждая транзакция будет выполнена полностью или не будет выполнена совсем, промежуточные состояния не допускаются;
- *Consistency* — консистентность (согласованность, разд. 6.6) означает, что транзакция, достигающая своего нормального

¹¹ ACID, #3-02

завершения (EOT – *end of transaction*) и тем самым фиксирующая результаты, сохраняет согласованность данных;

- *Isolation* – изолированность означает, что во время выполнения транзакции параллельные транзакции не должны влиять на результат;
- *Durability* – надежность транзакции означает, что, независимо от проблем на нижних уровнях (например, обесточивание системы или сбой в оборудовании), изменения, возникшие в результате успешно завершенной транзакции, должны остаться сохраненными после возвращения системы в работу. Если пользователь получил от системы подтверждение, что транзакция выполнена, он может быть уверен, что внесенные им изменения не будут утрачены из-за какого-либо сбоя.

В СРВ не принято¹² использовать транзакции. Это связано с тем, что процесс обработки транзакции чаще является синхронным. Когда узлов, участвующих в обработке транзакции, становится больше одного, транзакция становится распределенной (рис. 3-02).

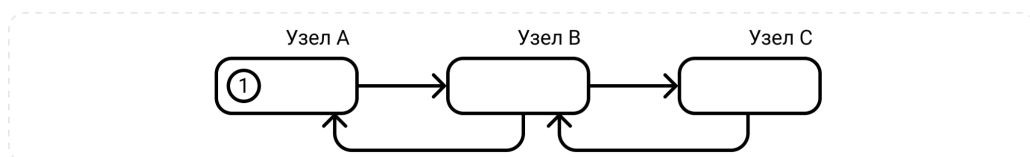


Рис. 3-02. Распределенная транзакция

Таким образом, узел А (инициатор транзакции) ждет, пока транзакция 1 будет обслужена всеми участвующими в процессе узлами. Результат выполнения транзакции вернется со всеми вытекающими задержками и блокировками. В таком случае задержки являются лишь частью проблемы. В ВНС транзакции обрабатываются очередями (разд. 5.4.1). При этом крайне выгодно задействовать все ресурсы вычислительного оборудования. Если спроектированное ПО является эффективным, задержка ожидания

¹² В ВНС тоже не принято, но есть решения, такие как 2PC, разд. 11.2.8

выполнения сложных распределенных транзакций может быть скомпенсирована: например, их количеством. Обработка новых транзакций из очередей может быть начата до завершения активной транзакции (разд. 11.1.3). В таком случае распределенным транзакциям придается некое свойство асинхронности обработки, а блокировки ожидания выполнения транзакций влияют только на их инициаторов. Такой механизм асинхронности позволяет эффективно задействовать значительное число распределенных узлов для децентрализации, распределения сколь угодно сложной логики (рис. 3-03) (разд. 11.2.4).

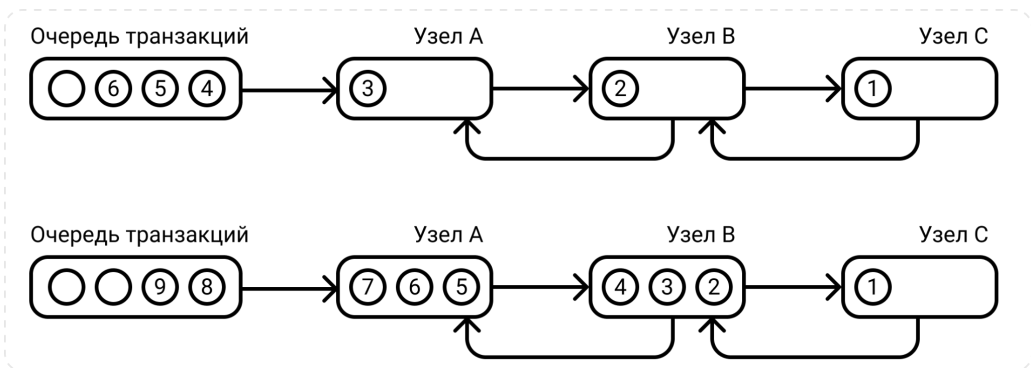


Рис. 3-03. Распределенная транзакция асинхронная

Такой процесс, называемый асинхронной обработкой распределенных транзакций, вместо обеспечения эффективности их обработки может привести к дисбалансу. В связи с тем, что обработка новых транзакций из очереди начинается до завершения уже начатой обработки предыдущих транзакций, среднее время обработки транзакции определенно будет увеличиваться. Таковую систему тяжело балансировать из-за неизвестных времени задержки и длительности блокировки, в особенности если целевая система позволяет использовать непредсказуемо сложную логику, задаваемую пользователем, а не жестко реализованную в системе.

3.4. Распределенность

Узлов-инициаторов, обработчиков и верификаторов транзакций в РСУ может быть множество. Такую систему необходимо непрерывно балансировать (Глава 8), чтобы обеспечить слаженность работы всех распределенных узлов (планировать их нагрузку). При условии, что обрабатывающие транзакции узлы не оказывают влияния на вычислительные процессы друг друга, балансировать такую конструкцию достаточно просто. Вычислительные алгоритмы, выполняемые в форме распределенных транзакций, могут использовать данные узлов, которые участвуют в обработке транзакций. Алгоритмически зависимые узлы могут непредсказуемо влиять на общую нагрузку, поэтому балансировать такую систему еще сложнее (рис. 3-04).

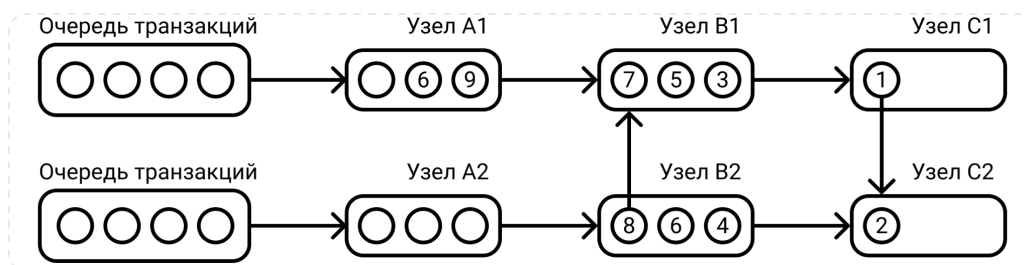


Рис. 3-04. Распределенная транзакция с зависимостью

Этот вопрос хорошо отработан в реляционных системах управления базами данных (СУБД), которые представляют собой специальные инструменты обработки распределенных транзакций¹³. Эффективность вычислений обеспечивают оптимизация структур обрабатываемых данных и оптимизация конфигурации СУБД, однако такая оптимизация практически не приближает обработку данных к режиму РВ. Скорость обработки в таких СУБД существенно ниже, чем в специализированных системах. Такого рода специальные системы становятся все более востребованными на современном рынке. Большинство реляционных СУБД осуществляют хранение данных на жестких дисках. Универсальный табличный (реляционный) формат требует постоянной

¹³ Каскадные транзакции, #3-03

дополнительной обработки данных. Следовательно, необходимо разбираться, откуда прочитать, куда записать, как найти, где обработать и куда сохранить индекс. Современные реляционные СУБД (РСУБД) позволяют хранить как данные, так и индексы в оперативной памяти, но сложность обработки реляционных структур накладывает свои ограничения. Транзакции, построенные на базе РСУБД, не отличаются особым быстродействием, которое необходимо СРВ и большинству ВНС. В СРВ стараются отказываться от транзакций в классическом понимании, используя более эффективные механизмы обработки данных и балансировки (планирования) распределенных взаимодействий. Для этого создана масса специализированных решений класса NoSQL, NewSQL — каждое из них имеет свои преимущества и недостатки. Естественно, разнообразные решения, эффективные для реализации целевых требований, принято комбинировать в единую взаимодействующую систему, интегрируя актуальные свойства каждого из них.

3.5. Ограничения распределенных систем

В любой распределенной системе действует принцип теоремы CAP¹⁴, которая гласит, что при реализации распределенных вычислений можно обеспечить не более двух из трех следующих требований:

- согласованность (консистентность) данных (*consistency*) — во всех не отказавших вычислительных узлах в один момент времени данные не противоречат друг другу, то есть являются одинаковыми;
- доступность (*availability*) — любой запрос к не отказавшим узлам возвращает корректный отклик, при этом нет гарантий, что ответы всех узлов системы совпадут;
- устойчивость к разделению (*partition tolerance*) — разделение распределенной системы на несколько изолированных сегментов в связи с нестабильностью соединения или его

¹⁴ CAP theorem, [#3-04](#), [#3-05](#) ►

полным отказом. В этом случае система должна продолжать функционировать, то есть перебои связи не должны приводить к отказу системы.

Типичные РСУ построены на сочетаниях требований CAP-теоремы (рис. 3-05):

- согласованные РСУ со всегда доступными узлами, но неустойчивые к разделению (CA);
- устойчивые к разделению РСУ со всегда с доступными узлами, но не согласованные (AP);
- согласованные и устойчивые к разделению РСУ, но с не всегда доступными узлами (CP).

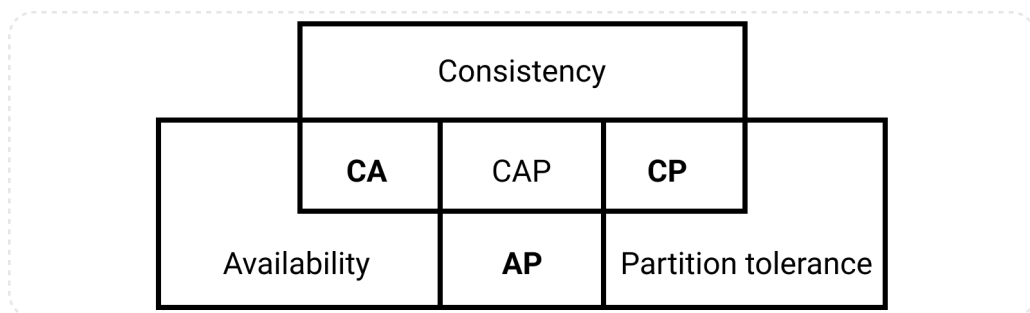


Рис. 3-05. CAP-теорема

В реальных задачах часто требуется, чтобы одновременно работали все три свойства теоремы. Возникает вопрос: как этого достичь, какие допущения и упрощения можно сделать, чтобы приблизиться к выполнению всех трех требований CAP? Опытным профильным архитекторам и разработчикам секрет давно известен: он заключается именно в свойствах консистентности, которые могут быть самыми разнообразными. Консистентность может быть как строгой, так и слабой, как длительной, так и периодической (разд. 6.6).

Например, если рассматривать РСУ в составе АСУ ТП, становится очевидно, что в таких системах недопустимо разделение узлов — может произойти потеря управления важной подсистемой.

Совершенно недопустима несогласованность данных. Например, если операторы будут работать с отличающимися данными, их действия будут несогласованными. Такая ситуация может привести к катастрофе. Естественно, в таких системах допустима потеря связи между узлами. В серьезных АСУ предусмотрена возможность автономной работы отдельных подсистем и даже автономного управления. Оборудование не будет повреждено даже при разделении целостной системы на подсистемы, однако такая АСУ долго не проработает в таком режиме. Существуют специальные регламенты, определяющие, сколько времени можно проработать в режиме автономности подсистем. После исчерпания регламентного времени технологический процесс следует остановить, что зачастую бывает очень дорого и небезопасно. Так, например, остановка ядерного реактора АЭС стоит десятки миллионов долларов.

Рассматриваемые системы, а именно ВНС, СРВ, РС(У), обладают различными свойствами, выполняют самые разнообразные задачи. ВНС часто являются распределенными (ВН РС), существуют ВНС, работающие в режиме РВ (ВНС РВ). Встречаются и РС(У), работающие в режиме РВ (РС РВ). Реже встречаются ВН РС, работающие в режиме РВ (ВН РС РВ): например, масштабные АСУ ТП. Как правило, СРВ является составной частью ВНС, которая, в свою очередь, может быть частью РС(У) (рис. 3-06).

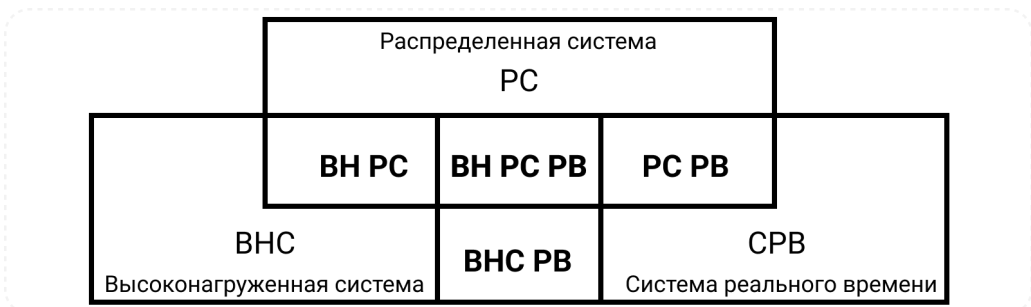


Рис. 3-06. Связь типов систем

Хороший пример противоположности принципов и задач, выполняемых такими системами, можно продемонстрировать,

сравнив, к примеру, системы промышленной автоматизации с банковскими системами. Например, если некий вычислитель вовремя не получит значение температуры реактора от одного или нескольких датчиков или получит его, но не вовремя, может возникнуть серьезная проблема. А вот если клиент банка получит сообщение о зачислении зарплаты немного позже, чем ожидал, или даже совсем его не получит, с ним ничего не случится — он зайдет в приложение и увидит, что деньги пришли. Ну или с банком поругается немного, не катастрофа. Обычно в зарплатные дни банковские системы испытывают высокие нагрузки: банки тщательно обрабатывают все транзакции и контролируют, чтобы все получили сообщения о начислении зарплаты и не звонили с вопросами. Банк не направляет клиенту более одного сообщения о начислении заработной платы, иначе клиент может обрадоваться раньше времени. В промышленной автоматизации, напротив, в АСУ ТП может поступать множество повторных сообщений о значениях температуры и ее изменениях. Если часть сообщений потеряется, ничего страшного не произойдет. В критически важных компонентах располагают множество избыточных резервных датчиков. Таким образом, подтверждающая информация о доставке значений температур с этих датчиков становится не совсем обязательной. При подаче команд управления, естественно, необходимо подтверждение их выполнения. В таких случаях команда управления обычно запускает параллельные каскадные изменения ряда параметров, которые формируют множество обратных связей. Множество таких связей могут привести к частому обновлению состояния переменных. Изменения в трафике, касающиеся команд, составляют долю процента от общего объема трафика. Следовательно, часть данных в АСУ ТП можно потерять. Вопрос о том, какой объем данных можно потерять, — это вопрос конкретных условий целевой задачи. Такое допущение позволяет «нарушить» CAP-теорему, и «выполнить» все три ее условия одновременно, так как потеря части данных не повлечет за собой утрату консистентности АСУ ТП. В правильно спроектированных РСУ АСУ ТП сообщения приоритизируются по

значимости, при этом допускается потеря наименее значимых данных, а наиболее приоритетные сообщения обрабатываются с подтверждением, почти как транзакции.

3.6. Единый формат данных

Основу быстродействия любой системы составляет отсутствие задержек при лишних преобразованиях форматов данных. Это значит, что в идеальной ситуации все узлы РСУ взаимодействуют между собой в едином формате в рамках формализованного протокола. Чтобы можно было быстро и эффективно обрабатывать, считывать и записывать (перезаписывать) данные, структура их хранения должна быть максимально упрощенной и соответствовать выбранному протоколу.

Данные, участвующие в обмене между узлами, могут быть отделены от метаданных, кроме отдельных специфических случаев. В реальных РСУ редко возникают ситуации, при которых в нагрузке изменяется структура данных: обычно меняются только значения переменных. Как правило, структура данных системы статична. Конфигурация структуры может быть загружена из файла или получена из РСУБД (RDBMS¹⁵) при старте узлов или в режиме минимальной нагрузки. Конфигурация структуры данных РСУ обычно находится в составе метаданных. Все сложные связи, определяющие структурную принадлежность, иерархичность и связность, могут находиться в метаданных. Все, что можно убрать из трафика обмена данными между узлами в режиме эксплуатации под нагрузкой, рекомендуется определять как метаданные.

3.7. Операции с данными

В высоконагруженных системах, в особенности в системах РВ, операции CRUD¹⁶ принято приоритизировать по значимости. Наиболее частыми (востребованными) являются операции чтения

¹⁵ Relational Database Management System, [#3-06](#)

¹⁶ CRUD — операции Create, Read, Update, Delete, [#3-07](#)

(*read*) и перезаписи (*update*). Операции создания сущности (*create*), как правило, сопровождаются выделением динамической памяти, а операции удаления (*delete*) — высвобождением динамической памяти. Данные операции не рекомендуется выполнять в режиме РВ, так как они являются затратными с точки зрения ресурсов. Если *create* — достаточно безопасная операция, то *delete* таковой совсем не является, поэтому в СРВ рекомендуется отказаться от этой операции и вместо физического удаления пометить структуру как удаленную. Физически операцию удаления рекомендуется проводить в режиме сервисного обслуживания или хотя бы в период, когда система достаточное время не будет под нагрузкой. В РСУ рекомендуется избегать любых динамических операций с данными, способных привести к нарушению целостности.

3.8. Распределенные операции с данными

Операции CRUD следует рассматривать с точки зрения распределенности узлов. Какой бы ни была распределенная структура узлов, все распределенные операции должны выполняться с учетом того факта, что узлов более одного. Все распределенные операции должны осуществляться с требуемой эффективностью, с минимальными задержками и, возможно, даже с поддержкой параллелизма.

Если в распределенной системе выполняется операция *create*, значит, должно появиться требуемое число копий сущностей, определяемых конфигурацией целевой системы. Следовательно, при выполнении операции *delete* эти копии должны быть уничтожены. Если осуществляется операция *read*, считывание данных должен выполнять узел, ближайший к источнику запроса. В некоторых случаях при новом запросе ячейка чтения может быть не заполнена, а в случае долгого отсутствия запроса она может перестать обновляться. Возникает дилемма: что выгоднее — поддерживать в актуальном состоянии ячейку данных или обновлять ее при запросе, при определенной частоте запросов или, например, по расписанию?

Операцию *update* в распределенной системе следует рассмотреть подробнее. По сути, *update* отвечает именно за поддержание актуальности той самой ячейки памяти. Однако поддержание ее актуальности может осуществляться с различным приоритетом в зависимости от значимости обрабатываемой переменной. В CPB рекомендуется выделять три вида таких операций:

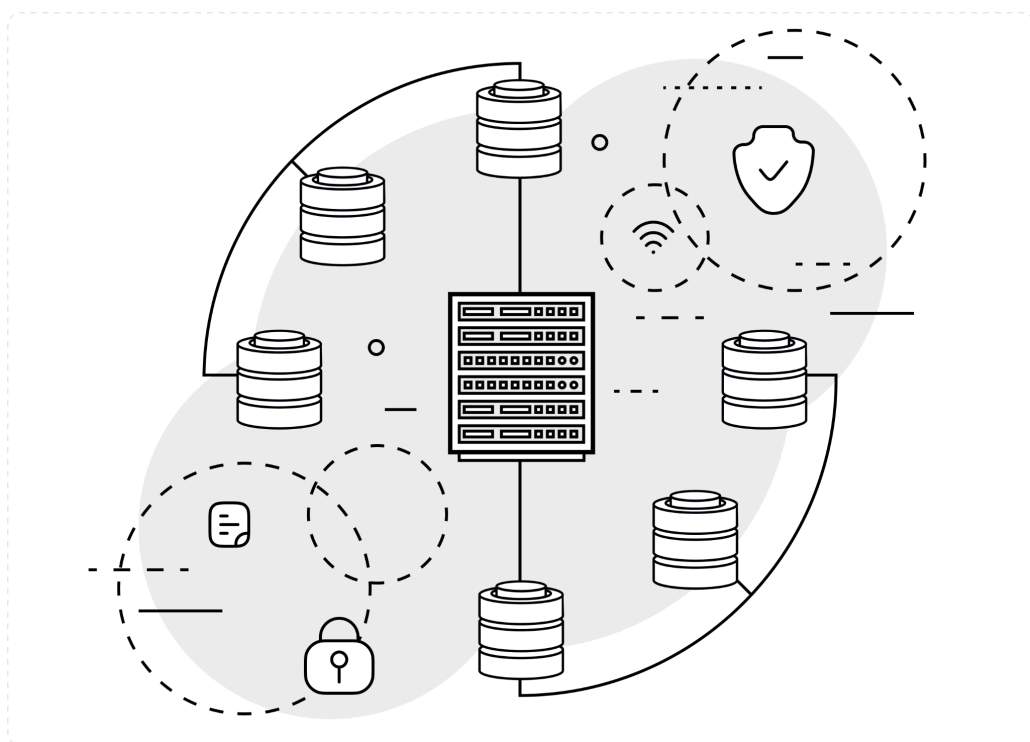
- негарантированный *fast update*¹⁷;
- гарантированный *strict update*;
- гарантированный с подтверждением *hard update*.

Простой негарантированный *update* не гарантирует доставку на целевые узлы и не предполагает подтверждения операции. Это самый быстрый способ распределенной модификации ячейки данных, применяемый для переменных, потеря значений которых может быть скомпенсирована следующим циклом обновления значений.

Гарантированный *update* гарантирует распространение изменений по всем распределенным целевым узлам. Гарантированный *update* с подтверждением обеспечивает самый надежный способ обновления данных с подтверждением. Выполнение такой операции будет происходить медленнее остальных. Ее следует использовать для работы с переменными, отвечающими за критически важные данные: например, когда нужно передать команду управления. В связи с тем, что такие операции являются наиболее ресурсозатратными, рекомендуется свести к минимуму их применение.

Перечисленные выше распределенные операции представляют собой механизмы распространения данных в распределенной системе, обеспечивающие их системную избыточность. Именно такие операции обеспечивают поддержание данных в актуальном состоянии при их изменении, по сути, осуществляя синхронизацию изменений (разд. 6.5).

¹⁷ Best effort [#3-08](#)



Глава 4. Принципы распределения данных

4.1. Распределенная обработка данных

В современных системах, сколь бы мощными ни были ресурсы узла (сервера), они в какой-то момент заканчиваются. В серьезных системах данные целесообразно обрабатывать на совокупности распределенных узлов. Это значит, что сначала данные необходимо распределить таким образом, чтобы их можно было удобно и эффективно обработать в распределенном режиме. Каждый такой узел обладает определенным объемом оперативной и постоянной памяти, определенными мощностями процессоров и пропускной способностью сетевых карт. В СРВ не принято задействовать постоянную память (ПЗУ), какими бы производительными ни были устройства.

Распределение данных между узлами может осуществляться в зависимости от их аппаратных конфигураций:

- совокупность узлов с одинаковой конфигурацией;
- совокупность относительно равных по конфигурации узлов;
- совокупность групп, относительно равных по конфигурации узлов;
- совокупность неравных по конфигурации узлов.

Наиболее эффективным для систем является первый вариант — по разным причинам, которые будут рассмотрены далее в разделах [4.3](#), [4.4](#). Жизненный цикл любой системы включает периодическое масштабирование (разд. [10.1](#)), модернизацию парка оборудования и его компонентов, поэтому второй вариант встречается чаще. Если речь идет о задачах, в которых встречаются граничные (*edge*¹⁸) и туманные (*fog*¹⁹) вычисления, то такие системы будет описывать третий вариант. Если в системе встречается множество разнообразных источников и потребителей, например, устройств интернета вещей (IoT), то такую систему будет описывать четвертый вариант.

Распределение данных между узлами может осуществляться с формированием избыточности. Принцип избыточности широко известен и применяется в различных информационных системах. Например, в системах хранения данных (СХД) для надежности используют разнообразные избыточные массивы независимых дисков (RAID²⁰). Существует целый ряд конфигураций таких массивов, в которых используются разнообразные способы организации избыточности. RAID эффективно работают, когда устройства хранения данных (ПЗУ, HDD, SSD), на базе которых они созданы, одинакового размера. В распространенных решениях СХД RAID-массивами управляют один или два узла, работающие в режиме *active-active* или *active-passive*. Решения с более чем двумя управляющими узлами (контроллерами) RAID встречаются редко

¹⁸ Edge compute, [#4-01](#)

¹⁹ Fog compute, [#4-02](#)

²⁰ Redundant Array of Independent Disks, [#4-03](#)

в связи с их сложностью и дороговизной. Обычно это два сервера, подключенные по резервируемым портам к специальным устройствам (полкам²¹) с дисковыми массивами. Современные RAID-системы, основанные на технологии All Flash на базе NVMe-OF²², могут быть созданы деагрегированными. А именно: они могут быть разнесены по нескольким узлам группы серверов, каждый из которых управляет своими сегментами (разд. 4.4) NVMe²³ SSD. Для этого может быть использован специальный протокол RoCE²⁴. Такие решения следует считать более близкими к программно-определяемым.

Существуют решения, которые представляют собой программно-определяемые хранилища, устроенные иначе, нежели классические RAID. Такие программно-определяемые хранилища могут быть созданы на базе узлов с разнообразной аппаратной конфигурацией. Конструкции подобного рода принято называть избыточными массивами независимых узлов (RAIN²⁵).

В PCY активно применяют как классические СХД, так и программно-определяемые СХД на базе RAIN. В любом случае до того, как данные попадут на физические носители таких решений, они сначала окажутся в оперативной памяти. Вопрос обмена данными между оперативной памятью и массивами дисков не является задачей РВ и осуществляется в режиме ограничения скоростей, доступных в соответствующих решениях.

PCY могут использовать для обработки данных оперативную память распределенных узлов как общее пространство. Такая конструкция, предназначенная для формирования общей оперативной памяти узлов, представляет собой особый вид RAIN. Задачи создания общей оперативной памяти лежат в области асимметричного и симметричного мультипроцессирования (AMP²⁶,

²¹ JBOD, JBOF, FBOF, [#4-04](#)

²² NVMe over Fabric, [#4-05](#)

²³ Non-Volatile Memory Host Controller Interface Specification, [#4-06](#)

²⁴ RDMA over Converged Ethernet, [#4-07](#)

²⁵ Redundant Array of Independent Nodes, [#4-08](#)

²⁶ Asymmetric multiprocessing

SMP²⁷), аналогично задаче превращения распределенных процессоров в единый вычислительный модуль (кластер). AMP и SMP — это технологии объединения находящихся на одной материнской плате процессоров, задачей которых является обеспечение разграничения доступа к оперативной памяти.

Сейчас, в эпоху тотальной виртуализации, задачу объединения ресурсов распределенных узлов формулируют как задачу виртуализации SMP²⁸. Задача объединения ресурсов распределенных узлов в единую конструкцию (разд. 4.2), по сути, является задачей, обратной классической виртуализации, в рамках которой решается задача разделения большого ресурса на меньшие с возможностью программного управления их активностью..

Именно такую конструкцию предлагается рассмотреть для управления распределенной обработкой данных в PCY. Такая конструкция по своей сути является распределенной разделяемой памятью.

4.2. Распределенная разделяемая память

Эффективным механизмом организации распределенного хранения данных является организация распределенной разделяемой памяти (РРП, DSM²⁹). РРП представляет собой разновидность технологии программно-определяемой памяти (SDM³⁰). РРП образует виртуальное адресное пространство, разделяемое всеми узлами PCY (программно-определяемое). Узловые сервисы (программы, сценарии) в PCY получают доступ к данным в РРП. РРП может быть организована статически, когда данные РРП не перемещаются между узлами, или динамически, когда они перемещаются между узлами (разд. 4.2.2).

²⁷ Symmetric multiprocessing, #4-09

²⁸ Следует обратить внимание на vSMP от ScaleMP, #4-10

²⁹ Distributed shared memory, #4-11

³⁰ Software defined memory

При организации РРП необходимо решить несколько основных вопросов:

- поддержки актуального состояния (целостности, разд. 6.6) структуры распределения данных;
- снижения латентности при доступе и обработке данных;
- обеспечения доступности данных на нескольких узлах одновременно для повышения производительности.

С точки зрения избыточности данных РРП может работать в нескольких вариантах режимов:

- данные на узлах не повторяются;
- данные повторяются на части узлов;
- данные повторяются на всех узлах.

В первом и втором вариантах данные могут быть распределены по узлам в зависимости от задачи системы и требуемой логики обработки данных. В отличие от первого варианта, второй подразумевает (одновременно обеспечивает) резервирование данных (разд. 6.1). Такой способ распределения данных называют сегментированием (разд. 4.4). Третий вариант обеспечивает многократное (кратное числу узлов) резервирование данных в системе. Это подразумевает наибольшие затраты на аппаратную конфигурацию узлов, при этом узлы должны иметь одинаковую конфигурацию. Обычно такие конструкции применяют для построения кластерных систем обработки данных (Глава 9).

Сегментирование РРП реализует прикладные цели:

- обеспечение эффективного резервирования данных, чтобы при отказе узлов данные не пропали;
- обеспечение оптимального использования (минимизации) аппаратных ресурсов при синхронизации сегментов;
- обеспечение большей доступности данных для узлов, осуществляющих их обработку, или иных потребителей.

Существует целый ряд алгоритмов организации РРП, основанных на следующих структурах данных:

- РРП на основе разделяемых страниц (блоков данных);
- РРП на основе разделяемых переменных;
- РРП на основе разделяемых объектов (структур данных).

Иных конструкций организации РРП не существует. Не имеет значения, какая логика взаимодействия задана на базе приведенных возможных архитектур РРП. Как теоретически, так и практически логика распределения данных может быть изолирована от логики целевых задач РСУ. Следовательно, на базе РРП можно одинаково успешно реализовать распределенную (*key-value*), или графовую СУБД, или иную конструкцию ([Глава 12](#)).

4.2.1. Алгоритм с центральным узлом или без

Алгоритм с центральным узлом представляет собой самый простой способ организации РРП. Центральный узел обеспечивает координацию всех запросов клиентов РРП (*single-leader*). В этом случае центральный узел является слабым звеном («узким местом»). Чтобы повысить эффективность такого алгоритма, целесообразно разделить функцию центрального узла между несколькими корневыми узлами (*multi-leader*, [разд. 5.1](#)) или выбрать способ координации без центрального узла (*leaderless*, [разд. 5.3](#)).

Когда данные распределены по нескольким узлам, необходимо уметь определять, на каких именно узлах находятся необходимые потребителю данные. Потребитель может осуществлять итеративный перебор узлов запросами, пока не получит доступ к нужным данным. Клиент может одновременно послать несколько запросов на все узлы, однако это может привести к перегрузке сети. Более эффективным решением является ведение адресации РРП и последующее предоставление доступа потребителям с опорой на адресацию. При наличии нескольких центральных узлов необходимо обеспечить поддержку актуальности адресации на всех узлах, чтобы не нарушить целостность системы.

4.2.2. Миграционный алгоритм

В отличие от алгоритма с центральными узлами, в котором запрос клиента в итоге адресуется узлу, обеспечивающему хранение необходимых клиенту данных, миграционный алгоритм обеспечивает перемещение данных на тот узел, в котором возник соответствующий запрос. Такой алгоритм позволяет взаимодействовать с локальными сервисами (программами, сценариями), интегрированными непосредственно в узлы. Он позволяет избежать перемещения клиентов между узлами. Миграционный алгоритм, подразумевает, что один узел в отдельный момент времени имеет возможность обратиться только к одному элементу данных.

При миграции данных обычно перемещается целый сегмент, содержащий выбранный элемент данных. По этой причине разумнее обеспечить уменьшение размера сегментов данных (разд. 4.4). Постоянное перемещение избыточных данных может привести к перегрузке системы. Такая перегрузка обусловлена пробуксовкой³¹.

Пробуксовка возникает в случаях, когда объем данных, требуемых для работы системы, существенно ниже, чем объем данных, используемый при миграции целых сегментов в действительности. То есть, по сути, частый свопинг данных между узлами мешает работе системы, перегружая ее ресурсы. Проблему пробуксовки могут решить аппаратные средства, поддерживающие технологию RDMA³², которая обеспечивает доступ к необходимым элементам удаленной памяти без необходимости перемещения, однако система может воспринять это как миграцию. В этом случае любой отказ узла повлечет за собой утрату данных.

³¹ Trashing, #4-12

³² Remote Direct Memory Access, #4-13

4.2.3. Алгоритм размножения для чтения

В отличие от миграционного алгоритма, который обеспечивает перемещение сегментов памяти между узлами, алгоритм размножения³³ не перемещает данные от узла к узлу, а размножает (создает реплику) их на требуемый узел. Обычно размножение происходит путем копирования целого сегмента данных (переменной или объекта), как в случае с миграционным алгоритмом. При этом данные становятся доступны с нескольких узлов.

Такой алгоритм можно сравнить с кэшированием (разд. 6.2) на ближайшем к потребителю корневом узле при условии, что такой кэш будет использован только для чтения. В самом простом исполнении модификация данных может быть осуществлена только на исходном узле (рис. 4-01).

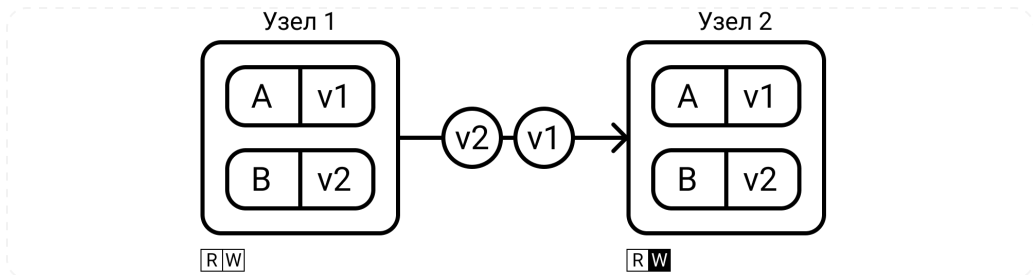


Рис. 4-01. Алгоритм размножения

В более сложном варианте алгоритм должен быть реализован с блокировками записи размножаемых переменных. Модификация может быть осуществлена только исходной переменной на исходном узле. На других узлах допускается исключительно чтение (рис. 4-02). В таком случае серьезной проблемой станет утрата узла источника. На всех оставшихся узлах переменные будут разблокированы для записи. Естественно, можно применить алгоритмы выбора нового лидера, назначить один из доступных узлов главным для модифицируемых переменных и разблокировать переменные на запись.

³³ Replication algorithm

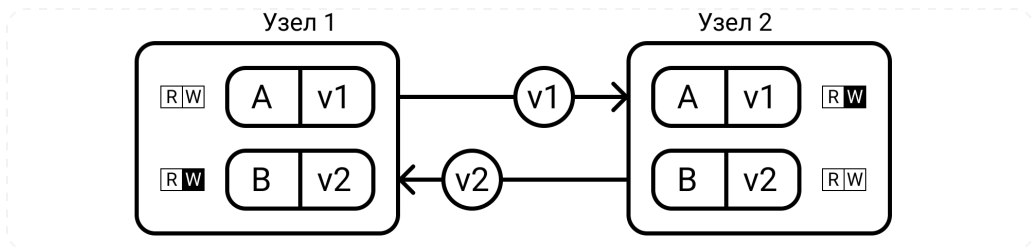


Рис. 4-02. Алгоритм размножения

Производительность системы пропорционально увеличивается за счет возможности организации одновременного доступа в режиме чтения с нескольких узлов. Однако в режиме записи имеют место существенные затраты на коррекцию (перезапись, обновление) всех устаревших блоков данных и обеспечение их консистентности (разд. 6.6). Такой вариант алгоритма называют алгоритмом полного размножения.

4.2.4. Алгоритм полного размножения

Алгоритм полного размножения представляет собой более сложный вариант алгоритма размножения для чтения. Такой алгоритм требует эффективного управления коллизиями. Может случиться так, что на двух узлах в размноженную переменную одновременно будут записаны разные значения. Необходимо будет принять решение, какое изменение считать корректным (рис. 4-03).

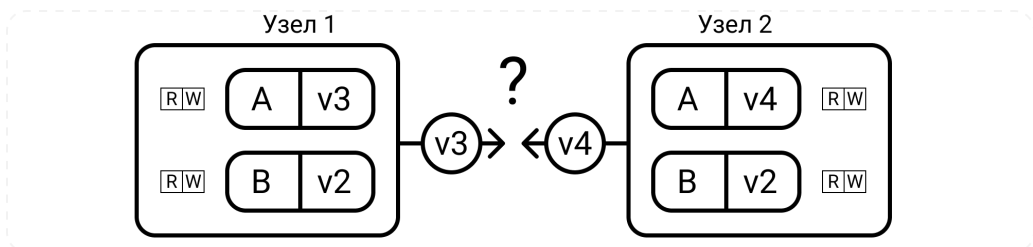


Рис. 4-03. Коллизия

Особенно острым вопрос может стать в случае если такие переменные отвечают за управление критически важными системами. Подобные проблемы решают путем превентивного выбора допустимых источников изменений в проектной логике целевой PCY.

4.3. Избыточность данных

Избыточность данных (*redundancy*) обусловлена их размножением. Она является важным свойством РСУ, обеспечивающим эффективную (повышенную³⁴) доступность данных, скорость их обработки и возможность резервирования.

Избыточные данные необходимы для повышения эффективности работы систем. Например, один узел способен обработать определенное количество запросов в единицу времени, но нужно обрабатывать больше или быстрее. В этом случае можно скопировать часть обрабатываемых данных на второй узел, обеспечить поддержку актуальности этих данных (как-то синхронизировать с первым узлом, разд. 6.5). Далее обработку данных можно частично переключить на второй узел (рис. 4-04).

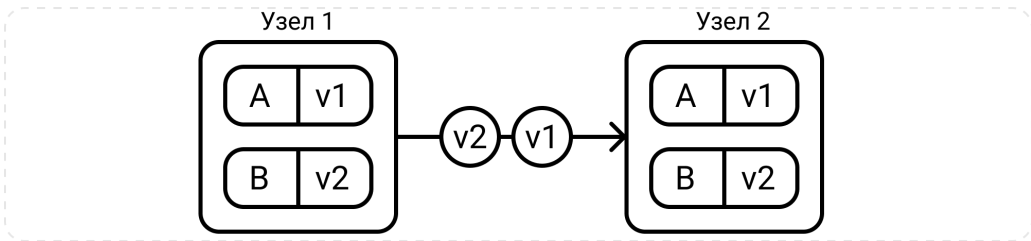


Рис. 4-04. Избыточность данных

Очевидно, что процесс синхронизации данных вносит свой вклад в нагрузку на аппаратные ресурсы системы. Следовательно, необходимо планировать, как эту нагрузку выделить в отдельную сеть, как нагрузить отдельные ядра процессора, — тогда эффективность такой конструкции увеличится. Если узлов больше двух и используются широковещательные механизмы обмена данными, то общая эффективность конструкции становится выше (рис. 4-05).

³⁴ По сравнению с единичным узлом

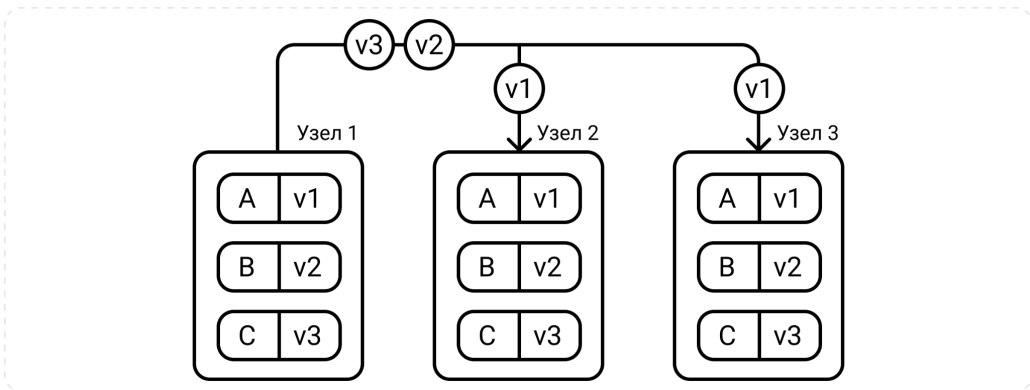


Рис. 4-05. Широковещательный обмен

4.4. Сегментирование данных

Сегментирование (шардинг³⁵) путем создания избыточности в РРП является важной оптимизационной задачей. Схемы сегментирования могут быть заданы вручную или автоматически. Вручную можно явно указать узлы, на которых будут представлены необходимые сегменты (переменные, объекты, страницы) (рис. 4-06).

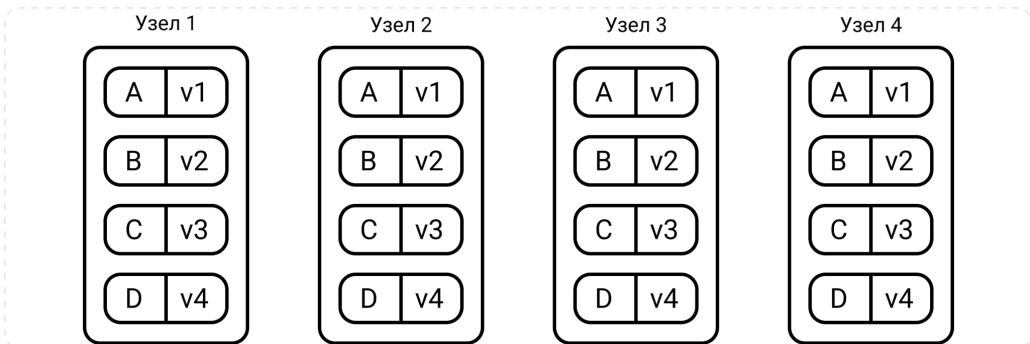


Рис. 4-06. Избыточность данных

Важным параметром сегментирования является фактор избыточности — он определяет, сколько раз сегмент будет представлен (реплицирован, разд. 6.4) в РРП.

Схема сегментирования может быть определена автоматически в соответствии с заданной целевой задачей.

³⁵ Sharding, [#4-14](#), [#4-15](#) ▶

Схема может динамически изменяться в зависимости от ситуации. Например, автоматическое сегментирование может определять планировщик нагрузки (Глава 8), формируя оптимальную схему распределения сегментов в зависимости от близости узлов к источнику и/или потребителям данных (рис. 4-07).

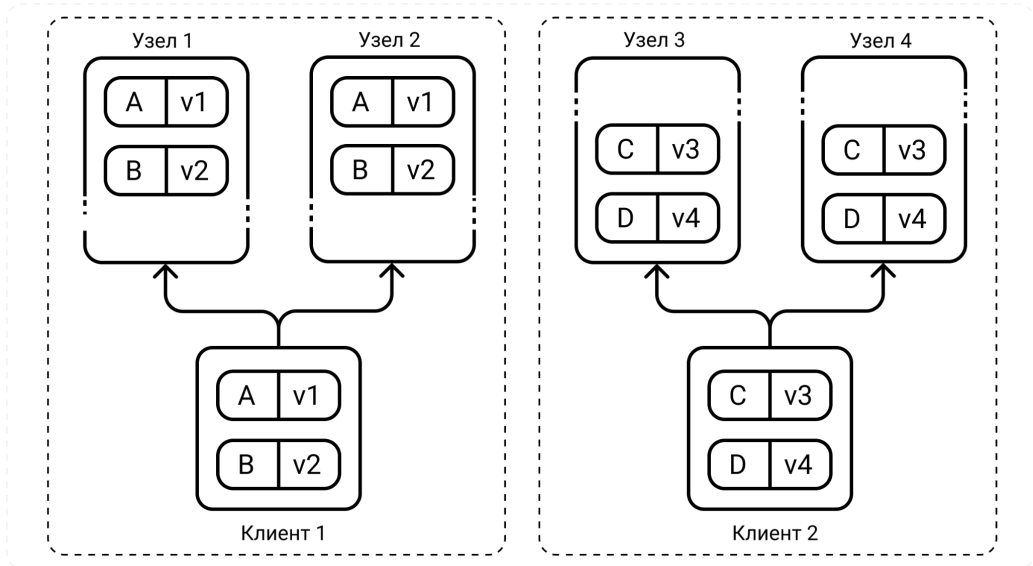


Рис. 4-07. Сегментирование данных

Переменные $v1$, $v2$, необходимые для работы клиенту 1, сегментированы и представлены на узлах 1 и 2. Переменные $v3$, $v4$, необходимые для работы клиенту 2, сегментированы и представлены на узлах 3 и 4. В случае отказа одного узла (например, узла 1), нужные клиенту 1 копии (реплики) сегментов (данных) остаются в единственном экземпляре на узле 2. При отсутствии в системе горячего резерва для дальнейшего обеспечения резервирования данные могут быть скопированы на узел 3 (ближайший), на котором будут находиться данные, необходимые клиенту 1 в случае утраты узла 2 (рис. 4-08).

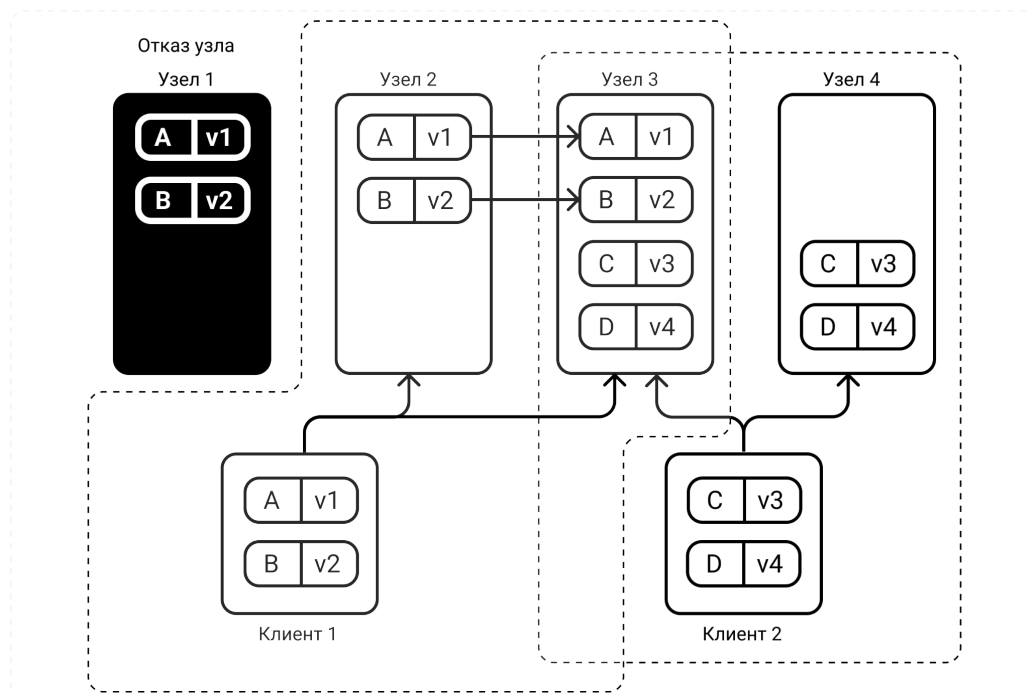
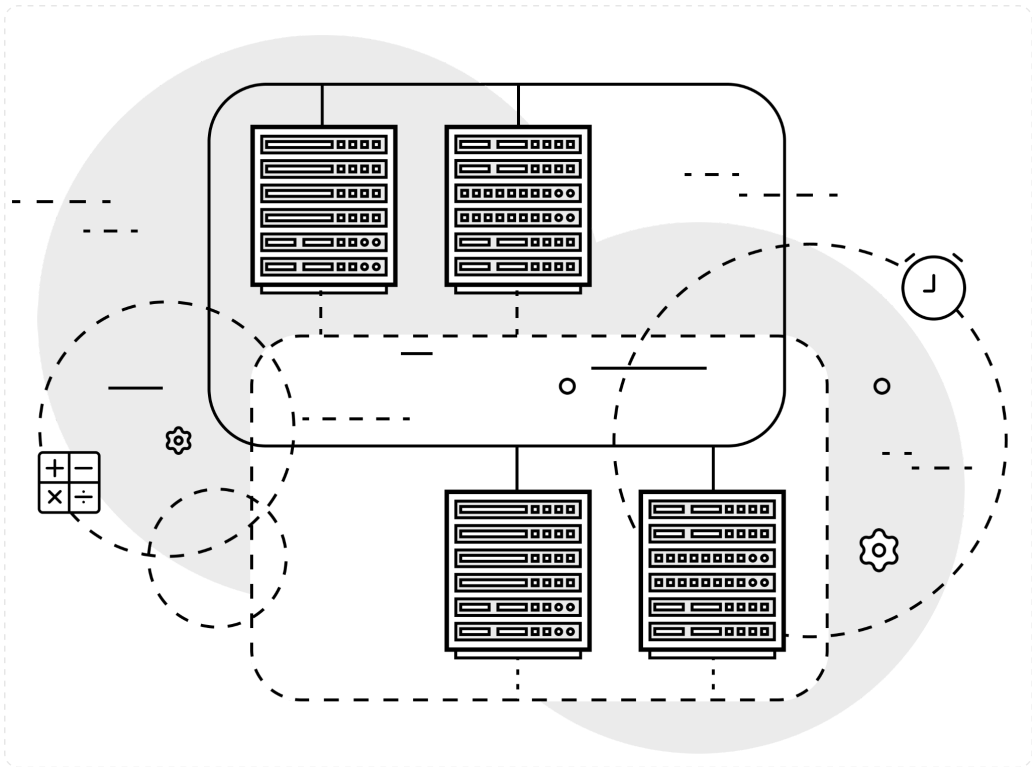


Рис. 4-08. Восстановление избыточности

4.5. Консолидация данных

При сегментировании данных с избыточностью необходимо отслеживать актуальное состояние избыточности сегментов. Может возникнуть такая ситуация, что при утрате узлов будут утрачены и избыточные данные, не говоря уже о данных в единственном экземпляре. При сегментировании данных необходимо оценивать фактор избыточности сегмента. В случае утраты требуемой избыточности необходимо ее восстановить.

Совокупность всех сегментов данных без избыточности образует консолидацию. Если каждый сегмент имеет единичную избыточность, по сути, возникает полная реплика данных в системе. Таким образом, формируются две консолидации, при этом определяется фактор избыточности консолидации. При утрате узлов необходимо следить за консолидацией и восстанавливать ее при необходимости — в зависимости от фактора консолидации (разд. 6.4).



Глава 5. Распределенная обработка данных

Большинство современных ВНС представляют собой распределенные системы. Например, обширный класс распределенных ВНС, СВВ можно отнести к распределенным системам управления (РСУ). Современные вычислительные комплексы расчетного моделирования физических процессов построены как распределенные кластеры, а системы операторского сервисного обслуживания построены как распределенные системы обслуживания.

Такие системы отличаются друг от друга классом задач, возможностью смены конфигураций выполняемых задач «на лету» и целевыми потребителями (пользователями).

Следует выделить следующие классы задач, решаемых РСУ:

- одна большая распределенная задача;
- множество задач, для выполнения каждой из которых достаточно одного узла;
- множество задач, для выполнения каждой из которых требуется более одного узла.

Современные РСУ могут состоять из множества узлов с разнообразными аппаратными характеристиками. Они должны обладать следующими свойствами и возможностями:

- резервирование — должны поддерживать ввод горячего, теплого и холодного резерва (разд. 6.3);
- масштабирование за счет добавления аппаратной инфраструктуры (разд. 10.1);
- балансировка, распределение задач для эффективного использования аппаратных ресурсов (Глава 8);
- превентивное переключение, перебалансировка;
- устойчивость к отказам, деградации с сохранением гарантий качества обслуживания (Глава 7);
- самодиагностика;
- полная автоматизация (оркестрация, разд. 11.2.5).

РСУ могут обрабатывать данные, принадлежащие как одному пользователю, так и множеству пользователей, поэтому они должны обладать эффективными свойствами изоляции, мультитенантности (разд. 11.1) и защиты от несанкционированного доступа (НСД).

Эффективность обработки данных в РСУ зависит от оптимального распределения данных и вычислений между узлами, балансировки их нагрузки и корректности резервирования узлов. Эффективное резервирование должно решать следующие задачи:

- компенсация внезапно отказавшего, ставшего недоступным узла;
- компенсация деградирующего узла;

- компенсация роста нагрузки на систему;
- компенсация резкого роста нагрузки³⁶;
- обеспечение масштабирования при плановом росте нагрузки.

Конструкция РСУ зависит от топологии узлов. С точки зрения статичности, выделяют два варианта топологии: статическая топология, при которой узлы сохраняют сетевую адресацию, расположение и связность с остальными узлами, и динамическая топология, при которой узлы могут перемещаться, адресация может изменяться, а связность узлов может изменяться динамически. Применение динамических топологий в СРВ является наиболее сложной задачей — такие задачи встречаются редко, в основном в узкоспециализированных областях.

С точки зрения сетевой организации возможны следующие топологии узлов РСУ (рис. 5-01):

- кольцо, двойное кольцо;
- звезда, двойная звезда;
- ячеистая топология (*mesh*);
- гибридная топология.

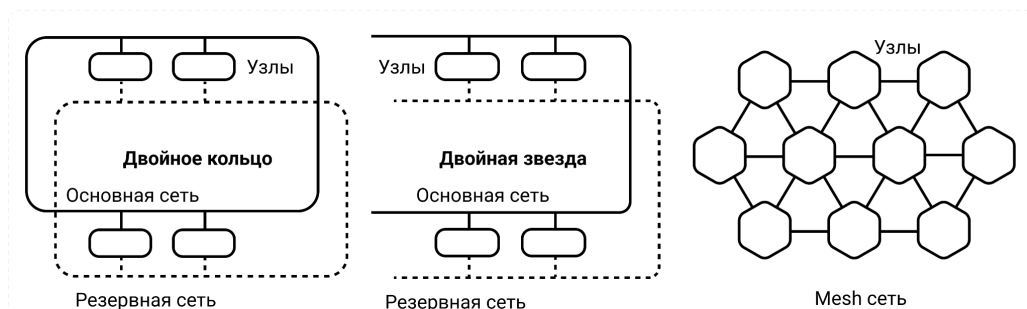


Рис. 5-01. Топологии

Современные РСУ могут поддерживать самые разные сетевые топологии, применение которых влияет на способы обработки данных. В качестве примера можно привести современную гибридную инфраструктуру АСУ ТП.

³⁶ Широковещательный шторм, лавина, [#5-01](#)

Такая инфраструктура может включать ЦОД, в котором корневые узлы объединены в топологию двойной звезды. К этой же сети могут быть подключены пограничные узлы, отвечающие за граничные вычисления (*edge*) и туманные вычисления (*fog*). К пограничным узлам, зачастую выполняющим функцию шлюзования, подключаются полевые проводные сети контроллеров типа «двойное кольцо» и беспроводные устройства (например, IIoT³⁷), работающие в *mesh*-сетях.

5.1. Узловая модель

Обычно PCY состоят из многочисленных узлов, которые могут включать широкое разнообразие (Глава 2) аппаратных конфигураций. В связи с этим узлы принято разделять, исходя из функциональности выполняемых ими задач. Как правило, узлы делят на два типа. Первый тип — это более мощные узлы, задача которых состоит в обработке нагрузки, формируемой менее мощными подключенными узлами. Более мощные узлы принято называть серверными. Например, в задачах СРВ, АСУ ТП подобные обрабатывающие нагрузку узлы не всегда являются серверными в классическом понимании. Функцию таких узлов могут выполнять специальные вычислители, контроллеры, поэтому для удобства описания любые узлы, обрабатывающие распределенную нагрузку, в этой книге обозначены как корневые.

Корневые узлы обеспечивают работу ПО, обрабатывающего нагрузки, монолитные сервисы (разд. 11.2.3) или микросервисы (разд. 11.2.4). Остальные узлы (второй тип) обозначены как клиентские (по отношению к корневым). Такие клиенты корневых узлов могут быть достаточно мощными. Они могут обслуживать свой пул клиентов, стоящих ниже в иерархии обслуживания.

³⁷ Industrial Internet of Things, #5-02

5.1.1. Корневые узлы

Корневые узлы (*root nodes*) выполняют обработку основной нагрузки РСУ, обмениваются данными между собой — иногда по выделенному соединению (интерконнекту).

Обычно корневые узлы посредством выделенного высокоскоростного соединения объединяет сеть типа «двойная звезда» (рис. 5-02). Обмен данными между корневыми узлами осуществляется с применением специализированного протокола, учитывающего особенности высокоскоростного интерконнекта.

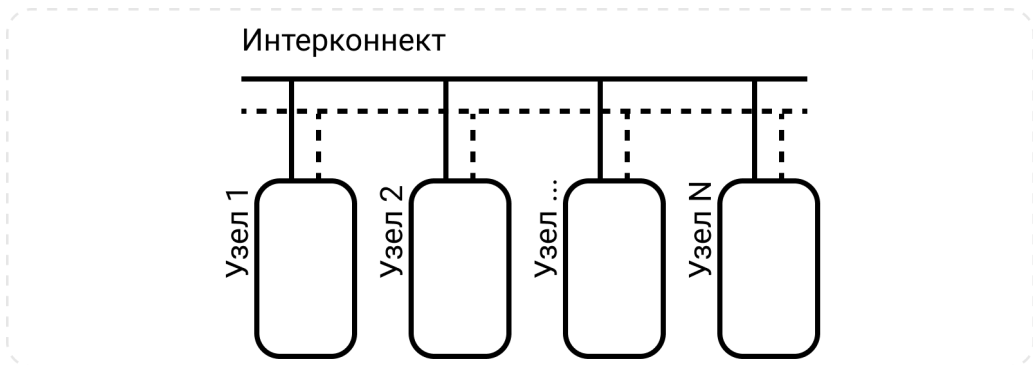


Рис. 5-02. Интерконнект резервированный

Подразумевается, что именно корневые узлы обеспечивают обработку большого объема данных целевой системы, как бы данные ни были распределены между узлами.

5.1.2. Клиентские узлы

Клиентскими узлами (*client nodes*) принято называть узлы, являющиеся клиентами одного или нескольких корневых узлов, имеющие с корневыми узлами одно или более соединений по заданному протоколу.

В отсутствие прямого соединения клиентского узла с корневым узлом может быть применена ретрансляция данных через другие клиентские узлы путем использования специальных протоколов ретрансляции. Для этого промежуточные узлы

должны быть напрямую соединены с корневым узлом. Такая ретрансляция актуальна в случаях, когда клиентский узел теряет прямое соединение с корневым узлом или когда узел является частью *mesh*-сети (*mesh node*, рис) (5-03).

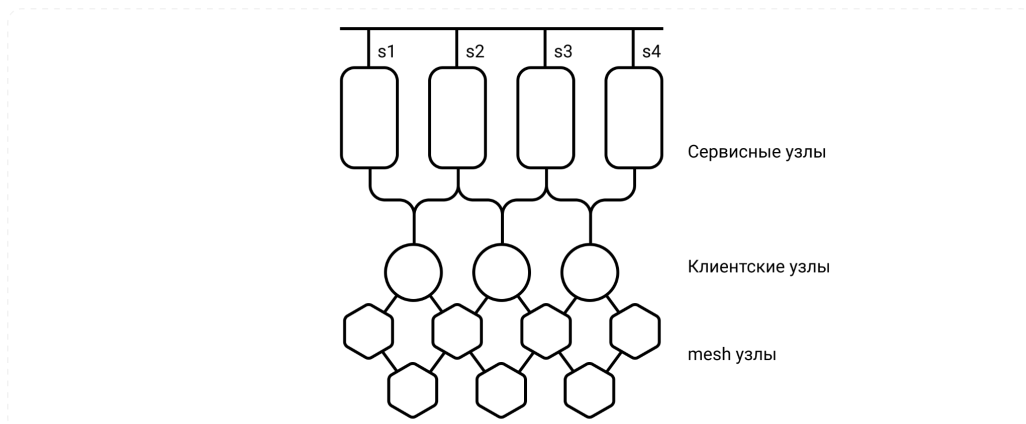


Рис. 5-03. Узлы и связи между ними

Скорость соединения между *s*-узлами и *r*-узлами обычно существенно ниже, чем *r*-интерконнект. Возможны задачи, в которых скорость *r*-с-соединений может быть достаточно высокой, однако в ВРСУ такая конструкция подразумевает, что не все данные, обрабатываемые на клиентских узлах, поступают на корневые узлы. Например, так могут быть устроены задачи предварительной обработки и фильтрации данных на клиентских узлах. В таких случаях *r*-с-соединения могут пропускать небольшой объем данных, но с высокой скоростью.

5.2. Способы обмена данными между узлами

В зависимости от задачи для каждой узловой конфигурации существует оптимальный способ организации обмена данными. Эффективным способом обмена данными между корневыми узлами (разд. 3.2, разд. 6.5) является использование широковещательных протоколов. Инициировать обмен рекомендуется в момент изменения значения или группы значений переменных. В качестве примера ниже будет рассмотрена организация РРП на основе переменных.

Обмен данными между *г-с*-узлами эффективно строить на базе устойчивых соединений, чтобы вовремя отслеживать разрывы соединений и быстро переключаться на активные (доступные) резервные соединения. В нормальном режиме эксплуатации *г-с*-соединения редко бывают настолько загруженными, чтобы приходилось обеспечивать условия их обработки в режиме высокой нагрузки или РВ. Встречаются ситуации, когда возникают соответствующие требования и появляется необходимость учитывать такие соединения при планировании нагрузки.

Обычно *т-т*-, *с-т*-соединения еще менее требовательны к режимам нагрузки и РВ — это связано с тем, что такие соединения обеспечивают физические беспроводные соединения, которые демонстрируют более высокие задержки, чем физические кабельные соединения.

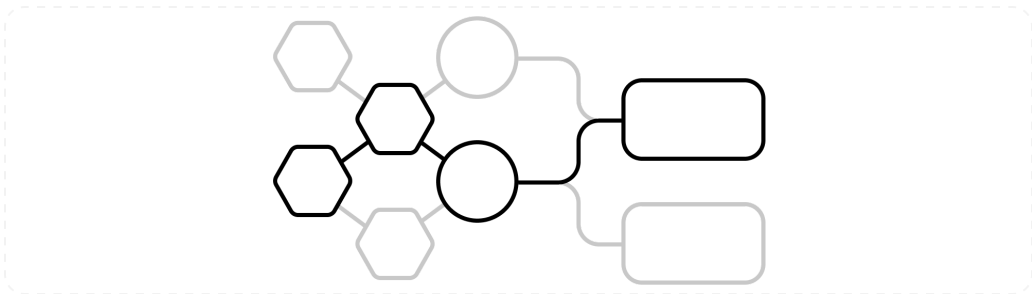


Рис. 5-04. Ретрансляция сообщения

Наиболее эффективным способом получения изменений значений переменных в клиентских узлах и, следовательно, в *mesh*-узлах, как производных, является механизм уведомлений и подписок³⁸ (разд. 11.1.7). В таких случаях при изменении значений переменных узел получает уведомление об этих изменениях и обновленные значения. Такой механизм работает в двух направлениях, позволяя эффективно реализовать ретрансляцию сообщений через цепочку узлов (цепная ретрансляция) (рис. 5-04).

³⁸ Например, применяется в протоколе *mqtt*, #5-03

5.3. Способы координации узлов

В РСУ можно выделить три конструкции взаимодействия между взаимодействующими узлами (рис. 5-05):

- *single-leader*: подразумевает наличие центрального корневого узла — этот узел координирует остальные корневые и клиентские узлы;
- *multi-leader*: подразумевает наличие нескольких центральных корневых узлов, каждый из которых координирует свой пул корневых и клиентских узлов;
- *leaderless*: подразумевает отсутствие центрального корневого узла, координация узлов осуществляется в соответствии с иными принципами (разд. 11.2.5).

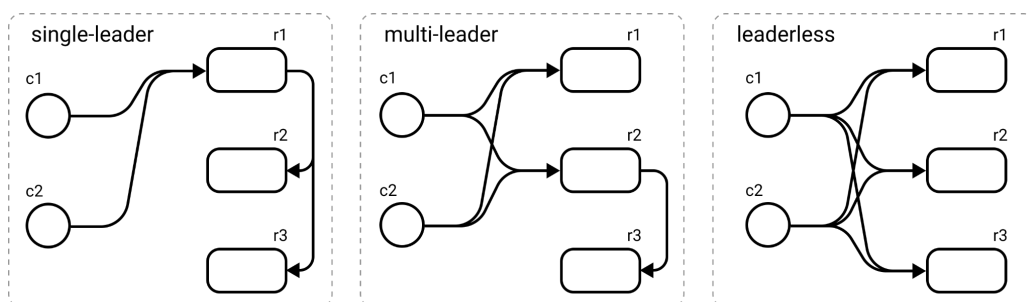


Рис. 5-05. Взаимодействие узлов

В случае утраты центрального узла координатора должен быть выбран новый узел, который получит роль координатора. Для этого может быть применен один из распространенных алгоритмов выбора лидера³⁹, например, алгоритм забияки⁴⁰.

5.4. Обработка сообщений

Обмен данными в РСУ чаще всего осуществляется путем обмена многочисленными небольшими по объему сообщениями. Источниками и приемниками сообщений могут быть как мало-нагруженные, так и высоконагруженные узлы.

³⁹ Leader election, #5-04

⁴⁰ Bully algorithm, #5-05

Сообщения обычно поступают в конструкции типа «очередь сообщений»⁴¹. Для планирования нагрузки в PCY может быть выгодно разделить очереди сообщений по типам содержащихся в них сообщений.

5.4.1. Обработка очередей

Важнейшей задачей ВНС является эффективная обработка очередей сообщений (*message queue*, MQ). Существует масса вариантов алгоритмов обработки очередей сообщений и их реализации в виде готовых решений. Каждое решение имеет свои плюсы и минусы. Задачей эффективной ВНС является получение сбалансированного решения, поэтому при разработке серьезных систем реализацию обработки очередей принято осуществлять самостоятельно — или подстраивать под целевую задачу готовые решения.

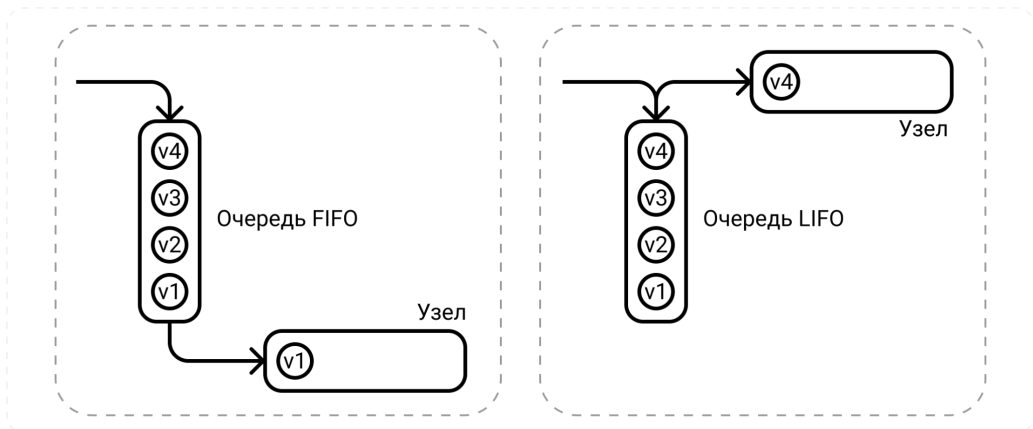


Рис. 5-06. Способ обработки очередей

Очереди сообщений отличаются способом организации и манипулирования сообщениями относительно времени и приоритетов. Классическая очередь сообщений использует простейший принцип обработки данных «первым пришел — первым обслужен» (FIFO⁴²). Однако в CPB принцип FIFO не всегда эффективен.

⁴¹ Message queue, #5-06

⁴² First input — first output

CPB всегда стремится оперировать наиболее свежими данными, поэтому для обработки очередей PB часто применяется принцип «последним пришел — первым обслужен» (LIFO⁴³, рис. 5-06).

В случаях, когда нагрузка на очередь большая, задержавшиеся надолго, устаревшие элементы можно не обрабатывать, а удалять.

5.4.2. Брокер очередей

Конструкцию, отвечающую за обработку сообщений, обычно называют брокером (диспетчером) сообщений⁴⁴ (очередей). В зависимости от архитектуры целевой системы на брокер сообщений возлагают функции маршрутизации сообщений от источников к приемникам, разбиения сообщений и их агрегации (рис. 5-07).

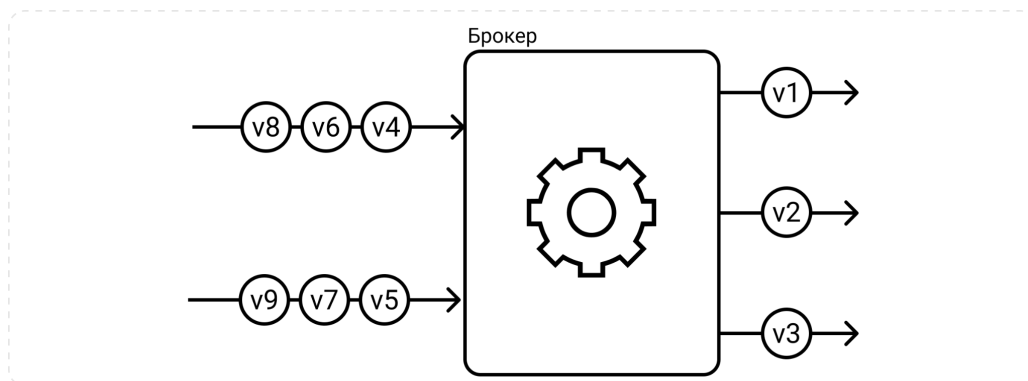


Рис. 5-07. Брокер сообщений

В брокер сообщений могут быть встроены функции валидации контроля целостности данных и их предобработки. Брокер может отвечать за преобразование данных согласно целевой спецификации, например, преобразования протоколов в формат, понятный приемникам.

⁴³ Last input — first output, [#5-07](#)

⁴⁴ Queue broker, message broker, разд. [11.1.4](#), [#5-08](#)

5.4.3. Приоритизированная обработка очередей

В РСУ могут быть использованы и более сложные алгоритмы обработки очередей. Например, может быть задан приоритет попадающих в очередь сообщений, согласно которому они должны быть обработаны. В таких случаях принято использовать брокер с приоритетом, включающий приоритизированные очереди (*priority queue, pq*).

Брокер с приоритетом управляет совокупностью очередей, каждая из которых отвечает за обработку сообщений с соответствующим приоритетом. Такой брокер прежде всего обрабатывает очереди с наивысшим приоритетом (рис. 5-08).

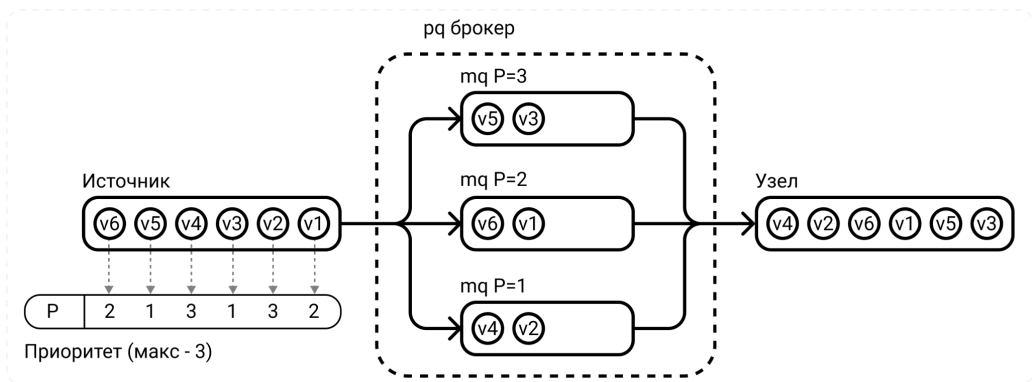


Рис. 5-08. Брокер с приоритетом

5.4.4. Многопоточная обработка очередей

Обычный брокер с приоритизацией очередей обеспечивает значимое преимущество в обработке сообщениям с наивысшим приоритетом. Сообщения с более низким приоритетом необходимо обрабатывать таким образом, чтобы они не накапливались в очередях с более низким приоритетом.

Современные ВНС подразумевают использование мощных многоядерных процессоров, по ядрам которых можно распределить потоки. Можно работать с несколькими потоками в режиме одного ядра, но это менее эффективно. Потоки имеют встроенное

свойство приоритетов, которое как нельзя лучше подходит для приоритетной обработки очередей сообщений. Если приоритет сообщения известен заранее и таких приоритетов разумное количество, а именно так часто бывает в реальных системах, удобнее создать конструкцию из статического числа потоков, равного числу приоритетов. В таких случаях возможно организовать эффективное распределение (маршрутизацию) сообщений (данных) согласно их приоритетам, причем в такой конструкции может быть сколько угодно входных очередей. В качестве примера можно рассмотреть следующий вариант: некий сервис получает сообщения из двух источников, $s1$ и $s2$, из которых сообщения маршрутизируются в очереди сообщений $mq1$, $mq2$, $mq3$. Указанные очереди обрабатываются потоками $T1$, $T2$, $T3$, отвечающими за обработку очередей с соответствующим приоритетом (рис. 5-09).

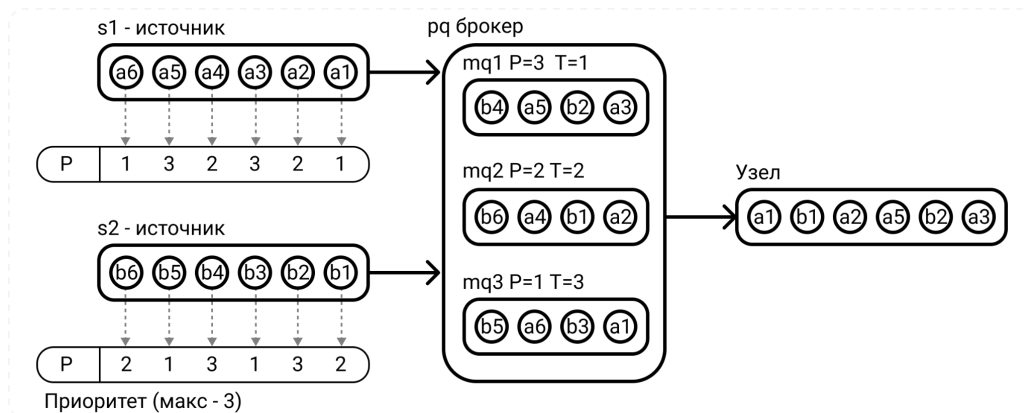


Рис. 5-09. Брокер многопоточный

Такая конструкция актуальна для обработки данных, поступающих с устройств (источников) $s1$, $s2$: например, с различными протоколами обмена. Брокер обеспечивает передачу данных для обработки в нужный поток. Брокер может выполнять предварительное преобразование данных (предобработку) в единый формат (разд. 3.6). В данном примере поток $T1$, согласно своему приоритету, успевает обработать три сообщения ($a3$, $b2$, $a5$), поток $T2$ успевает обработать два сообщения ($a2$, $b1$), а поток $T3$ — только одно ($a1$).

Во многих случаях используют поточные очереди⁴⁵ (*thread pool*), которые в классическом варианте представляют собой динамическую очередь потоков, которые могут быть созданы или уничтожены в случае необходимости (например, балансируя нагрузку на узел). Как было отмечено выше, в ВНС при нагрузке, в высокой динамике не рекомендуется создавать и тем более удалять любые динамические конструкции, объекты, потоки, переменные. По этой причине в таких случаях рекомендуется создавать статические пулы потоков.

В такой конструкции имеет место неэффективность: если число сообщений с различными приоритетами будет неравномерным, каждый из потоков будет загружен неравномерно. В этом случае выделение под потоки отдельных ядер при неравномерной загрузке неэффективно для ВНС. Закрепление потока за определенным ядром является общепринятой практикой в ВНС и тем более СРВ.

В распространенных операционных системах (ОС) потоки могут иметь доступ к общей разделяемой памяти. Для эффективного использования аппаратных ресурсов следует обеспечить оптимальную равномерную нагрузку на ядра, насколько этого возможно достичь средствами ОС, компиляторами и программной логикой.

Выгодно наделять одинаковой логикой обработчика данных (сообщений) в потоках, обеспечивая их равномерным доступом к разделяемой памяти и избавляясь от лишней логики маршрутизации, которая может стать источником задержек и «узким местом» алгоритма. В этом случае необходимо определиться с точкой входа обработчика (*entry point*) и методом передачи (ретрансляции) управления обработкой данных разделяемой памяти между потоками (рис. 5-10).

⁴⁵ Thread pool, #5-09

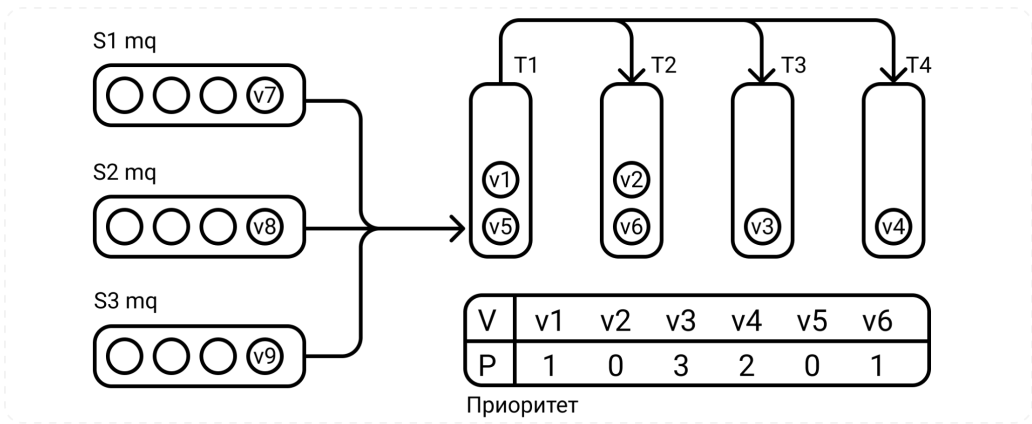


Рис. 5-10. Брокер на лидер узле

Вопрос выбора точки входа всегда остро стоит в системах распределенной обработки данных, но он становится еще более острым в ситуациях, когда в распределенной обработке данных участвует множество распределенных сетевых узлов.

5.4.5. Распределенная обработка очередей

Важной задачей PCY является обработка очередей сообщений на распределенных узлах. Задача обработки очередей PCY наиболее критична для корневых узлов, обрабатывающих большую часть нагрузки. Брокер должен обеспечивать управление распределением обрабатываемых сообщений по вычислительным узлам согласно заданному алгоритму целевой системы.

Выделяют несколько вариантов построения брокера, обслуживающего обработку очередей сообщений, основываясь на способах координации взаимодействия узлов (разд. 5.3).

В самом простом варианте выделяют один отдельный узел, который назначают брокером. Такой узел может быть назван прокси, так как он отвечает исключительно за маршрутизацию сообщений. На нем может быть определена элементарная логика распределения таких сообщений. В распределенных системах подобную конструкцию принято называть *single-leader* брокером (рис. 5-11).

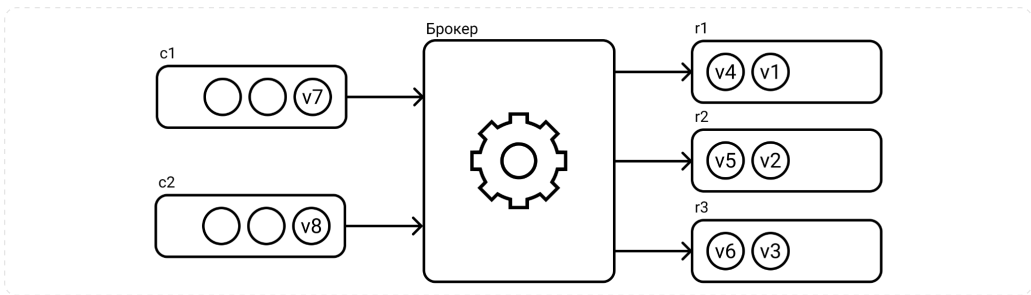


Рис. 5-11. Single-leader брокер

Такая конструкция крайне уязвима, так как она является «узким местом» — наиболее слабым звеном системы с точки зрения надежности. Отказ брокера повлечет за собой отказ всей системы. Поэтому в РСУ может одновременно применяться несколько таких брокеров. При отказе одного брокера клиенты могут переключаться на оставшиеся в работе. Если считать несколько подобных брокеров единой конструкцией обработки сообщений, такой брокер принято называть *multi-leader* брокером (рис. 5-12).

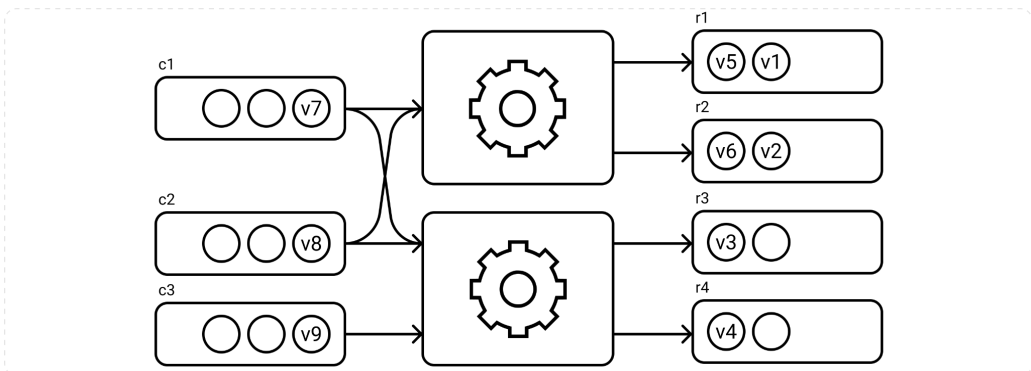


Рис. 5-12. Multi-leader брокер

Бывают ситуации, когда выделять отдельные узлы нет необходимости. В таких случаях брокер может быть встроен в клиентский узел. Каждый подобный узел может самостоятельно осуществлять маршрутизацию сообщений внутри доступного ему пула соединений с корневыми узлами. Это некая разновидность *leaderless* брокера (рис. 5-13).

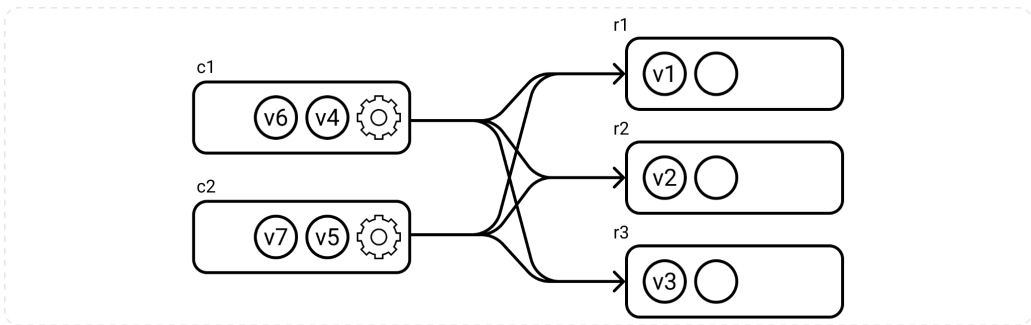


Рис. 5-13. Leaderless брокер

5.4.6. Распределенный брокер очередей

Для обеспечения взаимодействия корневых узлов друг с другом может быть использован распределенный брокер. На основе распределенного брокера могут быть построены синхронизация корневых узлов (разд. 6.5, 11.2.1) и их балансировка (разд. 8.2.4). На каждом корневом узле, участвующем в процессе, размещается брокер, обслуживающий обработку исходящих (*snd*) и входящих (*rcv*) сообщений (очередей). Все узловые брокеры должны координироваться между собой (рис. 5-14).

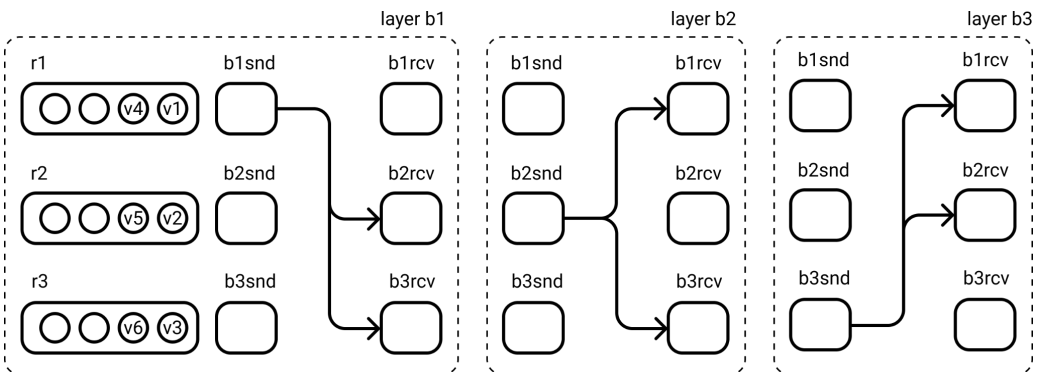


Рис. 5-14. Брокер распределенный

Распределенный брокер наиболее сложен в реализации в связи с тем, что вариантов реализации и алгоритмов работы такого брокера может быть очень много.

Например, самая простая ситуация, отраженная на схеме выше, подразумевает, что все узлы общаются друг с другом.

Следовательно, в зависимости от способа обмена данными (разд. 3.2) будут поддерживаться следующие соединения для соответствующих протоколов обмена (рис. 5-15).

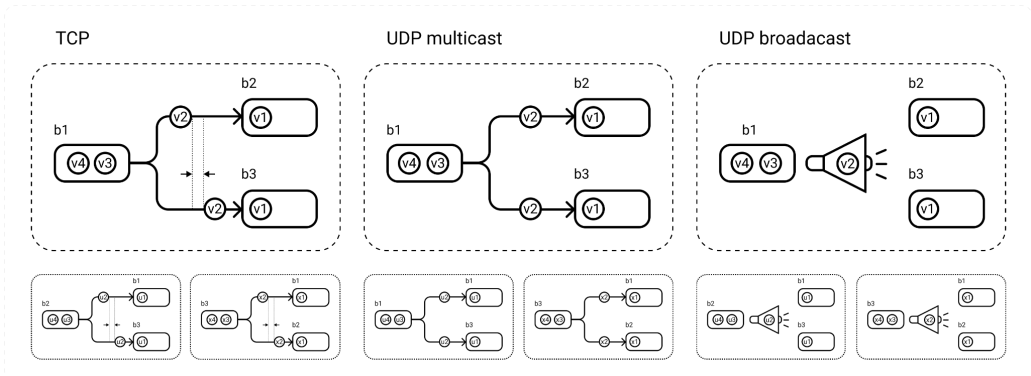


Рис. 5-15. Сравнение реализации способов обмена данными

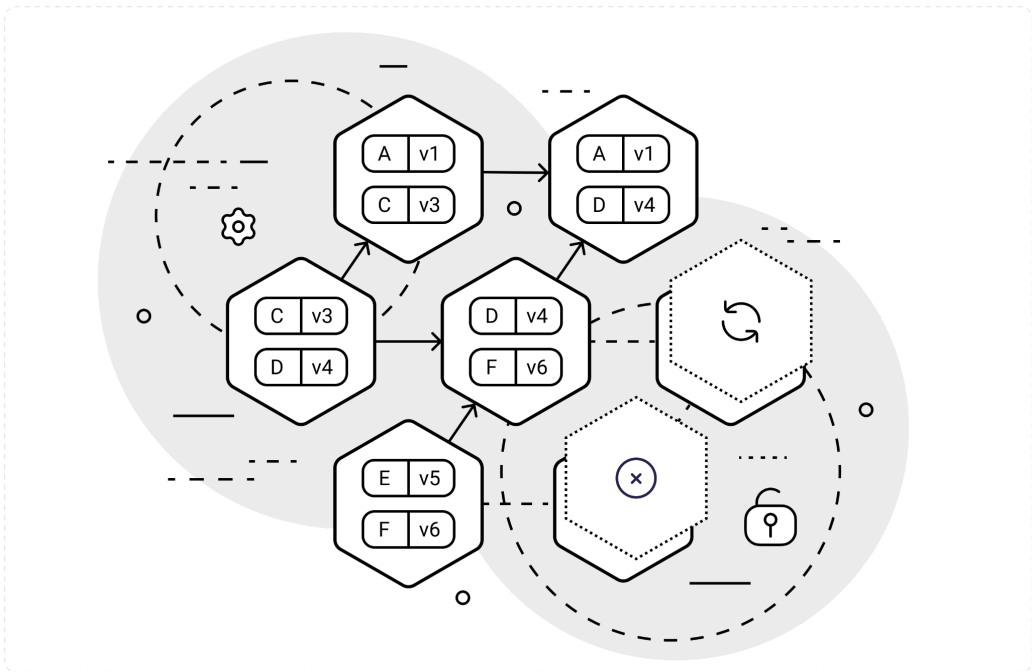
На схеме видно, что простейшая конфигурация распределенного брокера из трех узлов демонстрирует существенное преимущество использования мультивещательных и широковещательных способов обмена данными. Число соединений меньше, чем при использовании множества $r2r$ соединений. При увеличении числа узлов разница будет более существенной.

Важной задачей являются выбор схемы обмена данными, выбор синхронного или асинхронного режима обмена, определение необходимости применения блокировок и задержек для компенсации потери времени, связанной с тем, что мощность узлов может быть неодинаковой.

Необходимо определить способ инициации передачи. Например, может быть реализована событийная модель, в рамках которой отправка данных осуществляется по событию (разд. 11.2.2). Может быть реализована временная модель, в которой отправка данных осуществляется строго в определенные временные интервалы. При этом могут быть отправлены только те данные, которые изменились, а могут быть отправлены все данные, чтобы не тратить время на обход и идентификацию переменных, подвергшихся модификации. Для уменьшения числа

сетевых пакетов и увеличения их объема может быть реализована накопительная модель. Например, в этом случае выгоднее осуществлять проверку контрольных сумм пакетов: чем больше переменных упакованы в накопительный пакет, тем меньше будут засорять трафик подтверждающие пакеты с контрольными суммами.

Брокер может формировать диагностическую информацию об узлах, которые он обслуживает. Такая диагностика позволяет фиксировать обратные связи при балансировке РСУ (разд. [8.5](#)).



Глава 6. Надежная распределенная архитектура

Надежная архитектура подразумевает избыточность (разд. 4.3) узлов и данных. Чтобы получить избыточность, необходимо выполнить ряд действий. Можно создать резервную копию на отключаемом ресурсе. В этой ситуации появляется некоторая избыточность данных, но после отключения ресурса работать с такой резервной копией уже не представляется возможным.

Избыточность возникает, когда на пути от источника к потребителю данные проходят через промежуточный узел, где происходит кэширование (разд. 6.2). Такие кэшированные данные вполне можно использовать как резервные. Чтобы намеренно создать избыточность данных одного узла на другом, следует провести принудительную репликацию данных (разд. 6.4). Репликация обычно представляет собой одноразовый процесс, если не предполагается создание полностью персистентной системы, в которой будут сохранены все изменившиеся со временем реплики.

В последнем случае репликация обеспечивает резервирование данных (разд. 6.1). Если на одной реплике данные изменились, их нужно актуализировать с остальными, устаревшими репликами: перезаписать всю реплику или только изменившиеся части реплики. Такой процесс называют синхронизацией (разд. 6.5).

Системы классифицируют по частоте операций репликации и синхронизации. Выделяют системы, в которых в основном осуществляются операции репликации, — таковыми, например, являются СХД, в которых изменения претерпевают целые блоки данных (крупные или мелкие). Выделяют системы, в которых чаще осуществляются операции синхронизации, — таковыми являются разнообразные системы мониторинга, диагностики, диспетчеризации и управления, в них изменения претерпевает только часть данных реплики, например, значения переменных.

6.1. Резервирование: горячее, теплое и холодное

В программно-аппаратных системах (ПАК) резервирование всегда рассматривают комплексно. Например, в сервере или контроллере необходимо предусмотреть резервный блок питания. Никакая эффективная программная архитектура не спасет этот узел, если на нем произойдет отказ питания, так как он в любом случае станет недоступен для целевой системы. Если узлов в сети много, то данные можно распределить между узлами — в этом случае утрата одного или нескольких аппаратных узлов не повлияет на работоспособность всей системы.

Резервирование (*backup*) неразрывно связано с понятием избыточности (*redundancy*, излишество). Понятие *backup* принято применять к какой-либо сущности, не используемой в процессе эксплуатации системы. Например, это может быть архив данных, который периодически обновляют, чтобы в случае утраты эти данные можно было восстановить.

К ситуации, когда резервируемые компоненты находятся в режиме непрерывного ожидания включения в работу, принято

применять понятие *redundancy*. Все это — спор о терминах. Для ясности в книге используется термин «резервирование».

Выделяют два типа резервирования:

- аппаратно-определяемое резервирование;
- программно-определяемое резервирование.

Типичным примером аппаратно-определяемого резервирования является резервируемый блок питания.

Как уже было отмечено выше, в СРВ для обработки данных эффективно используют оперативную память (ОЗУ). Встречали ли вы где-нибудь резервирование оперативной памяти в пределах одного устройства? Существуют экзотические системы с поддержкой избыточности памяти (RAIM)⁴⁶, но в большинстве систем, даже очень серьезных, такую опцию не применяют. Поэтому, если на узле целевой системы происходит аппаратный отказ ОЗУ, это однозначно говорит о потере этого узла. По этой причине в РСУ принято компенсировать невозможность резервирования ОЗУ узла путем организации РРП (разд. 4.2).

С точки зрения активности узла, выделяют горячее, теплое и холодное резервирование. Узел, находящийся в горячем резерве, может быть нагружен равнозначно остальным узлам и может участвовать в распределении нагрузки между всеми активными узлами. Узел, находящийся в теплом резерве, является активным, но несет меньшую нагрузку, чем узлы в горячем резерве. В случае отказа узла узел из теплого резерва переходит в горячий резерв. Узлы в горячем и теплом резерве должны включаться в процесс обслуживания с минимальными задержками. Узел, находящийся в холодном⁴⁷ резерве, пассивен, то есть не несет нагрузку и готов в случае необходимости перейти в состояние теплого резерва. Резервирование корневых узлов может быть выполнено в соответствии с разнообразными схемами резервирования.

⁴⁶ Redundant Array of Independent Memory, #6-01

⁴⁷ Есть мнение в ИТ, что холодный резерв лежит на складе

6.1.1. Резервирование корневых узлов

Для обеспечения эффективной обработки данных, надежности, масштабирования необходимо обеспечить возможность резервирования корневых узлов. Активные корневые узлы, участвующие в обработке данных, могут обеспечивать горячее резервирование друг друга автоматически.

Некоторые узлы могут работать в режиме теплого резервирования, например, осуществляя синхронизацию (разд. 6.4) данных с других активных узлов для того, чтобы при отказе одного из них взять на себя нагрузку обслуживания клиентов. В таком случае необходимо начать ввод имеющихся в наличии узлов в холодном резерве в режим теплого резервирования. Для выполнения этой операции может потребоваться много времени.

В РСУ могут быть использованы несколько схем резервирования: несколько активных узлов (мультиактивный режим), мультиактивный режим с теплым и/или холодным резервированием.

6.1.2. Режим мультиактивный

Мультиактивный режим предполагает, что при отказе одного или нескольких узлов оставшиеся активные узлы продолжают осуществлять обслуживание целевой системы. Активных узлов при этом становится меньше, а нагрузка пропорционально распределяется между оставшимися корневыми узлами — при этом нагрузка на каждый из оставшихся узлов растет (рис. 6-01).

Мультиактивный режим работы узлов обуславливает ряд преимуществ и недостатков РСУ с большим количеством клиентских узлов. Такой режим работы корневых узлов подразумевает проектную избыточность (разд. 4.3). При этом он подразумевает рост общей нагрузки на систему в случае утраты узлов, а это не всегда хорошо.

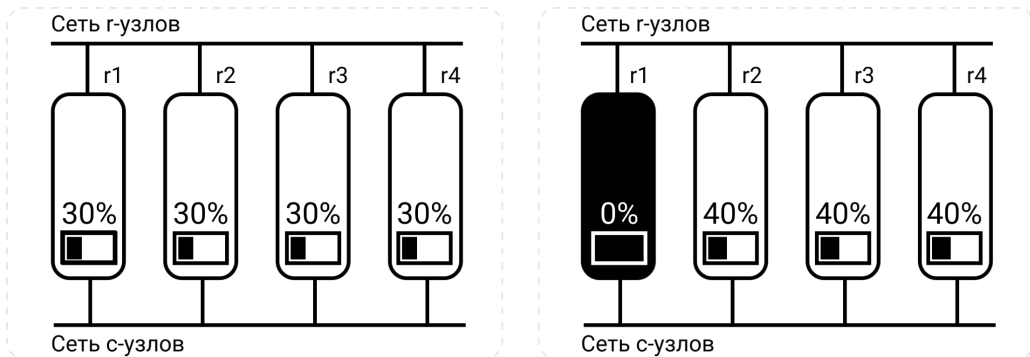


Рис. 6-01. Мультиактивный режим (% — загрузка сру)

6.1.3. Режим мультиактивный с резервированием

В отличие от мультиактивного режима без резервирования, этот режим не способствует увеличению нагрузки на доступные активные узлы. Отказ одного узла повлечет за собой ввод узла из теплого резерва — это будет происходить до тех пор, пока не останется ни одного подходящего доступного резервного узла (рис. 6-02).

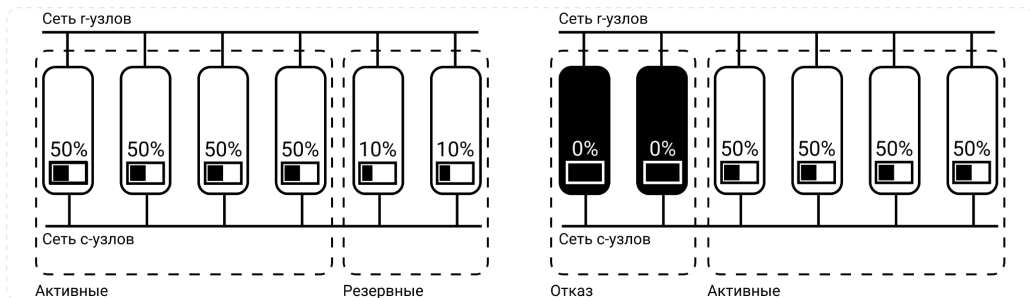


Рис. 6-02. Мультиактивный режим с теплым резервом

Существуют разнообразные схемы резервирования узлов. Классической схемой является $N+1$, в которой присутствует только один резервный узел. Более эффективна схема $N+M$, в которой доступны несколько резервных узлов. В качестве резервных узлов часто используют временные резервные узлы, которые вводят в работу до момента восстановления основных отказавших узлов. Следует отметить, что эти схемы одновременно относят как к теплому, так и холодному резервированию.

6.2. Кэширование данных

Кэширование — это эффективный инструмент ускорения доступа к данным, который представляет собой процесс создания промежуточного буфера (кэша⁴⁸). Доступ к такому буферу осуществляется быстрее, чем к исходному источнику, в связи с тем, что кэш формируется ближе к потребителю данных и его принято создавать в более быстрой памяти.

Различные определения кэша гласят, что объем кэша сильно ограничен по сравнению с объемом хранилища исходных данных, однако это утверждение содержит существенную ошибку. Она заключается в том, что современная техника позволяет нивелировать утверждение об ограничении кэша по сравнению с исходным хранилищем. Подразумевается, что ОЗУ всегда быстрее ПЗУ, а объем его оперативной памяти меньше, чем у ПЗУ. В СРВ все взаимодействия рекомендуется осуществлять в ОЗУ, поэтому вся память или ее сегмент (разд. 4.4) представляет собой кэш относительно локального узла.

Однако в РСУ понятие кэша обретает значение, вполне соответствующее его определению.

Рассмотрим простой пример (рис. 6-03): два клиента, $s1$ и $s2$, запрашивают данные у корневого узла $r1$, не имея к нему прямого доступа, но при этом имея доступ к узлу $r2$. Предположим, что вместо узла $r2$ был бы обычный сетевой ретранслятор. В этом случае при запросе (или передаче сообщения) с $r1$ будут переданы значения $v1$ и $v2$, а при запросе с $r2$ будут переданы значения $v2$ и $v3$. При двух таких запросах от узлов $s1$ и $s2$ трафик бы дважды включал значение $v2$.

⁴⁸ Cache, #6-02, #6-03 ►

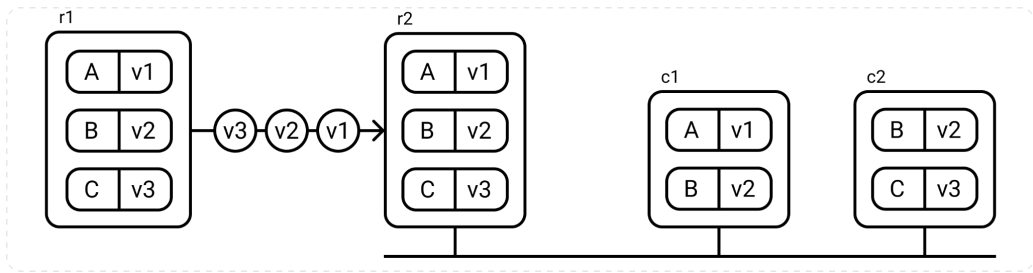


Рис. 6-03. Кэширование

При большом количестве переменных трафик был бы излишне нагружен, поэтому эффективная организация кэширования — крайне важная задача РСУ. Важной задачей является обеспечение узлам *c1* и *c2* гарантии актуальности (целостности, консистентности, разд. 6.6) кэшированных данных узла *r2*. Решение такой задачи осуществляют специальные алгоритмы, отвечающие за обеспечение консистентности данных — в данном случае консистентности кэшей.

Для описания процесса распределенного обмена данными относительно кэшей применяют следующую терминологию:

- экземпляр данных — блок данных, доступный на том или ином узле и актуальный в заданный момент времени;
- первичный экземпляр данных (оригинал) — блок данных, с которым идет работа и которому предстоит быть скопированным (реплицированным, разд. 6.4);
- вторичный экземпляр данных (реплика) — блок данных, скопированный с первичного экземпляра данных.

Когда скорость работы узла (устройства) снижается в связи с обслуживанием клиентов, удаленность узла, содержащего первичный экземпляр данных, высока, в связи с чем возникают существенные задержки и необходимость ускорения доступа к данным. В таких случаях эффективным решением является создание кэша, вторичного экземпляра данных на меньшем расстоянии и/или более быстром устройстве. Различают кэши *write-back* и *write-through*.

Кэш *write-through*, называемый *read-only*, — это кэш, предназначенный только для чтения, все данные в котором консистентны по определению (разд. 6.6), поскольку запись идет непосредственно в первичный экземпляр, после чего происходит обновление состояния вторичного экземпляра в кэше. Такой кэш ускоряет только чтение, что позволяет значительно повысить эффективность в ситуации преобладания операций чтения.

Кэш *write-back* позволяет ускорить операции записи, но при этом ставит под вопрос консистентность данных. Для организации кэшей *write-back* применяют специальные батареи, поддерживающие питание оперативной памяти до нескольких часов, либо специальные комплексы, позволяющие поддерживать питание оперативного кэша и дисков в течение времени, достаточного для передачи всего объема кэша по назначению. В случае СХД имеет место запись на постоянные энергонезависимые носители (диски). Для ускорения этого процесса рекомендуется применять быстрые флэш-накопители.

6.2.1. Когерентность кэша

Кэш может обслуживать множество экземпляров данных, то есть быть избыточным, или обслуживать один экземпляр данных. При этом остро встает вопрос целостности кэша. Такую целостность кэша называют когерентностью кэша.

Существует три основных алгоритма обеспечения когерентности кэшей:

- справочник (*directory*);
- отслеживание (*snooping*);
- перехват (*snarfing*).

В первом случае создается справочник, в котором размещена информация о состоянии блоков памяти. Справочник может быть распределен по узлам системы. В данном случае избыточность не допускается (разд. 4.3), иначе пришлось бы

следить за консистентностью. Такой алгоритм не очень эффективен в РСУ из-за наличия централизации.

В случае отслеживания каждый кэш оснащается метаданными, в которых размещена информация о его состоянии. Подразумевается, что узлы, содержащие кэш, находятся в одном разделяемом интерконнекте данных. Взаимодействуя с узлами по интерконнекту, контроллер (брокер) кэшей определяет наличие копии соответствующего блока данных. В связи с отсутствием централизации данный алгоритм массово используют в РСУ.

В случае перехвата, при перезаписи данных из определенного кэша, контроллеры остальных кэшей получают об этом событии уведомление, то есть перехватывают информацию об изменении данных и при необходимости изменяют соответствующие данные в своих кэшах.

Необходимо обеспечить условия, при которых вторичный экземпляр данных во всех кэшах был бы идентичен первичному и консистентен (разд. 6.6). Для этого применяют специальные протоколы когерентности и ограничивают области действия кэшей, чтобы не допустить случайных конфликтов (коллизий). В РСУ за обеспечение когерентности кэшей отвечает процесс синхронизации узлов (разд. 6.5).

Протоколы обеспечения когерентности отвечают за поддержание корректности данных между всеми кэшами в системе с РРП (разд. 4.2). Протокол поддерживает когерентность памяти согласно выбранной модели консистентности (разд. 6.6.1). Большинство аппаратных протоколов соответствуют модели последовательной консистентности (разд. 6.6.6), а программные протоколы соответствуют моделям слабой консистентности (разд. 6.6.3) и консистентности по выходу (разд. 6.6.10).

6.2.2. Архитектура NUMA

Современные PCY строятся с применением распределенной памяти. Все данные PCY распределены между узлами, доступ к памяти которых осуществляется с неравномерной задержкой. Стандартизирует такую архитектуру технология неоднородного доступа к памяти — NUMA⁴⁹.

Архитектура NUMA системы позволяет строить PCY, узлы которой имеют доступ как к собственной локальной памяти, так и к памяти других узлов (удаленной памяти). Очевидно, что доступ к удаленной памяти (*Remote Memory Access*) имеет существенно большую задержку, чем доступ к локальной памяти (*Local Memory Access*), отсюда возникает неоднородность. Важно отметить, что в PCY используется программно-определяемая NUMA. Классическую NUMA относят к аппаратной реализации (рис. 6-04).

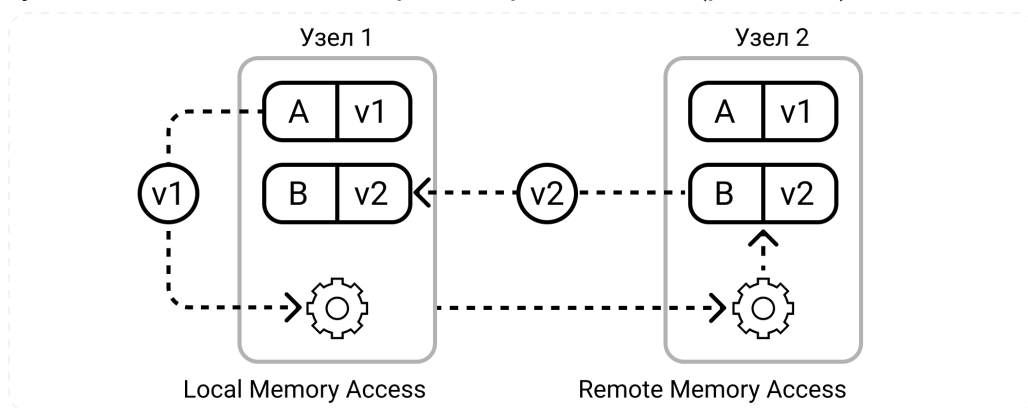


Рис. 6-04. NUMA

NUMA системы построены на основе одинаковых по аппаратной конфигурации распределенных узлов с небольшим числом процессоров и объемом оперативной памяти, которая отчасти используется в качестве кэша. Как правило, кэш представляет собой статический раздел в памяти узла. Таким образом, все кэши идентичны по размеру, в связи с чем алгоритмически несложно реализуется когерентность таких кэшей. Подобную NUMA называют кэш-когерентной NUMA (ccNUMA).

⁴⁹ Non-Uniform Memory Access, #6-04

Существует масса аппаратных ускорителей для NUMA-систем. В качестве альтернативы NUMA применяют *zero-copy*⁵⁰ протоколы RDMA, например, протокол RoCE, работающий поверх *ethernet*.

Важно отметить, что в некоторых системах одновременно используются преимущества NUMA и RDMA. Например, синхронизация данных кэшей осуществляется по протоколу RDMA. Таким образом, кэш и *zero-copy* в одной системе.

6.2.3. Алгоритмы кэширования

В задачах PCY проблем с наполнением кэша данными не возникает. Основные сложности возникают при управлении им, что связано с ограничением аппаратных ресурсов. Важным этапом является освобождение пространства кэша от неиспользуемых или редко востребованных данных, то есть вытеснение для заполнения кэша новыми данными. Сколько бы памяти, доступной для кэширования, ни было, рано или поздно всем разработчикам ВНС придется столкнуться с необходимостью вытеснения кэша.

Стратегия управления кэшем зависит от логики целевой системы. В PCY критическими системами принято приоритизировать переменные, участвующие в управлении технологическим процессом, по значимости.

Наиболее значимые переменные обязательно будут кэшированы на участвующих в управлении узлах. К ним может сколько угодно долго отсутствовать обращение, но они всегда должны быть наготове. Следовательно, такие переменные вытеснять не следует ни при каких обстоятельствах.

Рассмотрим простейшие алгоритмы вытеснения. Все они требуют отслеживания статистики обращения к кэшу, что в свою очередь достаточно ресурсозатратно.

⁵⁰ Не осуществляется копирование данных, [#6-05](#)

Выделяют следующие простейшие алгоритмы вытеснения:

- вытеснение давно неиспользуемых данных (*Least Recently Used*, LRU);
- вытеснение недавно использовавшихся данных (*Most Recently Used*, MRU);
- вытеснение редко используемых данных (*Least-Frequently Used*, LFU).

Более сложные алгоритмы кэширования, такие как многопоточное кэширование, могут быть точно адаптированы под проектную логику целевой системы.

6.2.4. Вытеснение давно неиспользуемых данных

Самой распространенной стратегией управления кэшированием в большинстве информационных систем является алгоритм вытеснения давно неиспользуемых данных (переменных) (*Least Recently Used*, LRU).

На этом этапе следует определиться, как формируется кэш. Кэш может быть сформирован несколькими способами. Самая распространенная ситуация подразумевает, что некий пользователь запросил данные с удаленного узла, а данные кэшировались на промежуточном. Тогда LRU однозначно работает. Происходит удаление данных, которые давно не запрашивали.

Но возможна совершенно иная ситуация. Например, для формирования избыточности (разд. 4.3) по требованию удаленного узла на промежуточном узле между целевым клиентом и источником данных был сформирован кэш. Система предполагает, что к этому кэшу может быть осуществлен запрос. Однако запрос может долгое время не поступать. Подобные ситуации весьма часто возникают в разнообразных РСУ. Важно отметить, что формирование избыточности не ограничивается единоразовым формированием кэша, он может интенсивно обновляться в связи с синхронизацией (разд. 6.5) или по подписке.

Как в таком случае следует воспринимать давно неиспользуемые данные? Возможны два варианта: данные, запрашиваемые удаленным узлом, и данные, запрашиваемые клиентом. Ответ однозначен. Всегда следует ориентироваться на инициатора формирования кэша. LRU следует использовать согласно следующему правилу: вытесняются данные, сформированные удаленным узлом и давно не обновляемые, и/или данные, которые давно не запрашивал клиент. На это влияет целевая логика системы.

Разновидностью алгоритма LRU является алгоритм вытеснения редко используемых данных (*Least-Frequently Used*, LFU). LFU обсчитывает формирование статистики обращений к данным (переменным). Те переменные, обращения к которым происходят реже, вытесняются в первую очередь.

Еще одной разновидностью алгоритма LRU является сегментированный LRU (*Segmented LRU*, SLRU). SLRU подразумевает создание нескольких сегментов кэша. К каждому сегменту могут быть применены другие правила вытеснения: например, основанные на времени переменной в сегменте кэша. В целом, сегментирование кэша является достаточно распространенной эффективной практикой. Выделяют разновидность *Midpoint* LRU, представляющую собой SLRU из двух сегментов. Наиболее эффективным в PCY видится применение многопоточной реализации LRU (разд. 6.2.6).

6.2.5. Вытеснение недавно использовавшихся данных

Не менее часто встречается стратегия вытеснения недавно использованных данных (переменных) (*Most Recently Used*, MRU). Такой алгоритм востребован в разнообразных техниках циклического обхода, сканирования. Данный алгоритм обеспечивает сохранение данных, сформированных LRU.

Во многих случаях алгоритмы LRU и MRU прекрасно уживаются в одной системе. Существует специальный алгоритм управления (балансировки) одновременной работы LRU и MRU,

который называют алгоритмом адаптивной замены (*Adaptive Replacement Cache*⁵¹).

6.2.6. Алгоритм многопоточного кэширования

В РСУ могут возникать разнообразные ситуации, в которых кэшированные элементы долгое время могут оставаться не востребованными. Отсутствие этих переменных в кэше в ответственный момент может привести к задержкам управления. Как было отмечено ранее, переменные могут быть приоритизированы по значимости.

Переменные могут быть распределены между сегментами кэша в зависимости от приоритета. Каждым сегментом могут управлять разнообразные стратегии вытеснения. Вытеснение может быть организовано в многопоточном режиме. Для этого применяют алгоритм многопоточного кэширования (*Multi Queue Caching Algorithm*, MQCA).

Например, переменные, запрашиваемые клиентом через кэширующий узел, размещаются в сегменте, управляемом LRU. Переменными, сформированными на кэширующем узле удаленным узлом, в целях формирования избыточности управляет LFU. Переменные, кэшированные вследствие циклического сканирования, размещаются в сегменте, управляемом MRU. Следовательно, кэш будет точно настроен под задачи целевой системы со всеми преимуществами применения многопоточности в многоядерных средах.

6.2.7. Статическое кэширование

В РСУ часто встречаются ситуации, когда осуществляется управление большим разнообразием устройств. Обычно такие устройства взаимодействуют через выделенный шлюз (сервер, контроллер). Подобный шлюз вполне может выполнять роль

⁵¹ ARC, #6-06

кэширующего узла. К этому шлюзу в обозримый период времени не будут подключены новые устройства. Следовательно, такому кэширующему узлу не требуется стратегия вытеснения. Он может самостоятельно запрашивать обновление кэша переменных согласно заранее определенному списку.

6.2.8. Стратегическое кэширование

PCY предназначены для обслуживания самых разнообразных технологических, производственных и прочих процессов. Например, на объектах генерации электроэнергии во время нормальной эксплуатации обслуживающая PCY задействует не более 10% доступного функционала. Остальной функционал необходим для эффективного ввода в эксплуатацию, технического обслуживания, то есть отладки разнообразных подсистем. Такая PCY содержит значительное число сценариев, каждый из которых оперирует неким определенным перечнем переменных. При использовании каждого из сценариев эффективным является обеспечение кэширования этого перечня переменных на участвующих в выполнении сценария узлах.

Сценарии могут состоять из составных сценариев. Каждый сценарий может иметь заранее известную продолжительность. В целом, статистика прохождения таких сценариев известна заранее. Кэши могут быть наполнены соответствующими переменными заблаговременно, перед началом нового сценария. После завершения сценария из кэшей могут быть вытеснены переменные, которые не предполагается использовать в дальнейшем. Следовательно, в такой PCY может быть достаточно точно определена стратегия кэширования, которая может быть задана как проектная логика целевой системы.

6.3. Резервирование данных

Резервирование данных, или резервное копирование, — это процесс копирования данных для использования в случае потери или уничтожения оригинала.

Необходимо различать распределенное резервирование данных и процесс создание кэшей. Распределенное резервирование данных — это процесс создания нескольких синхронных равноправных первичных экземпляров данных в пределах группы узлов (кластера). Например, это может быть создание двойной или тройной избыточности (разд. 4.3), позволяющей выдержать отказ одного или сразу двух узлов в группе.

Распределенное резервирование может осуществляться в соответствии с проектными алгоритмами, в которых могут быть заданы правила автоматического распределения резервируемых блоков. Например, алгоритм распределенного резервирования может инициировать распределение требуемого перечня блоков согласно предварительно заданной схеме. Именно такая схема распределенного резервирования данных может определять схему узлового сегментирования (разд. 4.4). По аналогии с конфигурациями разнообразных RAID, РРП из корневых узлов может обеспечивать хранение переменных с заданной схемой их распределения. При этом каждая переменная имеет свой адрес и располагается в оперативной памяти соответствующего узла согласно схеме.

Для РРП актуальны различные схемы резервирования: от полных копий блоков (переменных, объектов) на всех узлах до заданного числа избыточных копий каждого блока (рис. 6-05).

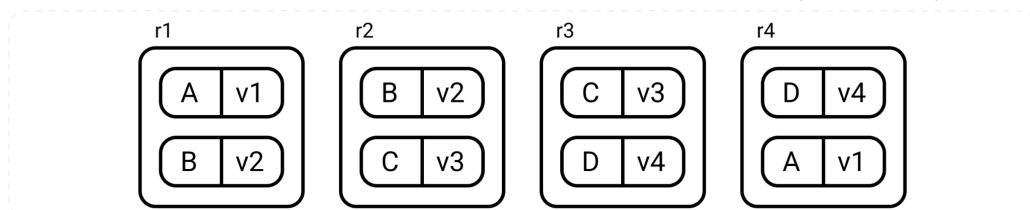


Рис. 6-05. Резервирование данных

Схема резервирования может быть задана в метаданных для отдельных блоков (переменных, группы переменных и более сложных композитных конструкций).

При отказах узлов, естественно, необходимо обеспечить перестройку (перераспределение, разд. 4.2.2) блоков на активных узлах с учетом доступного режима резервирования узлов.

Первичное распределение и дальнейшие этапы перераспределения блоков данных сопровождаются выполнением операций выделения памяти, которые не выполняют в режиме РВ. Первичное распределение данных на узлах, выделение памяти для блоков рекомендуется осуществлять при активации новых узлов. Например, самый простой и эффективный вариант предполагает, что каждый узел может содержать полную резервную копию (реплику) всех остальных узлов. В таком случае при активации нового узла конфигурация переменных может быть скопирована из любого активного узла. Если схема распределения блоков подразумевает сегментирование данных на узлах, данные могут быть собраны с активных распределенных узлов.

Резервирование – это процесс создания вторичного экземпляра данных, актуального в определенный момент времени. Принципиальным отличием от кэша или распределенного резервирования является его предназначение служить первичным доверенным экземпляром данных при восстановлении после аварии, разрушившей все первичные экземпляры. Процесс создания резервной копии, устройство(а) для хранения резервной копии, количество копий и стадии дальнейшей репликации определяет матрица рисков, согласованная до начала проектирования.

Различают два вида резервных копий: *crash consistent* и *application consistent*.

Crash consistent: копирование данных осуществляется как есть, невзирая на консистентность и завершенность транзакций. Такие копии обычно производятся быстрее, они практически не

затрагивают производительность сервиса, но не гарантируют консистентность данных. Поэтому считается, что это «лучше, чем ничего».

Application consistent: перед созданием резервной копии производится запуск сценария передачи кэшей из оперативной памяти по назначению, например, сброса на диск. Происходит завершение текущей транзакции и приостановки обработки новых — до фиксации состояния данных. Такие копии гарантированно консистентны, однако процесс передачи кэшей и остановки транзакций может оказаться слишком затратным для системы под нагрузкой, поэтому копии часто формируют в специальных окнах обслуживания либо при минимальной нагрузке на систему.

6.4. Репликация данных

Репликация представляет собой процесс создания копии данных (реплики), а именно переменной, объекта, блока, сегмента данных — иными словами, вторичного экземпляра данных. Принципиальным отличием репликации от распределенного резервирования является подчиненность реплики. Если в рамках группы узлов распределенной системы зачастую трудно разделить кэш, реплику и первичный распределенный экземпляр данных, то в случае с географически разнесенными репликами все иначе.

Например, сетевые задержки локальной сети могут составлять от 10 мкс до 1 мс, что зачастую позволяет считать распределенную систему синхронной и строго консистентной (разд. 3.3). При этом в случае протяженности в десятки километров (метро-расстояние) задержки возрастают до 5–10 мс, а при большей протяженности (500+ км) они могут составлять десятки и сотни миллисекунд. В таких случаях реплика однозначно становится асинхронной и слабо консистентной (разд. 6.7). Важно отметить, что узлы РСУ зачастую находятся на небольшом удалении друг от друга. При возникновении непрогнозируемых

высоких нагрузок на узлы могут возникать задержки, влияющие на эффективность репликации.

Обычно репликация не является задачей РВ и представляет собой задачу низкого приоритета в СРВ. Репликация может быть полной, она может осуществляться одновременно с нескольких узлов для ускорения процесса репликации и последующей активации узла (рис. 6-06).

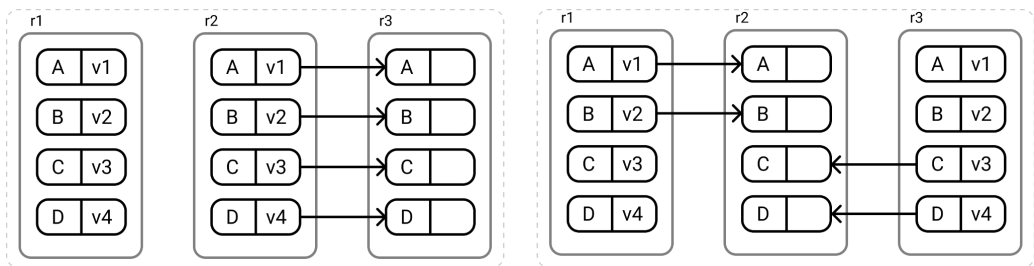


Рис. 6-06. Репликация данных

Репликация может быть частичной (сегментарной, шардированной), если данные предварительно сегментированы (рис. 6-07).

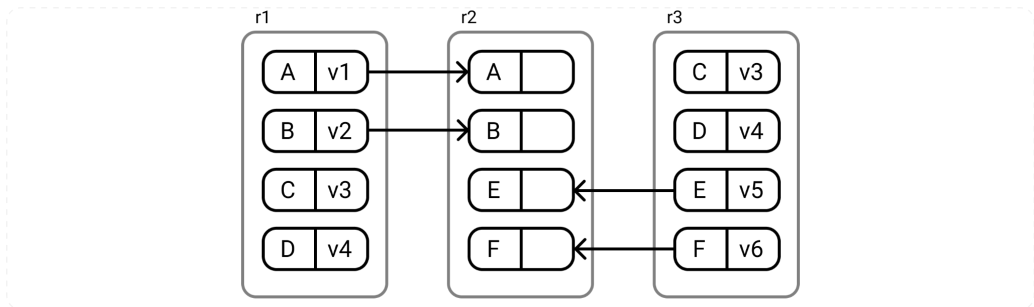


Рис. 6-07. Сегментарная репликация

Механизм репликации одинаково эффективен как для плоских распределенных структур, так и для более сложных конструкций, таких как *mesh* (рис. 6-08).

Репликация может быть выполнена в случае изменения структур данных целевой системы в режиме эксплуатации. При этом необходимо тщательно следить, чтобы не допустить утраты целостности и связности структур данных (разд. 6.6).

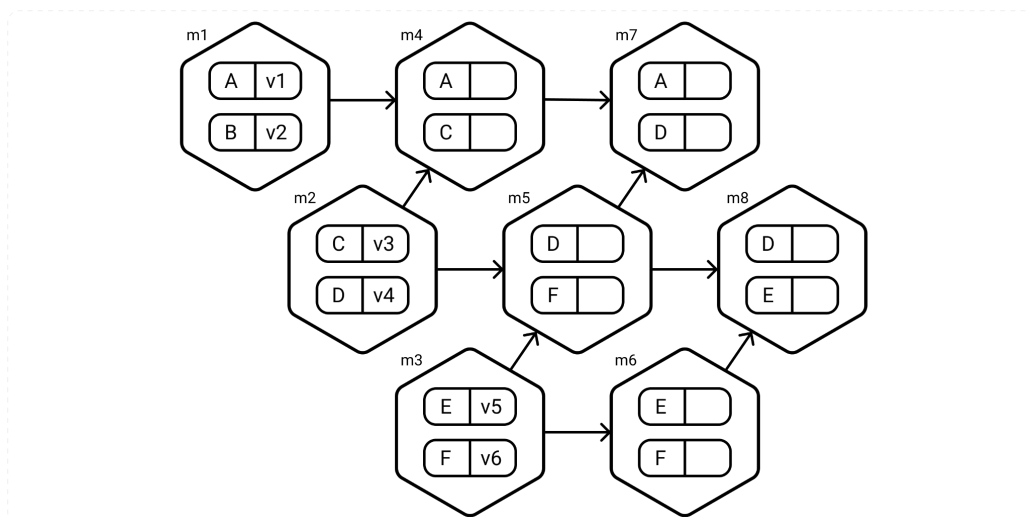


Рис. 6-08. Репликация mesh

Принципиальное отличие резервирования данных от репликации заключается в том, что в абсолютном большинстве случаев репликация — это непрерывно бегущая «почти консистентная копия». В случае логического разрушения данных реплика будет разрушена по истечении задержки репликации (сетевой задержки). Резервная копия, напротив, гарантированно консистентна в заданный момент времени; при этом часто бывает, что по состоянию на разные моменты времени сохраненных резервных копий не одна, а целая библиотека (разд. 12.1.5). Одной из особенностей резервирования можно назвать распространенное требование наличия резервной копии на отчуждаемом носителе (например, магнитной ленте) для ответственного хранения в отдельном помещении или даже отдельном здании.

6.5. Синхронизация данных

Синхронизация данных — это непрерывный процесс приведения к состоянию идентичности в заданный момент времени двух копий (реплик) данных. В СРВ данные, обрабатываемые в режиме РВ, зависят от времени. По этой причине каждый элемент, блок данных должен иметь временную метку или несколько (разд. 12.3.2). Наличие временной метки существенно упрощает алгоритмическую реализацию синхронизации (рис. 6-09).

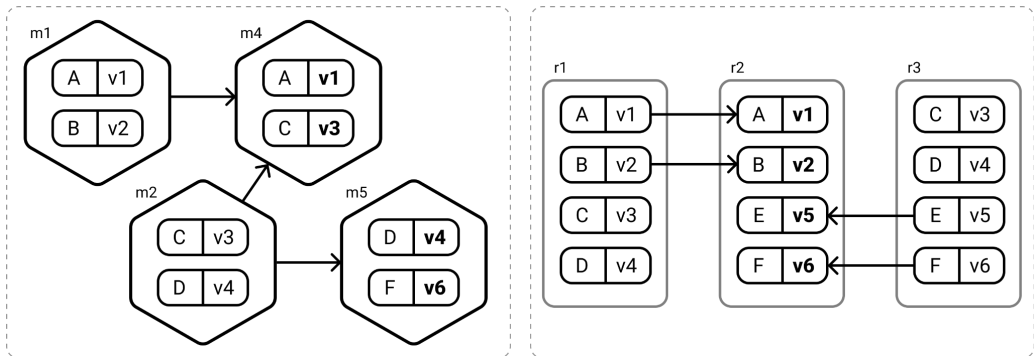


Рис. 6-09. Синхронизация данных

Возможны конфликты (коллизии, ошибки) синхронизации — ситуации, при которых одну переменную на разных узлах одновременно пытаются модифицировать данными из разных источников. В таких случаях приходится решать, каким изменениям доверять и какие считать актуальными.

В некоторых случаях такие коллизии синхронизируют, оставляя обе, а решение проблемы делегируют вышестоящей логике. Если, например, целевая система представляет собой АСУ ТП, а синхронизируются показания датчиков, то такие коллизии обрабатывают обычные фильтры. Однако если это команда управления, которая поступила от двух разных источников (разд. 6.7.2) и которая подразумевает разные результаты, то такая коллизия должна вызвать тревогу. Рассмотренная ситуация, очевидно, представляет собой проблему архитектуры автоматизированной системы или следствие ошибки несанкционированного доступа

(например, DDoS атаки). Такие коллизии не должны обрабатываться в режиме РВ.

Возможной проблемой синхронизации данных в СРВ является то, что скорость изменения данных источников может быть выше эффективной скорости синхронизации. Это значит, что синхронизация не обеспечивает передачу всех изменений переменных. Однако это проблема архитектуры целевой системы или какое-либо проявление деградации (разд. 7.3) целевой системы.

Синхронизация обычно включает в себя непрерывные такты синхронизации (распределенные транзакции). В РСУ для синхронизации обычно применяют узловые очереди сообщений, обрабатываемые распределенным брокером. Каждый такт синхронизации подразумевает изъятие из очереди части сообщений, упаковку сообщений в сетевой пакет и отправку сетевого пакета в соответствии с выбранным способом обмена данными (разд. 3.2), получение некоего периодического статуса об отсутствии ошибок.

Синхронизация может осуществляться в различных режимах в зависимости от архитектуры целевой системы. С точки зрения времени синхронизация может быть периодической — это значит, что в определенный интервал времени однократно запускается такт синхронизации. Она может быть событийной — это значит, что такт синхронизации запускается по событийной модели, например, по факту изменения переменной на одном из узлов целевой системы (разд. 5.4.5).

В РСУ синхронизация данных может осуществляться с учетом приоритета (разд. 5.4.3) данных. Например, при периодической синхронизации на узлах могут накапливаться серии изменений данных⁵². В СРВ зачастую бывают важны самые свежие изменения — при синхронизации именно такие данные доставляются первыми. Например, алгоритмам целевой системы для обработки могут быть нужны текущие и ближайшие экстремальные значения изменений. Эти значения должны быть

⁵² Time series

доставлены в порядке приоритета, а за ними все остальные, сформировавшие соответствующие очереди синхронизации (рис. 6-10).

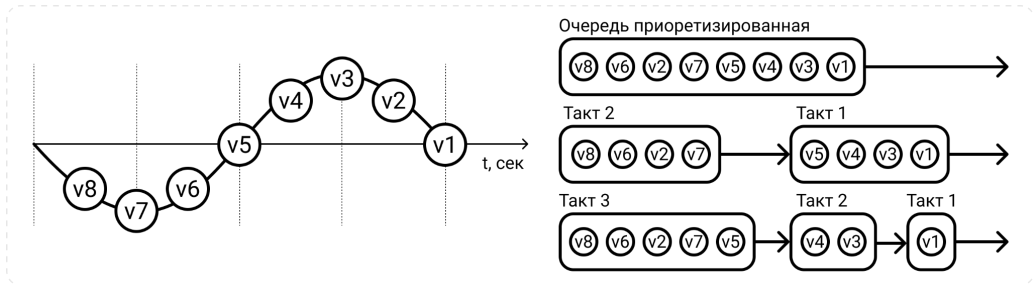


Рис. 6-10. Приоритет сообщений

Локальный алгоритм сортировки очереди определенно будет применен быстрее, чем распределенный алгоритм синхронизации, особенно если необходимо подтверждение консистентности распределенных данных. Управление эффективностью алгоритмов обработки данных как локальных, так и распределенных, обеспечение их непротиворечивости с точки зрения управления задержками целиком и полностью находятся в зоне ответственности архитектора.

Важной задачей является управление размерами пакетов синхронизации. В зависимости от архитектуры целевой системы они могут как быть одинаковыми, так и различаться по длине.

В PCY синхронизация данных может осуществляться между любыми типами и любым количеством узлов. Как правило, даже в самых нагруженных системах синхронизации корневых узлов придают большее значение, нежели синхронизации данных между корневыми и клиентскими узлами — особенно в случае синхронизации *mesh*-узлов.

Для обеспечения в PCY режима РВ достаточно соблюдать следующее правило: нагрузка трафика на общую сеть (совокупность всех соединений) не должна превышать пропускную способность этой сети. Если расчетная нагрузка не предполагает каких-либо скачков, можно предусмотреть резервную сеть,

которая может быть использована для сглаживания таких эффектов. Например, если к сети⁵³ 10GbE будут подключены 10 клиентов со скоростью 1GbE, которые дадут максимальную нагрузку, такой сети не хватит пропускной способности для синхронизации, поэтому начнется деградация (разд. 7.3) обмена данными. Совсем не факт, что резервной сети хватит для сглаживания такой лавины. Следовательно, как уже было сказано выше (разд. 5.1.1), для синхронизации корневых узлов необходим выделенный интерконнект — еще лучше, если он будет резервированный (разд. 6.3) (рис. 6-11).

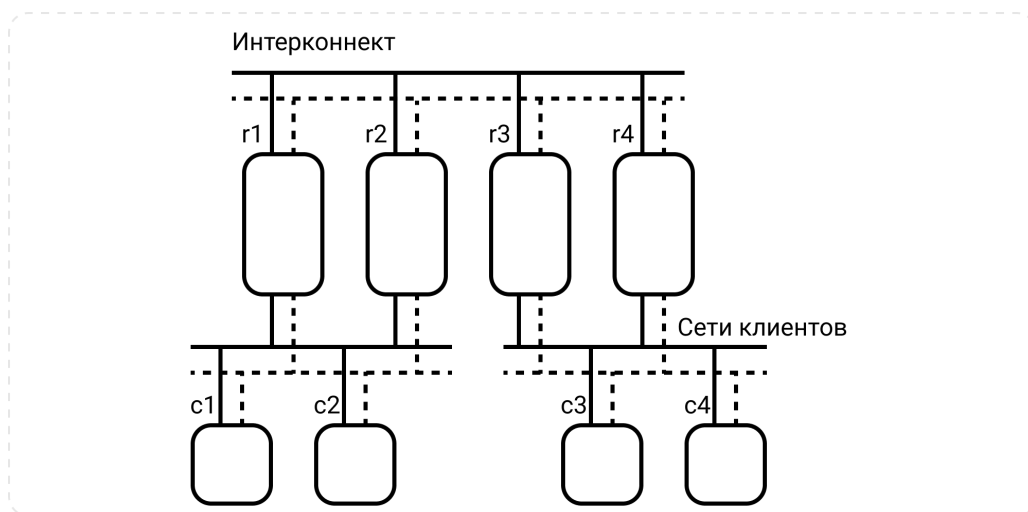


Рис. 6-11. Интерконнект синхронизации

Таким образом, для обеспечения эффективности синхронизацию корневых и клиентских узлов следует разделить. В реальной жизни при такой сетевой архитектуре избежать проблем не удастся. В качестве примера рассмотрим следующую конфигурацию: в интерконнекте из пяти корневых узлов использован интерконнект 400GbE, к каждому корневому узлу выполнено подключение 100GbE. Следовательно, в момент пиковой нагрузки суммарная нагрузка клиентских соединений будет выше, чем способен обработать интерконнект корневых узлов; и это если не вдаваться в подробности структуры трафика сетевых пакетов,

⁵³ С эффективной топологией для конкретного решения

которая может содержать дополнительную информацию, увеличивающую общую нагрузку на сеть (рис. 6-12).

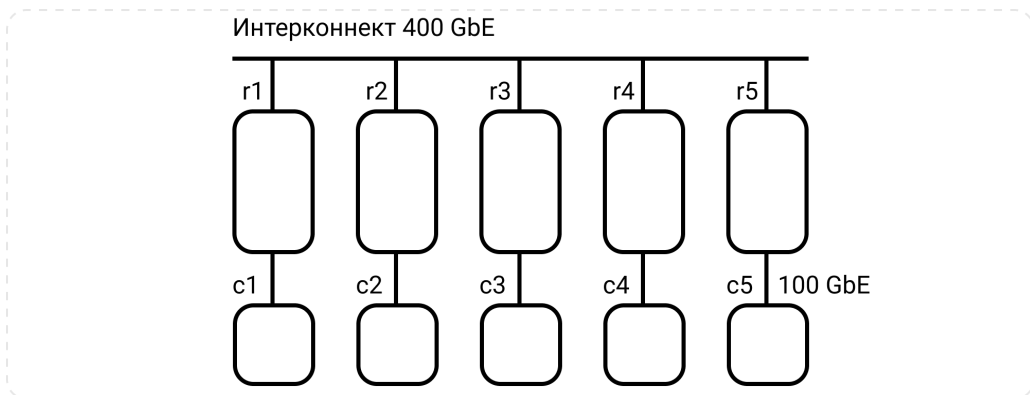


Рис. 6-12. Синхронизация, скоростные ограничения

Итак, для построения эффективных РСУ необходимо, чтобы было выполнено базовое правило конфигурации сетей: пропускная способность интерконнекта корневых узлов не должна быть меньше суммы пропускных способностей всех сетей, обеспечивающих подключения к корневым узлам клиентов.

6.5.1. Стратегии синхронизации

Важно отметить необходимость выбора стратегии синхронизации данных. Иногда выгоднее синхронизировать только изменившиеся данные, а иногда — все данные без разбора.

Как правило, стратегию синхронизации выбирают по принципу снижения нагрузки на систему. В некоторых ситуациях дольше разбираться, какие данных изменились, а какие нет, поэтому проще синхронизировать все сразу.

Важно выбрать порядок синхронизации данных. Нужно определить, требуется ли при синхронизации приоритизация. Анализ любых метрик (разд. 7.1.1), например, метрик приоритета, занимает дополнительные ресурсы.

6.6. Консистентность данных

Консистентность, или согласованность данных, — это состояние целостности и непротиворечивости данных в системе. Статус консистентности данных в сети распределенных узлов системы определяет консенсус. Формирование (установление) консенсуса сводится к процессу получения согласованного результата и устранения конфликтов группой узлов, участвующих в синхронизации (обмене данными). Консенсус может быть установлен только при наличии кворума.

Кворум, как объект принятия решения, весьма неоднозначен, он может применяться достаточно гибко. Разнообразные источники совершенно по-разному объясняют, что такое кворум. Прежде всего, существует весьма распространенное заблуждение, что в кворуме участвуют равнозначные узлы. Основываясь на этом предположении, приводят формулу, где кворум системы из N узлов формируется $\frac{1}{2}N + 1$ узлами. Во-первых, предположение о равнозначности узлов далеко не всегда является неверным. В реальных условиях работы РСУ узлы могут иметь самую разную ценность. Во-вторых, существуют алгоритмы консенсуса, при которых в рамках принятия решения о статусе консистентности могут быть созданы специализированные узлы с расширенными полномочиями, которые будут участвовать в кворуме самостоятельно или с привлечением части узлов РСУ. Поэтому кворум может состоять из минимального числа узлов, достаточного для выполнения РСУ целевой задачи. Это следует даже из определения кворума: *quorum praesentia sufficit* — «которых присутствие достаточно». Если система разделена на подсистемы, то у каждой подсистемы может быть своя группа узлов, участвующих в кворуме, а состав этой группы может формироваться динамически. Таким способом можно эффективно задействовать большинство узлов РСУ, распределив между ними нагрузку, связанную с процессом формирования консенсуса, снизив задержки в системе в целом.

В РСУ алгоритм консенсуса бывает необходим во многих случаях: например, при создании новых переменных и их репликации, при инициализации новых узлов и их синхронизации, при обновлении значений переменных.

Алгоритм консенсуса наиболее критичен для синхронизации корневых узлов, когда требуется обеспечить гарантированное обновление данных с подтверждением (*hard update*, разд. 3.8). Очевидно, что для обеспечения диагностики целевой системы требуется периодическая проверка консистентности данных при выполнении остальных операций (*fast update*, *strict update*), и, если позволяют мощности системы, ее лучше проводить в режиме РВ, однако это необязательное условие. Наиболее распространенными алгоритмами консенсуса для распределенных систем⁵⁴ являются Paxos⁵⁵, Raft⁵⁶, Zab⁵⁷.

Важно отметить, что обрабатываемые СРВ данные почти всегда имеют временную метку, а чаще — несколько таких меток. В самых простых случаях задача установки консистентности узлов при их синхронизации подразумевает проверку того, что на каждом из узлов действительно было установлено актуальное (последнее) значение данных, соответствующее источнику изменений. В более сложных случаях задача может быть расширена до необходимости контролировать синхронизацию серий данных (цепочек, временных рядов) (рис. 6-13).

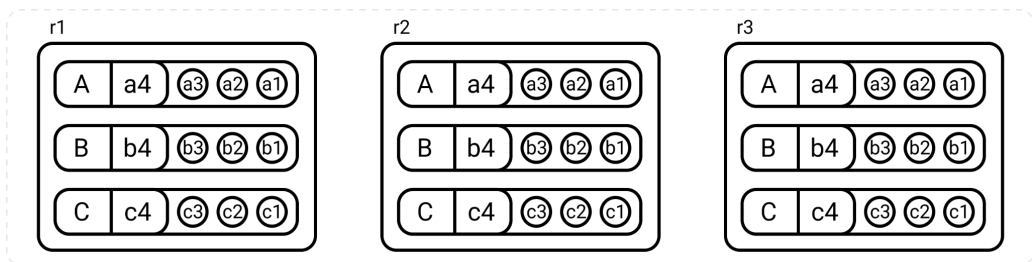


Рис. 6-13. Консистентность

⁵⁴ Сравнение алгоритмов консенсуса для РС, #6-07

⁵⁵ Paxos, #6-08

⁵⁶ Raft, #6-09

⁵⁷ Zab, #6-10

В CPV серию данных следует ограничить фиксированным размером, чтобы не допустить роста нагрузок, связанного с обработкой постоянно увеличивающегося объема данных (разд. 12.1.5). В рассматриваемых системах консенсус может быть представлен диагностическим параметром. Этот параметр может действовать периодически и иметь ограниченный срок действия. Система может быть, а может и не быть в консистентном состоянии. Необходимо постоянно оценивать состояние консистентности в соответствии с принятой моделью консистентности.

6.6.1. Модели консистентности

Модели консистентности делят на модели без синхронизации и модели с синхронизацией. Подразумевается, что заводят некие специальные переменные, которые необходимо синхронизировать, чтобы обеспечить принципиальную возможность работы таких моделей. Существует масса моделей консистентности данных PCY, основные из которых приведены ниже:

- строгая консистентность (*strict consistency*)⁵⁸;
- слабая консистентность (*weak consistency*);
- консистентность в конечном счете (*eventual consistency*);
- причинная консистентность (*causal consistency*);
- последовательная консистентность (*sequential consistency*);
- консистентность по времени (*timeline consistency*);
- линеаризуемая консистентность (*linearizable consistency*);
- консистентность по выходу (*release consistency*);
- консистентность по входу (*entry consistency*);
- строгая консистентность в конечном счете (*strict eventual consistency*).

6.6.2. Строгая консистентность

Это самая надежная, но при этом трудозатратная и медленная модель консистентности. Установка строгой консистентности

⁵⁸ Целевое состояние ACID системы (разд. 3.3)

может быть выполнена только после завершения синхронизации всех узлов. Любой последующий запрос к данным любого узла РСУ вернет обновленное (актуальное) значение. До завершения синхронизации будут установлены блокировки на чтение данных в течение максимально допустимой задержки. Если в течение задержки ожидания консистентное состояние не будет установлено, процесс будет запущен снова. Во время выполнения попытки установки консистентного состояния может потребоваться обновление данных. Таким образом, на запись могут быть установлены блокировки. Указанный подход никоим образом не удовлетворяет требованиям ни CPB, ни ВНС.

6.6.3. Слабая консистентность

Слабая модель консистентности — это компромисс по отношению к строгой модели. Такая модель консистентности не гарантирует, что запросы к синхронизируемым узлам вернут актуальные значения. Перед тем, как будет возвращено актуальное значение, должно быть выполнено несколько условий (компромиссов). Период между обновлением и моментом, когда каждый узел гарантированно получит актуальное значение, называется окном неконсистентности (*inconsistency window*).

Слабая консистентность обеспечивает более низкую латентность и более высокую доступность данных по сравнению со строгой консистентностью. При этом только строгая консистентность гарантирует подтверждение обновления (актуализации) данных. По этой причине модель слабой консистентности может быть использована для реализации гарантированного *strict update*, а модель строгой консистентности — для реализации гарантированного *hard update* с подтверждением (разд. 3.8).

Слабая модель консистентности имеет массу вариантов, которые могут быть использованы в зависимости от целевой задачи.

6.6.4. Консистентность в конечном счете

Консистентность в конечном счете⁵⁹ представляет собой разновидность слабой консистентности. Если в заданный временной интервал данные не подверглись изменению, следует считать, что РСУ находится в консистентном состоянии. Любые запросы (подписки) к любому узлу будут возвращать актуальные значения данных. В любой момент времени РСУ находится в возможно-консистентном состоянии.

На схеме (рис. 6-14) представлены три узла. Узел *c1* является клиентом, а узлы *r1* и *r2* – корневыми узлами. Сначала клиент *c1* записывает данные на корневой узел *r1*, далее *r1* синхронизируется с *r2*. Как видно из схемы, существует определенная задержка обмена данными между всеми узлами. В момент времени *T2* данные на узле *c1* обновляются, но обновление достигает узла *r1* в момент *T3*, а узла *r2* – в момент *T4*. В конечном итоге все данные синхронизируются.

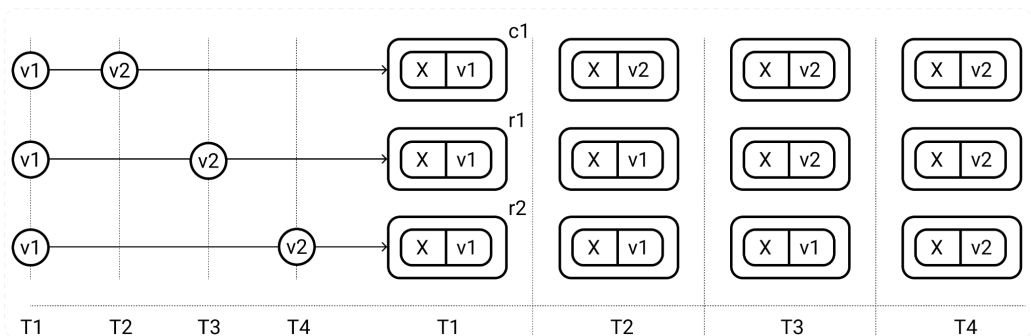


Рис. 6-14. Консистентность в конечном счете

Важно отметить, что достигнутое таким образом консистентное состояние является целевым для BASE (разд. 6.7) системы.

Разновидностью консистентности в конечном счете является конвейерная (*pipelined*, разд. 11.1.1) консистентность. Такая консистентность подразумевает, что все операции записи,

⁵⁹ Eventual consistency, #6-11

выполняемые на одном узле, видны всем остальным узлам в порядке выполнения, но операции записи, выполняемые на разных узлах, могут быть видны в произвольном порядке. Такая консистентность при некоторых допущениях может составить основу строгой консистентности в конечном счете (разд. 6.6.12).

6.6.5. Причинная консистентность

Модель причинной консистентности (*causal consistency*) используют в случаях, когда в РСУ имеет место очевидная зависимость (причина) изменения одних переменных от изменения других. Причинная консистентность может быть реализована как разновидность последовательной консистентности.

Рассмотрим простой пример (рис. 6-15), в котором модифицируемое значение переменной синхронизируется без причинной взаимосвязи. Клиентский узел *c1* в момент времени *T1* записывает значение переменной *v1*, а клиентский узел *c2* в момент времени *T2* записывает значение переменной *v2*. Узлы *c1* и *c2* синхронизируются с сервисными узлами *r1*, *r2*. В связи с обстоятельствами синхронизации в момент времени *T3* узел *r1* видит значение переменной *v1*, а узел *r2* видит значение *v2*. В момент времени *T4* узел *r1* видит значение переменной *v2*, а узел *r2* видит значение *v1*.

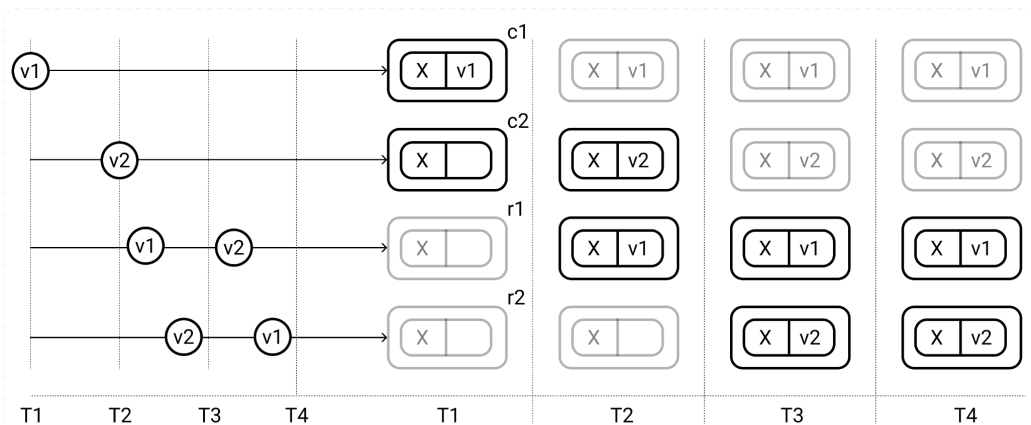


Рис. 6-15. Консистентность без причинной взаимосвязи

Предположим, что проектная логика системы подразумевает взаимосвязь модификации переменной. Таким образом, логически известно, что узел *c2* может внести изменение значения переменной только после узла *c1*. Такую логику необходимо учитывать при выполнении синхронизации данных (рис. 6-16).

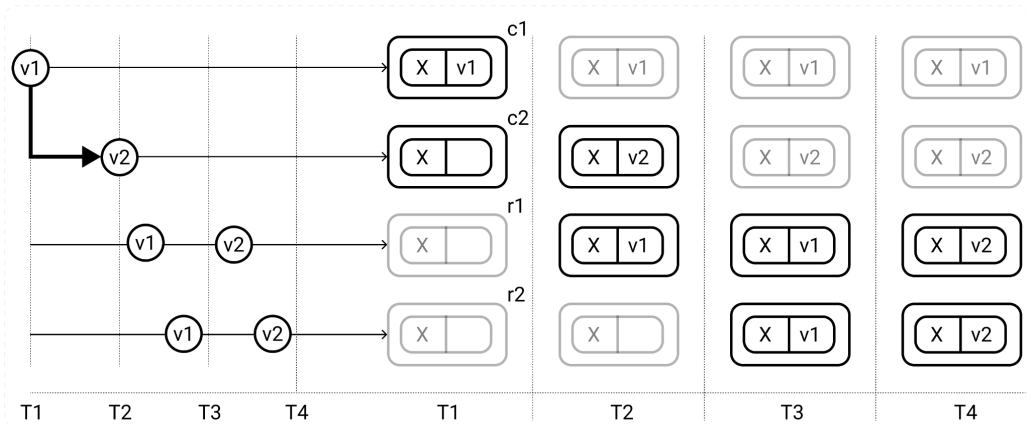


Рис. 6-16. Консистентность с причинной взаимосвязью

Такая конфигурация устанавливает причинный порядок между соответствующими операциями. Если последняя операция синхронизируется быстрее, чем предыдущая, она станет видимой только в последовательности, установленной логикой взаимосвязи. Таким образом, последовательность изменений выстраивается логически, без необходимости анализа временных меток.

В качестве простого примера можно привести следующий: в РСУ, управляющей промышленным оборудованием без команды управления, объект управления, такой как двигатель, не может включиться сам по себе. Если же такое случилось и система видит активность двигателя, но не видит команды управления, это повод разобраться, кто управляет двигателем.

6.6.6. Последовательная консистентность

Модель последовательной консистентности (*sequential consistency*) подразумевает, что установка консистентности РСУ предполагает упорядоченную последовательную установку консистентности

отдельных ее операций. Например, в РСУ все операции (сообщения) могут проходить через единый брокер очередей, поэтому такая модель является самой простой в реализации. Нужно понимать, что в РСУ и тем более в СРВ операции проходят достаточно динамично, поэтому процесс последовательной консистентности априори становится непрерывным и является слабо консистентным.

Алгоритм последовательной консистентности наиболее просто реализовать совместно с балансировкой нагрузки операций, выполняемых в виде очередей.

Модель последовательной консистентности можно применять при синхронизации данных, содержащих временные ряды. Данная модель гарантирует, что все синхронизируемые узлы будут видеть изменения в одинаковой последовательности. Такую модель называют консистентностью по временной шкале.

Такую консистентность еще называют сериализуемой консистентностью (*serializable consistency*) в связи с тем, что в случае применимости данной модели узлы распределенной системы могут обмениваться сериализованными данными.

6.6.7. Консистентность по времени

Выделяют консистентность по времени (*timeline consistency*). В этой модели консистентности сообщения синхронизируются строго в том порядке, в котором они были получены. Такая модель консистентности может быть использована для сообщений, в которых отсутствует временная метка или какая-либо возможность проверки временной метки в процессе обработки, но в которых требуется обеспечить структуру данных *time series* (рис. 6-17).

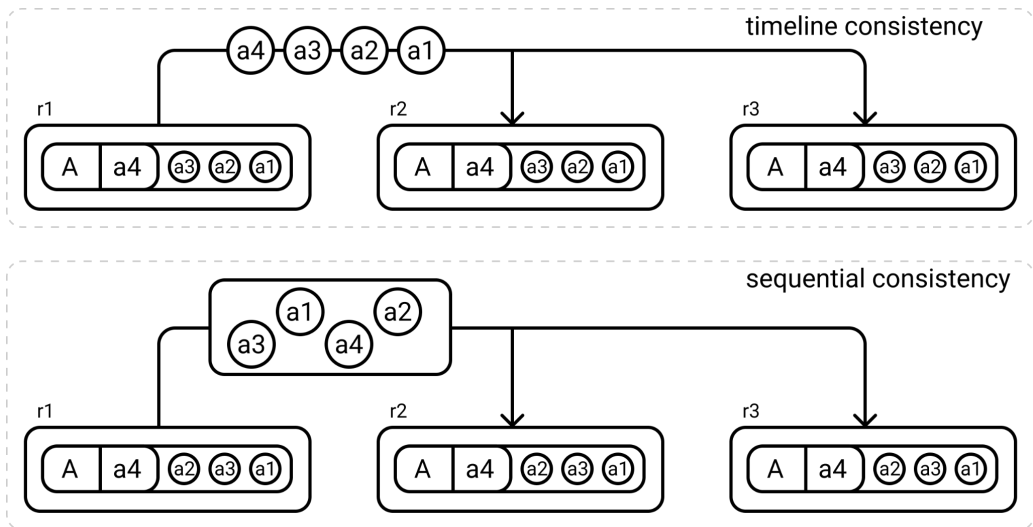


Рис. 6-17. Консистентность последовательная и по времени

6.6.8. Линеаризуемая консистентность

Модель последовательной консистентности тесно связана со свойством линеаризуемости⁶⁰. Это возможность выстроить в последовательность (очередь) любые операции, включая параллельные. Алгоритмическая реализация подразумевает отсутствие необходимости каких-либо блокировок.

Линеаризуемая консистентность (*linearizable consistency*) должна являться неотъемлемым свойством СРВ, потому что при эффективной синхронизации и разбиении времени все операции последовательно должны выстраиваться в очередь *time series*. Значения времени операции будут отличаться хотя бы на наносекунду (рис. 6-18).

Например, изменения на узле $r1$ инициируют синхронизацию широковещательным сообщением с узлами $r2, r3, r4$. В теории синхронизация должна идти параллельно, но физика берет свое, поэтому в момент времени $T3$ наблюдаются небольшие задержки.

⁶⁰ Linearizability, #6-12

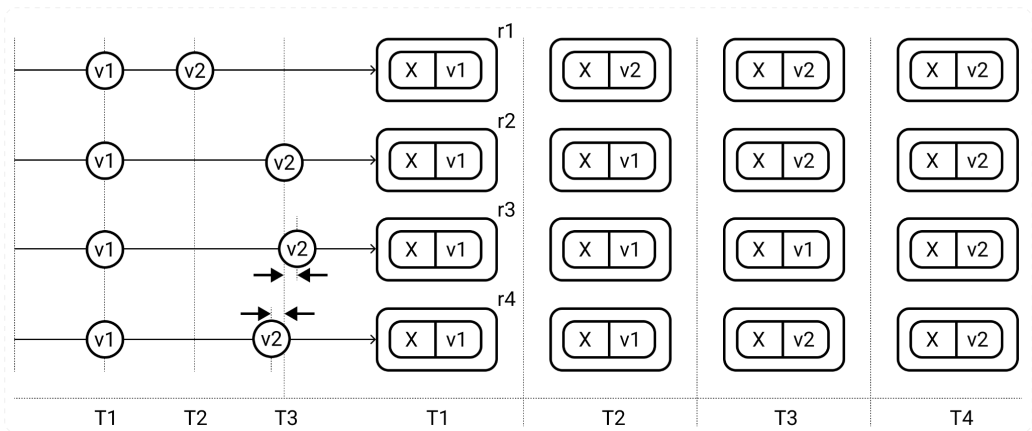


Рис. 6-18. Линеаризуемость

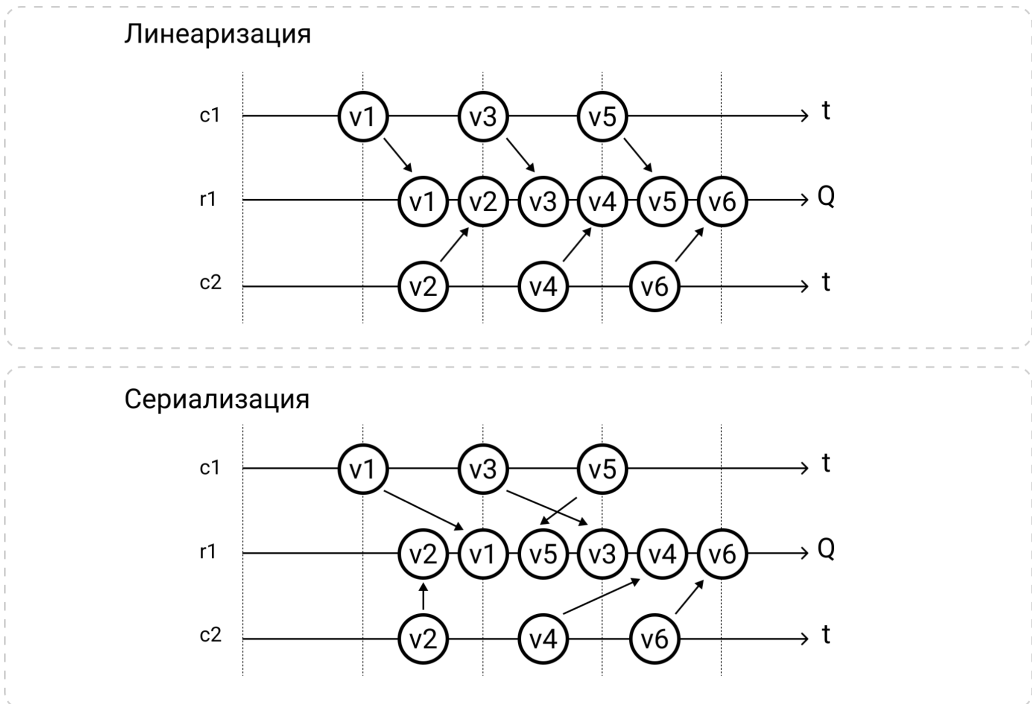


Рис. 6-19. Линеаризация vs сериализация

Отличие линеаризуемой консистентности от сериализуемой заключается в том, что для первой существует лишь единственно возможный вариант чередования сообщений в очереди Q , а для второй может существовать некое пространство вариантов чередования сообщений в очереди (рис. 6-19).

6.6.9. Проблемы оценки консистентности

В ВНС со слабой консистентностью возникают очевидные проблемы. Алгоритм оценки консистентности может не успевать срабатывать, а скорость изменения данных может мешать алгоритму оценки. Чтобы упростить задачу оценки консистентности, система может выделять время на синхронизацию, блокируя возможность записи новых значений в модифицируемые переменные. Таким образом, модель слабой консистентности еще больше упрощается, так как в момент синхронизации с системой ничего не происходит. Конечно, для CPB такой подход скорее вреден, однако если необходим порядок в оценке консистентности, то он вполне применим. Применяют две модели ослабления слабой консистентности: консистентность по выходу и консистентность по входу.

6.6.10. Консистентность по выходу

Консистентность по выходу основана на блокировке доступа к синхронизируемым переменным в моменты синхронизации в целях непредоставления неконсистентных данных. Для этого создается специальный объект управления доступом (блокировки) к синхронизируемым переменным, а именно критическая секция. Синхронизация осуществляется в пределах критической секции, при этом все остальные операции система блокирует. Модификацию значений переменных и их чтение переводят в режим ожидания. На выходе из критической секции наступает консистентность по выходу, то есть система считается консистентной. В таком случае необходимо аккуратно настраивать частоту циклов синхронизации, чтобы успевать обрабатывать все изменения, без ухудшения характеристик ВНС. В моменты, отличные от выхода из цикла синхронизации, система существует в окне неконсистентности.

6.6.11. Консистентность по входу

Каждой синхронизируемой переменной ставят в соответствие связанную специальную синхронизационную переменную. Затем осуществляется синхронизация с блокировками — как в случае с консистентностью по выходу, лишь с этой конкретной переменной. Таким способом можно осуществлять синхронизацию части переменных, блокируя доступ не ко всем переменным сразу.

Такая модель позволяет осуществлять непрерывную синхронизацию данных в системе, обрабатывая лишь изменившиеся данные. Запускать синхронизацию конкретной переменной можно при изменении статуса синхронизационной переменной, то есть по факту ее изменения или по входу.

Такой подход удобно использовать в случаях модификации переменных на разных узлах в разнообразных системах сбора и обработки информации из разных источников.

6.6.12. Строгая консистентность в конечном счете

В некоторых случаях можно найти золотую середину между строгими и слабыми моделями консистентности. Подобной золотой серединой является строгая консистентность в конечном счете. Такая модель консистентности допустима при использовании специального типа данных с некоторыми ограничениями⁶¹. Такие данные можно реплицировать на множество узлов и обновлять параллельно, без координации между узлами. Важно отметить, что подобной системе консенсус не нужен, при этом теоретически выполняются все три условия CAP-теоремы. Указанные ограничения совсем не противоречат требованиям реально существующих и создаваемых РСУ. Самым простым примером таких данных являются любые энтропийные переменные в РСУ. Под энтропийными данными могут подразумеваться инкрементные переменные, которые никогда не уменьшаются. Таковыми являются

⁶¹ Conflict-Free Replicated Data Type (CRDT), [#6-13](#)

разнообразные счетчики, используемые в разных целях, например, в целях обслуживания статистики в РСУ. При синхронизации таких переменных между узлами нет необходимости осуществлять сравнение — необходимо осуществлять исключительно операцию сложения. Более сложным примером таких данных являются переменные, изменение которых может происходить только на одном узле, то есть данные, управляемые алгоритмом размножения для чтения (разд. 4.2.3), а точнее — данные, в которых невозможны коллизии.

Естественно, в РСУ не все данные могут относиться к CRDT, но значительная их часть точно может. Следовательно, применяя стратегию использования строгой консистентности в конечном счете для части переменных, можно существенно оптимизировать производительность в целом.

6.7. Современная классификация РСУ

Как уже было сказано выше (разд. 3.5), CAP-теорема применима только к строгой консистентности — она не классифицирует современные РСУ, допускающие слабую консистентность. Существуют принципы, которые более точно описывают проблематику ограничений РСУ. Наиболее известным принципом считают BASE⁶², который описывает следующие требования к РСУ:

- *Basically available* — базовая доступность подразумевает, что сбой незначительного числа узлов не приведет к утрате работоспособности всей РСУ и отказу в обслуживании значимого числа взаимодействий;
- *Soft-state* — неустойчивое состояние (разд. 7.3.1) означает, что РСУ некоторое время может находиться в состоянии, близком к деградации (разд. 7.3), теряя часть наименее значимых для ее работы данных;
- *Eventually consistent* — РСУ находится в возможно-консистентном состоянии (разд. 6.6.4).

⁶² BASE, #6-14

Таким образом, принцип BASE является обратным принципу ACID. Подразумевается, что в реальных условиях в РСУ не может быть достигнута строгая консистентность. ACID подразумевает строгую консистентность, но невысокую скорость. BASE, напротив, подразумевает высокую скорость, но слабую консистентность.

Существует принцип, уточняющий CAP-теорему, — теорема PACELC⁶³ (рис. 6-20). Теорема учитывает латентность (*L*), позволяющую выстроить более точную классификацию требований к РСУ, чем в CAP (*E-Else* — означает иначе, когда система работает в нормальном режиме, в отсутствие разделения).

PA/EC	EC	PC/EC
PA	PACELC	PC
PA/EL	EL	PC/EL

Рис. 6-20. Теорема PACELC

Теорема подводит к существованию четырех классов РСУ :

- PA/EL, при разделении (*P*) обеспечивается доступность (*A*), консистентность менее важна. Иначе (*E*), важнее латентность (*L*), жертвуя консистентностью (*C*);
- PA/EC, при разделении обеспечивается доступность, консистентность менее важна. Иначе, важнее консистентность, жертвуя латентностью;
- PC/EL, при разделении обеспечивается консистентность, доступность менее важна. Иначе, важнее латентность, жертвуя консистентностью;

⁶³ PACELC, #6-15, #6-16 ►

- PC/EC, при разделении обеспечивается консистентность, доступность менее важна. Иначе, важнее консистентность, жертвуя латентностью.

Современные зрелые распределенные системы (например, те же СУБД) могут поддерживать разные конфигурации в зависимости от состояния⁶⁴, поэтому такие системы нельзя отнести к тому или иному классу PCY.

6.7.1. Типы PCY в зависимости от консистентности

На основании требований принципов CAP (разд. 3.5), PACELC, BASE могут быть выделены типичные представители соответствующих типов решений.

Типичным представителем решений CA (PC/EC) являются PCY, поддерживающие принцип ACID (разд. 3.3). Таковыми являются PCYБД. Большинство известных решений нацелены на оптимизацию данных при их распределенном хранении (не всегда с резервированием). Реляционные связи таблиц и язык запросов SQL обуславливают необходимость высокой доступности всех узлов и строгой консистентности данных (разд. 6.6). Именно по этой причине PCYБД обычно требуют мощных серверов в едином ЦОД с хорошими каналами связи.

Типичным представителем решений AP (PA/EL) являются PCY, поддерживающие требования принципа BASE. Проектирование именно таких систем описано в книге.

Типичным представителем решений AP (PC/EC) являются PCY с высокой степенью автономности узлов. Например, такими системами являются системы распределенных вычислений GRID (разд. 9.1.4). Узлы системы такого типа взаимодействуют друг с другом только при получении задания и выдаче результатов — в момент осуществления расчетов они не взаимодействуют.

⁶⁴ Конфигурации PACELC, #6-17

6.7.2. Византийская задача

В большинстве систем, какими бы они ни были, за изменение каждого конкретного элемента данных отвечает отдельный источник. Например, изменение значения датчика температуры приводит к изменению соответствующей переменной. Если для работы системы необходимо обеспечить реакцию на основе измерений нескольких датчиков, можно исключить из решения определенные результаты (например, с большим отклонением), пересчитать среднее и двигаться дальше.

Бывают ситуации, когда необходимо принять решение на основе команд, одновременно поступающих из нескольких источников. Например, легко принять решение об остановке ядерного реактора, потому что если хотя бы одна из подсистем, от которых зависит его безопасность, вызывает необходимость остановки, указанный процесс будет запущен на основании голоса одной подсистемы. Такие системы обычно относятся к СРВ, где все узлы принимают участие в обработке данных, поэтому можно успеть много раз проанализировать ситуацию.

Что делать, если не все узлы РСУ участвуют в решении задачи принятия решения? Каналы связи нестабильны. Доверие между узлами отсутствует, некоторые из них могут предоставлять заведомо ложную (ошибочную) информацию. Такие задачи вполне реальны. Например, самоорганизующийся рой несущих вооружение беспилотников, часть из которых была взломана противником с соответствующими последствиями, — это типичный пример задачи о византийских генералах⁶⁵, хорошо известной и подробно описанной. Именно так формулируют задачи в блокчейнах.

⁶⁵ Византийская задача, #6-18

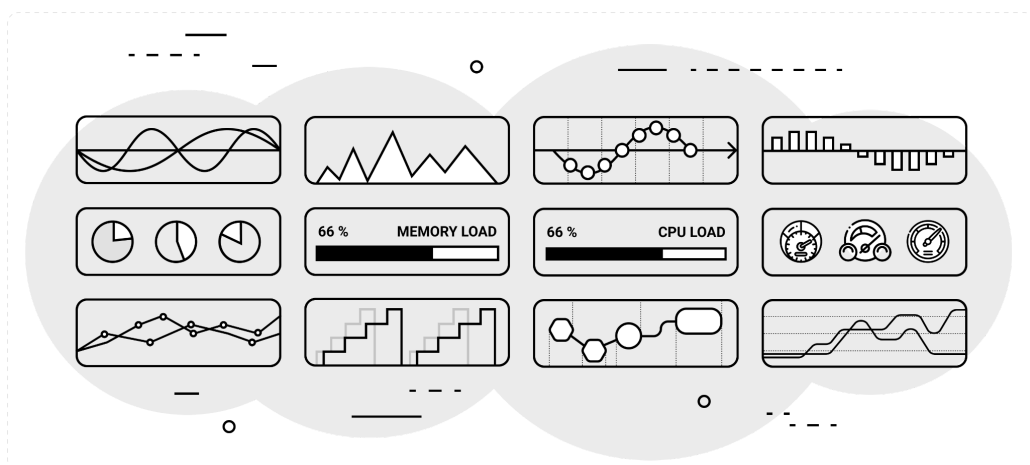
6.7.3. Блокчейн

В блокчейнах используют специальные алгоритмы подтверждения новых блоков данных, при этом совсем не быстрые. Классические блокчейны подразумевают необходимость хранения всей цепочки изменений блоков, поэтому объем цепочек постоянно растет. Создание новых элементов и использование нестатических структур данных в СРВ неконструктивны. Существуют решения с фиксированными размерами цепочек блоков и даже быстрые алгоритмы консенсуса, разработанные специально для блокчейна⁶⁶. Такие решения можно применять в качестве сопровождающих систем для РСУ, например, для поддержки архивных данных, их защиты от модификации (подделки, искажения).

6.8. Персистентность данных

В серьезных системах важно уметь сохранить состояние данных узла перед его отказом. РРП РСУ может быть персистентной. Совокупность распределенных узлов с высоким фактором репликации (разд. 4.5, консолидации) данных может обеспечивать следующее: при утрате узла и вводе вместо него нового данные, необходимые для работы нового узла, могут быть загружены с активных узлов. Конечно, для этого нужно, чтобы РСУ обеспечивала эффективную длительную консистентность. Такая форма организации распределенной памяти с избыточностью данных и разумными свойствами РВ вполне тянет на полноценную персистентную память. Это чрезвычайно важное свойство, так как оно не может быть реализовано на одном локальном узле. Таким образом, распределенная персистентная оперативная память целевой системы всегда доступна, вне зависимости от факта утраты даже значимого числа узлов. Если РСУ поддерживает сохранение изменений состояния данных, то может быть реализована и полная персистентность, позволяющая восстановить работу, например, узла в состоянии до сбоя или любого требуемого события.

⁶⁶ 65 алгоритмов консенсуса для блокчейна, [#6-19](#)



Глава 7. Отказоустойчивость и высокая доступность

7.1. Диагностика и качество сервиса (QoS)

Обеспечение качества сервиса QoS⁶⁷ (обслуживания) является важнейшей задачей любой ВНС, в особенности СРВ. Существует множество разных определений QoS — например, «принцип предоставления различным классам трафика различных приоритетов в обслуживании», или «вероятность того, что сеть связи соответствует заданному соглашению об уровне обслуживания (SLA⁶⁸)», или «обозначение вероятности прохождения пакета между двумя точками сети».

PCY обычно взаимодействуют не с трафиком низкого уровня, а с протоколами высокого уровня: начиная с протоколов транспортного уровня модели OSI⁶⁹ активно применяют протоколы сеансового уровня. В этом случае перечисленные выше определения можно свести к следующему: QoS — принцип

⁶⁷ Quality of Service, [#7-01](#)

⁶⁸ Service Layer Agreement, [#7-02](#)

⁶⁹ OSI, [#7-03](#)

предоставления различным классам сообщений различных приоритетов в обслуживании (разд. 5.4). Для присвоения сообщению определенного класса необходимо задать приоритет данных (переменных, блоков), которые будут упакованы в соответствующее сообщение. Очевидно, что с точки зрения архитектуры возможно групповое присвоение приоритетов переменным, то есть имеется возможность задать общие, групповые правила (политику) приоритизации переменных.

Приоритизация сообщений может быть применена в планировании общей нагрузки системы и отдельных ее узлов с опорой на метрики качества сервиса.

7.1.1. Метрики качества сервиса

Все метрики целевой системы, метрики качества сервиса (метрики QoS) можно разделить на несколько типов:

- задающие (*set metrics*) — задают параметры компонентов системы;
- диагностические (*diag metrics*) — используются для определения диагностических параметров компонентов системы;
- композитные (*composite metrics*) — сложные метрики, описывающие поведенческую модель отдельных компонентов системы и всей системы в целом, которые могут быть использованы для адаптивной динамической подстройки системы.

7.1.2. Метрики приоритета

Метрики приоритета относят к задающим метрикам. Они позволяют задать приоритет данных для использования в алгоритмах их обработки, для работы с очередями сообщений, подготовки, упаковки и отправки сетевых пакетов.

7.1.3. Метрики состояния

Диагностические метрики позволяют оценить текущее состояние целевой системы и определить метрики, связанные с длительной непрерывной работой системы.

Выделяют несколько типов метрик, например, метрики, связанные с работой конечных узлов. Для оценки состояния и корректировки работы узлов применяют следующие узловые метрики:

- загрузка процессоров;
- загрузка памяти;
- загрузка сетевых адаптеров;
- объем трафика (пакетов);
- скорость обработки очередей сообщений.

Для диагностики эффективности синхронизации данных между узлами системы могут быть заданы следующие метрики консистентности:

- время консистентности;
- время неконсистентности;
- среднее время неконсистентности.

Метрики сообщений (очередей):

- общее и среднее число отказов доставки сообщений;
- метрики дедупликации данных;
- метрики эффективности обработки очередей;
- время доставки сообщения (*message latency*).

Метрики отклика:

- дедлайн (*deadline*) — предельный срок операции;
- латентность (*latency*) — время отклика (задержки) операции;
- джиттер (*jitter*) — разброс значений времени отклика.

7.1.4. Композитные метрики

Композитные метрики представляют собой сложные вычислительные конструкции, необходимые для определения сложного состояния целевой системы и управления им. Например, метрика задержки сообщения является типичной композитной метрикой, так как представляет собой сумму задержек всей цепочки узлов. Задержка прохождения (ретрансляции) сообщения L от узла $r1$ до $m2$ – это сумма задержек доставки по сети между узлами $L1, L3, L5$ и задержек обработки $L2, L4$ (рис. 7-01).

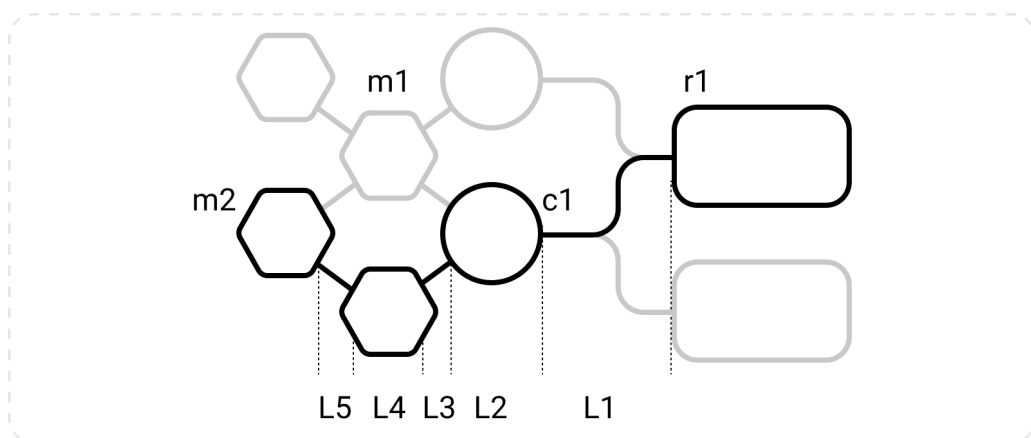


Рис. 7-01. Задержка ретрансляции

7.1.5. Метрика латентности *heartbeat*

Время отклика (латентность) является важнейшим параметром ВНС. Измерение отклика позволяет оценить реальное быстродействие системы, ее компонентов, реализованных алгоритмов и сценариев. При желании можно провести измерение откликов всех компонентов, вычислительных сегментов – при этом необходимо обеспечить хранение соответствующих метрик.

В диагностических измерениях необходим баланс. Если ввести большое количество диагностических метрик, их обслуживание системой создаст дополнительную нагрузку, которая может оказаться нежелательной. Поэтому необходим разумный подход к проектированию средств диагностики.

Встречаются критически важные системы, в которых лишь 10% данных являются полезными. Остальное — диагностика; система рассчитана на ее обслуживание, но без такой тщательной диагностики работать не может.

Измерение отклика — значимая задача. Можно выполнить измерения напрямую или ввести косвенные метрики, которые будут отражать поведение отдельного компонента или всей системы в целом.

В диагностике PCY применяют метрики типа *heartbeat* («сердцебиение») — это специальные диагностические сигналы, которые позволяют оценить латентность системы или ее компонента при определенных условиях.

Наиболее простой *heartbeat* используют для измерения отклика узла на простейший запрос (типа *ping*). Отклик зависит от загруженности узла.

Эффективнее всего использовать более сложные *heartbeat*. Обычно генерируют специальный сигнал в виде лесенки, пилы, синусоиды или иной показательной специальной функции — такой сигнал пропускают через наиболее показательные для диагностики маршруты обмена данными и нагруженные узлы. По изменениям параметров *heartbeat* можно судить об изменениях системы. Чтобы точно определить изменения (для набора статистики), при вводе системы в эксплуатацию измеряют эталонные *heartbeat* в разных условиях нагрузки. В реальной работе осуществляют сверку с полученной статистикой.

Наборы сигналов *heartbeat* отлично поддаются статистической обработке. Как правило, анализ корреляции изменений *heartbeat* различных компонентов системы позволяет точно определить источник возникших или потенциальных проблем.

7.2. Отказоустойчивость и надежность

Какой бы ни была архитектура РСУ, важнейшей задачей архитектуры является обеспечение высокой доступности и надежности целевой системы. Надежность — это способность системы сохранять свои функции в заданном временном интервале. Это композитный параметр: надежность любой системы определяет надежность ее отдельных компонентов.

В этой главе целесообразно рассмотреть несколько терминов теории надежности, из которых можно вывести совокупность диагностических параметров, присущих РСУ, СРВ.

Надежность напрямую связана с показателем безотказности, а именно безотказной работы системы, то есть такой системы, в которой отсутствуют отказы. Отказ – один из основных терминов теории надежности, обозначающий событие, при котором система или ее компонент целиком или частично перестает выполнять свои функции. Отказ является случайным событием, он может быть единичным. Множественные отказы часто бывают напрямую связаны с единичным отказом, их рассматривают именно как единичный отказ. Например, если заявлено, что система должна обработать 1000 сообщений в секунду (MPS⁷⁰), но по каким-то причинам это не выполняет и при этом стабильно показывает, что справляется только с 999 MPS, такая ситуация однозначно является отказом.

Отказоустойчивость⁷¹ — способность системы сохранять работоспособность после единичного отказа или множественных отказов одного или нескольких компонентов системы. Отказоустойчивость определяет количество единичных отказов компонентов системы, после которых сохраняется работоспособность всей системы в целом. Базовый уровень отказоустойчивости предполагает защиту от отказа одного любого элемента.

⁷⁰ Message Per Second

⁷¹ Fault-Tolerance, FT, [#7-04](#)

Таким образом, основным способом повышения отказоустойчивости является избыточность (разд. 4.3).

С точки зрения теории надежности, избыточностью называют функциональность, которая не является необходимой для безотказной работы системы.

Термин «отказ» применим только к аппаратному обеспечению. К ПО он неприменим, так как в ПО не может возникнуть отказ. ПО может плохо работать из-за ошибок разработчика или внешнего воздействия, в особенности вредоносного. Таким образом, отказ, связанный именно с аппаратной частью, может иметь место в ПАК, АПК, ПТК. Например, всем известный DoS (отказ в обслуживании) возникает не из-за плохого ПО, а прежде всего из-за недостатка ресурсов аппаратного обеспечения.

В правильно спроектированные ВРСУ, СРВ закладывают характеристики для диагностики всех возможных признаков отказа. Если система введена в действие (эксплуатацию), считается, что вероятность наступления события отказа минимизирована на этапе проектирования и отладки системы.

Вероятность отказа — это показатель надежности, характеризующий вероятность того, что в пределах заданного времени работы система откажет хотя бы один раз. Такое время называют временем наработки между отказами (OTBF⁷²). Вероятность отказа всей системы сразу, естественно, зависит от вероятностей отказа всех элементов системы — даже самого маленького винтика. Далее вспоминаем курс теории вероятностей и математической статистики.

Важным параметром системы является вероятность безотказной работы — это вероятность того, что в пределах заданной наработки или в заданном временном интервале отказа системы (компонента) не возникнет.

⁷² Operating Time Between Failures, #7-05

Средняя наработка на отказ (MTBF⁷³) представляет собой параметр, характеризующий надежность восстанавливаемой системы или компонента. Средняя продолжительность работы системы между отказами, то есть MTBF, демонстрирует среднюю наработку, которая приходится на один отказ. MTBF измеряют в часах. Часто применяют термин «наработка на отказ».

Хорошие системы принято проектировать так, чтобы значение MTBF превышало время между запланированными ремонтами или срок эксплуатации оборудования.

Самый неприятный тип отказа в системах — это постепенный отказ. Такой отказ характеризуется медленным (постепенным) ухудшением параметров компонентов, приводящим к деградации всей системы (разд. 7.3).

7.2.1. Отказоустойчивая архитектура

Отказоустойчивость обеспечивается прежде всего избыточностью компонентов системы. Для продуктивного использования свойств избыточности необходимо быстро и эффективно диагностировать отказы — в особенности постепенные.

Отказ одного или нескольких компонентов системы не должен приводить к отказу в обслуживании клиентов системы. Система должна сохранять способность обслуживать клиентов все время, пока остается работоспособной хотя бы минимальная часть ее компонентов. В случае с РСУ необходимо соблюдать кворум узлов (разд. 6.6).

Какой бы ни была избыточность, за эффективное переключение, задействование избыточных аппаратных компонентов в итоге отвечает ПО. ПО может управлять переключением и задействованием избыточных ядер процессора, процессоров целиком, сетевых карт и ресурсов вычислительных узлов в целом.

⁷³ Mean Time Between Failures

Для серьезного оборудования и каждого его компонента производитель определяет значения наработки на отказ, однако это не позволяет избежать необходимости диагностировать состояние каждого компонента системы в режиме эксплуатации. Нужно непрерывно проверять, не происходит ли постепенный отказ какого-нибудь компонента системы.

7.2.2. Время восстановления

Любая техническая система может отказать полностью. Необходимо учитывать время, необходимое для восстановления системы. Время восстановления системы RTO^{74} не должно превышать время, в течение которого могут наступить катастрофические события, возникнуть существенные финансовые и/или репутационные потери.

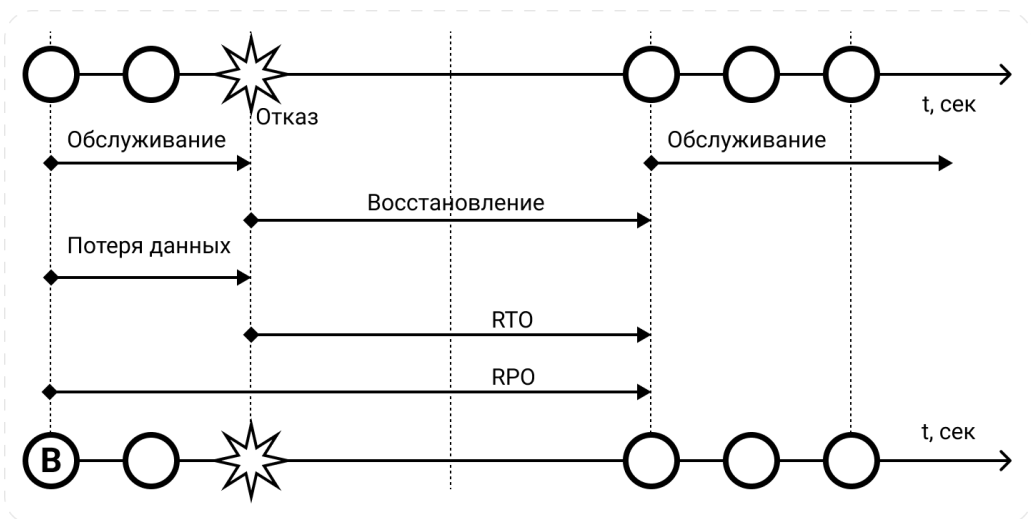


Рис. 7-02. RTO, RPO

В критических системах важно выполнять резервирование данных в режиме реального времени. Эти данные могут потребоваться для восстановления после полного сбоя. Показатель точки восстановления (RPO^{75}) определяет время

⁷⁴ Recovery Time Objective, #7-06

⁷⁵ Recovery Point Objective #7-07

восстановления системы после сбоя с момента последнего резервирования. Показатели RTO, RPO должны быть заданы на этапе проектирования системы как требования (рис. 7-02).

7.2.3. Катастрофоустойчивость

Если отказоустойчивость представляет собой способность системы сохранять работоспособность после единичного или множественных отказов, то катастрофоустойчивость⁷⁶ — это способность к восстановлению и продолжению функционирования после катастрофы, то есть устойчивость к воздействию природных катаклизмов и террористических актов, а точнее к влиянию человека, человеческого фактора.

Например, события на Чернобыльской АЭС говорят о том, что объект не был катастрофоустойчивым. Сейчас АЭС строят так, что в них допускаются падение на энергоблок крупного авиалайнера, 9-ти балльные землетрясения — даже в таких случаях не произойдет повреждение систем, важных для расхолаживания реактора, в особенности противоаварийной защиты⁷⁷ (ПАЗ) АСУ ТП.

Обеспечение катастрофоустойчивости РСУ (АСУ ТП) — непростая задача, однако она крайне важна для критической информационной инфраструктуры (КИИ), в особенности для значимых объектов КИИ (ЗОКИИ). Например, задачу обеспечения катастрофоустойчивости ЦОД реализуют путем географического распределения, что невозможно сделать, например, с объектом генерации. Очевидно, что катастрофоустойчивость еще меньше связана с ПО, чем отказоустойчивость. Однако современные РСУ в некоторых особо эффективных конструкциях могут решить и эту задачу (разд. 9.2.2).

⁷⁶ Disaster Tolerance, #7-08

⁷⁷ Safety Instrumented Systems (SIS), #7-09

Время восстановления системы управления после катастрофы⁷⁸ не должно превышать временной интервал, по истечении которого начнется катастрофа на физическом объекте управления. Для обеспечения гарантированного восстановления сервиса должен быть разработан план аварийного восстановления DRP⁷⁹, который необходимо регулярно актуализировать. На его основе нужно регулярно проводить учения персонала по отработке плана. Учения можно проводить как на макетах (тренажерах, цифровых двойниках), так и на реальных объектах. Например, на некоторых предприятиях внутренний распорядок предусматривает регулярное «боевое» переключение между различными группами узлов ЦОД.

Управление процессом аварийного восстановления может входить в состав систем управления как отдельное решение DRS⁸⁰. Такие решения могут обеспечить доступность критичных систем в случае возникновения крупных распределенных аварий. Одним из способов обеспечения катастрофоустойчивости является предупреждение и предотвращение аварий.

В рамках концепции «недопущения катастроф⁸¹» могут быть реализованы схемы защиты от «медленно наступающих катастроф», таких как постепенное затопление или медленно распространяющийся пожар. Если в ходе проектирования системы были реализованы схемы резервирования (например, географически разнесенного) работы групп (кластеров) узлов, то активная нагрузка может быть перенесена между группами в режиме РВ без прерывания обслуживания. Каждая отраслевая система может содержать ряд метрик обнаружения событий, связанных с предупреждением катастроф, и методов их предотвращения.

⁷⁸ Disaster Recovery, #7-10

⁷⁹ Disaster Recovery Plan, #7-11

⁸⁰ Disaster Recovery Solution, #7-12

⁸¹ Disaster Avoidance, #7-13

7.2.4. Архитектура высокой доступности

Высокая доступность⁸² — архитектурный подход к проектированию системы, позволяющий достичь высоких показателей доступности сервиса. Доступность сервиса⁸³ — это безразмерный числовой показатель отношения времени доступности сервиса для клиентов к общему времени работы. Чаще всего доступность измеряют в «девятках», то есть две девятки — это 99%, а три — 99,9%. Отказоустойчивость представляет собой частный случай высокой доступности, при согласованной матрице рисков и заданной доступности равный 100%. В общем случае под высокой доступностью сервиса подразумевают его способность пережить отказ компонента и за минимальное время вернуться в рабочее состояние. Например, в РСУ КИИ доступность не должна быть ниже шести девяток 99,9999%, это не более 30 секунд недоступности в год. Только представьте, что может произойти за эти 30 секунд на опасном химическом производстве. Таким образом, существуют системы с существенно более высокими требованиями к доступности, реализация которых — совсем не тривиальная задача.

7.2.5. Доступность сервиса SLA

Соглашение об уровне сервиса (SLA) — одно из наиболее спорных понятий в современных системах. Как следует из ранних изданий методики управления инфраструктурой ИТ (ITIL⁸⁴), такое соглашение состояло из набора целевых показателей сервиса SLI⁸⁵ и штрафных санкций за неисполнение. Однако в ITIL v4 ключевым моментом является отсутствие в шаблоне SLA каких-либо штрафных санкций за невыполнение показателей, а само SLA становится «меморандумом о намерениях» без указания ответственности провайдера сервисных услуг.

⁸² High availability, #7-14

⁸³ Uptime, #7-15

⁸⁴ IT Infrastructure Library, #7-16

⁸⁵ Service Level Indicators, #7-17

Тем не менее SLA в первом прочтении чрезвычайно актуально для серьезных ВНС и PCY, которые обычно имеют собственную серверную инфраструктуру, собственный ЦОД или даже используют распределенные ресурсы нескольких ЦОД. Обычно, когда подразумевают SLA, имеют в виду доступность серверной инфраструктуры для оказания сервисных услуг. Важно отметить, что SLA относится к любым PCY и к каждому отдельному их компоненту. Конечно, при оценке SLA не следует брать в расчет не резервированные компоненты, например, какие-либо датчики. Однако совокупность каналов связи с крупноузловым оборудованием необходимо учитывать.

При работе с серьезными PCY в соглашении об уровне сервиса необходимо указать все значимые показатели латентности, связанные как с сетевой частью PCY, так и с вычислительными компонентами. Например, в АСУ ТП удобно считать два показателя: SLA серверной инфраструктуры и всей остальной инфраструктуры, являющейся клиентской по отношению к серверной. Для PCY в составе АСУ ТП весьма показательно, насколько система является распределенной в действительности и должна быть доступна. Интегральные и предельные показатели SLA могут быть рассчитаны на основании расчетов надежности отдельных составных компонентов и всей системы в целом.

7.3. Деградация

Деградация (регрессия) представляет собой процесс ухудшения характеристик системы или отдельных ее компонентов с течением времени — постепенное ухудшение, снижение качества, разрушение аппаратных компонентов в связи с воздействием окружающей среды.

Самым очевидным примером деградации в PCY является деградация датчиков, применяемых в АСУ ТП. Материалы, из которых изготовлен датчик, со временем деградируют, вследствие чего изменяются характеристики генерируемого им сигнала.

С деградацией давно научились эффективно бороться, поэтому немедленной необходимости замены таких датчиков нет. О материалах, из которых изготовлен датчик, известно многое. Известно, как со временем будет меняться (плыть) сигнал вследствие деградации.

Существуют специальные настроечные таблицы, формулы, которые называют тарифовочными. Они позволяют оценить процесс деградации и учесть его в расчетах. Тарифовка — это компенсация погрешности измерительной системы от входа измеряемой физической величины до выхода измеренного значения этой величины. При деградации погрешность увеличивается. Обычно на обслуживающем датчик контроллере заводят функции или таблицы для дальнейшего перерасчета переменных и передачи их по назначению. Бывает так, что перерасчет осуществляется на более высоком уровне, например, уже в ПО РСУ.

Типичный пример: дорогостоящие датчики измерения температуры (термопары), в состав которых входят драгоценные металлы, находясь в активной зоне ядерного реактора, разрушаются под действием радиации, нейтронного излучения, высокой температуры теплоносителя. Их невыгодно часто менять, поэтому датчики тарируют.

В РСУ деградации, при которой ухудшается качество сигнала, наиболее подвержены материалы проводки подключения датчиков. Со временем в негодность приходят контроллеры, коммутаторы, серверы и прочее оборудование. Вендоры приводят соответствующие параметры надежности, долговечности, которые учитывают при проектировании РСУ. Кроме того, такие параметры учитывают при страховании рисков возникновения последствий отказов РСУ. Если отказ приведет к неприятностям, которые будут связаны с качеством продукции определенного вендора, то у него есть шанс получить очень серьезные проблемы, а в случае с КИИ проблемы не ограничатся только финансовой составляющей.

В хороших РСУ, которые проектируют на десятилетия, потенциальные признаки деградации закладывают в проект заранее. В проект вносят различные критерии для наблюдения за деградацией, сценарии поведения системы при выполнении таких критериев деградации: например, ввод резерва (аварийного). Кроме этого, прописывают регламент действия персонала.

7.3.1. Неустойчивое состояние

Неустойчивое состояние системы проявляется в момент пиковых нагрузок, различных отказов компонентов, при длительной неконсистентности и прочих проблемах. Система некоторое время может находиться в состоянии, близком к деградации, однако если она выходит из такого состояния самостоятельно или с какой-либо помощью извне, можно сказать, что система некоторое время находилась в неустойчивом состоянии. Длительность пребывания в таком состоянии является самостоятельной метрикой. Если система находится в состоянии неустойчивости дольше предельного времени, это может быть прямым проявлением деградации.

7.3.2. Элегантная и изящная деградация

Появление признаков деградации, не приводящих к отказу компонентов системы или отказу в обслуживании всей системы в целом, называется элегантной⁸⁶ деградацией. Если деградация привела к отказу компонентов системы, но не привела к отказу в обслуживании всей системы в целом, такую деградацию называют изящной⁸⁷. Указанные виды деградации должны обеспечить возможность продолжения эксплуатации системы до планового ремонта. В таком случае система будет сохранять свою работоспособность. Деградация не будет заметна пользователю до тех пор, пока регресс в работе системы не приведет к полному отказу в обслуживании системы, то есть до момента достижения порога элегантной деградации.

⁸⁶ Elegant degradation, [#7-18](#)

⁸⁷ Graceful degradation, [#7-19](#)

7.3.3. Управляемая деградация

Своевременная диагностика позволяет идентифицировать проявления деградации заблаговременно — до наступления отказа. Управление деградацией заключается в том, чтобы обеспечить требуемый сценарий управления технически регрессирующим оборудованием с помощью РСУ. Существует множество сценариев. Например, самый распространенный сценарий предполагает снижение нагрузки на оборудование путем балансировки нагрузки с целью снизить динамику деградации. И наоборот, можно перегрузить узел, чтобы окончательно израсходовать ресурс и заменить на новый. Такие сценарии актуальны для РСУ дискретных производств.

Эффективная диагностика позволяет точно определить причины деградации и принять необходимые меры для замедления этого процесса.

7.3.4. Деградация отклика

Бывает, что по каким-то причинам система или ее компонент начинает работать медленнее (тормозит). Такая ситуация свидетельствует об увеличении отклика компонента системы.

Обычно для измерения откликов системы используют метрики типа *heartbeat* (разд. 7.1.5). Если *heartbeat* со временем увеличивается, можно говорить о деградации отклика (рис. 7-03).

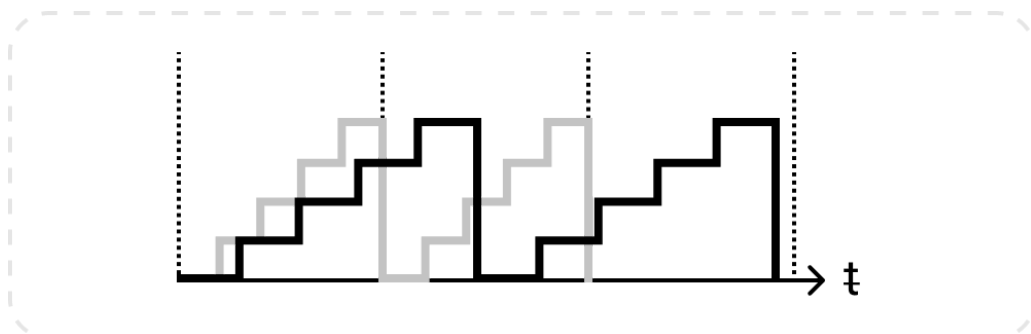


Рис. 7-03. Деградация отклика

7.3.5. Деградация очередей

Значимым видом деградации в РСУ является перегрузка очередей сообщений. Такая проблема может проявиться в том случае, если объем входящих сообщений больше, чем узловые аппаратные ресурсы в состоянии обработать за единицу времени. В таких случаях эффективно применяют алгоритм управляемой деградации очередей. Указанный алгоритм предполагает игнорирование сообщений с метриками приоритета наименьшей значимости. Обычно подобные алгоритмы реализуют в планировщиках нагрузки.

Возможно применение более сложных конструкций — например, при деградации очередей сообщений. Можно активировать выполнение алгоритма разряжения (фильтрации, разд. 12.3) данных на узле источников.

Для контроля эффективности очередей вводят специальные метрики — от замеров производительности CPU до расчета средней задержки обработки сообщения. Отклонения этих метрик позволяют судить о причинах деградации очереди сообщений. Когда метрики возвращаются к приемлемым для системы значениям, исчезает необходимость в управляемой деградации очереди.

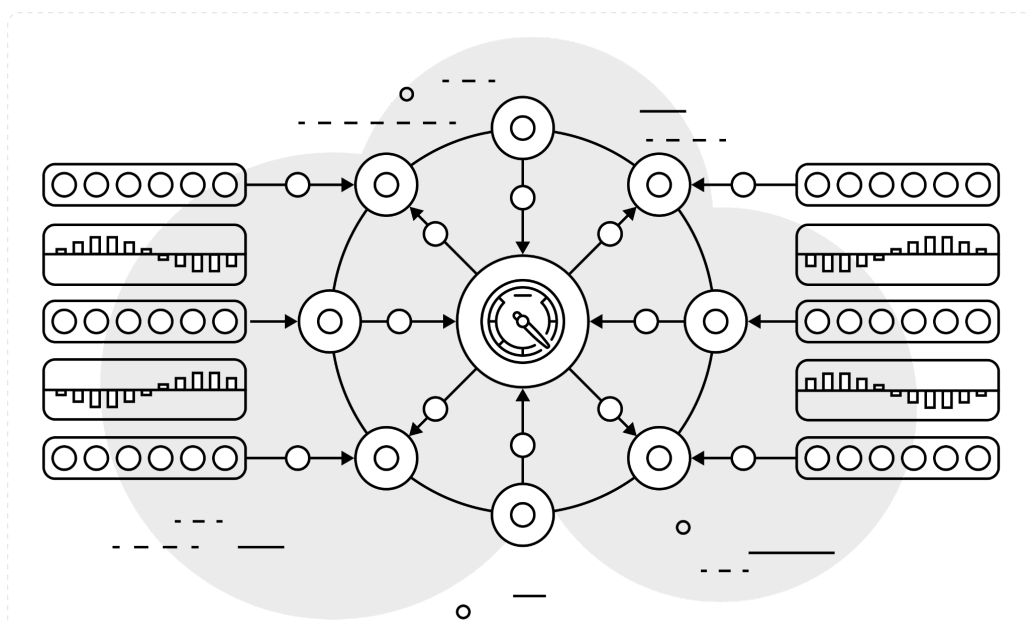
7.3.6. Деградация при информационном шторме

В РСУ, таких как АСУ ТП, возникают ситуации, когда компоненты системы самостоятельно начинают генерировать большой объем данных, кажущихся системе полезными — например, диагностических. При этом происходит общая перегрузка системы. Часто такие явления объясняются неэффективным проектированием компонентов системы, описанием логики обработки данных или ошибками при разработке программного кода.

В таких случаях обязательно возникают деградация очередей и деградация отклика компонентов, отвечающих за

логическую обработку данных. В ряде случаев такой информационный шторм может проявляться и нарастать очень быстро (лавинообразный эффект).

Для сохранения функции управления необходимо быстро и эффективно установить факт появления шторма и найти источник его возникновения. Нужно запустить управляемую деградацию источников, фиксируя обратную связь до полного восстановления баланса в системе.



Глава 8. Балансировка нагрузки

Балансировка нагрузки⁸⁸ (планирование нагрузки) РСУ является одной из сложнейших задач современных инфраструктур. Балансировка нагрузки — это метод распределения заданий между несколькими распределенными узлами. Целью балансировки являются:

- оптимизация использования ресурсов, сокращение времени отклика;
- горизонтальное масштабирование (разд. 10.2) узлов РСУ (динамическое добавление/удаление устройств);
- предотвращение перегрузки узлов;
- обеспечение отказоустойчивости (резервирования).

Архитектура схем и алгоритмов балансировки нагрузки весьма разнообразна, она зависит от целевой архитектуры системы. Например, архитектура балансировки нагрузки на узлы

⁸⁸ Load balancing, #8-01

mesh-сети сложнее, чем архитектура балансировки нагрузки на узлы сети типа «звезда». В большинстве РСУ узлы, обеспечивающие основную нагрузку (на сервер), обслуживают сети типа «звезда», «двойная звезда». В предлагаемой архитектуре РСУ таковыми являются корневые узлы.

Управление нагрузкой корневых узлы должны обеспечивать специальные балансировщики нагрузки, которые осуществляют распределение обработки данных между этими узлами в соответствии с заданным алгоритмом.

8.1. Задачи балансировки

Все задачи балансировки неразрывно связаны с оптимизацией. Самая распространенная задача балансировки — это задача оптимизации и управления распределением нагрузки на ресурсы вычислительных узлов. В этом случае учитывают аппаратные особенности узла, такие как мощность и загрузка ядер процессоров, интегральная загрузка, пропускная способность и загрузка сетевых устройств, скорость работы и загрузка оперативной памяти, ПЗУ. Может быть учтена эффективность работы ПО на узле с конкретными аппаратными характеристиками, так как оно может работать по-разному в зависимости от аппаратной конфигурации.

Оптимизация при балансировке может учитывать распределение активных соединений между узлами, например, в зависимости от удаленности узлов, качества соединения. В случае необходимости распределения нагрузки между узлами с учетом политики безопасности объектов и субъектов она может коснуться вопросов безопасности.

Различают следующие уровни балансировки: сетевой, транспортный и прикладной. Естественно, при проектировании РСУ, ВНС внимание уделяют прикладному уровню, хотя некоторые решения балансировки могут быть вынесены за пределы

функциональности целевой системы и развернуты на сетевом и транспортном уровне.

Задачи балансировки нагрузки тесно связаны с задачами планирования очередей, которые обеспечивают переключение между обрабатываемыми очередям на основе обратных связей. РСУ должны обеспечивать распределенную обработку очередей сообщений (разд. 5.4.5) с помощью специальных брокеров сообщений (очередей). Целевые алгоритмы (сценарии) балансировки задают в соответствующих брокерах, они являются их неотъемлемой частью.

8.2. Типы балансировки

Исходя из возможных конструкций брокеров сообщений можно выделить несколько типов брокеров-балансировщиков:

- встроенный в прокси-узел;
- встроенный в клиентский узел;
- синхронизации с координатором;
- синхронизации распределенный.

8.2.1. Балансировщик на прокси-узле

Брокер-балансировщик, встроенный в промежуточный (прокси) узел, является самым простым инструментом оптимизации нагрузки, формируемой клиентскими подключениями (рис. 8-01).

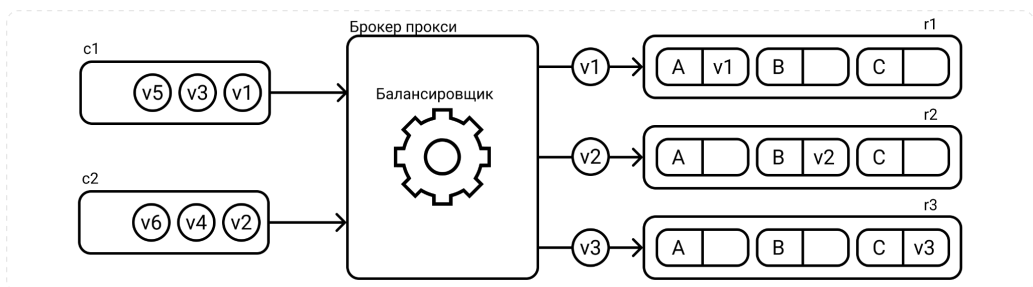


Рис. 8-01. Single-leader балансировщик

Проблемой таких единичных *single-leader* брокеров, которые представляют собой лишь посредники соединений, является то, что отказ брокера становится «узким местом» любой системы (разд. 5.4.2). В зависимости от целевой конфигурации прокси-брокеров может быть несколько (*multi-leader*) (рис. 8-02).

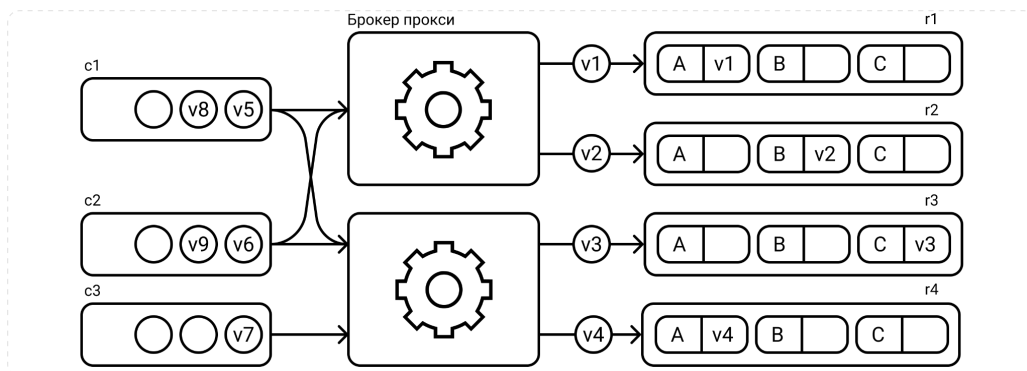


Рис. 8-02. Multi-leader балансировщик

В подобной ситуации клиент может иметь подключения к нескольким брокерам. В случае отказа одного из брокеров клиент переключается на вновь выбранный (разд. 5.3) или доступный брокер. Для того, чтобы было технически возможно реализовать такое переключение на уровне клиента, на самом клиенте должен присутствовать элементарный функционал проверки доступности брокера. Без такой проверки сообщения, доставляемые без подтверждения, могут потеряться.

8.2.2. Балансировщик на узле клиента

Брокер может быть встроен в клиентский узел по схеме без центрального узла (*leaderless*, разд. 5.4.2) (рис. 8-03).

Для того, чтобы реализовать логику балансировщика на узле клиента, клиент должен обладать функционалом получения информации о состоянии корневых узлов. Такая ситуация не всегда возможна в связи с тем, что некоторые РСУ создают не с нуля, а в рамках модернизации отдельных подсистем, поэтому

часто бывает нужно подключать компоненты, не обладающие соответствующим функционалом. Для таких случаев разумным решением является перенос брокера на корневые узлы (разд. 8.2.4).

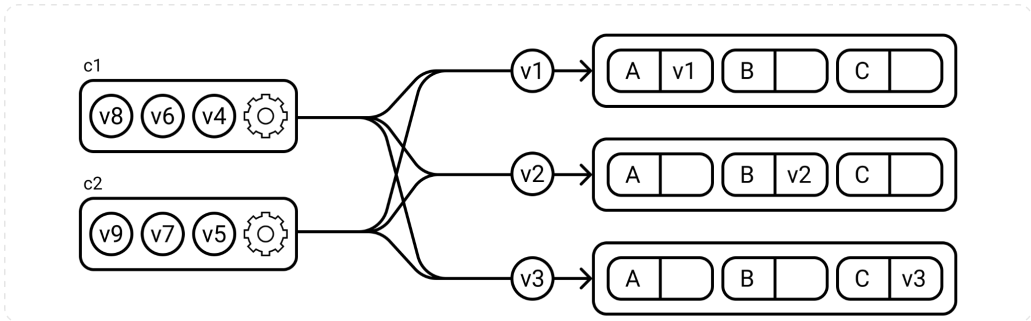


Рис. 8-03. Leaderless балансировщик

8.2.3. Балансировщик синхронизации с координатором

Сложную РСУ необходимо эффективно балансировать. Балансировки клиентов не всегда достаточно для эффективной балансировки РСУ. Клиентский узел может осуществить сложный запрос, который потребует координированной работы нескольких корневых узлов. При выполнении таких запросов участвующие в них данные необходимо будет синхронизировать, а узлы, обрабатывающие нагрузку, — балансировать.

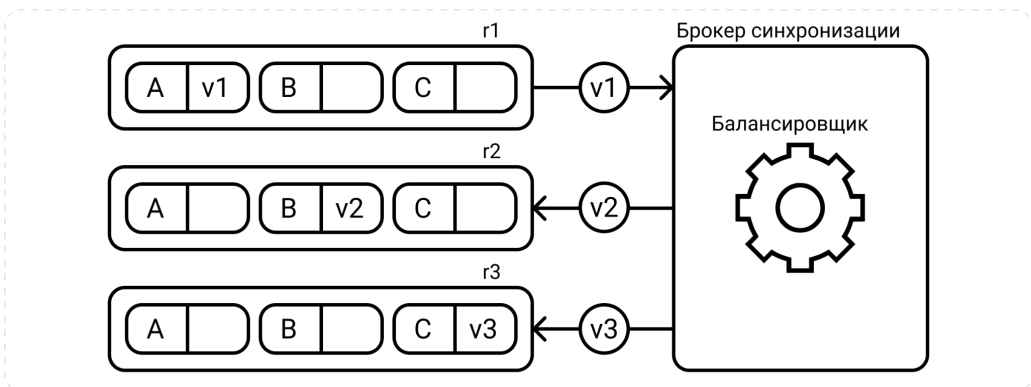


Рис. 8-04. Брокер синхронизации с координатором

Для обеспечения синхронизации с балансировкой корневых узлов может быть реализован специальный брокер синхронизации (рис. 8-04). Такой брокер, как правило, бывает реализован на специально выделенном узле, назначенном координатором синхронизации. В аппаратных системах такого рода координатор может быть встроен в шасси (бэмплейн), обеспечивающее выделенный интерконнект. Например, именно так проектируют некоторые модельные контроллеры. Подобная конструкция наиболее эффективна для модульных устройств. Рассмотренный брокер представляет собой типичное *single-leader*-решение. Отказ узла координатора или аппаратного шасси однозначно приводит к утрате работоспособности такой конструкции.

8.2.4. Балансировщик синхронизации распределенный

Брокер синхронизации может быть реализован в распределенном виде. Такая конструкция обычно бывает реализована на базе распределенного брокера (разд. 5.4.6), в основе которого заложена типичная *leaderless* архитектура. Брокер такого рода рекомендуется обслуживать отдельным, выделенным и изолированным от клиентов серверным интерконнектом. Подобное решение удобно для организации мощных вычислительных систем, причем такая конструкция брокера удобна как для модульных, так и для распределенных узловых конструкций (рис. 8-05).

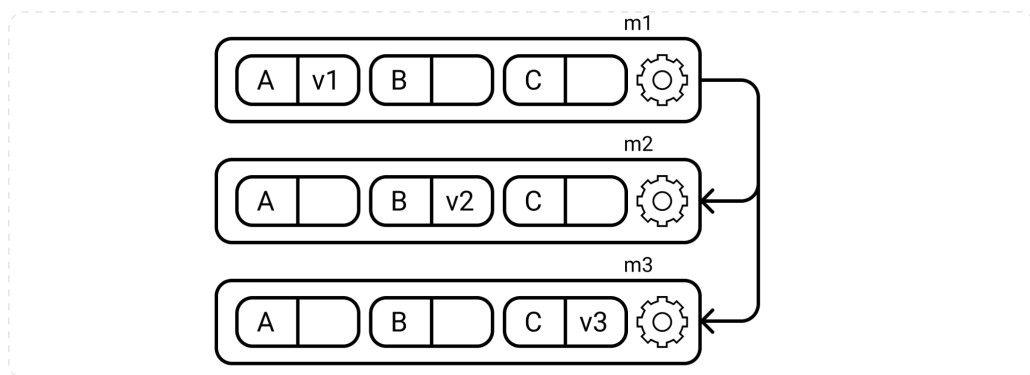


Рис. 8-05. Брокер синхронизации, встроенный в корневой узел

Такая конструкция балансировщика синхронизации является максимально эффективной и надежной для серьезных PCY, в которых требуются динамическая оптимизация, многопараметрическая балансировка.

Таким образом, в одной PCY могут быть одновременно использованы несколько брокеров: например, брокеры, встроенные в клиентские узлы, и брокер синхронизации. В подобной конструкции брокеры могут взаимодействовать одновременно, каждый из них берет на себя часть функций распределенного управления нагрузкой всей системы. Естественно, такие конструкции являются достаточно сложными в реализации и в отладке, но при этом максимально гибкими и эффективными (рис. 8-06).

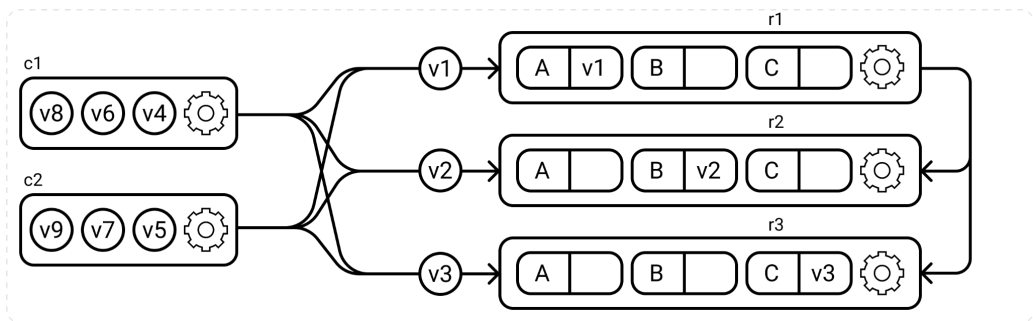


Рис. 8-06. Брокер клиентов с брокером синхронизации

8.3. Логика балансировки

Логику (алгоритм⁸⁹) балансировки задают на этапе технического проектирования целевой системы. Существует множество алгоритмов реализации планирования нагрузки — их делят в соответствии с рядом принципов.

Для алгоритмов планирования важным принципом является наличие или отсутствие обратной связи. Наличие обратной связи при планировании нагрузки позволяет алгоритму учесть состояние балансируемых узлов.

⁸⁹ Top 6 Load Balancing Algorithms, [#8-02](#) ►

8.4. Балансировка без обратной связи

Наиболее эффективным решением для выполнения балансировки без обратной связи является использование метрик приоритета в сообщениях.

При балансировке нагрузки важно получать информацию о состоянии балансируемых узлов, однако это не всегда возможно. Для таких случаев разработано множество полезных алгоритмов — например, циклическое распределение.

8.4.1. Циклическое распределение

Базовым алгоритмом планирования является алгоритм циклического обхода *round robin*⁹⁰ (RR). Циклический алгоритм подразумевает последовательное перенаправление (распределение, маршрутизацию) сообщений (задач), начиная с первого и заканчивая последним. По достижении последнего балансирующего узла из списка обход узлов начинается сначала. Обычный алгоритм *round robin* предполагает, что все узлы имеют примерно одинаковые свойства производительности.

Термин *round robin* — отсылка к XVII веку. В переводе с французского это документ с кольцевой подписью — коллективное письмо, которое подписывают по кругу, чтобы было невозможно определить инициатора. Такие письма с коллективной ответственностью, которую можно считать своего рода круговой порукой, использовали во всем мире. Подобным образом балансировщик нагрузки обеспечивает некую форму коллективной ответственности узлов: отказал один узел — нагрузку разделяют все.

8.4.2. Циклическое распределение взвешенное

Для узлов, не одинаковых по производительности, существует модификация базового циклического алгоритма RR, *weighted round*

⁹⁰ Round-robin algorithm, [#8-03](#)

robin (WRR), которая учитывает поправку на разницу технических характеристик и производительности узлов. В этом случае каждому узлу присваивается весовой коэффициент, который отражает его производительность. Узлы с разной производительностью обрабатывают пропорциональное их производительности число запросов.

8.4.3. Разновидности циклических алгоритмов

Существует множество специальных усовершенствованных RR-алгоритмов циклического планирования. Например, алгоритм *round robin with subset* (RRSS) позволяет точно описать фиксированный список узлов, участвующих в балансировке конкретной очереди сообщений. Алгоритм *round robin with arrival time* (RRAT) позволяет точно указать задержку, необходимую для обработки сообщения (задания) назначенным узлом. Алгоритм *round robin with priority* (RRWP) позволяет учесть приоритеты обработки очередей и отдельных сообщений при распределении по вычислительным узлам. Существует бесчисленное множество алгоритмов планирования распределения сообщений и задач — информацию о них читатель может найти самостоятельно.

8.4.4. Минимум соединений

Алгоритмы *round robin* не учитывают количество активных подключений к узлу. Рассмотрим случай, когда есть два корневых узла. К первому узлу подключено меньше клиентов, чем ко второму, при этом оказывается, что первый узел нагружен сильнее. Например, это может быть связано с тем, что клиенты первого узла дольше поддерживают соединение. В таких случаях для балансировки узлов с учетом количества активных клиентских соединений применяют алгоритм «минимум соединений с весовыми коэффициентами», *least connections* (LC). Каждое следующее сообщение передается узлу с наименьшим количеством активных соединений.

Как в случае с RR и WRR, существует усовершенствованный вариант алгоритма LC, который предназначен для использования в PCY, состоящих из узлов с разными техническими характеристиками и, следовательно, с разной производительностью. Это алгоритм *weighted least connections* (WLC) — при распределении нагрузки он учитывает количество активных соединений с поправкой на соответствующий узловой весовой коэффициент.

Алгоритмы LC и WLC эффективны для PCY, в которых длительность соединений клиентов с узлом может вызывать излишние непрогнозируемые нагрузки — например, нагрузки, связанные с обслуживанием накопленного кэша с нефиксированным буфером. В CPB все такого рода эффекты обычно нивелирует статическая архитектура, поэтому в них подобные алгоритмы будут неэффективны.

Существуют и другие варианты алгоритма LC, которые стоит упомянуть: например, алгоритм распределения, основанный на размещении и минимуме соединений (*locality-based least connection scheduling*, LBS). Он предназначен специально для кэширующих (прокси) узлов или для корневых узлов, которые, в отличие от остальных корневых узлов, содержат избыточные данные. Его суть заключается в следующем: наибольшее количество запросов передается корневым узлам с наименьшим количеством активных соединений. За каждым из клиентских узлов закрепляют группу клиентских сетевых адресов (*ip*). Запросы с этих сетевых адресов направляются на закрепленный (назначенный по умолчанию) корневой узел, если он не загружен полностью. В противном случае запрос будет перенаправлен на другой корневой узел, при этом он должен быть загружен менее, чем на заданное предельное значение. Список перенаправления приоритизирован.

В алгоритме «распределение, основанное на размещении и минимуме соединений с репликацией» (*locality-based least connection scheduling with replication scheduling*, LBSR) каждый сетевой адрес или группу сетевых адресов закрепляют не за отдельным корневым узлом, а за целой группой корневых узлов.

Сообщение передается наименее загруженному корневому узлу из группы. Если же все корневые узлы из закрепленной группы перегружены, будет зарезервирован новый корневой узел. Этот узел будет добавлен к группе, обслуживающей сетевой адрес, с которого было отправлено сообщение. Из этой группы будет исключен наиболее загруженный корневой узел. Такой алгоритм позволяет избежать избыточности, если это необходимо.

8.5. Балансировка с обратной связью

В ВНС абсолютные параметры нагрузки корневых узлов могут меняться весьма динамично. Для эффективной балансировки необходимо непрерывно отслеживать изменения заданных параметров и учитывать их в алгоритме балансировки (рис. 8-07).

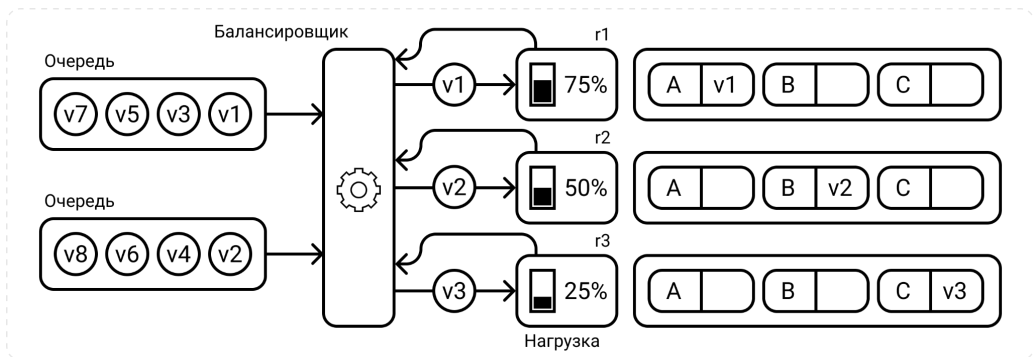


Рис. 8-07. Балансировщик с обратной связью

В таких случаях применение взвешенных алгоритмов WRR, WLC и их модификаций не имеет никакого смысла, поскольку алгоритм может получить точную информацию обо всех требуемых параметрах.

Рассмотрим пример: корневые узлы $r1$, $r2$, $r3$, $r4$ находятся в поле интерконнекта синхронизации; узел $r4$ назначают балансирующим прокси в клиентской сети; переменные $L1$, $L2$, $L3$ отражают нагрузку узлов $r1$, $r2$, $r3$ соответственно; клиенты $s1$, $s2$, $s3$ обеспечивают нагрузку (рис. 8-08).

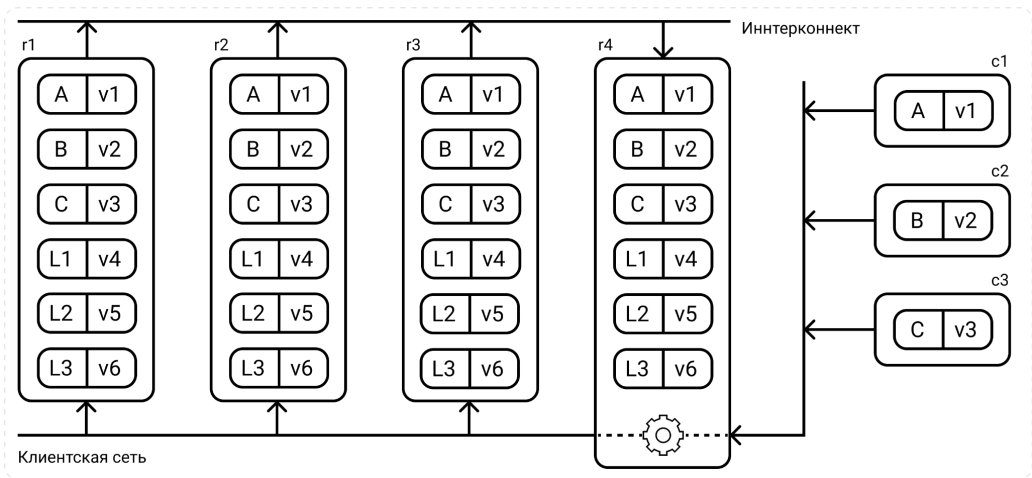


Рис. 8-08. Схема балансировки

За счет синхронизации узел *r4* получает необходимые данные о загрузке узлов *r1*, *r2*, *r3*, он должен учитывать их в применяемом алгоритме в качестве обратной связи (ОС).

8.5.1. Циклическое распределение с ОС

Циклический алгоритм RR может быть применен с учетом обратной связи (*RR with feedback*, RRFB). Обратная связь формируется на основе изменения метрик нагружаемых балансируемых узлов. В качестве обратной связи можно рассматривать изменения параметра абсолютной нагрузки узла. RRFB нагружает наименее нагруженные узлы до тех пор, пока какой-нибудь другой узел не окажется наименее нагруженным, после чего перенаправляет нагрузку на него (рис. 8-09).

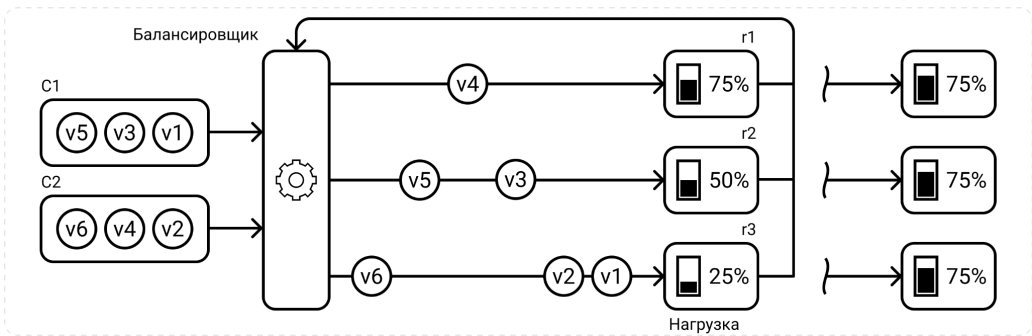


Рис. 8-09. Циклическое распределение

8.5.2. Циклическое распределение взвешенное с ОС⁹¹

Существенное повышение эффективности обычного WRR может обеспечить обратная связь от балансируемых узлов с различными техническими характеристиками и производительностью (WRRFB). В качестве весового коэффициента могут быть использованы метрики, отражающие доступные ресурсы для обработки нагрузки. Весовой коэффициент может быть рассчитан на основании композитных метрик, таких как доступная мощность, пропускная способность сети, ее загрузка и др.

8.6. Балансировка с приоритизацией

Балансировщик нагрузки может учитывать метрики приоритета. В этом случае очереди или сообщения в очередях должны быть обработаны в соответствии с заданным приоритетом в пределах заданного временного окна. Сначала должны быть обработаны более значимые данные, а потом менее значимые. Балансировщик может учитывать загрузженность узлов.

Возможны различные схемы работы с композитными метриками приоритета и обратной связи на основе данных динамики нагрузки узлов. Например, схема балансировки узлов с учетом приоритизации сообщений и уровня загрузки узлов может быть реализована с применением циклического алгоритма RR с приоритизацией и обратной связью (комбинация RRFD и RRWP) (рис. 8-10).

В данном примере для простоты будем считать, что обработка одной переменной занимает 25% нагрузки узла. Значение переменной $v5$ направляется на обработку в наименее загруженный узел $r3$ в порядке наивысшего приоритета $P4$. Загрузка узла $r3$ становится равной 50%. Рассмотренный узел $r3$ имеет аналогичную нагрузку, как узел $r2$, поэтому на $r2$ не следует осуществлять ретрансляцию для снижения потерь скорости.

⁹¹ С обратной связью

Следовательно, значение переменной v_2 направляется на обработку на узел r_3 в порядке приоритета P_3 . Загрузка узла r_3 становится равной 75%. Очевидно, что для обработки следующего значения необходимо выбрать наименее загруженный узел, например, с помощью циклического алгоритма (разд. 8.5.1). Следовательно, обработка значения v_3 с приоритетом P_2 маршрутизируется на узел r_2 . Обработка переменных v_1, v_4, v_6 осуществляется аналогичным образом.

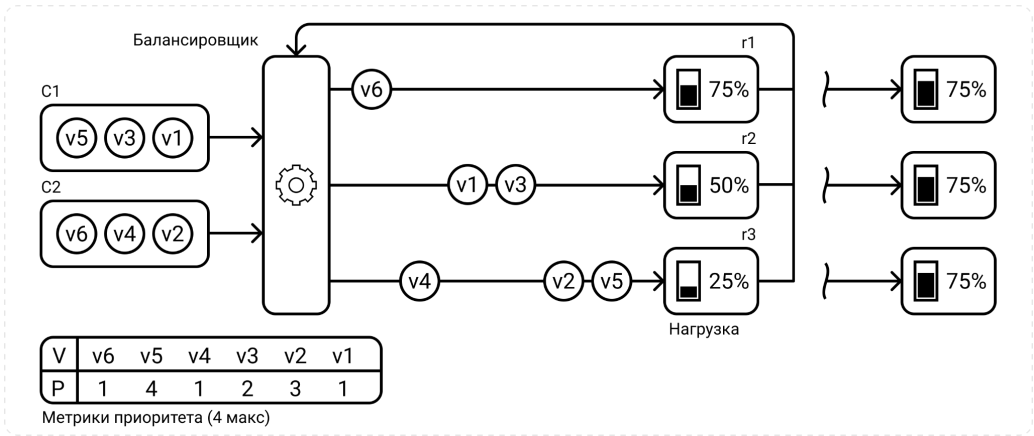


Рис. 8-10. Распределение с приоритизацией

8.7. Теория массового обслуживания

Распределенная обработка очередей и их балансировка (распределение) подкреплены мощным математическим аппаратом.

В 1906 году датский математик Агнер Эрланг разработал теорию массового обслуживания (теорию очередей⁹²). Занимаясь оптимизацией обслуживания телефонных соединений, Эрланг выявил и сформулировал закономерности, описываемые законами теории вероятности. Теория позволяла определить вероятность отказа в обслуживании абонента в случаях, когда все линии были заняты и абонент не мог совершить звонок. Благодаря техническому прогрессу появился функционал постановки

⁹² Queueing theory, #8-04

абонентов в очередь ожидания обслуживания. Была рассчитана вероятность того, что поступивший вызов не будет обслужен немедленно, а поставлен в очередь. Время ожидания в очереди не должно было превышать среднее время обслуживания (вызова). За это время должен был освободиться один из каналов, на который переключали вызов, в противном случае вызов сбрасывался. Вероятность отказа означала, что вызов из очереди мог не дожидаться свободного канала. Исходя из этого операторы планировали оснащение телефонных станций оборудованием для поддержки высокой доступности сервиса — например, позволяющего гарантировать обслуживание 99% вызовов. Теория Эрланга давала возможность оценить количество оборудования, необходимое для выполнения таких гарантий⁹³.

В 1954 году была сформулирована теорема Литтла⁹⁴, которая применима к устойчивой (стабильной) системе любой сложности. Теорема определяет среднее время отклика (латентность, время прохождения запроса через систему) как отношение среднего числа поступающих в систему запросов к скорости их поступления в систему. Например, эту теорему использовали для оценки пропускной способности трафика в автоматических телефонных станциях.

На основе теории массового обслуживания создан целый ряд систем массового обслуживания и инструментов их разработки.

В 1986 году компания Ericsson выпустила специализированный язык программирования для разработки телекоммуникационных систем массового обслуживания, который назвали в честь Эрланга. *Erlang*⁹⁵ обеспечивает возможность разработки распределенных вычислительных систем.

⁹³ Гарантии в теории Эрланга, [#8-05](#)

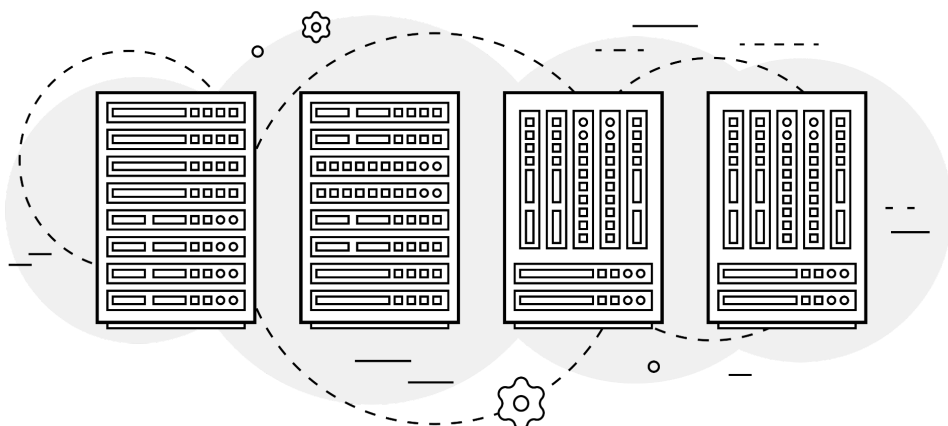
⁹⁴ Little's law, [#8-06](#)

⁹⁵ Erlang (ERicsson LANGUage), [#8-07](#)

Работая «из коробки», язык включает средства управления легковесными параллельными процессами, обменивающимися асинхронными сообщениями. Он позволяет создавать отказоустойчивые ВНС, РСУ, кластерные параллельные СРВ. За счет легковесности процессов язык может обслуживать миллионы подключений на узел, а в режиме кластера — сотни миллионов подключений. *Erlang* не стал модным и широко распространенным языком из-за того, что не обеспечивал высокую эффективность на разном железе.

В 1990-е годы, в период развития и становления отрасли, стандартом де-факто стал язык Си, который лег в основу всех известных ядер современных операционных систем, так как обеспечивал поддержку максимального аппаратного разнообразия. *Erlang* проиграл битву за железо и не лег в основу разработки ни в одной современной корпорации. Очевидно, что заложенные в этом языке концепции намного опередили свое время, поэтому *Erlang* до сих пор применяют в специализированных системах. Парадигмы языка *Erlang* сильно повлияли на развитие языка Go⁹⁶, на который следует обратить внимание при разработке ВНС, РСУ в случаях отсутствия необходимости работать с железом напрямую.

⁹⁶ Google Oberon, #8-08



Глава 9. Кластеризация

В некоторых задачах РСУ, СРВ возможным и эффективным решением является объединение корневых узлов в единый вычислительный кластер. Кластерные вычисления необходимы для выполнения самых разнообразных задач, прежде всего для ускорения вычислений путем распределения задач по узлам.

9.1. Принципы построения

Вычислительным кластером называют объединение нескольких вычислительных узлов с помощью высокоскоростных каналов связи, с точки зрения пользователя представляющее собой единый аппаратный ресурс. Кластеризация подразумевает ряд жестких ограничений, применяемых к аппаратной составляющей задействованных узлов. Все узлы должны иметь идентичную аппаратную конфигурацию. Кластер всегда предполагает наличие выделенного интерконнекта, соединяющего кластерные (корневые) узлы. РСУ можно считать кластером, пока обеспечивается кворум кластера, то есть пока РСУ содержит достаточное число активных узлов, позволяющих кластеру выполнить поставленную задачу. Выделяют несколько разновидностей кластеров на основании доступности и типов выполняемых ими задач.

9.1.1. Кластеры высокой доступности

Кластеры высокой доступности (НАС⁹⁷), или отказоустойчивые кластеры, — это РСУ с высокой избыточностью узлов, гарантирующие предоставление сервиса в случае отказа нескольких узлов. В зависимости от требуемой надежности оказания сервиса такие кластеры делят на три типа: мультиактивные, мультиактивные с резервированием и мультиактивные с модульной избыточностью (Глава 6).

В отличие от РСУ, которые, как кластер, могут работать в обоих мультиактивных режимах, в режиме с модульной избыточностью работают только кластеры высокой доступности. Такие кластеры применяют в системах высокой критичности, когда недопустимы как отказ, так и эффекты увеличения латентности отдельных узлов. В кластерах подобного типа все узлы или несколько узлов выполняют обработку одной и той же задачи. Отказ любого узла не повлияет на результат: перебалансировка нагрузки не произойдет, иных задержек в работе такого кластера не будет. В благоприятной ситуации результаты обработки данных каждого из таких узлов должны быть одинаковыми. На случай, если есть какие-либо сомнения, существуют разнообразные эффективные схемы приоритизации выбора результата. Например, алгоритм тройной избыточности (*triple modular redundancy*), который сравнивает все результаты и выбирает два из трех ($\frac{2}{3}$), три из четырех ($\frac{3}{4}$), и другие варианты.

9.1.2. Кластеры распределенной нагрузки

Выделяют разновидность кластеров балансировки нагрузки⁹⁸. Такие РСУ обладают всеми преимуществами кластера. Для решения поставленных перед кластером целевых задач обеспечивают управление нагрузкой с применением одного из известных методов (Глава 8).

⁹⁷ High-availability clusters, #9-01

⁹⁸ Load balancing clusters, #9-02

9.1.3. Вычислительные кластеры

Отдельной разновидностью кластеров являются сверхпроизводительные кластеры (НРС⁹⁹). Их отличительными особенностями являются сверхэффективная аппаратная инфраструктура, быстрый ввод-вывод, низкая латентность всех аппаратных компонентов, высокопроизводительные CPU, GPU, разнообразные специализированные сопроцессоры, высокоскоростные сетевые интерконнекты. Такая архитектура предназначена прежде всего для научных вычислений и применения в СРВ. Именно этот вид кластеров представляет собой то, что принято называть суперкомпьютерами. Он является предметом гордости разработчиков и занимает место в рейтинге мощнейших суперкомпьютеров ТОП-500¹⁰⁰. Для построения высокопроизводительных вычислительных кластеров применяют многослойную (разд. 11.1.5) распределенную конвейерную архитектуру (разд. 11.1.3).

9.1.4. Системы распределенных вычислений GRID

Решения, позволяющие использовать обычную РСУ с низкой связностью узлов с различной аппаратной конфигурацией под управлением разнообразных ОС, которые применяют для организации решения задач, присущих классическому НРС, принято называть GRID¹⁰¹-вычислениями. Такие вычисления могут быть эффективно реализованы в любых типах сетей, прежде всего в сетях типа *mesh*.

Для построения систем такого типа применяют специальное ПО, обеспечивающее реализацию программно-определяемого кластера. Такое ПО является своего рода оркестратором, но со специальными функциями эмуляции (виртуализации) кластерных вычислений. По сути, GRID представляет собой виртуальный (программно-определяемый) суперкомпьютер.

⁹⁹ High performance computing clusters, #9-03

¹⁰⁰ TOP-500, #9-04

¹⁰¹ GRID, #9-05

9.2. Кластерные вычисления

Кластерные вычисления (задачи) очень разнообразны. Виды кластерных вычислений можно ранжировать по длительности задач, синхронности их выполнения и нагрузке узлов, их созависимости с другими задачами. Если созависимость кластерных задач высокая, разумнее организовать их как параллельно выполняемые задачи.

9.2.1. Параллельные вычисления

Параллельные вычисления¹⁰² — это способ выполнения вычислений, при котором они организованы как набор взаимодействующих вычислительных процессов, работающих параллельно (одновременно). Такие вычисления эффективнее всего организовывать на базе кластерных инфраструктур.

Параллельные вычисления тесно связаны с понятием параллельных алгоритмов, которые могут быть реализованы по частям на множестве различных вычислительных узлов с последующим объединением полученных результатов и получением итогового результата вычислений. Это свойство параллельных алгоритмов обеспечивает возможность существенного сокращения времени вычислений. С помощью параллельных алгоритмов могут быть решены задачи распознавания текущего состояния системы и ее компонентов, задачи предиктивной аналитики и предиктивного управления.

Выделяют два типа параллельных алгоритмов с точки зрения синхронности вычислений: синхронные и асинхронные. Синхронный параллельный алгоритм требует дождаться результатов вычислений от узла, вычисления на котором идут с самой высокой задержкой, что приводит к простоям остальных узлов. В качестве простейшего примера такого алгоритма можно привести следующее: узлы *r1-r6* непрерывно синхронизируются в поле интерконнекта (рис. 9-01).

¹⁰² Parallel computing, #9-06

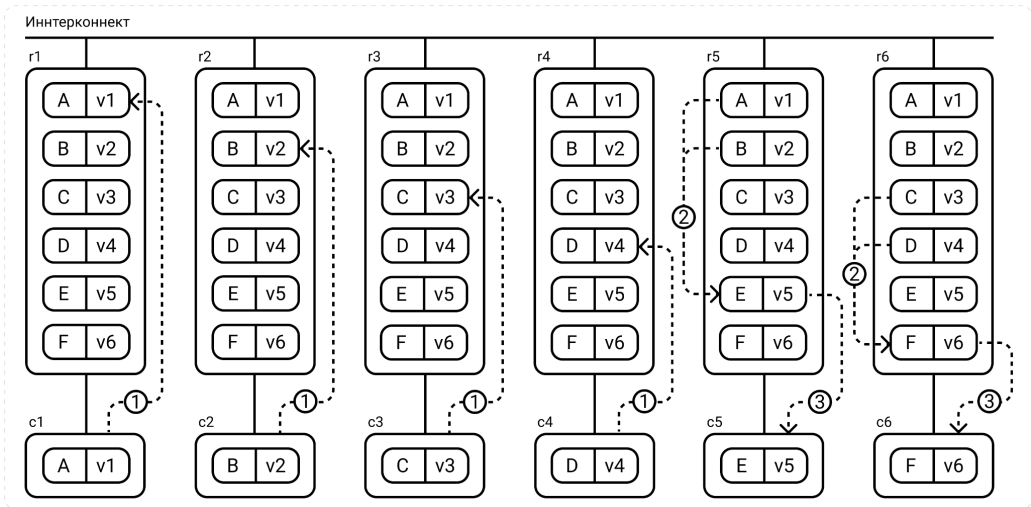


Рис. 9-01. Параллельный алгоритм

Далее выполняют три шага – сначала узлы $r1-r4$ обрабатывают входящие изменения значений $v1-v4$ переменных $A-D$ от клиентов $c1-c4$, затем узлы $r5, r6$ параллельно осуществляют вычисления, записывая значения результатов $v5, v6$ в переменные E, F . После этого клиенты $c5, c6$ получают необходимые им для дальнейшего использования значения.

Асинхронный параллельный алгоритм не требует ожидания какого-либо из участвующих в вычислениях узлов, что обеспечивает максимальную эффективность загрузки узлов.

Например, такой алгоритм больше всего подходит для задач обработки данных в РСУ. Если какой-либо физический параметр любой физической системы, связанный с изменением состояния технологического процесса, меняется, то остальные связанные с ним физические параметры меняются с аналогичной динамикой. Предположим, что все эти физические параметры задействованы в параллельном асинхронном алгоритме: если по каким-либо причинам один из этих физических параметров будет слегка устаревшим относительно других параметров, на итоговое качество вычислений это не повлияет.

В качестве примера можно привести такую ситуацию. Насос прокачивает теплоноситель через теплообменник со скоростью Q , при этом температура теплообменника T зависит от Q . PCY обеспечивает эффективный механизм обработки данных. По какой-то причине в определенный момент времени переменная $T2$ не успевает синхронизироваться на узле $r1$, при этом она задействована в асинхронном параллельном алгоритме. Если в ходе технологического процесса данные поступают с высокой частотой, то на итоговое качество вычисления алгоритма такие отклонения не повлияют (рис. 9-02).

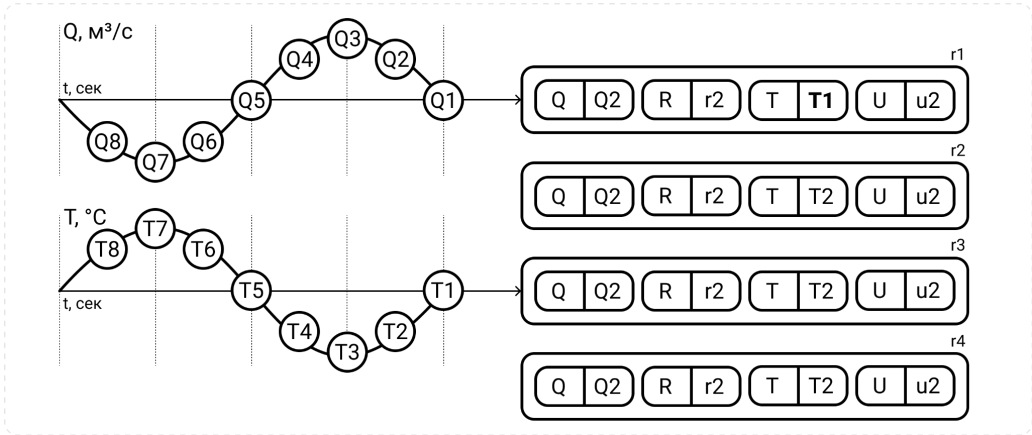


Рис. 9-02. Влияние задержек

Таким образом, алгоритмы параллельных вычислений можно разделить по длительности такта узловых вычислений, которая измеряется временем отклика. Выделяют низкочастотные и высокочастотные параллельные алгоритмы. Алгоритм считают хорошо распараллеленным, если все узлы вычислительного такта имеют одинаковую латентность (рис. 9-03).

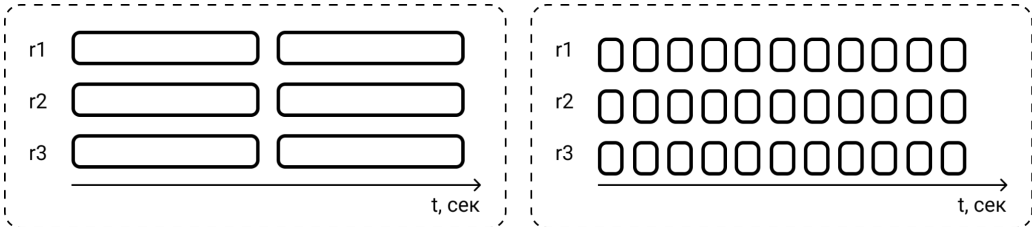


Рис. 9-03. Длительность вычислительных тактов

Если латентности существенно различаются, параллельный алгоритм неэффективен.

Алгоритмы параллельных вычислений делят по типам взаимодействия. Они могут взаимодействовать посредством сообщений или через разделяемую память. Обычно параллельные алгоритмы, взаимодействующие через обмен сообщениями, работают в синхронном режиме, так как вычислительный такт такого алгоритма зависит от факта получения сообщения. А вот параллельные алгоритмы, взаимодействующие через разделяемую память, могут быть как синхронными, так и асинхронными. Вычислительные такты таких параллельных алгоритмов могут синхронизироваться. Синхронизация тактов асинхронных параллельных алгоритмов, взаимодействующих в разделяемой памяти, может осуществляться при использовании соответствующего функционала планировщика нагрузки.

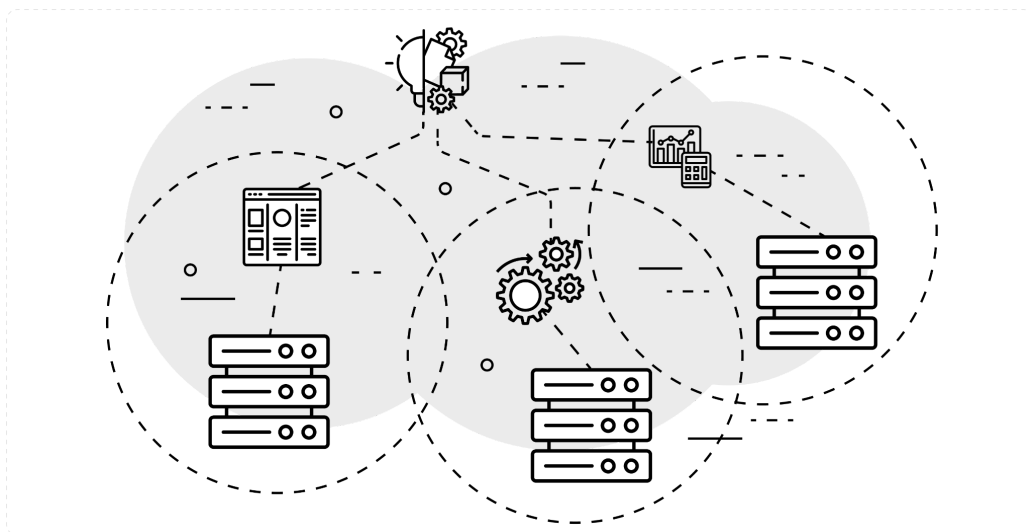
Существует множество стандартных решений для реализации параллелизма процессов. Самыми известными из них являются основанные на передаче сообщений стандарты MPI¹⁰³ и OpenMP¹⁰⁴, которые имеют массу разнообразных и специальных вариантов реализации. Предложенный в книге вариант реализации параллелизма несколько отличается от этих стандартов тем, что для синхронизации используется РРП (разд. 4.2). Принципиально разделены задачи синхронизации данных между узлами и использования данных в алгоритмах самих параллельных вычислений. MPI, OpenMP чаще предполагают создание систем со строгой консистентностью. Вариант на основе РРП позволяет собрать вычислитель в режиме слабой консистентности.

¹⁰³ Message Passing Interface, #9-07

¹⁰⁴ Open Multi-Processing, #9-08

9.2.2. Катастрофоустойчивые кластеры

В сложно организованных РСУ параллельные алгоритмы могут взаимодействовать в гибридном режиме, когда часть взаимодействий осуществляется через распределенную разделяемую память, а другая часть осуществляется путем обмена сообщениями, содержащими данные и параметры организации распределения вычислительных тактов. Фактически может быть реализована группа устойчивых к разделению кластеров с зарезервированными каналами связи для быстрого восстановления соединения (разд. [12.2.1](#)).



Глава 10. Масштабирование

Серьезная РСУ представляет собой большой и дорогостоящий инфраструктурный объект, который проектируют на десятилетия. Не исключено, что со временем система будет модернизирована, появятся новые дополнительные устройства, контроллеры и прочее оборудование. Это значит, что нагрузка на эксплуатируемую систему будет выше.

Необходимо заранее подготовиться к возможному масштабированию системы. Обычно масштабирование бывает необходимо в следующих случаях:

- увеличивается общий объем данных;
- увеличивается количество источников и разнообразие данных;
- усложняется логика обработки данных.

Во всех случаях масштабирование осуществляют путем добавления в систему новых узлов.

Может случиться так, что каждый акт масштабирования, нацеленный на пропорциональное увеличение нагрузки, потребует непропорционально больших ресурсов. Подобная ситуация является прямым основанием для критики целевой архитектуры и признания ее неэффективной. Хуже всего, если это обнаружится на этапе первого масштабирования.

10.1. Проблемы масштабирования

Проблемы масштабирования обычно проявляются из-за неэффективной проработки целевой архитектуры.

Выделяют аппаратное, а именно вертикальное и горизонтальное масштабирование. Вертикальное масштабирование подразумевает увеличение мощности и прочих ресурсов единичных узлов, а горизонтальное масштабирование предполагает увеличение их числа.

Вертикальное масштабирование, например, в виде увеличения оперативной памяти без добавления дополнительного процессорного модуля, может привести к тому, что ПО не справится с обработкой большого объема памяти, в связи с чем может увеличиться латентность узла.

При горизонтальном масштабировании ПО может потерять эффективность. Если ПО рассчитано на некое предельное число распределенных узлов, то после превышения предельного числа узлов ПО может тратить больше времени на обеспечение взаимодействия между узлами в ущерб обслуживанию клиентов. Для масштабирования обслуживания клиентов потребуются непропорционально большее число узлов. Такие ситуации становятся причиной нелинейности масштабирования¹⁰⁵.

¹⁰⁵ Vertical vs horizontal scaling, #10-01►

10.2. Линейное масштабирование

Правильно спроектированная система должна обеспечивать линейное масштабирование. Перед вводом целевой системы в эксплуатацию должно быть известно, как клиентские и внутрисистемные взаимодействия (операции) нагружают целевую систему. Для этого существуют соответствующие метрики (Глава 7), которые измеряют в ходе нагрузочного тестирования¹⁰⁶.

Предположим, что система спроектирована правильно — так, что она канонически поддерживает линейно-горизонтальное масштабирование. В некий момент времени начинается деградация (разд. 7.3) — почему-то для выполнения расчетного, спрогнозированного числа операций не хватает ресурсов. Если система спроектирована грамотно, клиенты будут частично перераспределены по менее нагруженным, более свободным узлам. Однако возникает вопрос: что происходит? В таких случаях необходимо следить за тем, как развивается функционал системы.

10.3. Функциональное масштабирование

Все системы со временем развиваются. Необходимо учитывать, что при добавлении функционала система станет более требовательной к ресурсам. Бывают и более сложные конструкции, которые позволяют пользователям системы самим генерировать дополнительную незапланированную нагрузку. Например, пользователи могут описывать некоторые внутрисистемные сценарии, которые вносят существенный вклад в рост производительности. При проектировании целевых систем необходимо учитывать эту возможность и закладывать ее в архитектуру, не забывая включить соответствующие метрики.

При функциональном масштабировании необходимо учитывать возможность возникновения функциональной регрессии. Например, регрессия может возникать, когда клиенты

¹⁰⁶ Нагрузочное тестирование, #10-02

или внутрисистемные сервисы продолжают использовать устаревшие прикладные программные интерфейсы (ППИ), что вызывает ошибки и проблемы, для решения которых требуются дополнительные ресурсы. Еще одним характерным примером функционального масштабирования является возможность задать логические алгоритмы на внутреннем языке системы, что может вызывать повышенную потребность в аппаратных ресурсах.

Очевидно, что от таких событий спасает регрессионное тестирование¹⁰⁷, которое является самым сложным среди типов тестирования систем, так как разработка и тестирование происходят на стенде, а данные пользователей, их сценарии с устаревшими ППИ находятся на боевой системе.

Например, для отладки ПО КИИ, такой как АСУ ТП АЭС, требуются модели (цифровые двойники) всех энергоблоков, на которых функционирует целевая система. А их получение — это очень сложная организационная задача, потому что всегда нужно делать выбор — обеспечить техническую поддержку нескольких версий и сложных конфигураций или распространить единую версию на все объекты. Например, в случае, если это SCADA¹⁰⁸ общего назначения, пользователями которой являются десятки тысяч клиентов, функциональное регрессионно-стойкое масштабирование является не менее сложной задачей. Упростить задачу можно, спроектировав систему так, чтобы ее можно было разбить на минимальные взаимодействующие друг с другом части — например, микросервисы.

10.4. Гипермасштабирование

Эволюция вычислительных систем не останавливается. Современные облачные технологии достигли высокого уровня зрелости. Драйверами развития облачных технологий, включающих в себя

¹⁰⁷ Регрессионное тестирование, [#10-03](#)

¹⁰⁸ Supervisory Control And Data Acquisition, [#10-04](#)

технологии граничных и туманных вычислений, являются интернет вещей (IoT), развитие новых поколений связи 5G, 6G.

Вызовы современного мира лежат в плоскости решения задач сопряжения многочисленных взаимосвязанных систем и гармонизации их влияния друг на друга. Такой процесс называют гиперавтоматизацией. В России в кругах автоматизаторов нередко можно услышать термин «цифровой госплан». Для решения задач гиперавтоматизации требуются соответствующие технологии, которые, в зависимости от потребностей, будут осуществлять эффективное масштабирование программных и аппаратных ресурсов автоматически, то есть без вмешательства человека.

Таковыми являются технологии гипермасштабирования (гиперскейлинг). Как правило, гипермасштабирование применяют в специально созданных под эту задачу ЦОД. Такие конструкции принято называть гиперскейлерами. Отдельные подходы к гипермасштабированию могут быть применены и за пределами ЦОД, на отдельных узлах, составляющих разнообразные РСУ.

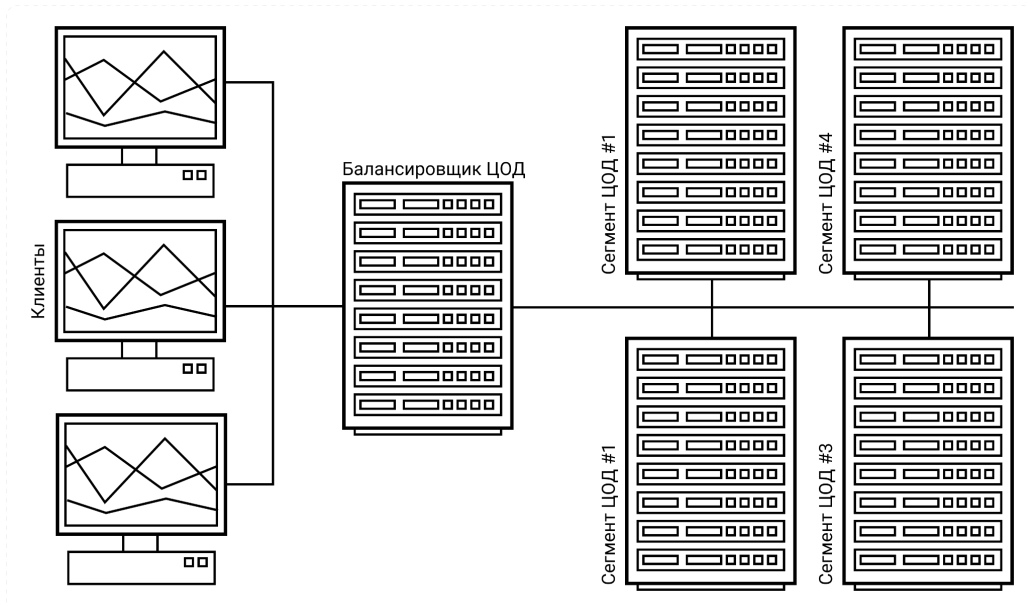


Рис. 10-01. Гиперскейлер

Гиперскейлер оперирует двумя сущностями — данными и сервисами, участвующими в их обработке. Он обеспечивает инфраструктуру с поддержкой бесшовного масштабирования и низкими задержками для взаимодействия сервисов друг с другом (рис. 10-01).

В отличие от кластеров высокой доступности (разд. 9.1.1), в которых каждый узел является участником единого вычислительного процесса, гиперскейлер управляет множеством сервисов, каждый из которых может решать свою задачу. Отличие гиперскейлера от кластера высокой доступности заключается в том, что он обеспечивает взаимодействие не всех сервисов друг с другом, а лишь некоторых с некоторыми. Таким образом, гиперскейлер обеспечивает сервисную ячеистую топологию взаимодействия сервисов друг с другом (*service mesh*). Гиперскейлеры преимущественно обслуживают сервисы и микросервисы, находящиеся под контейнерной виртуализацией, и могут обеспечивать виртуализацию каналов связи между узлами с помощью SDN. Средства управления такими сервисами, оркестраторы в гиперскейлере, называют сервисными фабриками¹⁰⁹.

Данные, привязанные к отдельным сервисам гиперскейлера называют *data mesh*¹¹⁰. Зачастую необходимо, чтобы данные взаимодействующих сервисов в гиперскейлере были доступны в виде единообразного прикладного программного интерфейса, протокола или формата данных. В такой ситуации принято говорить, что гиперскейлер представляет собой озеро данных¹¹¹. Средства управления данными гиперскейлера принято называть фабрикой данных¹¹².

Если *data mesh* — это архитектура, при которой сервисы взаимодействуют со своими источниками данных (каждый со своим), то архитектура *data lake*, является архитектурой более

¹⁰⁹ Service fabric

¹¹⁰ Data mesh, [#10-05](#)

¹¹¹ Data lake, [#10-06](#)

¹¹² Data fabric, [#10-07](#)

высокого уровня, она включает в себя дополнительный слой виртуализации. Такой слой виртуализации обеспечивает инфраструктуру хранения метаданных (разд. 3.1) и единый ППИ доступа к данным. Подобную инфраструктуру называют хабом данных¹¹³. Для создания высокоэффективных хабов данных применяют специальные СХД, обеспечивающие эффективные механизмы работы с метаданными. Для того, чтобы разобраться, какие данные доступны в хабе данных, применяют архитектуру каталогизации данных¹¹⁴, которая обеспечивает возможность классификации данных и навигации по ним. Разнообразные сервисы аналитики и визуализации структур озер данных используют именно такую архитектуру.

10.5. Жизненный цикл целевой системы

Все перечисленные выше вопросы чрезвычайно важны — от них зависят общее число возникающих проблем и стоимость их устранения. Если что-либо не учтено, не запланировано, именно оно впоследствии станет источником проблем. Очевидно, что запланировать абсолютно все не представляется возможным, не следует забывать об информационной безопасности: важно иметь возможность своевременно обновлять прошивки компонентов оборудования при выявлении в них соответствующих уязвимостей (Глава 13).

Важно учитывать, что отказоустойчивость необходимо планировать на всех этапах жизненного цикла целевой системы, включая этапы проектирования и разработки. На этапах эксплуатации это находится в зоне ответственности службы эксплуатации и технической поддержки.

Существует множество методик оценки эффективности работы ИТ системы и отдельных ее компонентов на всех этапах ее

¹¹³ Data hub, #10-08

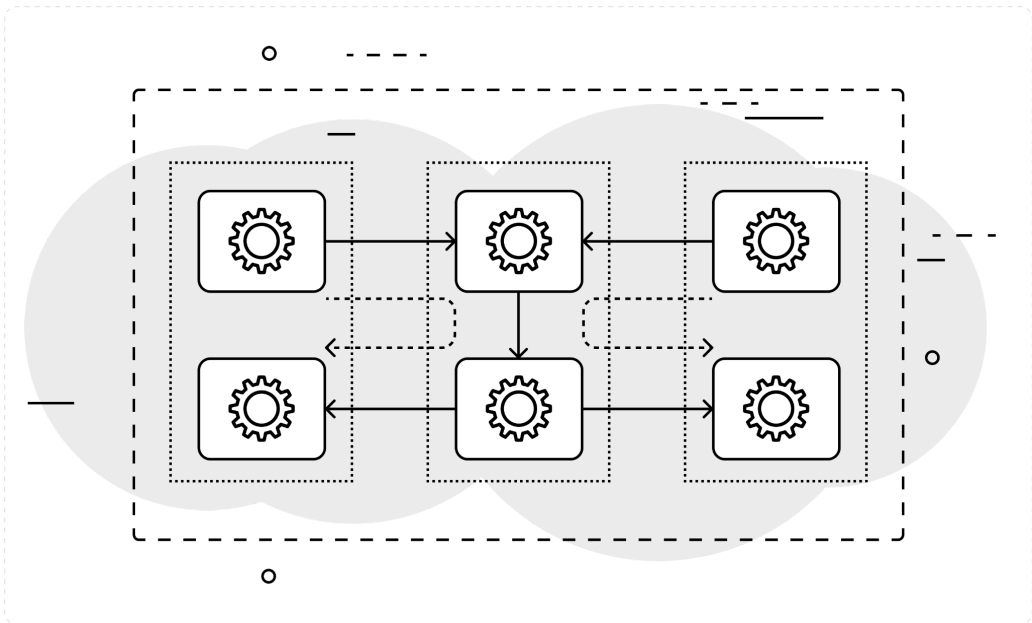
¹¹⁴ Data catalog, #10-09

жизненного цикла. Например, методика ITIL в разделе описания процесса управления доступностью приводит следующие метрики:

- доступность (*availability*);
- производительность (*performance*);
- надежность (*reliability*);
- сопровождаемость (*maintainability*);
- обслуживаемость (*serviceability*);
- безопасность (*security*).

SLA регламентирует отношения между системой и внешним потребителем. Существуют подходы к оценке внутреннего межкомпонентного взаимодействия системы, которые позволяют установить общую ответственность всех участников процессов разработки и эксплуатации за доступность сервиса. Например, методика SRE¹¹⁵ формализует подход к формированию таких метрик и оценке их изменений на всех этапах жизненного цикла. Обычно требования к качеству сервиса, установленные внутренним SRE-стандартом, выше указанных в SLA. При проектировании и разработке целевой системы необходимо заранее учитывать необходимость включения соответствующих диагностических инструментов в ее состав.

¹¹⁵ System Reliability Engineering, #10-10



Глава 11. Архитектурные шаблоны

От архитектуры разрабатываемой платформы и реализации задач целевой системы зависит очень многое: возможности эффективной эксплуатации, диагностики, обслуживания, адаптации к новым условиям, модернизации, тестирования.

Важно определить задачи системы и приоритизировать их, определить, что важнее — надежность или скорость обработки, целостность данных или их доступность, как будет осуществляться взаимодействие с клиентами, что собой представляют клиенты системы. Например, клиентами IoT платформы будет большое число небольших устройств, каждое из которых обеспечивает достаточно скромную нагрузку на систему. В качестве противоположного примера можно привести СХД: чем выше ее уровень, тем, как правило, серьезнее клиенты. В роли клиентов выступает ограниченное число серверов (например, серверы ВНС).

Для решения конкретных классов задач в мировой практике накопилась масса отработанных, проверенных подходов к созданию архитектуры РСУ. Такие подходы принято называть архитектурными паттернами¹¹⁶ или шаблонами. Использование подобных шаблонов позволяет создать целевую архитектуру системы с прогнозируемыми свойствами.

Важно понимать, что архитектурные шаблоны — это прежде всего практика, зачастую чужая. Простые архитектурные шаблоны могут быть сгенерированы средствами интегрированной среды разработки. Шаблоны посложнее могут быть представлены в виде отдельных готовых библиотек. Наиболее сложные шаблоны могут иметь вид фреймворков или даже готовых решений. Но не стоит забывать, что каждый архитектор, разработчик отвечает за себя сам, и использование чужих решений может появиться непредсказуемо. Наиболее часто возникающие проблемы связаны с линейным масштабированием (разд. [10.2](#)).

Искусство программной архитектуры заключается в умении архитектора сочетать разнообразные архитектурные практики для получения целевого решения доступными ресурсами. Не менее важным является то, что на всех этапах жизненного цикла система должна вести себя соответственно заложенной архитектуре — без существенных отклонений.

Важно отметить, что можно разработать архитектуру, реализация которой будет невозможна или невыгодна. Например, может не быть доступных разработчиков, способных реализовать заданную архитектуру, или разработчики просто не смогут договориться между собой. При разработке архитектуры важно учитывать командный фактор.

Можно привести массу отрицательных примеров, когда группа архитекторов намеренно проектировала сложную, ресурсоемкую в реализации архитектуру, чтобы сохранить старую архитектуру, а, следовательно, команды и финансовые потоки.

¹¹⁶ Software architectural pattern, [#11-01](#)

В таких ситуациях владельцы продуктов через некоторое время обнаруживали, что они отстали от рынка и не способны конкурировать с новыми решениями. Как правило, подобные вещи происходят со зрелыми продуктами, которым не один десяток лет. Например, так было с реляционными СУБД. Это происходит со многими продуктами, которые все мы видим каждый день и сейчас.

Для того, чтобы не скатиться в такую ситуацию, рекомендуется разрабатывать гибкую архитектуру. Гибкость архитектуры позволяет многое: замену слоя сопряжения отдельных компонентов и сервисов, их резервирование и балансировку. Важно отметить, что гибкая архитектура нужна далеко не всегда, как, например, в АСУ спутника или марсохода, выпускаемых в единственном экземпляре.

Назначение архитектурных шаблонов весьма разнообразно. Например, структура хранения данных и доступа к ним, иерархическая или графовая, это готовый архитектурный шаблон. Микросервисная архитектура (разд. [11.2.4](#)) представляет собой типичный архитектурный шаблон распределения и управления сервисами на разных узлах системы.

Важно отметить, что шаблоны не существуют сами по себе. Шаблоны или шаблонное поведение наблюдаются относительно конкретных обрабатываемых данных. С разных точек зрения одна и та же архитектура может быть описана как отличающиеся наборы шаблонов.

С точки зрения управления памятью и структурирования данных, шаблоны делят на три типа:

- порождающие;
- структурные;
- поведенческие.

Порождающие шаблоны предназначены для создания экземпляров объектов или групп связанных объектов.

Такие шаблоны решают задачу управления памятью. Композиционные (структурные) шаблоны предназначены для управления структурированием объектов и определения связей между структурами объектами. Поведенческие шаблоны управляют правилами взаимодействия между объектами. Такие шаблоны задают правила взаимодействия объектов, то есть то, как объекты будут обмениваться данными.

С точки зрения области применения, следует выделить следующие типы шаблонов:

- базовые архитектурные;
- потоковой и конвейерной обработки;
- распределенных систем;
- параллельной обработки;
- выполнения задач и команд.

Рассмотрение таких шаблонов необходимо начать с базовых.

11.1. Базовые архитектурные шаблоны

Базовые архитектурные шаблоны представляют собой общеизвестные практики¹¹⁷, которые успешно применяют в РСУ в особенности для решения задач построения гибких масштабируемых архитектур.

В предыдущих разделах, по сути, уже были описаны некоторые распространенные архитектурные шаблоны:

- раздел 4.2, шаблон «Разделяемая база данных»¹¹⁸;
- раздел 4.2.1, шаблон «Общие данные»;
- раздел 4.2.1, шаблон «Единая точка входа»;
- раздел 4.3, шаблон «Избыточность»;
- раздел 4.4, шаблон «Шардинг»;
- раздел 5.4.1, шаблон «Очередь сообщений»;

¹¹⁷ Top 5 most used architecture patterns, [#11-02](#) ►

¹¹⁸ Shared database, [#11-03](#)

- раздел 5.4.2, шаблон «Брокер очередей»;
- раздел 5.3, шаблон «Выбор лидера»¹¹⁹;
- раздел 5.4.4, шаблон «Поточная очередь»;
- раздел 6.1, шаблон «Ведущий-ведомый»¹²⁰;
- раздел 6.2, шаблон «Кэш на стороне»¹²¹;
- раздел 8.2.2, 8.2.4, типичный повод применить шаблон «Амбассадор» как частный случай шаблона «Прицеп»¹²²;
- раздел 10.4, упоминаются шаблоны «Фабрика данных»¹²³, «Фабрика сервисов», «Каталог данных».

Многие архитектурные шаблоны представляют собой сочетание нескольких более простых шаблонов. Ниже будут рассмотрены те шаблоны, которые представляют интерес с точки зрения проектирования РСУ.

11.1.1. Каналы и фильтры

Базовым шаблоном для распределенных систем является шаблон «Каналы и фильтры»¹²⁴, задающий конвейерную (*pipeline*) архитектуру. Основными элементами этого шаблона являются стандартная очередь сообщений (разд. 5.4.1) и фильтр. Под фильтром подразумевают некий алгоритмический блок, осуществляющий обработку входящей очереди сообщений. Фильтр может быть источником новой очереди сообщений, являющихся результатом процесса обработки.

Минимальный элемент данного шаблона называют конвейером очереди (сообщений). Он включает в себя одну очередь и один фильтр (рис. 11-01). Более сложный элемент шаблона называют брокером очереди (сообщений). Применительно к этому шаблону брокер обеспечивает преобразование сообщений согласно заданному в фильтре алгоритму (рис. 11-02).

¹¹⁹ Leader election, #11-04

¹²⁰ Leader-follower, master-slave, active-passive, #11-05

¹²¹ Cache-aside, #11-06

¹²² Sidecar, #11-07

¹²³ Data fabric, #11-08

¹²⁴ Pipes & filters, pipeline #11-09, #11-10

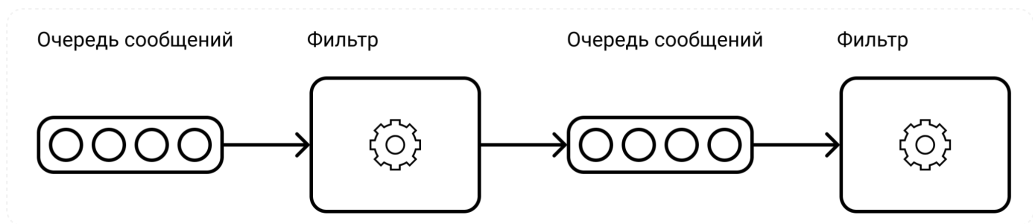


Рис. 11-01. Архитектурный шаблон «Каналы и фильтры»

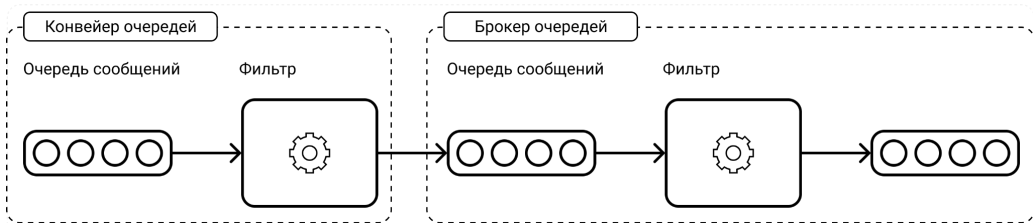


Рис. 11-02. Конвейер и брокер

Конвейерную архитектуру задает шаблон «Каналы и фильтры», который может включать в себя шаблоны «Конвейер» и «Брокер очередей».

Важно отметить, что обычно в архитектурных схемах очередь сообщений не указывают, но всегда подразумевают, что она есть и обозначают стрелками (рис. 11-03).

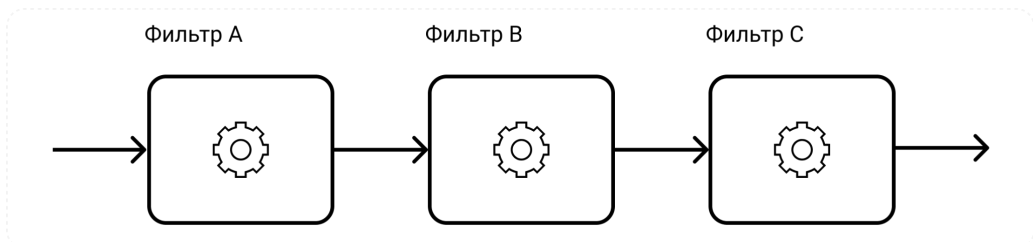


Рис. 11-03. Конвейерная архитектура

11.1.2. Многоуровневая архитектура, Клиент–сервер

Архитектурный шаблон «Клиент-сервер»¹²⁵ известен всем. Объяснять, как он устроен, не нужно. Важно отметить роль этого шаблона в построении распределенных систем.

¹²⁵ Client-server, #11-11

Прежде всего «Клиент-сервер» является разновидностью шаблона более высокого ранга «Многоуровневая архитектура»¹²⁶. В нем выделяют одноуровневую (локальные приложения), двухуровневую (клиент-сервер, сервер-сервер), трехуровневую (рис. 11-04) и N-уровневую архитектуру.

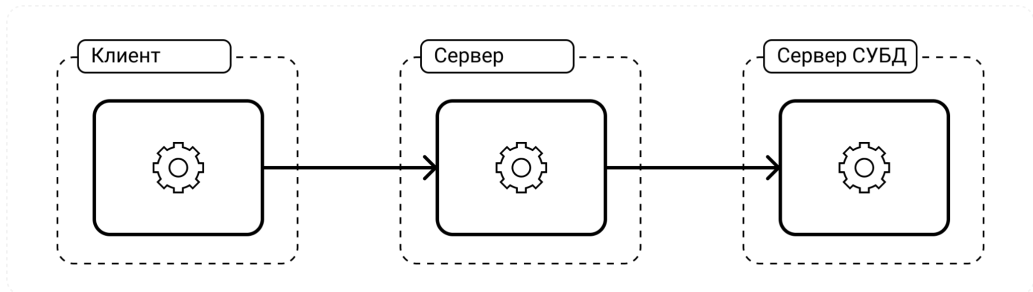


Рис. 11-04. Трехуровневая архитектура

«Клиент-сервер» необходимо рассматривать как более сложный вариант *pipeline* архитектуры. Как минимум, клиент с сервером обмениваются данными в одном направлении, но чаще обмен происходит в обоих направлениях. При этом, как правило, сервер отвечает на запрос клиента, образуя классический архитектурный конвейер (рис. 11-05).

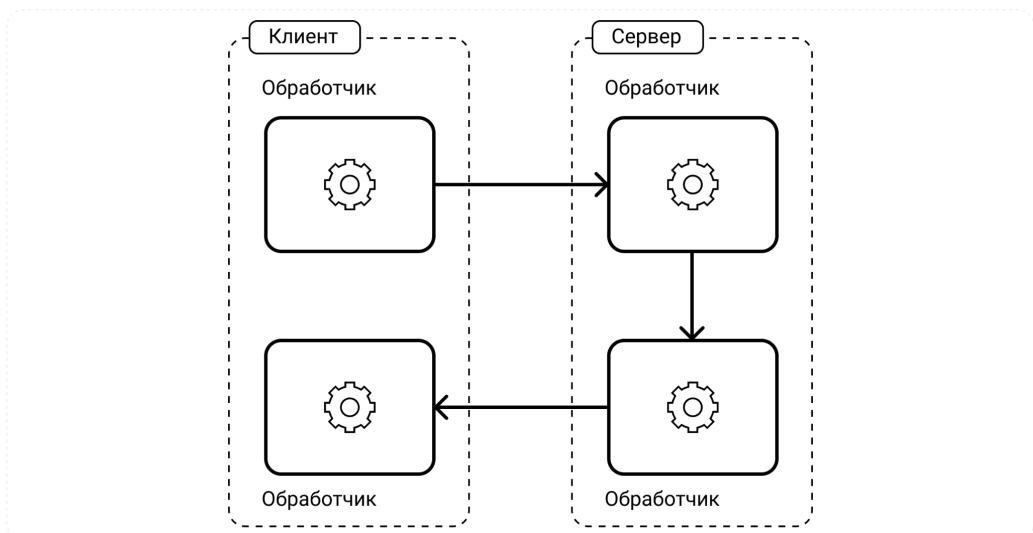


Рис. 11-05. Клиент-сервер, конвейер

¹²⁶ Tiered architecture, [#11-12](#)

Данный архитектурный шаблон подразумевает наличие множества клиентов у одного сервера. Если сервер не многопоточный, то в интервал времени обработки взаимодействия будет действовать один *pipeline*. Многопоточная реализация позволит обеспечить одновременно необходимое количество обработчиков (рис. 11-06).

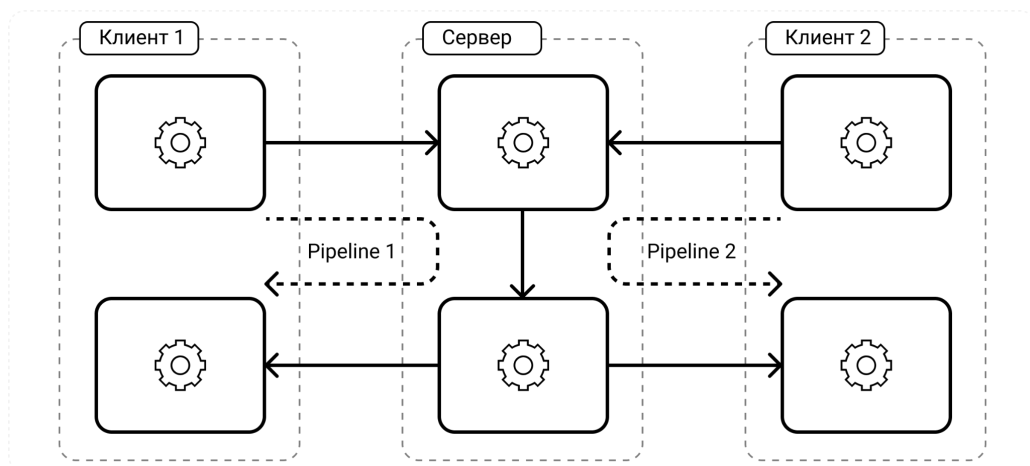


Рис. 11-06. Клиент-сервер, *pipeline* приоритет

Важным моментом является место клиент-серверного шаблона в РСУ. В любой схеме взаимодействия сервисов РСУ можно найти множество сегментов, в которых сервисы взаимодействуют в формате «клиент-сервер». В связи с этим данный архитектурный шаблон можно считать составной частью любой РСУ (рис. 11-07).

«Клиент-сервер» является основой для создания самых разнообразных топологий узлового взаимодействия, таких как сети типа «кольцо», «звезда» ([Глава 5](#)).

Существует симметричная модификация шаблона «Клиент-сервер», известная как «Одноранговый шаблон» или «Одноранговая сеть». В ней каждый узел одновременно является и клиентом, и сервером по отношению к другому узлу. Каждый узел имеет право инициировать запрос к другому узлу и получить ответ.

Такую конструкцию обозначают как $P2P^{127}$. Этот шаблон составляет основу для создания слабосвязанных топологий, например, таких, как ячеистая топология (*mesh*) и хаотичная самоорганизующаяся топология.

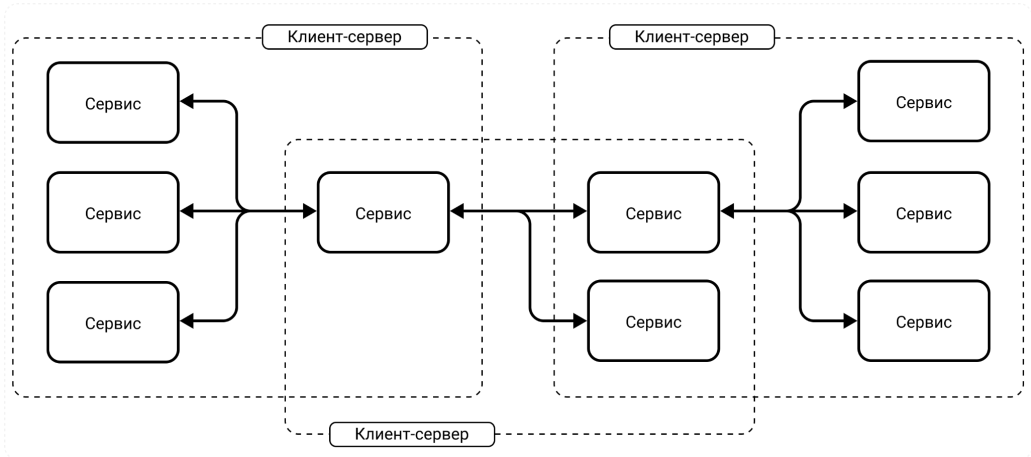


Рис. 11-07. Место клиент-серверной архитектуры в РСУ

11.1.3. Распределенная конвейерная архитектура

Существует способ повысить эффективность обычной конвейерной архитектуры, обеспечив непрерывность использования (задействования) участвующих распределенных в системе узлов.

Рассмотрим схему, при которой узлы $A1$, $A2$, $A3$ взаимодействуют в однопоточном режиме (рис. 11-08).

Узел $A1$ отправляет сообщение узлу $A2$, от обработчика $A1q$ к $A2q$. В этот момент узел $A3$ отправляет сообщение узлу $A2$. Затем возможны варианты. При плохой архитектуре узел $A3$ будет ждать до тех пор, пока узлы $A1$ и $A2$ не завершат цикл обмена сообщениями, пока узел $A2$ не отправит ответное сообщение узлу $A1$, от $A2a$ к $A1a$. Таким образом, сначала узел $A2$ обслужит *pipeline* 1 ($A1q \rightarrow A2q \rightarrow A2a \rightarrow A1a$), и только потом *pipeline* 2 ($A3q \rightarrow A2q \rightarrow A2a \rightarrow A3a$).

¹²⁷ Peer to peer — равный к равному, #11-13

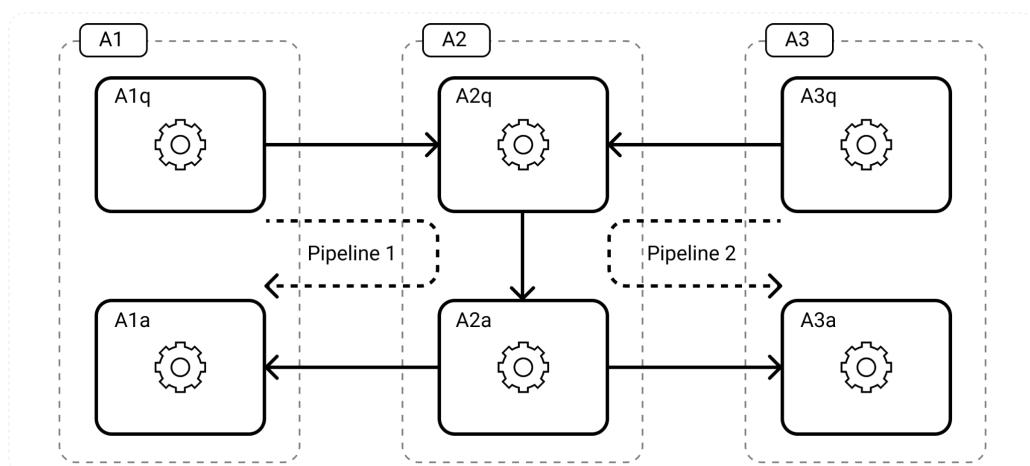


Рис. 11-08. Взаимодействие узлов

Рассмотренную схему можно существенно ускорить, позволив $A2q$ сразу после обработки сообщения от $A1q$, не дожидаясь завершения обработки всей цепочки сообщений от узла $A1$, приступить к обработке сообщений от узла $A3$. Таким образом, узел $A2$ в некий момент времени будет обрабатывать сообщения сразу с двух узлов, $A1$ и $A3$, сократив общее время обработки. В таком случае необходимо, чтобы сообщения содержали метаданные, а именно метку принадлежности источнику и потребителю. Важно отметить, что обработка метаданных займет дополнительное время.

Такая архитектура известна как суперконвейерная, а шаблон называют «Суперконвейер»¹²⁸. Подобная архитектура позволяет существенно повысить эффективность обработки данных в распределенной системе.

Этот архитектурный шаблон состоит из нескольких экземпляров конвейеров, «обернутых» клиент-серверными шаблонами. В данном случае в качестве клиента конвейера рассматривают восемь узлов ($N1, N2, \dots, N8$). В цепочке конвейера задают пять операций. В каждый момент времени ($t1, t2, \dots, t8$) начинается обработка соответствующей операции с данными соответствующего

¹²⁸ Super pipeline, hyper pipeline, #11-14

узла конвейера. Таким образом, в определенный временной интервал одновременно для пяти узлов выполняется конвейерная операция, при которой суперконвейер полностью загружен (рис. 11-09).

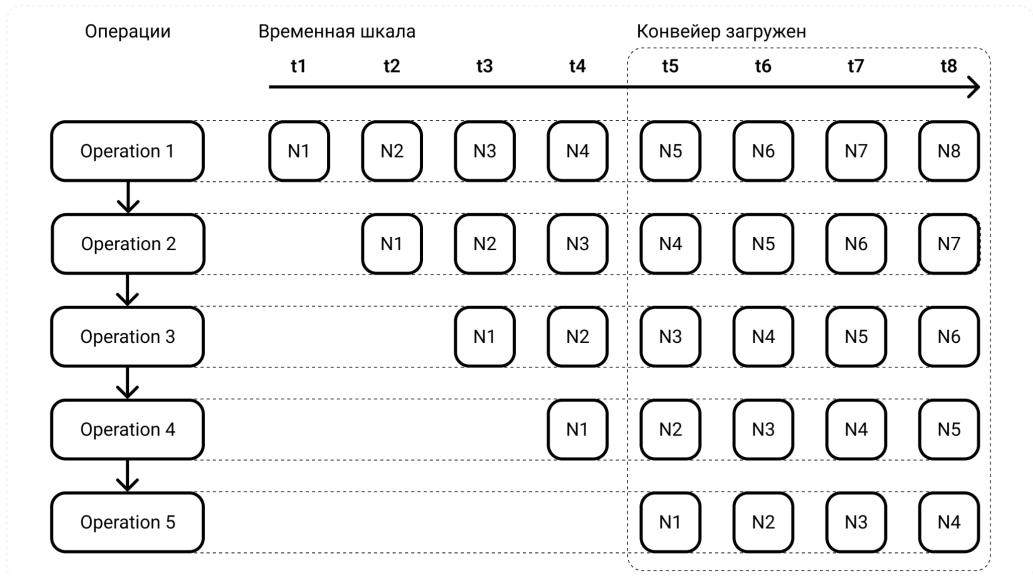


Рис. 11-09. Суперконвейер

Важно отметить, что современные процессоры имеют множество ядер, и такой конвейер может быть существенно оптимизирован по скорости, если обработчик операций распределить по разным ядрам, не говоря о возможности распределить операции между разными процессорами в одном сервере, создав локальный конвейерный вычислительный кластер ([Глава 9](#)).

Современные технологии построения распределенных систем позволяют создать распределенный конвейер на разных сетевых узлах. Такую конвейерную архитектуру применяют для построения высокопроизводительных вычислительных кластеров (разд. [9.1.3](#)).

Распределенный конвейер может быть построен на основе шаблона «Многослойная архитектура» (разд. [11.1.5](#)).

11.1.4. Брокер очередей

В предыдущих разделах уже не раз упоминался брокер очередей¹²⁹ (брокер сообщений, или просто брокер, разд. 5.4.2). Определенно брокер необходимо рассматривать как архитектурный шаблон. Основной функцией брокера очередей, как шаблона, является маршрутизация сообщений.

Естественно, брокер может содержать некую логику, но эта логика может быть декомпозирована в другие шаблоны. Как правило, для построения брокера очередей используется конвейерная архитектура.

Брокеры могут быть классифицированы по разнообразным признакам:

- с точки зрения наличия признаков приоритета сообщений брокер может быть приоритизированным (брокер с приоритетом, разд. 5.4.3);
- с точки зрения наличия обратной связи брокер может не иметь (разд. 5.4.3, 8.4) или иметь (брокер с обратной связью, разд. 8.5) обратную связь. Логика брокера может опираться на синхронизированные переменные в кэше (разд. 6.2), отражающие состояние узлов РСУ. В режиме динамических изменений эти переменные могут меняться. Такие изменения будут источником обратной связи;
- с точки зрения участия брокера в процессе балансирования системы брокеры могут быть классифицированы как маршрутизирующие (обычные) и балансирующие (брокер-балансировщик, разд. 8.2);
- с точки зрения шаблона «Ведущий-ведомый» выделяют брокер-лидер, брокер-мультилидер и брокер без лидера (разд. 5.1);
- с точки зрения узлового размещения брокера в РСУ выделяют встраиваемый в узел брокер (брокер встраиваемый) и

¹²⁹ Queue broker, message broker, #11-15

брокер, размещаемый на отдельном выделенном узле (брокер-прокси, разд. 5.4.2);

- с точки зрения масштабирования РСУ как отдельный шаблон можно выделить брокер распределенный, брокер синхронизации (разд. 5.4.6);

Архитектурные шаблоны брокеров могут сочетаться по-разному в зависимости от целевой задачи.

11.1.5. Многослойная архитектура

Распространенной практикой является разделение архитектуры на разные слои¹³⁰. Классический подход позволяет разделить любую систему на слои. В каждой системе можно выделить слои данных, логики и интерфейса. Слой может быть как прикладным, так и пользовательским. Обычно один из таких слоев выделяют как слой сопряжения составляющих подсистем (рис. 11-10).

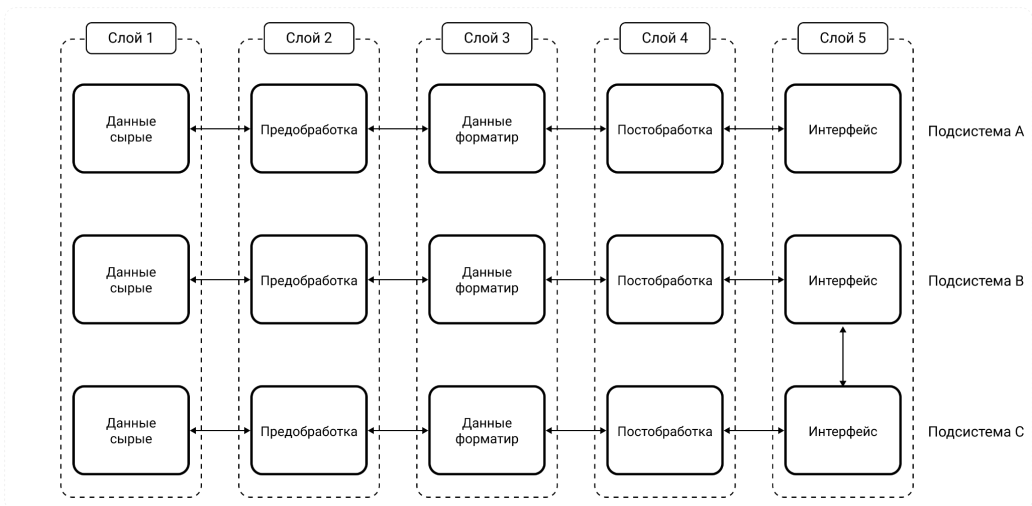


Рис. 11-10. Архитектурные слои и уровни

В отличие от конвейерной архитектуры данный шаблон предназначен для более сложного архитектурного структурирования. Многослойная архитектура позволяет выделить схожие по смыслу и реализации слои разных подсистем и компонентов

¹³⁰ Layered Architecture, #11-16

целевой системы. Всегда можно выделить архитектурные слои, как минимум, слои данных, логики, интерфейса.

Многослойная архитектура позволяет осуществить послойное структурирование с последующей гармонизацией взаимодействия составляющих подсистем. Гармонизация (оптимизация) взаимодействия подсистем обеспечивает формирование ППИ сопряжения подсистем.

ППИ многослойной архитектуры может по-разному осуществлять сопряжение подсистем. Может быть выделен единый слой взаимодействия, например, слой данных. В этом случае все подсистемы обращаются к одному слою, отвечающему за предоставление доступа к данным. Такой подход применяют при использовании разнообразных СУБД и для создания сервисов на основе доступа к ППИ. Для сопряжения может быть выделен слой пользовательского интерфейса, в который пробрасывают все функции каждой из подсистем. В этом случае взаимодействие подсистем полностью обеспечивает пользователь.

В некоторых случаях подсистемы могут взаимодействовать на нескольких слоях одновременно. Выделяют слои сильного сопряжения, в которых сконцентрирована большая часть функционала ППИ, и слои слабого сопряжения. В ВНС принято гармонизировать ППИ многослойной архитектуры таким образом, чтобы снизить задержки. В РСУ принято гармонизировать ППИ многослойной архитектуры так, чтобы были выполнены заданные проектные требования относительно CAP (разд. 3.5) и PACELC (разд. 6.7).

Такая гармонизация ППИ, как правило, обеспечивает возможность выделения ядра многослойной архитектуры. Подобная практика сама по себе является шаблонной.

Важно отметить, что архитекторы и разработчики часто путают многослойную архитектуру с многоуровневой (разд. 11.1.2).

11.1.6. Выделенное ядро

Распространенной практикой является объединение наиболее часто используемых функций вокруг одного или нескольких слоев сопряжения в единое функциональное ядро¹³¹ системы или нескольких ядер. Например, в промышленных РСУ принято выделять технологические и бизнес-ядра, потому что объединять их в одно нет никакого смысла, но обеспечить сопряжение необходимо.

Ядро как правило обеспечивает весь функционал, связанный с сопряжением подсистем РСУ (рис. 11-11).

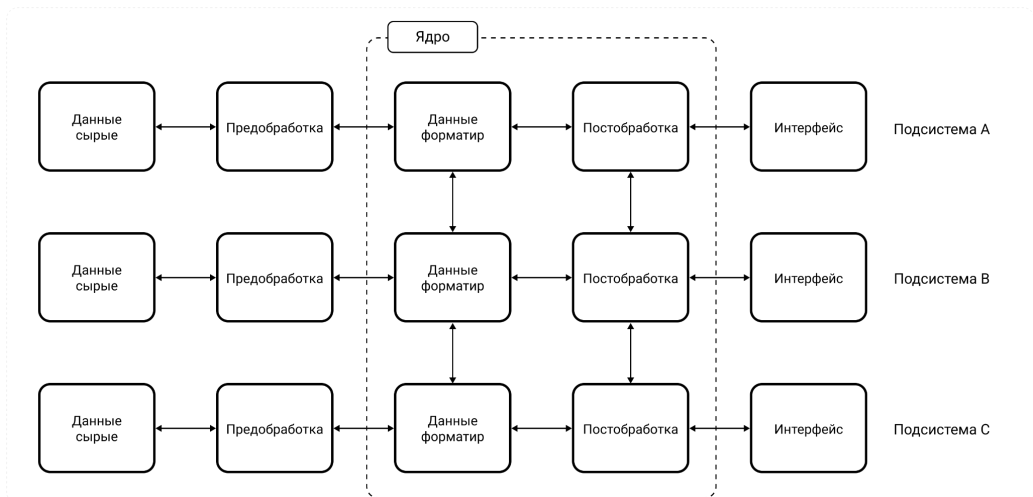


Рис. 11-11. Выделение ядра

Ядро может состоять из укрупненных модулей, интегрирующих послойный функционал (рис. 11-12).

Ядро может быть как монолитным, так и модульным. Модульное ядро может быть создано на основе микросервисов. Ядро может быть распределенным.

Именно такой подход в основном используют для создания РСУ (разд. 11.2.6).

¹³¹ Dedicated core, [#11-17](#)

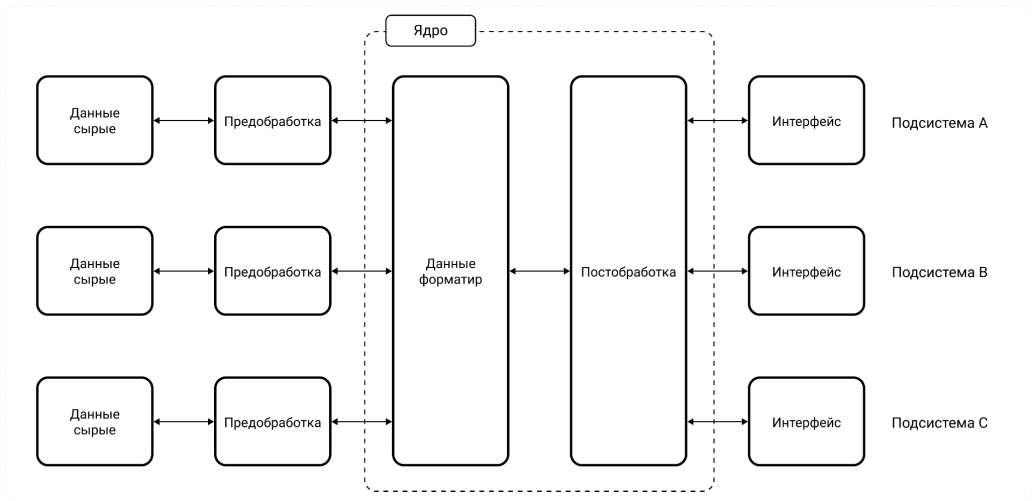


Рис. 11-12. Выделенное ядро

11.1.7. Издатель-подписчик

Этот шаблон необходимо упомянуть как важный для построения РСУ. Шаблон «Издатель-подписчик»¹³² обеспечивает механику подписки потребителя на изменения разнообразных источников. Множество потребителей подписываются на события, публикуемые множеством источников. Все взаимодействия осуществляются через единую шину событий. При этом все участники могут быть анонимными (рис. 11-13).

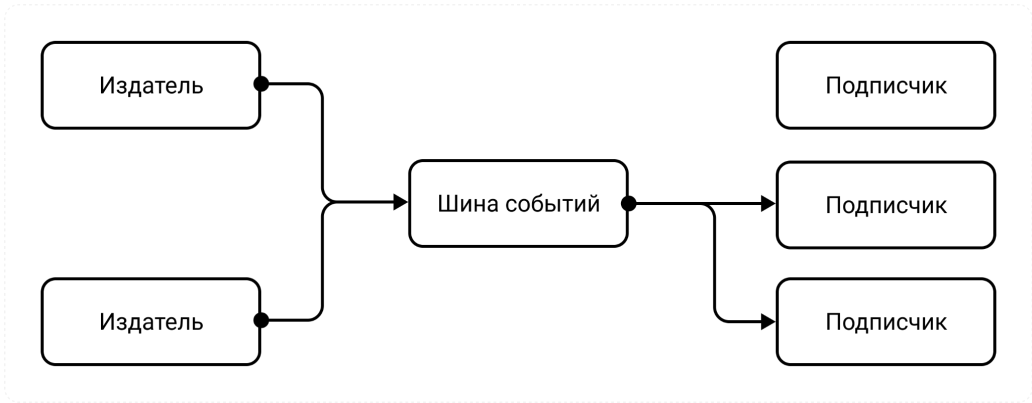


Рис. 11-13. Издатель-подписчик

¹³² Publisher-subscriber, #11-18

Использовать этот шаблон в ВНС следует с осторожностью. Подписка на событие — это очень удобно. Если в определенный момент времени все издатели начнут генерировать трафик, может возникнуть эффект лавины ([Глава 5](#)), поэтому придется применить управляемую деградацию (разд. [7.3](#)).

При наличии в системе организованной РПП (разд. [4.2](#)) использование шины событий становится намного более эффективным (разд. [11.2.1](#)).

11.2. Архитектурные шаблоны РСУ

В предыдущем разделе были приведены распространенные архитектурные шаблоны, которые применяют для построения любых систем. Эти шаблоны являются составными частями, кирпичиками для создания специализированных шаблонов РСУ. Шаблоны, приведенные в данном разделе¹³³, предназначены специально для проектирования РСУ.

Например, способы построения шаблона «Реплицированные сервисы с распределением нагрузки» (RLBS¹³⁴) отражены в значительной части материала этой книги.

11.2.1. Интерконнект

Неотъемлемой частью любой РСУ являются сетевые каналы для обмена данными. По ним осуществляются всевозможные виды взаимодействия (разд. [3.2](#)). Существует масса разнообразных типов физических каналов связи, такие как кабельные проводные, оптические каналы, беспроводные радиоканалы, инфракрасные каналы. Каждый из них имеет свои преимущества и ограничения. Современные каналы связи эффективно осуществляют сегментирование трафика.

¹³³ Top 7 most-used distributed system patterns, [#11-19](#) ►

¹³⁴ Replicated Load-Balanced Services, [#11-20](#)

Одной из современных технологий сегментирования трафика является использование программно-определяемых сетей (SDN¹³⁵). Использование SDN осуществляет виртуализацию работы с физическими сетевыми каналами. Современные технологии SDN очень развиты. Масса оборудования поддерживает работу с данной технологией и обеспечивает аппаратное ускорение SDN.

Интерконнект принято строить как на физических, так и на виртуальных каналах связи. Как правило, интерконнект представляет собой одно или несколько физических соединений. Интерконнект из нескольких соединений может быть применен для масштабирования канала связи, если одного соединения не хватает. Интерконнект может быть сегментирован и/или предоставлен для обслуживания SDN.

В пространстве физических и виртуальных сетей интерконнект может обеспечивать работы разнообразных шин обмена данными. Следует выделить следующие типы шин:

- шина P2P, равноправных взаимодействий точка-точка;
- шина подписки, обслуживания событийной архитектуры (разд. 11.2.2);
- шина RDMA, удаленного прямого доступа к памяти;
- шина широковещательного обмена (разд. 3.2).

Чтобы PCY работала эффективно, необходимо гармонично балансировать предельную нагрузку на интерконнекты системы. Для этого в PCY принято организовывать дополнительные типы специализированных шин:

- шина синхронизации, которую применяют для обслуживания высоконагруженных соединений для обмена данными между корневыми узлами (разд. 5.1);
- шина сопряжения, которую применяют для обслуживания менее нагруженных соединений, например, для обмена

¹³⁵ Software Defined Network, #11-21

данными между корневыми узлами и клиентами, mesh-узлами;

- шина диагностики, которую применяют для изоляции диагностического трафика;
- шина полевая, которую применяют в промышленных системах для обмена данными между устройствами низовой автоматики и разнообразными шлюзовыми устройствами;
- шина сервисная для взаимодействия сервисов, которую применяют для обмена данными между разнообразными сервисами.

В современных РСУ одновременно могут быть использованы несколько шин. P2P-конструкции в основном используют для обмена данными узлов, взаимодействующих по *mesh* топологии или осуществляющих функцию ретрансляции.

РСУ, построенные исключительно на шинах подписки (разд. 11.1.7), рискуют столкнуться с проблемами перегрузки. Однако существует механика, при которой шина событий в РСУ будет работать эффективно и безопасно в отношении потенциальных перегрузок. Шина событий эффективно сочетается с применением РРП (разд. 4.2). При наличии РРП шина событий лишь сообщает о факте изменения переменной в РРП. Такая конструкция более эффективна, чем реализация механики отслеживания изменений в РРП. Фактически по шине событий циркулируют адреса и индексы, указывающие на место изменений данных в РРП. Такая конструкция позволяет управлять переменными, придавать динамику разнообразным объектным надстройкам, собранным на базе РРП (разд. 12.1.2), управлять связями между объектами (разд. 12.1.3). При этом обмен данными возлагают на шину синхронизации РРП.

Для управления и синхронизации данных РРП РСУ применяют шины RDMA и широковещательные шины. RDMA обеспечивает доступ к памяти удаленных узлов. Шина широковещания обеспечивает массовый обмен данными между узлами. Практику применения RDMA и широковещания можно

отнести к шаблонам PCY. Обычно данные шины эффективно балансируются. Наиболее часто синхронизируемые данные кэшируются на необходимых узлах и затем синхронизируются по широковещательной шине. Широковещательная шина может работать в режиме максимальной нагрузки всегда — даже при низком объеме изменений данных. Такая конструкция полезна, когда нет смысла разбираться в составе изменений, а проще передать весь объем данных. RDMA-шину рекомендуется использовать в режиме, в котором не будет перегрузок шины.

Все рассмотренные шины обычно работают в рамках одного интерконнекта. В зависимости от целевого состояния системы можно осуществлять балансировку шин, динамически управляя предельной пропускной способностью каналов. Например, в режиме пусконаладки промышленной PCY объем диагностики существенно выше, чем в режиме эксплуатации, поэтому его имеет смысл расширить.

11.2.2. Событийно управляемая архитектура

Архитектура, управляемая событиями (EDA¹³⁶), основана на применении шаблона «Издатель-подписчик» в рамках PCY. Применение EDA существенно упрощает архитектуру целевой PCY. EDA эффективно поддается масштабированию. Например, большинство СУБД содержат EDA компоненты, такие как обработчики хранимых процедур реляционных СУБД или событийная модель NoSQL СУБД.

Обычно выделяют следующие объекты шаблона EDA:

- источник событий;
- брокер событий;
- шина событий;
- обработчик событий;
- генератор команд.

¹³⁶ Event-driven architecture, [#11-22](#)

Источником событий являются как внешние факторы, так и внутренние изменения. Брокер событий отвечает за идентификацию события, упаковку события в формат, понятный обработчику событий. Брокер пересылает сообщение по шине событий обработчику. Обработчик событий запускает логическую обработку. После завершения обработки возможны два варианта: происходят модификация данных, изменение состояния системы, за которыми ничего не следует, либо обработчик формирует команду управления, которая может быть источником нового события или даже каскада событий. Важно помнить, что каскадирование событий может привести к перегрузкам и известным негативным последствиям (разд. 11.2.1).

В связке с РПП (разд. 4.2) EDA показывает себя максимально эффективно. Данные в РПП обычно синхронизируются в синхронном режиме. EDA работает преимущественно в асинхронном режиме. Следовательно, такая связка обеспечивает полный контроль над РСУ.

В логике CAP (разд. 3.5) EDA представляет собой типичную слабо связанную, устойчивую к разделению систему. Это обусловлено тем, что событие как объект абсолютно автономно и не привязано к какой бы то ни было цели.

EDA эффективна для обработки потоков событий (ESP¹³⁷). Обработчик событий EDA может использовать ESP в качестве основного логического элемента. Обычно ESP предназначена для анализа потоков событий и изменений данных, связанных с событиями. ESP позволяет обнаруживать взаимосвязи между событиями, иерархию, корреляцию, выявлять причинность событий.

EDA удобна для описания сложной логики работы технологических и бизнес-систем. Например, функционально-групповое управление (ФГУ), активно применяемое в РСУ технологическими и производственными процессами для автоматизации оперативного управления, реализуется с помощью EDA.

¹³⁷ Event Stream Processing, #11-23

EDA активно применяют и в автоматических системах управления, в которых не принимает участие человек. Именно EDA в связке с технологиями искусственного интеллекта и машинного обучения позволит устранить человека как оператора сложных PCY в ближайшем будущем. ESP очень удобно использовать в промышленных PCY при анализе каскадных тревог для выявления первопричины. EDA и ESP применяют в разнообразных системах поддержки принятия решений (СППР), своего рода советчиках оператора, являющихся предвестниками полного автоматического управления.

11.2.3. Сервис-ориентированная архитектура

Сервис-ориентированная архитектура (COA, SOA¹³⁸) является достаточно старой практикой, известной еще с 1990-х годов. COA появилась как ответ на развитие технологий, связанных с появлением мощного железа, миграцией приложений в ЦОД, развитием каналов связи и возможностью их применения для организации взаимодействия таких приложений.

Со временем COA пережила массу изменений. Было апробировано множество актуальных и уже неактуальных технологий. Практика применения COA обеспечила зрелость принципов масштабирования и взаимодействия массовых распределенных слабо связанных систем. Она обеспечила появление общепринятых протоколов, которые используются в настоящее время, таких как REST¹³⁹. COA сильно повлияла на развитие современных сетевых сервисов и стиль управления ими.

Концепция COA обеспечила выделение такого класса обслуживающего ПО, как сервисы, и сформировала общепринятые требования к ним. Появление разнообразных сервисных шин сопряжения сервисов, например, сервисных шин предприятий (ESB¹⁴⁰), связано с развитием и зрелостью технологий COA.

¹³⁸ Service-Oriented Architecture, [#11-24](#)

¹³⁹ Representational State Transfer, [#11-25](#), [#11-26](#) ►

¹⁴⁰ Enterprise Service Bus, [#11-27](#)

Задача сервисной архитектуры заключается в том, чтобы «развалить» тяжелое и сложное монолитное ПО на множество более простых конструкций и обеспечить взаимодействие между ними. Такие конструкции принято называть сервисами. Сервис, как любое ПО, может быть разработан на распространенных языках программирования, скомпилирован и запущен как приложение целевой ОС или разработан как сценарий (скрипт) в соответствующей инфраструктуре. Таким образом, создание сложного ПО существенно упрощается в связи с возможностью эффективно распределить разработку отдельных сервисов между разными командами, которые могут работать в разных стилях. Разработчикам нужно лишь соблюдать правила ППИ сервисной шины.

Важно отметить, что практика создания сервисных шин и интерфейсов взаимодействия с сервисами представляет собой отдельный архитектурный шаблон. Как правило, сервисные шины были нагромождением разнообразных технологий, утяжеляющих общую целевую конструкцию, не всегда с эффективными механизмами сопряжения и высоким порогом входа в уже сложившиеся конструкции. Многие разработчики бизнес-шин намеренно разрабатывали проприетарные конструкции ради конкурентной борьбы на рынке. Именно по этой причине многие архитектурные практики остались проприетарными. Указанные обстоятельства привели к тому, что рынок потребовал существенного упрощения работы SOA.

Развитие систем на основе открытого исходного кода, драйвером которого стала открытая ОС Linux, привели к существенному снижению массового применения проприетарной сервисной архитектуры и, как следствие, проприетарной конкуренции продуктов SOA. Развитие технологий виртуализации, облачного развертывания сервисов на основе массово доступного зрелого открытого ПО, привели к принципиальной трансформации отрасли SOA. Конкуренция развернулась вокруг качества оказания услуг внедрения и управления сервисами на основе открытого ПО.

Это привело к тотальной стандартизации отрасли, под которую вынуждены подстраиваться даже технологические гиганты.

Одним из ключевых архитектурных стандартов подобного рода стала микросервисная архитектура, а COA является ее неоспоримым прообразом.

11.2.4. Микросервисная архитектура

Микросервисная архитектура (MCA, MSA¹⁴¹) является современным вариантом развития COA (разд. 11.2.3). Основная концепция данной практики лежит в повышении эффективности управления жизненным циклом сервисного ПО, а именно эффективности разработки, тестирования, развертывания, сопряжения.

Одним из ключевых факторов успешности стратегии MCA является зрелость технологий развертывания микросервисов в облачных инфраструктурах. При проектировании PCY на основе MCA необходимо заранее определиться с ППИ взаимодействия, чтобы не подвергать их дальнейшему изменению.

Сопряжение микросервисов может быть осуществлено разными способами. Один из них заключается в организации взаимодействия путем обмена сообщениями (разд. 5.4.1). Существует способ, предполагающий использование РРП (разд. 4.2). Координация работы микросервисов может осуществляться на основе событийно управляемой архитектуры (разд. 11.2.2), применения оркестраторов и хореографии (разд. 11.2.5).

PCY, созданную на основе микросервисов, проще эксплуатировать, развертывая отдельные микросервисы. Развертывание можно осуществлять в режиме эксплуатации, не отключая от инфраструктуры обновляемый узел. Обновление отдельного микросервиса потенциально вызывает небольшую задержку на время обновления. Если же в системе предусмотрено резервирование (разд. 6.1.1), задержек можно избежать.

¹⁴¹ Micro Service Architecture, #11-28, #11-29 ►

Микросервисы могут быть эффективно изолированы друг от друга, поэтому доступность микросервисов относительно других микросервисов и клиентов может быть тонко настроена (рис. 11-14). Изоляция может быть обеспечена путем виртуализации, присущей именно микросервисам. Такую технологию виртуализации микросервисов называют контейнеризацией, она заключается в размещении ПО микросервиса в легкой виртуальной машине под управлением специального гипервизора.

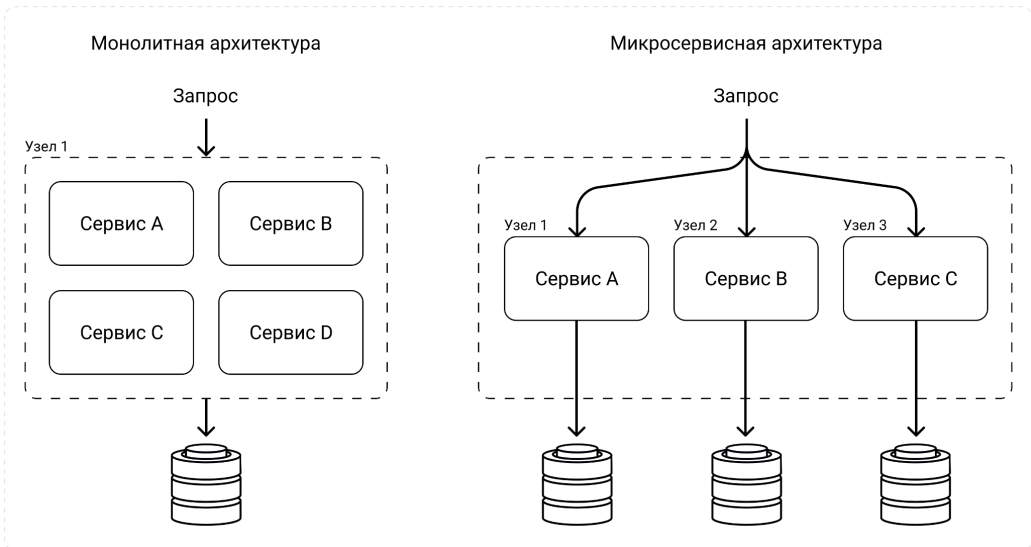


Рис. 11-14. Монолитная vs микросервисная архитектура

В подобной архитектуре может быть эффективно реализована мультитенантность¹⁴² (мультиарендность), заключающаяся в том, что для каждого клиента создается свой экземпляр микросервиса в изолированном контейнере. Таким образом, данные разных клиентов могут быть надежно изолированы и защищены (Глава 13). Все указанные моменты необходимо учитывать при проектировании и декомпозиции РСУ.

¹⁴² Multitenancy, #11-30

11.2.5. Оркестратор и хореография

Оркестратор¹⁴³ представляет собой достаточно распространенный архитектурный шаблон. Оркестратор координирует запросы клиентов к распределенным сервисам. С точки зрения декомпозиции, оркестратор является составным объектом. «Под капотом» у оркестратора есть и брокер очередей (разд. 5.4.2), и балансировщик нагрузки (разд. 8.2). Оркестратор координирует и их работу. Такую координацию называют оркестрацией.

Оркестратор может быть достаточно сложно устроенным. Он может обладать своей собственной СУБД, встроенными сервисами, диагностикой. В целом, существует целый класс продуктов-оркестраторов, предназначенных для сопряжения и координации других продуктов.

В задачах гипермасштабирования (разд. 10.4) оркестраторы сами по себе могут представлять сложно организованную распределенную систему, которая может координироваться специальным гипероркестратором.

Если рассматривать оркестратор как шаблон, его узким местом является централизация. Таким образом, координируемые оркестратором сервисы сильно от него зависят (рис. 11-15).

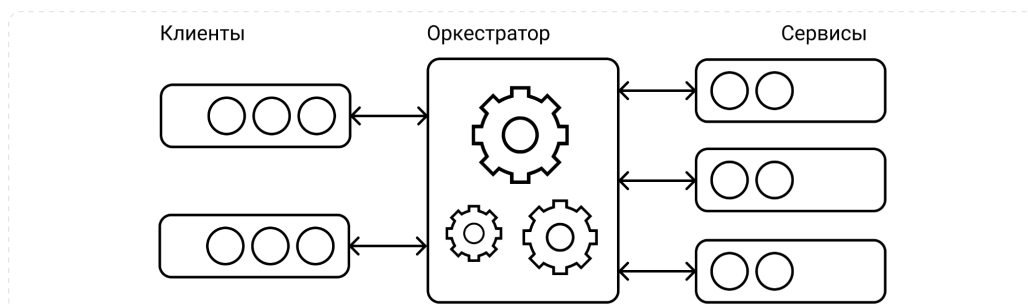


Рис. 11-15. Оркестратор

Оркестратор управляет группами сервисов подобно тому, как дирижер управляет оркестром, в котором есть группы

¹⁴³ Orchestrator, #11-31

музыкантов с одинаковыми инструментами. Если из оркестра убрать дирижера, вероятно, оркестр станет играть хуже. Представьте себе, если некий дирижер захочет управлять одновременно большим распределенным «гипер-оркестром» состоящим из сотен оркестров, то это будет достаточно непростой задачей, но только не в случае программной архитектуры.

В случае с танцорами все совершенно иначе. Постановку танца осуществляет хореограф, но в самом представлении он не участвует. Танцоры справляются самостоятельно и координируются друг с другом согласно заранее определенным правилам. Такой подход применим и к распределенным сервисам.

Шаблон «Хореография»¹⁴⁴ дает сервисам возможность не зависеть от централизованного управления, а координироваться самостоятельно. Данный подход позволяет обслуживать запросы при самостоятельной координации сервисов.

Варианты реализации шаблона «Хореография» могут быть весьма разнообразными. Самый простой вариант заключается в использовании некоего контейнера, в котором размещаются сообщения. На роль контейнера могут подойти очередь сообщений (разд. 5.4.1), РРП (разд. 4.2), СУБД. Сообщение может содержать информацию об адресатах, о том, какие операции какие сервисы должны выполнить и в каком порядке. Сообщения маршрутизируются в нужный сервис заданным способом: например, через РРП или путем подписки (разд. 11.1.7). Сервис обрабатывает поступившее сообщение, а при успешном выполнении своей операции возвращает сообщение с отметкой о выполнении в свой или другой контейнер, из которого сообщение будет передано следующему сервису. В случае неудачи сообщение может быть помещено в специальный контейнер для неудачно обработанных запросов.

Оркестратор представляет собой типичную *single-leader* (или *multi-leader*) архитектуру, а хореография — типичную *leaderless*

¹⁴⁴ Choreography, #11-32

архитектуру (разд. 5.2). *Leaderless*, распределенный брокер (разд. 5.4.6), *leaderless* балансировщик (разд. 8.2.2) являются типичными примерами практики использования шаблона «Хореография» (рис. 11-16, рис. 11-17).

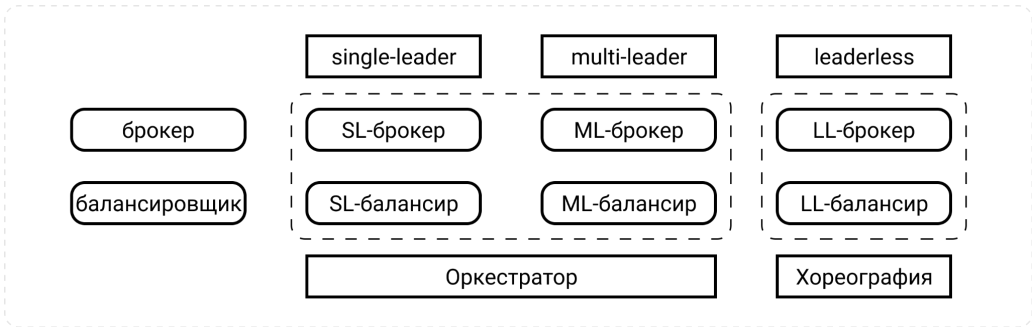


Рис. 11-16. Место Оркестратора и Хореографии

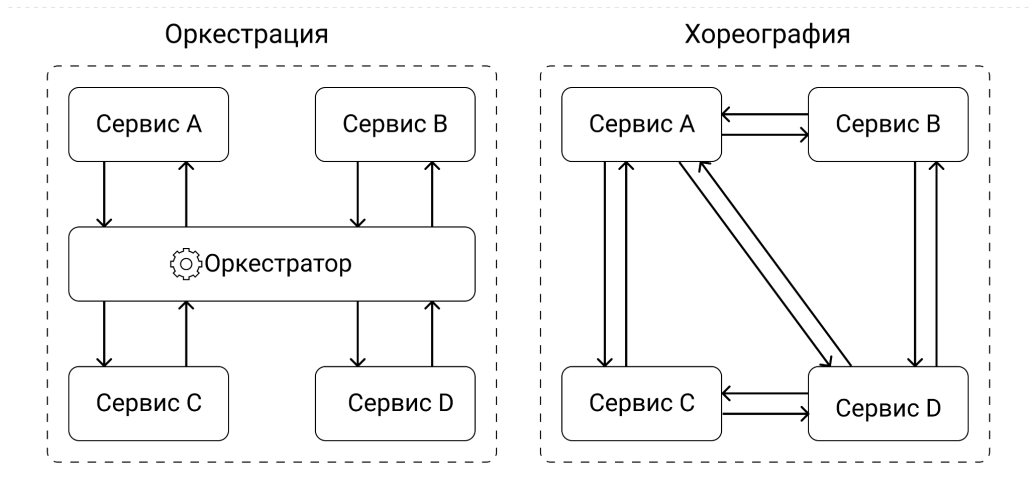


Рис. 11-17. Оркестратор vs Хореография

Очевидно, что применение шаблона «Хореография» обеспечивает более высокую надежность систем в связи с отсутствием одной или нескольких вероятных точек отказа в виде оркестраторов. Однако построить такую систему сложнее.

Существует специальный раздел программирования «Хореографическое программирование», описывающий стили и правила разработки подобных систем¹⁴⁵.

¹⁴⁵ Choreographic programming, #11-33

11.2.6. Распределенное ядро

Для реализации принципов распределения данных в РСУ может быть применен шаблон «Распределенное ядро»¹⁴⁶. Функциональное распределенное ядро (далее — ядро) обычно собирают из нескольких базовых шаблонов. Прежде всего, подразумевается, что функциональное ядро выделено. Это означает, что применен шаблон «Выделенное ядро» (разд. 11.1.6). Ядро размещается на распределенных узлах, а коммуникация может осуществляться путем использования шаблона «Интерконнект» (разд. 11.2.1). Интерконнект может содержать различные интерфейсы (протоколы) обмена данными.

Для распределения ответственности между узлами ядра могут быть использованы шаблоны «Оркестратор» и «Хореография» (разд. 11.2.5). Естественно, узлы ядра могут кэшировать часть данных. В такой ситуации возможно использование шаблона «Кэш на стороне» (разд. 6.2). Если узлы ядра являются частью РСУ, то команды управления целевой системой могут быть изолированы при условии применения шаблона «CQRS» (разд. 11.2.7).

11.2.7. Разделение ответственности команд и запросов

Этот шаблон принципиально важен для РСУ, в особенности для тех, которые обеспечивают управление критическими процессами. Шаблон «Разделение ответственности команд и запросов» (CQRS¹⁴⁷) делает именно то, что лежит в основе его названия.

Рассмотренная практика предлагает принципиально разнести операции чтения и записи всеми возможными способами. Данный подход подразумевает некоторые правила:

- запрос не содержит данных;
- запрос может указывать на блок данных (переменную);

¹⁴⁶ Distributed core

¹⁴⁷ Command Query Responsibility Segregation, [#11-34](#)

- запрос не возвращает ответ или подтверждение выполнения;
- запрос на запись не возвращает данные;
- запрос на чтение не может привести к изменению данных.

Если есть возможность разнести запросы на запись и запросы на чтение в разные компоненты (части), это нужно сделать. При наличии возможности в рамках практики применения данного шаблона обязательно нужно делать следующее:

- выполнять запросы в разных брокерах (разд. 5.4.2);
- выполнять запросы в разных конвейерах (разд. 11.1.1);
- выполнять запросы в распределенных конвейерах (разд. 11.1.3);
- разнести запросы по разным интерконнектам (разд. 11.2.1);
- разнести запросы по разным ядрам (разд. 11.1.6, 11.2.6);
- разнести запросы по разным узлам (разд. 5.1).

Как правило, CQRS архитектура бывает построена на распределенном брокере (разд. 5.4.6).

В качестве примера можно привести следующий (рис. 11-18). Клиентский узел *c1* отправляет команду на запись переменной *A* на корневой узел *r1*, обслуживающий клиентов только в режиме записи. Корневые узлы *r1* и *r2* синхронизируются. В связи с изменением переменной *A*, происходит некое вычисление, которое приводит к изменению переменной *B*. Узел *c1* подписан на изменение переменной *B*, он получает ее значение от узла *r2*, который предназначен для обслуживания клиентов в режиме чтения. Узлы *r1* и *r2* синхронизируются.

Важно заметить, что синхронизация корневых узлов происходит по двум отдельным каналам связи *Q12*, *Q21*. Каждый корневой узел отправляет данные и принимает по разным каналам. Клиентский узел взаимодействует с корневыми узлами по разным каналам.

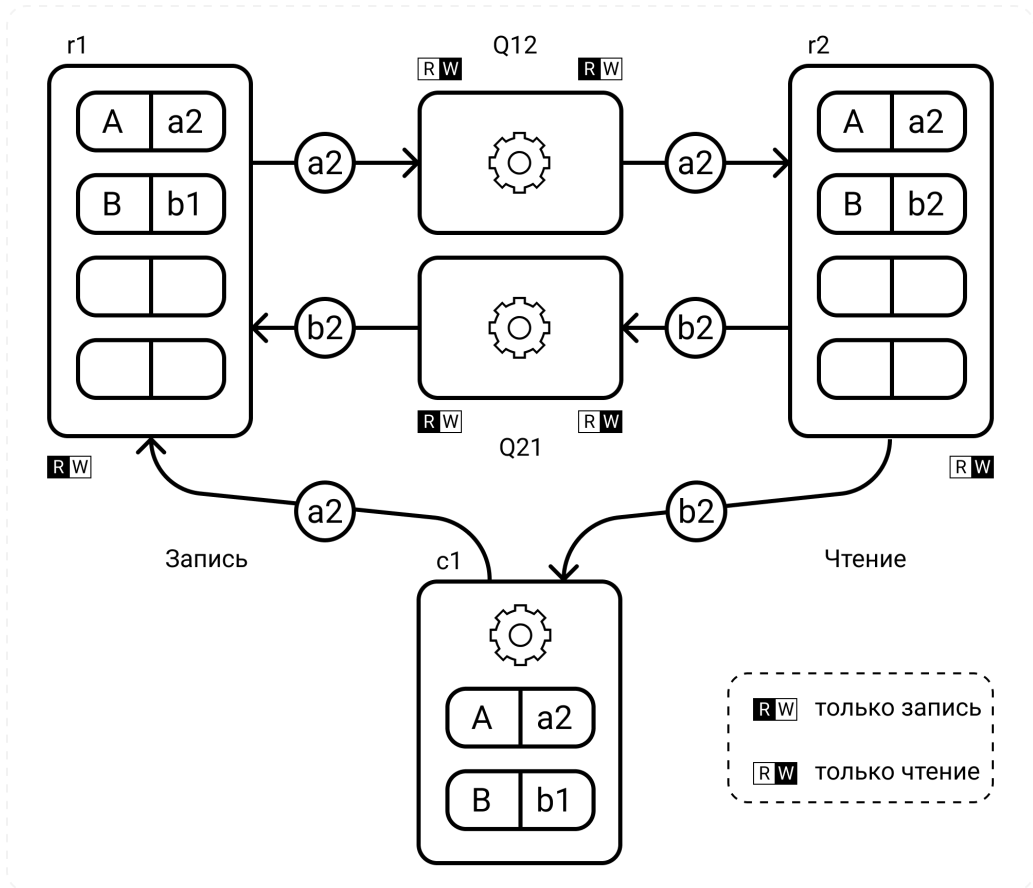


Рис. 11-18. CQRS обмен данными

Такая конструкция обеспечивает несколько существенных преимуществ. Команда управления в PCY явно занимает меньший объем, чем сопровождающийся поток данных в системе. Таким образом, команда управления не потеряется в общем потоке данных.

С точки зрения безопасности, повредить два канала всегда сложнее, чем один. В серьезных PCY каналы, в которых реализуется CQRS архитектура, резервируют, обеспечивая высокую надежность, отказоустойчивость и защищенность от вредоносных воздействий.

11.2.8. Двухфазная фиксация

Шаблон «Двухфазная фиксация» (2PC¹⁴⁸) нельзя не упомянуть в связи с его массовым использованием в распределенных системах. Данная практика позволяет разделить запрос клиента на задачи, распределить задачи между целевыми узлами, координировать их выполнение и гарантировать успешность выполнения запроса, если все участники готовы обработать назначенные им задачи.

Суть двухфазности заключается в следующем. В первой фазе все узлы, участвующие в процессе выполнения распределенной задачи, опрашивают на предмет готовности ее выполнить. Если все узлы подтвердили готовность, во второй фазе все получают команду на выполнение, а затем возвращают статус выполнения. Узел-инициатор процесса принято называть координатором (рис. 11-19).

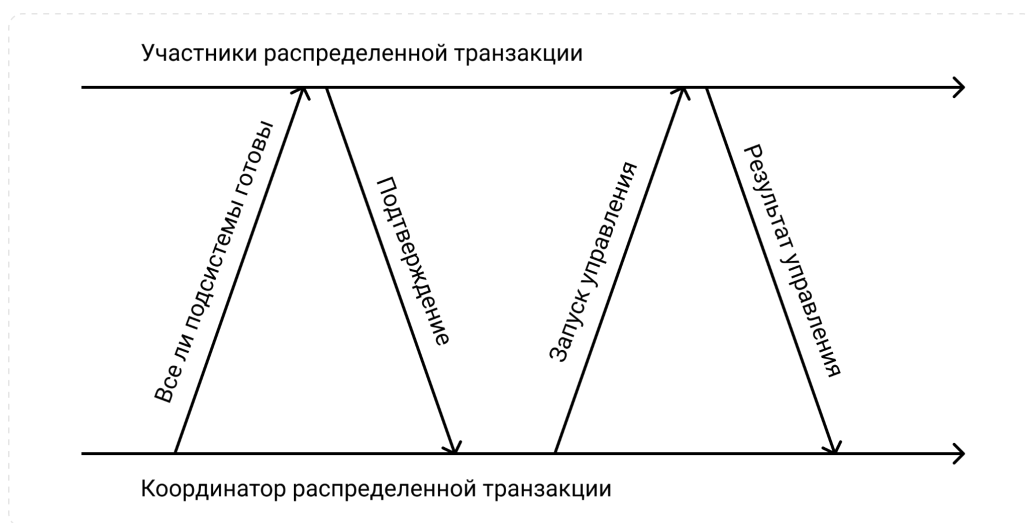


Рис. 11-19. 2PC

Основное преимущество применения этой практики заключается в том, что в первой фазе можно проверить

¹⁴⁸ Two-Phase Commit, [#11-35](#)

готовность всех узлов к выполнению конкретной задачи. Рассмотрим подробнее каждую фазу.

В первой фазе координатор посылает всем узлам условия выполнения задачи. Каждый узел сам проверяет выполнимость условий задачи.

При подтверждении готовности каждый узел возвращает координатору статус и блокируется для любых других действий, не связанных с выполнением этой задачи. Если хотя бы один из участников оказывается не готов, он возвращает координатору свой статус, а координатор посылает команду разблокировать все узлы, готовые к выполнению задачи.

Важно отметить, что узел, или узловой сервис, может работать в многопоточном режиме. Тогда все действия относительно блокировок касаются только участвующего в процессе потока.

Если все узлы готовы к выполнению распределенной задачи, запускается вторая фаза процесса. Все узлы получают команду выполнить поставленные задачи. При успешном выполнении каждый из узлов уведомляет об этом координатор. Если все узлы прислали положительное уведомление, распределенная задача считается выполненной. Координатор посылает команду разблокировать все узлы.

Механизм блокировки подразумевает, что каждый узел резервирует необходимый ресурс для выполнения задачи, обеспечит все условия. Однако может так случиться, что узел не выполнит свою задачу, например, если произойдет элементарный отказ питания узла или сетевого соединения. В такой ситуации процесс останавливается. Координатор посылает команду разблокировать все узлы, а распределенная задача считается невыполненной.

Основным минусом данной конструкции является транзакционная природа процесса выполнения распределенной задачи.

Каждый узел может выполнять свою задачу с разными значениями задержки (разд. 7.1). Таким образом, выполнение задач каждым из узлов будет происходить в асинхронном режиме относительно координатора (асинхронный 2PC). При необходимости можно обеспечить синхронность выполнения задач узлами, управляя последовательностью раздачи команд на выполнение (синхронный 2PC). При этом общая задержка синхронного алгоритма будет больше.

Таким образом, рассмотренный шаблон имеет существенные ограничения применения в ВНС. Подобная практика может быть применена в некоторых ответственных случаях, не требующих реакции в режиме РВ, но требующих аккуратности выполнения запроса. 2PC полностью соответствует принципу ACID (разд. 3.3), который может быть точно применен в РСУ. Например, при управлении критически важными процессами в АСУ ТП синхронность в управлении предельно необходима. Например, это касается чувствительных команд управления объектами автоматизации, требующих предварительной проверки состояния оборудования, и отслеживания обратных связей при осуществлении управления.

Не стоит недооценивать скоростные характеристики этого шаблона. Принцип распределенности выполнения задачи говорит сам за себя: несколько узлов выполнят задачу быстрее, чем один. При этом во время постановки задачи каждому узлу может быть оговорено условие о скорости выполнения с заданным значением величины задержки. Подобная практика дает возможность опросить РСУ о готовности узлов быстро, надежно и безопасно выполнить требуемую задачу (команду).

Шаблон 2PC эффективно масштабируется, он может быть построен в режиме избыточности (разд. 4.3). В таком случае в

выполнении распределенной задачи будут участвовать избыточные узлы. Таким способом может быть реализована мажоритарная логика, например, «2 из 3-х», когда результат отработки задачи разными узлами сравнивают, а решение принимают по принципу большинства. Подобные алгоритмы могут быть реализованы аппаратно для критических систем, например, в виде жесткой логики (рис. 11-20).

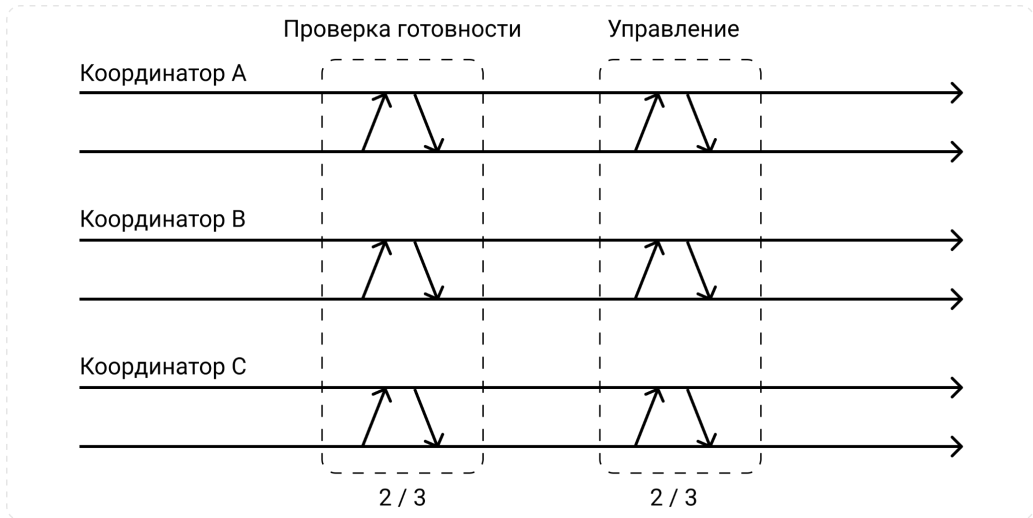


Рис. 11-20. 2PC

11.2.9. Saga

В больших распределенных системах, особенно географически распределенных, сложно обеспечить высокую консистентность транзакций 2PC. В какой-то момент что-нибудь обязательно пойдет не так. Транзакции (разд. 3.3) будут массово возвращать отрицательные результаты. Такая практика позволяет разделить запрос клиента на задачи, распределить задачи между целевыми корневыми узлами, координировать их выполнение и попробовать успешно выполнить запрос, не зная, готовы ли все участники обработать назначенные им задачи.

Saga¹⁴⁹ соответствует принципу BASE (разд. 6.7) и является противоположностью 2PC (разд. 11.2.8). Система, координируемая с применением данной практики, является слабоконсистентной, а точнее возможно-консистентной (разд. 6.6.4). Название говорит само за себя, saga — это длинная эпическая история о героическом достижении.

Запрос клиента разделяется на задачи, которые могут быть обработаны распределенными сервисами. Сервисы могут координироваться централизованно с помощью оркестратора¹⁵⁰ или децентрализованно с помощью хореографии¹⁵¹ (разд. 11.2.5). Как бы ни были скоординированы сервисы, рассмотренная практика подразумевает последовательное выполнение декомпозированных задач назначенными сервисами.

Выполнение задачи сервисом может привести к изменению данных. Сагу можно использовать только в случаях, когда есть возможность отменить внесенные изменения, то есть откатиться назад. Это обстоятельство является главным ограничением практики. Отмена узловой задачи в саге приводит к отказу выполнения всего запроса. Все изменения, связанные с обработкой задачи каждым сервисом, должны быть отменены. Реализация такой механики предполагает, что внутри каждого сервиса за обработку назначенной ему задачи отвечает сервисная транзакция. Если сервисная транзакция не выполняется, запускается выполнение компенсационной транзакции, осуществляется откат всех изменений. Сервис сообщает координатору о факте сбоя выполнения сервисной задачи. Затем координатор в обратном порядке обходит сервисы, успешно завершившие свои транзакции, и запускает на них компенсационные транзакции. Это происходит до тех пор, пока не будут компенсированы все изменения, внесенные всеми участвующими в запросе сервисами.

¹⁴⁹ Saga, #11-36

¹⁵⁰ Orchestration-based saga

¹⁵¹ Choreography-based saga

Таким образом, сага точно не подходит для обслуживания, например, функционально-группового управления. Сагой ни в коем случае не стоит управлять технологическими процессами непрерывного производства, например, АЭС. Компенсировать управляющее воздействие будет невозможно. Сага в большей степени предназначена для обслуживания бизнес-процессов, в которых можно отменить операцию, прервать и попробовать заново. Так как сага соответствует слабой модели возможной консистентности, в конечном счете после нескольких попыток запрос будет обработан.

Хорошим примером применения этого шаблона является сервис бронирования билетов (рис. 11-21).

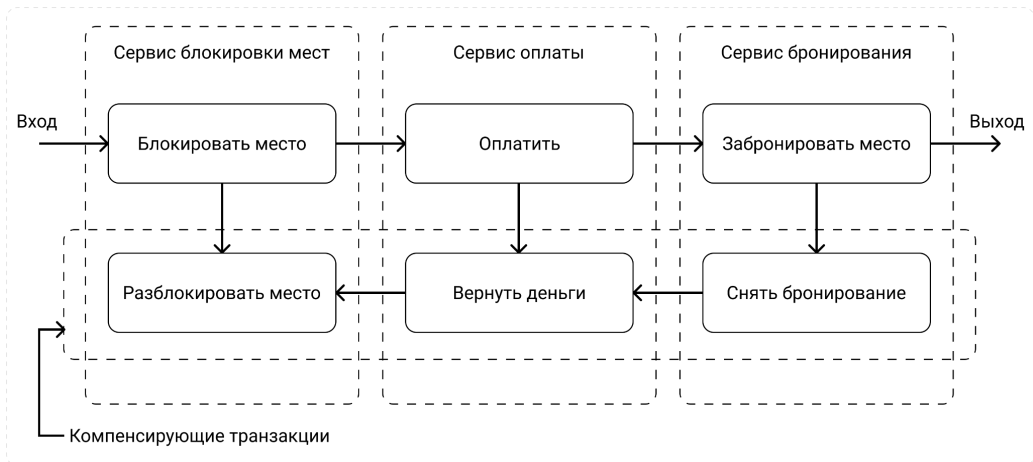
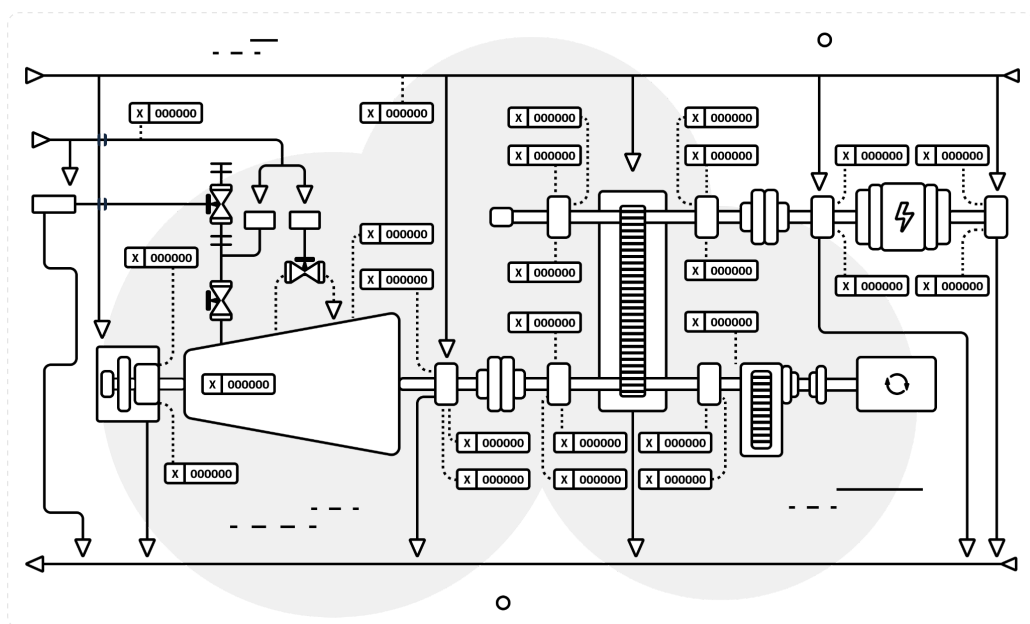


Рис. 11-21. Сага



Глава 12. Приемы реализации

12.1. Распределенные СУБД

В этой книге не будут рассмотрены механизмы организации локального хранения данных на жестких дисках — подразумевается, что архитектор найдет эффективный способ. Системы распределенного хранения и доступа к данным наиболее эффективно реализуют в архитектуре *in-memory*. Таким образом, при создании распределенной системы, на узлах которой будут задействованы жесткие диски, мы будем считать, что было обеспечено надлежащее кэширование данных в оперативную память для организации эффективной работы узла.

12.1.1. Ключ-значение

Эффективным способом организации хранения распределенных данных является конструкция типа ключ-значение (*key-value*), которая представляет собой реализацию типичной РРП (разд. 4.2)

на базе переменных. Формат хранения плоских структур пар типа «ключ-значение» лучше всего приспособлен для обработки в оперативной памяти. Такая конструкция одинаково эффективна для обработки данных на обычных классических процессорах в связке с оперативной памятью, на ПЛИС¹⁵² (FPGA) и других специализированных ИС¹⁵³ (ASIC), которые часто применяют в СРВ.

Под переменными подразумевают базовые конструкции, такие как целочисленные переменные (*int*), переменные с плавающей точкой (*float*), фиксированные массивы блоков (*array*) и строковые (*string*) переменные. Ничто не мешает хранить значительный блок данных, например, такой как JSON¹⁵⁴, чанки (постоянный блок данных), страница (временный блок данных), в поле *value*.

Подобная конструкция позволяет без лишних сложностей добавлять новые пары даже при наличии существенной нагрузки. Алгоритмы репликации и синхронизации на основе такой структуры данных реализовать проще всего. Плоские структуры пар «ключ-значение» наиболее понятные и простые в управлении.

12.1.2. Объектная надстройка

Проектировать распределенные системы, основанные на объектной модели, намного сложнее, чем основанные на плоской модели. Как правило, под объектом подразумевают некий состоящий из заранее определенного набора атрибутов контейнер. Каждый атрибут может представлять собой пару типа «ключ-значение». Контейнеру могут быть присвоены имя и прочие свойства. При эксплуатации целевой системы такая структура данных, как объект, является статической. Это как в случае с компилятором: после компиляции кода в режиме *runtime* невозможно изменить структуру классов объектов.

¹⁵² Программно-логическая интегральная микросхема, [#12-01](#)

¹⁵³ Интегральная схема специального назначения, [#12-02](#)

¹⁵⁴ Эффективный для разбора текстовый формат обмена данными, [#12-03](#)

Для PCY, CPB процесс создания новых объектов все равно достаточно сложен. Объекты могут содержать разнообразные наборы атрибутов различного типа, которые требуется идентифицировать в режиме *runtime*. Необходимо проводить массу проверок, влияющих на быстродействие целевой системы.

Существует множество ухищрений, позволяющих упростить работу с объектной моделью и свести все к работе с простейшей плоской структурой «ключ-значение». Самый простой подход к созданию объектовой структуры — это использование метаданных. По сути, можно создать статический справочник, в котором необходимо определить перечень ключей глобального списка, формирующих атрибуты объекта. Это может быть файл со схемой структурной разметки, который будет скопирован на требуемые узлы перед их запуском (рис. 12-01).

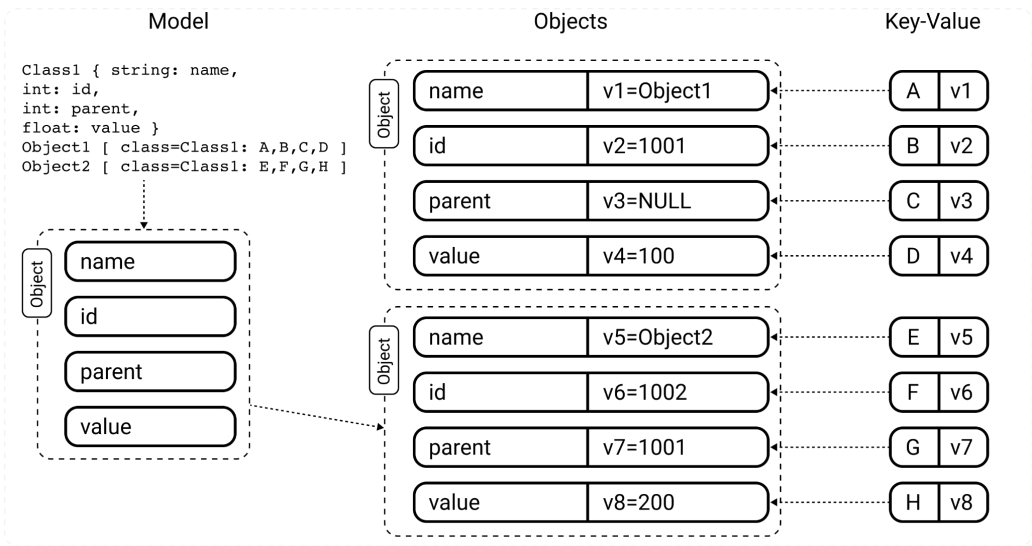


Рис. 12-01. Структура объекта

Это может быть специальная СУБД, к которой имеют доступ клиенты: они могут считать модель данных и сопоставлять нужные ключи с объектными атрибутами.

Можно поступить еще проще: ввести соглашение о наименовании ключей, отражающее принадлежность ключа к

конкретному объекту, интегрировав в состав наименования ключей уникальные префиксы. Естественно, такая конструкция накладывает некоторые ограничения, способствует появлению требований к дополнительным проверкам, однако благодаря ей исчезает необходимость в каком-либо отдельном внешнем механизме описания структур данных (рис. 12-02).

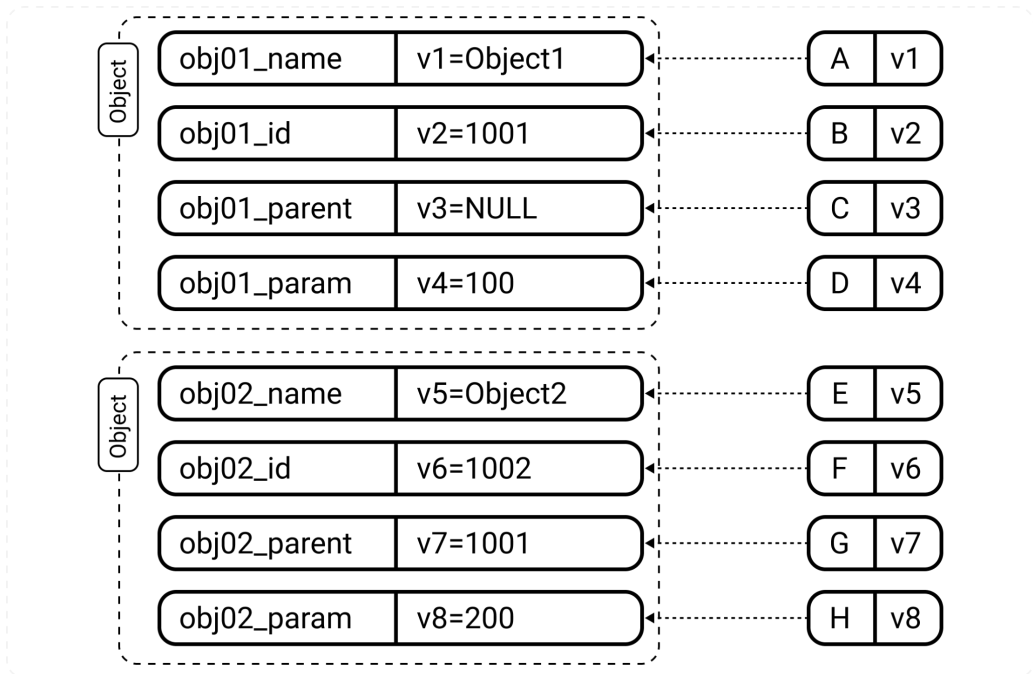


Рис. 12-02. Структура объекта с соглашением описания

Для того, чтобы понять, к какому элементу более сложной структуры данных (объекта, ветви дерева, узла графа и др.) относится ключ, какую именно переменную отражает, рекомендуется использовать метаданные.

12.1.3. Связи и отношения между объектами

Структура данных большинства СРВ в основном является статической. Данные, входящие в состав статических структур, могут меняться весьма динамично. В подобных системах изменение структуры данных происходит не в режиме РВ, а в режиме

нормального времени, однако в некоторых задачах возникает необходимость изменения самой структуры данных в режиме РВ.

Например, бывают задачи, в которых изменение связей между объектами, индексных и связанных ссылок между объектами, определяющих их структурную принадлежность (к списку, дереву, графу и др.¹⁵⁵), должно происходить очень быстро, иногда даже в режиме РВ — например, когда нужно описать большой распределенный граф с динамически изменяющимися связями и указанием направления векторов связи. В случае необходимости добавить новую связь в список ключ-значений добавляют новую пару. Например (рис. 12-03), ключи *Link* задают в качестве значения ссылку на ID целевого сегмента данных (объекта).

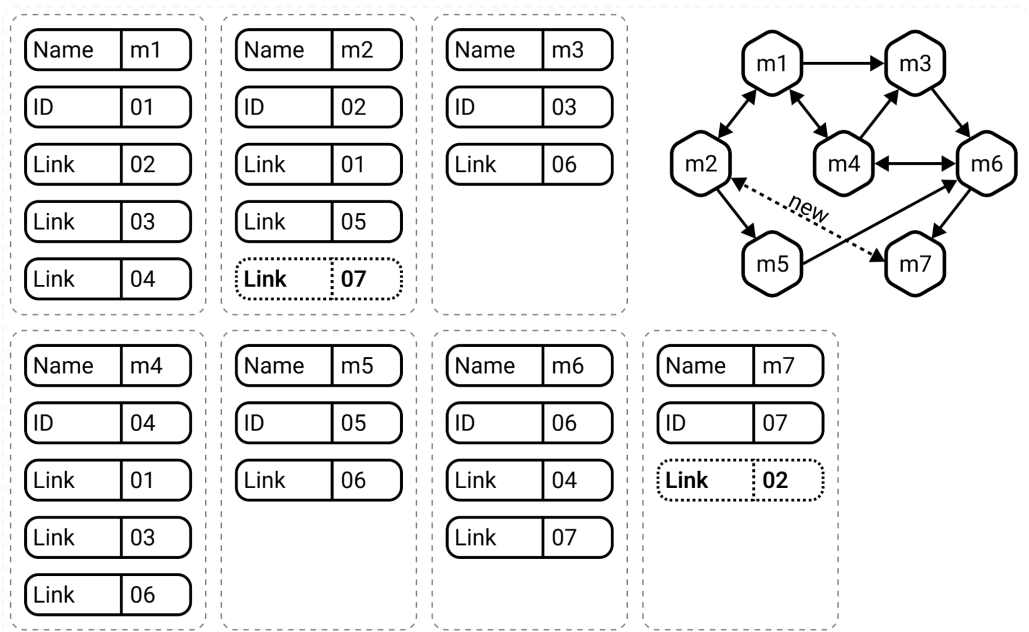


Рис. 12-03. Задание связей

Таким образом, данные, которые подвергаются изменению в режиме РВ, могут быть выделены в простейший список *key-value*. Список может быть сколь угодно большим — он может быть распределен на множество узлов, а ограничения будет определять доступная РРП.

¹⁵⁵ 10 key data structures, #12-04 ►

Пример без резервирования: для поиска переменной клиентам *c1*, *c2* необходимо выполнить поиск узла, содержащего запрашиваемую переменную, перебором (рис. 12-04).

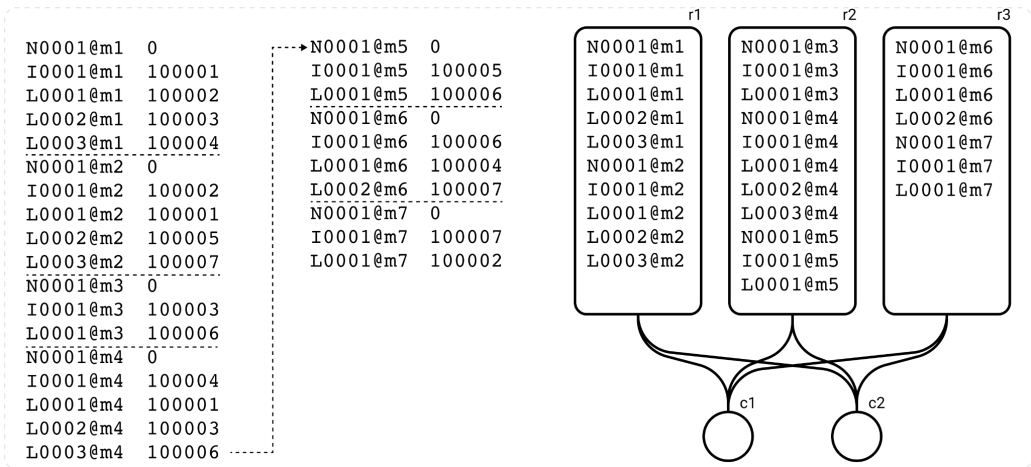


Рис. 12-04. Хранение связей

12.1.4. Индексирование

Индекс — это структура СУБД, необходимая для ускорения поиска данных. В СУБД разных типов индексы выглядят по-разному.

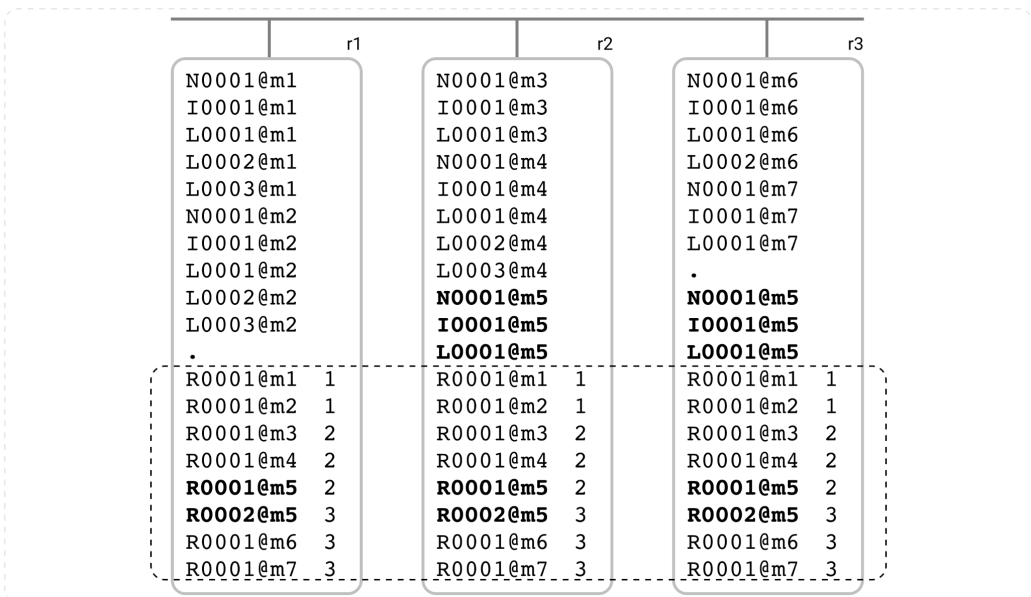


Рис. 12-05. Индексирование объектов

В распределенных конструкциях целесообразно указывать, на каком именно узле хранятся запрашиваемые данные, чтобы не выполнять перебор, приводящий к излишним нагрузкам на узлы и сеть. Как показано в примере (рис. 12-05), индексные переменные *R* реплицированы в режиме полной избыточности — об этом свидетельствуют переменные *R0001@m5*, *R0002@m5*. Данные, относящиеся к объекту *m5*, сегментарно реплицированы на узлы *r2*, *r3*.

12.1.5. Временные ряды

Когда нужно обеспечить хранение изменений данных во времени целесообразно использовать структуры для хранения данных временных рядов (*time series*). Временные ряды могут быть полезны для реализации краткосрочных, среднесрочных и долгосрочных архивов. При этом важно учитывать, что работа с временными рядами может быть организована в режиме РВ, но в такой ситуации структуры данных должны быть фиксированной длины. Обработку временных рядов СРВ и остальных лучше всего распределить между разными сервисами или микросервисами (разд. 11.2.4) таким образом, чтобы она выполнялась в разных ядрах или процессорах.

Хорошей практикой является организация хранения данных временных рядов в специализированных хранилищах, для этого существуют специальные решения, такие как СУБД для организации хранения временных рядов (TSDB¹⁵⁶). Многие современные решения содержат специальные встроенные структуры и сервисы для работы с временными рядами.

Существует специальный тип TSDB для хранения временных рядов с фиксированным количеством записей. Такие СУБД называют циклическими¹⁵⁷. Решения подобного рода оперируют циклическими структурами данных. Переключение между элементами, инкремент, осуществляется путем перезаписи

¹⁵⁶ Time Series Database, #12-05

¹⁵⁷ Round Robin Database, #12-06

первого элемента, который всегда является следующим по отношению к последнему.

Циклические СУБД часто содержат функцию записи через фиксированные временные промежутки. Для этого в таких решениях присутствует функционал ресемплирования (разд. 12.3.1). Например, если значение не изменилось, оно все равно будет перезаписано. В таком случае не будет затрачено время на поиск ближайших изменений. Такое свойство циклической СУБД позволяет эффективно и быстро осуществлять разнообразные вычисления благодаря одинаковой длине структур хранения и положению указателей. Описанные выше решения обеспечивают быстрый вывод данных на разнообразные тренды и диаграммы состояний, применяемые в системах диагностики.

Циклические структуры данных могут быть иерархически связаны между собой в соответствии с временным интервалом. Каждая следующая структура обеспечивает хранение обработанных данных, поступающих из предыдущей структуры. Обработка может включать дедупликацию, хранение только экстремумов, то есть применение большего временного интервала записи. Чем дальше от сырых данных, тем ниже частота хранения данных (рис. 12-06).

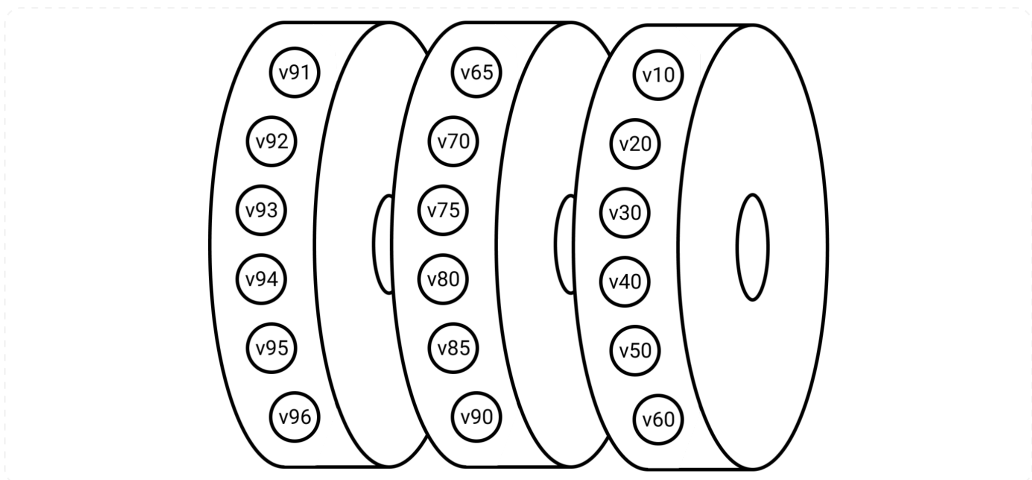


Рис. 12-06. Иерархическое циклическое хранение

12.2. Критические информационные системы

Современные АСУ, управляющие объектами КИИ, уже имеют высокий уровень автономности. Автоматизация является достаточно зрелой наукой. Со временем проблем становится все меньше, а решения существующих проблем прекрасно автоматизируются. Современное законодательство оперирует понятиями КИИ и ЗОКИИ. Государственные регуляторы уделяют пристальное внимание АСУ таких серьезных инфраструктур. Например, с точки зрения законодательства, банковские системы относятся к КИИ. Если такая система откажет, проблемы будут в основном организационного характера. Наиболее критичными являются вопросы, связанные с индустриальными системами управления. В связи с известными обстоятельствами последних лет российским разработчикам придется совершить настоящий прорыв в области создания новых продуктов автоматизации для достижения технологической независимости.

Когда говорят о промышленной автоматизации, очевидно подразумевают совокупность решений АСУ ТП, в состав которой входят РСУ, автоматические СУ, SCADA. Не следует забывать об управляющем ПО контроллеров, коммутаторов, разнообразных устройств интернета вещей. Когда речь идет об автоматизированной системе, это означает наличие оперативного управления, то есть оператора. Если говорят об автоматической системе, это значит, что она работает автоматически, без участия человека.

Самыми сложными объектами управления являются опасные производства — например, химическое производство, генерация электроэнергии на АЭС или ГЭС, передача и распределение электроэнергии, железнодорожные перевозки и другие отрасли. Как известно, самой большой проблемой для таких объектов управления является человеческий фактор, то есть неправильные решения. Современная тенденция к автоматизации подобных объектов всегда предполагает единственный сценарий, а именно снижение нагрузки на оператора.

Большинство возможностей современных средств автоматизации необходимы не для оперативного управления, а для тонкой настройки объекта, тестирования работы до ввода объекта в эксплуатацию после строительства или ремонта.

В некоторых странах и городах успешно функционируют полностью автоматические поезда, метро. Управление маневренными режимами объектов генерации осуществляется с помощью функционально-группового управления — например, когда оператор одной командой выводит объект на номинальную мощность. Многие подсистемы объектов генерации работают автономно, не требуя непрерывной синхронизации с остальными подсистемами. Такие системы являются устойчивыми к разделению и слабоконсистентными.

В ближайшем будущем АСУ объектов генерации однозначно ждет усиление устойчивости подсистем к разделению, которое существенно снизит нагрузку на оператора и превратит его в супервайзера (наблюдателя). Сначала оператор будет выведен за пределы энергоблока и начнет управлять всем объектом целиком — это произойдет примерно в 2035 году. Затем будут созданы единые резервированные диспетчерские центры, размещаемые за пределами объекта управления. Они будут обеспечивать централизованное управление объектами генерации следуя глобальному плану энерговыработки — этого следует ожидать примерно к 2050 году. К концу века такие объекты станут полностью автоматическими, синхронизируя планы по выработке электроэнергии, а человек будет отстранен от управления. Если задуматься, такие системы уже создают — например, системы управления первой АЭС с промышленным термоядерным реактором ИТЭР¹⁵⁸.

Наиболее ответственно нужно подходить к автоматизации объектов непрерывного производства, остановка которых может привести как к экономическим потерям и излишнему износу сложного в производстве оборудования, так и к проблемам энергоснабжения инфраструктурных объектов.

¹⁵⁸ ITER, #12-07

12.2.1. АСУ непрерывного производства

Для того, чтобы обозначить основную задачу управления и ее решения, рассмотрим следующий пример (рис. 12-07).

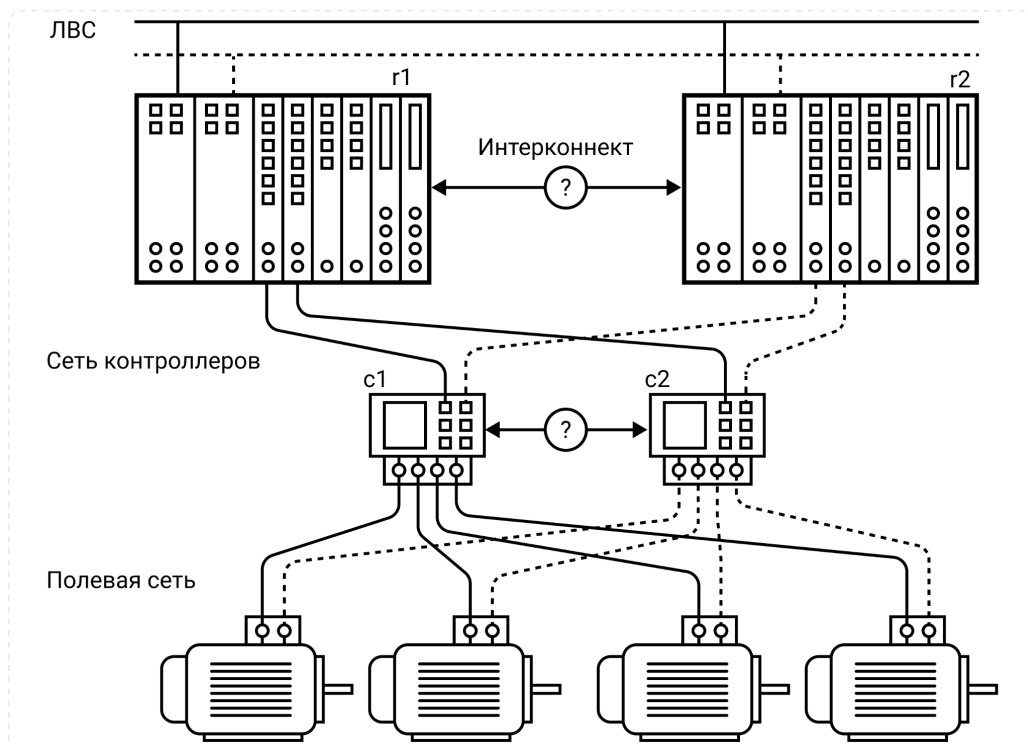


Рис. 12-07. Схема резервирования многоуровневая

Корневые контроллеры $r1$, $r2$ обслуживают контроллеры сопряжения $c1$, $c2$. Эти контроллеры могут управлять оборудованием — например, двигателем сервопривода задвижки. Современные системы управления устроены иерархически, а контроллеры сопряжения знают все о своем объекте управления и о конфигурации двигателя: параметры конечных положений, скорость вращения, время вращения между конечными положениями. Кроме того, контроллер сопряжения знает все о текущем состоянии объекта управления: находится ли двигатель под управлением или в ожидании команды, физическое положение задвижки и, возможно, абсолютное значение положения в процентах. Представим себе, что контроллер $c1$

отказал в процессе управления, поэтому контроллер $s2$ должен взять на себя управление и продолжить операцию. Это значит, что между контроллерами $s1$ и $s2$ должен быть консенсус. Большинство сервоприводов поставляют со встроенными контроллерами, встречаются и варианты с двухканальными контроллерами, однако это не является полноценным резервированием. На ответственных участках такие конструкции собирают под целевой проект. В подобных ситуациях корневые (шлюзовые, управляющие) контроллеры знают об объекте управления намного меньше: им известно положение задвижки и то, находится ли объект в режиме управления или ожидания. Этой информации достаточно для управления. Возникает вопрос: кто должен управлять консенсусом контроллеров сопряжения $s1$, $s2$? Они синхронизируются между собой, или их синхронизацией должны управлять корневые контроллеры ($r1$, $r2$)? Редко встречаются решения с прямой синхронизацией контроллеров сопряжения. Сложно представить, как ими всеми управлять и как обслуживать, если контроллеров достаточно много и они еще и изготовлены разными производителями.

Современные тенденции автоматизации развиваются в двух направлениях. Первая тенденция такова: происходит стандартизация протоколов диагностики, управления, конфигурирования контроллеров, что позволяет выстроить единую нагроможденную конструкцию. Вторая тенденция, более понятная, — это максимальное упрощение контроллеров уровня сопряжения и передачи всей информации об объекте управления корневым контроллерам ($r1$, $r2$).

В первом случае имеют место огромное разнообразие взаимодействий между контроллерами разного уровня и размазывание логики по контроллерам, а во втором случае — унифицированная стандартизация и вынос всей логики управления и конфигурации в единый слой, где могут быть реализованы сценарии резервирования, переключения, диагностики,

деградации, безопасности, изоляции, мультитенантности и другие полезные функции.

Есть мнение, что аппаратные контроллеры надежнее, что не стоит выносить логику управления на более высокий уровень. Такие суждения обусловлены прежде всего привычками автоматизаторов. В программно-логических контроллерах все равно спрятана масса программного кода. Этот программный код может отличаться по стилю написания, кодогенерации, используемым средствам программирования, потому что в больших РСУ всегда присутствует «зоопарк» решений. Перенос логики на контроллеры (узлы) более высокого уровня позволяет создать единый логический слой (разд. 11.1.5). Такой слой может оперировать единым форматом данных (разд. 3.6), обслуживаться привычными языками программирования. Подобная конструкция также позволит прозрачно осуществлять развертывание и контроль версий, что существенно упростит управление жизненным циклом инфраструктуры (разд. 10.5).

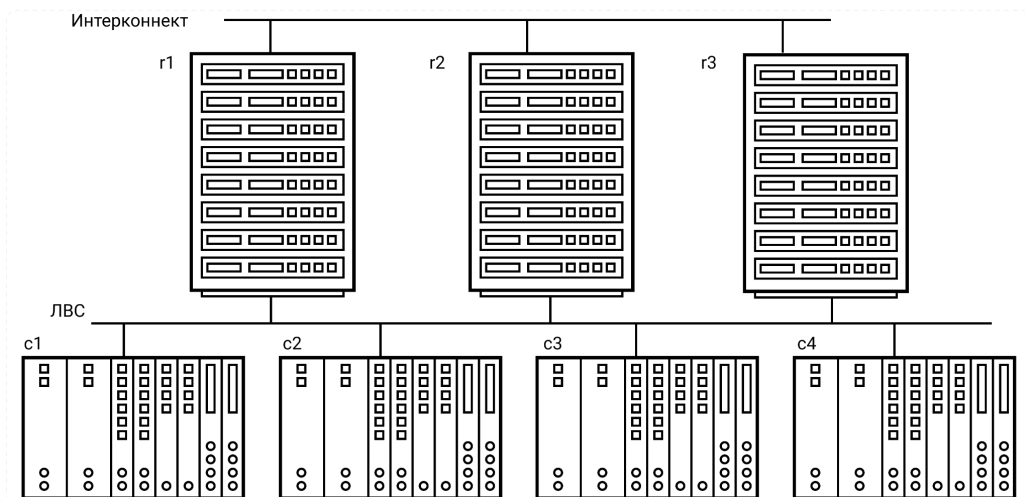


Рис. 12-08. Схема резервирования верхнего уровня

Существуют достаточно мощные контроллеры (рис. 12-08), управляемые современными процессорами, но для РСУ их может быть недостаточно. По этой причине в качестве корневых узлов

логично использовать производительные серверы (r), которые могут быть объединены в кластер. В кластер можно объединить и контроллеры (c). В зависимости от требований к целевой системе корневые контроллеры (c) и серверы (r) могут быть объединены одним интерконнектом или размещены в сети иначе.

Такая конструкция позволяет создавать масштабные РСУ с глобальным резервированием. Рассмотрим управление генерацией электроэнергии (рис. 12-09) на нескольких энергоблоках ($e1$, $e2$, $e3$). Распределенный кластер управления (r) состоит из корневых узлов $r11$ - $r24$. Клиентами кластера являются пограничные $edge$ -узлы $e11$ - $e23$. Пограничные $edge$ -узлы обслуживают корневые узлы подсистем $c11$ - $c34$, они объединены в локальные специализированные кластеры.

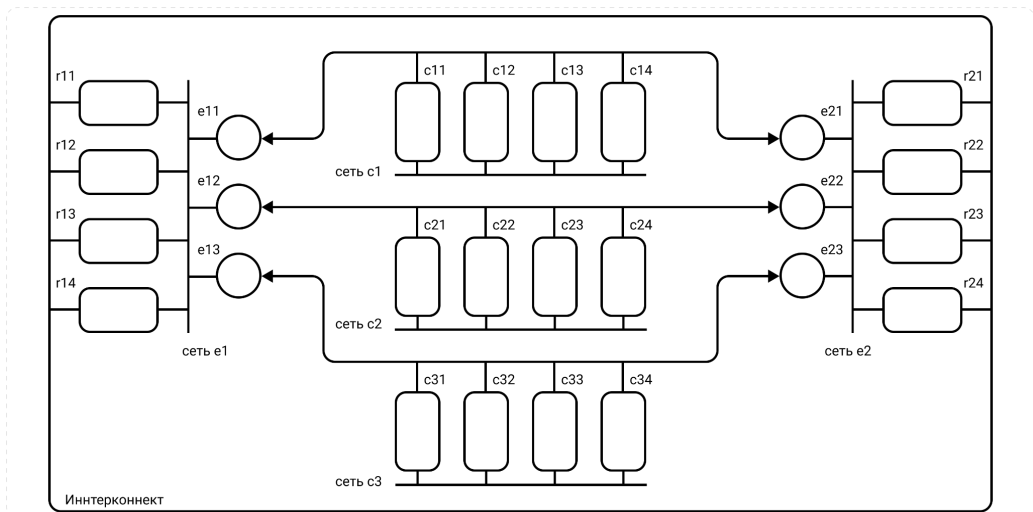


Рис. 12-09. Схема резервирования глобальная

Подобная конструкция позволяет вынести в единый плоский информационный слой $edge$ -кластера (e) информацию о локальной системе управления каждого из энергоблоков, синхронизировав ее с общей РСУ на основе кластера r . Узлы r , будучи консистентными по отношению к узлам e , могут выполнять множество полезных функций. Помимо функции резервирования вычислений (алгоритмов), узлы могут выполнять дополнительные

полезные расчеты — например, задачи сравнения с эталонными моделями на основе текущих показаний в связи с их доступностью в режиме РВ. Могут быть выполнены расчеты предиктивной аналитики, машинного обучения моделей на основе расхождения прогнозной модели с реальным поведением объекта как обратной связи управляющего воздействия. К такой инфраструктуре в нужный момент времени могут подключаться операторы, технологи, отвечающие за конкретный процесс наладки или маневра. Рассмотренная схема несколько отличается от классической схемы управления энергоблоком, однако именно она станет основной при конструировании новых объектов генерации ближе к 2035 году — хотя бы потому, что даже два диспетчерских центра (ДЦ, пульта управления) на три энергоблока (как на схеме выше) намного дешевле трех ДЦ. Именно в этом направлении двигаются современные РСУ непрерывных производств. Но РСУ дискретного производства не отстают.

12.2.2. АСУ дискретного производства

Дискретное производство, конвейерное производство, отличается от непрерывного тем, что конвейер можно остановить для замены производимой продукции. Основная сложность управления дискретным производством заключается в обслуживании такого же «зоопарка» оборудования, который отличается от оборудования для непрерывного производства тем, что его нужно перенастраивать «на лету». Любой сбой приводит к простою конвейера и, как следствие, к экономическим потерям. Отличие РСУ непрерывного производства от РСУ дискретного производства состоит в том, что вторые должны быть консистентны в момент конфигурации и перенастройки конвейера. При работе в режиме выполнения задания такие системы обычно устойчивы к разделению.

12.2.3. Цифровой двойник

С точки зрения управления цифровой двойник (ЦД) — это совокупность знаний об объекте и технологий автоматизации, позволяющая осуществлять цифровые (виртуальные) воздействия на объект (эксперименты) в целях выработки оптимальной функции управления и повышения его эффективности. Часть узлов РСУ или кластера можно выделить под расчетные модели цифрового двойника автоматизируемого производства. Как следует из примера (разд. 12.2.1), модели ЦД можно подгрузить в нужный момент — например, при маневрировании или испытаниях оборудования конкретного энергоблока, пока остальные работают в режиме нормальной эксплуатации. Основным инструментом ЦД АСУ является предиктивная аналитика.

12.2.4. Предиктивная аналитика

Предиктивная аналитика — это совокупность инструментов моделирования состояния объекта управления в режиме РВ на основании данных, получаемых в ходе его эксплуатации. Любое моделирование может включать массу созависимых физико-математических моделей — например, гидродинамическую, теплофизическую, нейтронно-физическую, химическую, радиационно-химическую, прочностную и другие модели. Эффективная среда предиктивной аналитики должна уметь использовать такие модели с поправкой на необходимость обсчитывать модели как в режиме РВ, так и в режимах, превышающих РВ. Для этого нужны достаточно серьезные вычислительные ресурсы. Строить инфраструктуру предиктивных вычислений, например, для каждого энергоблока, достаточно затратно. Как и в предыдущем примере, модели предиктивной аналитики, зачастую являющиеся частью ЦД, могут быть загружены и синхронизированы с объектом управления в момент маневрирования или возникновения иной сложной ситуации, которая требует модельной верификации для поддержки принятия решения в задачах управления.

12.2.5. Граничные и туманные вычисления

Граничные вычисления — это концепция выполнения вычислений на обладающих избыточной мощностью граничных узлах, наиболее близких к потребителю. Граничные вычисления обеспечивают снижение вычислительной нагрузки на ЦОД. В связи с тем, что граничные узлы обычно расположены ближе к потребителю, их латентность существенно снижена. На них можно провести промежуточные вычисления, которые должны быть выполнены в выделенном сегменте сети. Например, в РСУ АСУ ТП на граничных узлах сопряжения принято осуществлять разнообразные проверки и преобразование данных (протоколов) для снижения нагрузки на целевую систему.

Туманные вычисления — это концепция, близкая к граничным вычислениям. Отличие заключается в том, что вычисления осуществляются не на граничных, а на дополнительных узлах, которые расположены между потребителем и ЦОД. Как правило, граничные узлы присутствуют в системе как разнообразные шлюзы, они могут нести полезную нагрузку. Туманные узлы должны быть заранее спланированы в проекте целевой системы. Они могут быть нагружены в зависимости от ситуации и целевой задачи — например, именно туманные узлы могут нести нагрузку, обеспечивая вычисления, связанные с обслуживанием моделей цифровых двойников и предиктивной аналитики.

Важной задачей является управление заданиями и нагрузкой граничных и тем более туманных узлов. Описанные в книге подходы к созданию РСУ подходят для решения этой задачи.

12.2.6. Интернет вещей

Интернет вещей (IoT), промышленный интернет вещей (IIoT) динамично развиваются. Драйвером этих технологий является прежде всего развитие беспроводных технологий — 5G, 6G и других. Платформам сервисного обслуживания устройств IoT, IIoT требуется серьезная многосвязная распределенная инфраструктура.

Отрасль интернета вещей развивается за счет облачных технологий, граничных и туманных вычислений, виртуальных сетей – как классических, так и слабосвязанных *mesh*-сетей. Разнообразные платформы интернета вещей появляются как грибы после дождя, и их будет становиться все больше. Каждая платформа по-своему уникальна, она рассчитана на свой круг задач. Иногда выгоднее собрать (разработать) свою платформу, наделив ее требуемым функционалом. Рассмотренные подходы могут стать основой архитектуры такой платформы.

12.3. Обработка данных

Важной задачей в РСУ является обработка данных. Нехватка данных или их излишество могут повлиять на эффективность работы целевой системы. Приведение данных в порядок может значительно повысить эффективность и надежность РСУ, а также существенно упростить архитектуру.

Стратегия обработки данных может быть весьма разнообразной. Может требоваться предварительная обработка данных (предобработка), промежуточная или последующая обработка (постобработка). Обработка данных может осуществляться невзирая на высокую нагрузку системы или в промежутки ослабления нагрузки.

12.3.1. Обработка временных рядов

В ВНС и особенно в СРВ, основная нагрузка обычно наблюдается при обработке временных рядов (разд. [12.1.5](#)). Чем меньше данных системе придется обрабатывать, тем меньше будет нагрузка на систему. Существует ряд подходов к снижению нагрузки на систему, упрощению ее работы с данными, которые могут противоречить друг другу.

Самый очевидный способ снизить нагрузку на систему – отфильтровать лишние данные. Фильтрация (прореживание) данных (рис. 12-10) является эффективным способом снижения

нагрузки на процессы записи и чтения данных. Однако сам процесс фильтрации, очевидно, вызовет дополнительную нагрузку на систему.

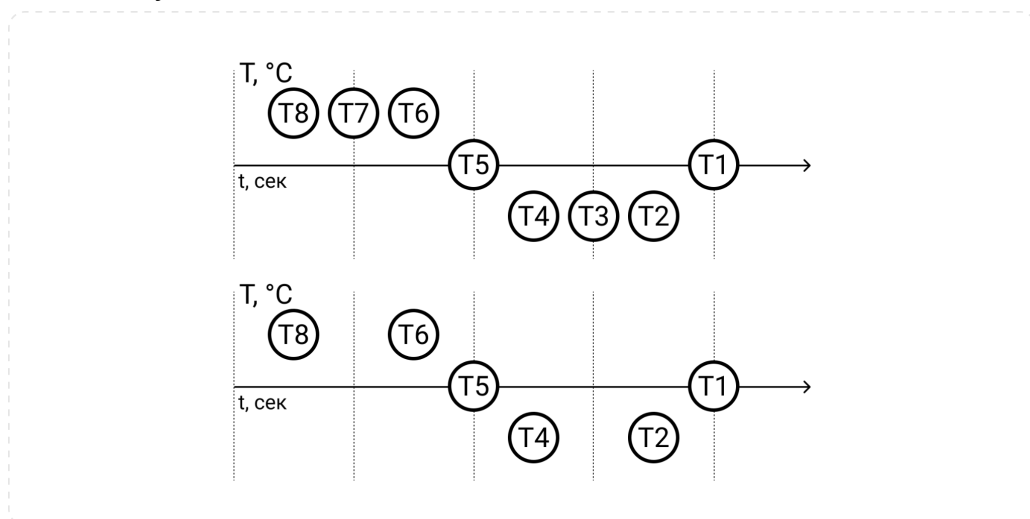


Рис. 12-10. Фильтрация данных

В РСУ данные зависимых изменений могут поступать с разными частотами, однако для их обработки полезно произвести выравнивание относительно требуемой частоты. Для такого выравнивания частот применяют подход, называемый ресемплированием данных.

Ресемплирование данных хорошо работает для временных рядов, отражающих протекание физических процессов, кроме высокочастотных. РСУ, оперирующие с высокочастотными процессами, характеризуются большими объемами данных. Когда нужно не потерять важные данные, в таких системах хорошо работают алгоритмы ресемплирования.

Ресемплирование временных рядов может осуществляться с применением различных стратегий, направленных на фильтрацию данных или, наоборот, на дополнение отсутствующих данных (рис. 12-11).

Ресемплирование может подразумевать комплексное смещение данных относительно требуемых опорных координат (частот)(рис. 12-12).

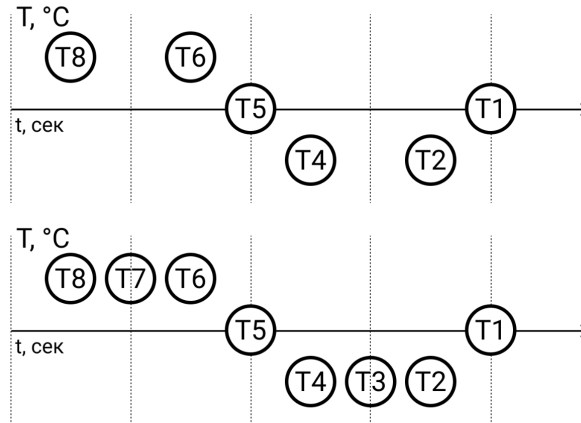


Рис. 12-11. Дополнение данных

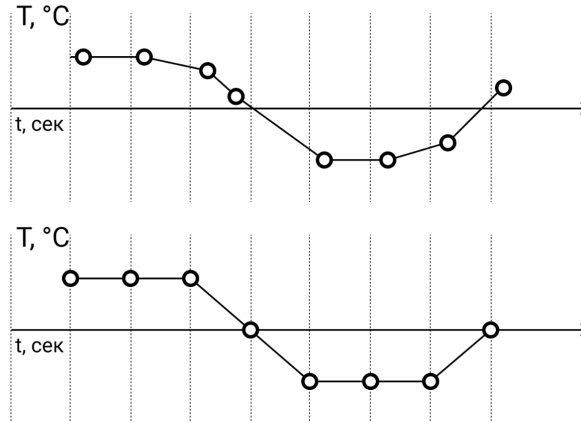


Рис. 12-12. Выравнивание данных

Алгоритмически ресемплирование может опираться как на статистические законы, так и на законы физики для более точного восстановления данных. Ресемплирование может быть весьма затратным применительно к ресурсам процессов, поэтому может осуществляться в периоды низких нагрузок на систему.

В зрелых РСУ всегда применяется ресемплирование, оно полезно прежде всего для работы с архивными данными. При запросе в архив сразу будут выданы данные, сопутствующие запрашиваемому срезу или диапазону с доступными частотами.

Важно отметить, что в случае ресемплированных данных частоту дискретизации запроса можно эффективно регулировать, по мере необходимости подгружая более детальные данные между временными срезами. Когда данные не ресемплированные, часто приходится запрашивать весь срез. РСУ, использующие в своей работе ресемплированные данные оперативного архива для выполнения задач РВ, например, предиктивной аналитики, очевидно будут более производительными.

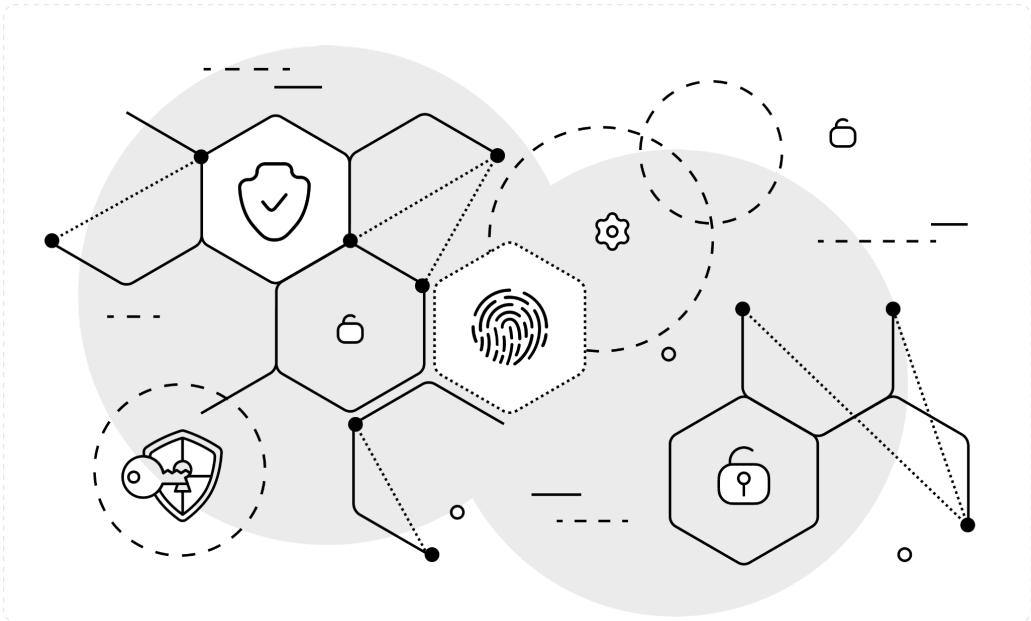
12.3.2. Маркировка данных метками времени

В РСУ важнейшим параметром является время возникновения данных, описывающих то или иное событие или состояние объекта управления. В процессе передачи данных между компонентами системы могут происходить задержки, обусловленные расстоянием между узлами, их нагруженностью и приоритизацией обработки данных узлами (разд. 7.1.4).

Для того, чтобы оценить актуальность данных, их полезно маркировать временными метками¹⁵⁹ на ключевых узлах прохождения данных. Например в РСУ полезно знать время появления данных, время поступления данных на узлы, в которых осуществляется обработка данных. В некоторых системах принято ставить метку времени, когда данные теряют свою актуальность.

Проставление меток времени очевидно усложняет структуру данных, увеличивает их объем и нагрузку на их обработку. Однако множество меток времени позволяет эффективно диагностировать РСУ и оптимизировать, балансировать (Глава 8) ее работу.

¹⁵⁹ Timestamp, #12-08



Глава 13. Кибербезопасность РСУ

Термин «кибербезопасность» не закреплен в каких-либо официальных документах. В настоящем тексте данный термин следует понимать как подмножество дисциплин, изучаемых наукой «информационная безопасность (ИБ)», гармонизированное и не противоречащее требованиям промышленной и функциональной безопасности. Эти требования очень важны для РСУ технологических и производственных процессов. Западные издания пользуются терминами *cybersecurity* и *computer security*, которые наиболее точно отражают смысл термина КБ.

Разделяют подходы к обеспечению КБ в области информационных и операционных технологий (ИТ и ОТ *cybersecurity*). Важно отметить, что РСУ однозначно следует в большей степени отнести к операционным технологиям, чем к информационным, даже в том случае, когда РСУ является составной частью ИТ инфраструктуры (а не промышленной системы), например, сервисной платформой, обслуживающей ИТ.

Такой подход не отменяет необходимость обеспечения КБ объектов РСУ применительно к обеим технологиям.

Задачей КБ является обеспечение непрерывности, безопасности и эффективности технологических и производственных процессов. КБ представляет собой набор средств, стратегий, принципов обеспечения безопасности, гарантий безопасности, подходов к управлению рисками, действий, профессиональной подготовки, страхования и технологий, которые используются для защиты ресурсов защищаемых объектов. Отдельно выделяют объекты критической информационной инфраструктуры (КИИ), в том числе значимые (ЗОКИИ), к РСУ которых предъявляют высокие требования в контексте КБ.

КБ подразумевает достижение и сохранение свойств киберзащищенности РСУ, направленных против соответствующих киберугроз. При обеспечении КБ основной задачей является обеспечение доступности, целостности, достоверности и конфиденциальность данных.

Для обеспечения защиты РСУ необходимо определить источники угроз, выявить уязвимости компонентов. Необходимо оценить векторы атак на РСУ и спланировать стратегию защиты.

13.1. Угрозы в РСУ

Выделяют угрозы целостности, доступности и конфиденциальности информации. Для РСУ угрозы конфиденциальности являются несущественными. Если произойдет утечка данных, РСУ функционировать не перестанет. А вот если произойдет нарушение целостности (разд. 6.6) и доступности (Глава 7) компонентов РСУ, то для автоматизации КИИ это будет существенной проблемой. По этой причине именно эти угрозы будут рассмотрены в настоящей книге.

Современные РСУ состоят из огромного количества разнообразного оборудования и ПО. РСУ крупных промышленных

предприятий могут управлять сопряжением оборудования сотен поставщиков, обеспечивать взаимодействие с десятками внешних программных систем. Каждое обслуживаемое РСУ устройство, сопрягаемое ПО, является потенциальным источником угроз в РСУ.

В жизненном цикле (разд. 10.5) объекта автоматизации доступ к поставляемому оборудованию и ПО имеет непредсказуемое число субъектов. Таким образом, количество техногенных и антропогенных источников¹⁶⁰ угроз весьма велико.

13.1.1. Техногенные источники угроз

Масштаб разнообразия оборудования и ПО в крупной РСУ может произвести впечатление. В качестве примера можно рассмотреть РСУ крупного промышленного предприятия. В составе такой РСУ обязательно будут представлены следующие компоненты:

- датчики и исполнительные механизмы;
- программно-логические контроллеры (ПЛК);
- устройства промышленного интернета вещей;
- оборудование управления сетями: коммутаторы, маршрутизаторы;
- серверное оборудование;
- клиентские устройства;
- устройства коллективного пользования: экраны, панели;
- устройства ввода: функциональные клавиатуры, панели;
- периферийное оборудование.

Оборудование содержит массу системного ПО, такого как:

- прошивки контроллеров;
- прошивки сетевых устройств;
- прошивки серверного оборудования: БСВВ¹⁶¹, ВМС¹⁶², прошивки серверных сетевых устройств;

¹⁶⁰ Классификация источников угроз, #13-01

¹⁶¹ Базовые средства ввода-выводы, #13-02

¹⁶² Baseboard Management Controller, #13-03

- прошивки клиентских устройств: БСВВ, прошивки клиентских сетевых устройств;
- микропрограммы процессоров;
- микропрограммы дисков HDD, SSD;

Функционирование общего ПО обеспечивает следующее оборудование:

- загрузчики;
- операционные системы (ОС);
- драйверы встраиваемых устройств;
- СУБД и шины данных;
- серверы преобразования протоколов (OPC, MQTT и др.).

PCY обычно содержит разнообразное прикладное ПО, системы диагностики, мониторинга, управления, бизнес-приложения. Оборудование и ПО, составляющие часть PCY, поступают от самых разных вендоров в разнообразных сочетаниях и чередованиях всего вышеперечисленного. Вместе они могут составлять десятки тысяч устройств и программных компонентов. Со временем часть оборудования отказывает, устаревает, аппаратные и программные компоненты проходят модернизацию. Каждый из указанных компонентов PCY может быть потенциальным источником угроз. Источниками угроз могут быть сама координирующая система PCY и ее компоненты. Самой большой проблемой является обеспечение доверия всех вышеуказанных компонентов (разд. [13.4.1](#)).

13.1.2. Антропогенные источники угроз

Антропогенные угрозы, или человеческий фактор, являются наиболее непредсказуемыми угрозами. В целом, с точки зрения расположения угроз относительно объекта автоматизации выделяют внешние и внутренние источники.

Внутренний источник может получить доступ к оборудованию и ПО. Он может внести в него изменения, которые приведут к неприятным последствиям. Внешний источник точно так же может получить доступ к оборудованию. Важно отметить,

что если внешний источник внес изменения в компоненты РС на этапе поставки, то применительно к этим компонентам угроза становится техногенной.

13.2. Уязвимости в РСУ

Уязвимости компонентов РСУ классифицируют относительно источника угроз. Разделяют уязвимости относительно техногенного и антропогенного, внешнего и внутреннего источников.

Что касается человеческого фактора, тут все понятно. Внешние и внутренние нарушители могут воспользоваться уязвимостями в технических средствах, и остановить их не так просто. Следовательно, важно устранить все возможные источники техногенных угроз или свести их к минимуму.

Уязвимости могут таиться в любом компоненте РСУ (разд. 13.1.1). Их можно классифицировать¹⁶³ как непреднамеренные и преднамеренные. Разработчик технических средств может намеренно заложить в архитектуру условия для возникновения уязвимостей, а может случайно пропустить ошибку, которая впоследствии может быть использована злоумышленником.

Жизненный цикл компонентов РСУ подразумевает этапы тестирования, которые включают разнообразные методы исследования технических средств на уязвимости.

13.2.1. Методы исследования уязвимостей

Уязвимости оборудования и ПО выявляются при проведении обычного тестирования¹⁶⁴, а именно модульного, интеграционного, функционального, сквозного, нагрузочного, регрессионного и некоторых специальных видов тестирования.

¹⁶³ Классификация уязвимостей, #13-04

¹⁶⁴ Виды тестирования, #13-05

Применяют различные специализированные виды тестирования, предназначенные для выявления уязвимостей. Тестирование на проникновение¹⁶⁵, заключающееся в целевом воздействии на компоненты РСУ всеми доступными методами, также предполагает анализ реакции.

Поиск уязвимостей может сопровождаться исследованием документации и исходного кода объекта исследования для поиска опасных конструкций, приводящих к их появлению. Это упрощает формулирование условий для тестирования на проникновение. Такое тестирование называют тестированием белого ящика. Если документация и исходный код недоступны, то подобное тестирование называют тестированием черного ящика. Если доступна только часть документации и исходного кода, такое тестирование называют тестированием серого ящика. В случае особой необходимости может быть проведен реверс-инжиниринг, который может заключаться в дизассемблировании ПО, исследовании аппаратных компонентов, вплоть до восстановления топологии микропроцессоров. Такой процесс является невероятно длительным и дорогостоящим.

Уязвимости могут проявляться в режиме сопряжения компонентов РСУ. Для выявления таких уязвимостей исследуют генерируемый сетевой трафик, моделируют тестовый трафик, который может вызвать целевое исследуемое поведение компонентов РСУ. Часть таких исследований можно автоматизировать, их следует периодически проводить в автоматизированном или автоматическом режиме.

Не стоит забывать, что когда объект является целью для атаки, такие исследования проводят как специализированные лаборатории, так и лаборатории злоумышленников. Возникает закономерный вопрос: кто окажется первым?

¹⁶⁵ Penetration test, [#13-06](#)

13.3. Методы проведения атак РСУ

Атаки на РСУ, обслуживающие объекты КИИ отрасли, могут готовиться десятилетиями. В подготовке таких атак могут участвовать целые специализированные институты. В программах подготовки целевых атак могут быть задействованы фабрики, интегрирующие уязвимости в технические средства. Для сокрытия таких уязвимостей могут быть затрачены колоссальные ресурсы. В подобных программах могут участвовать международные корпорации, разведслужбы государств. Исследования такого рода уязвимостей на порядки дороже, чем их инкапсуляция в поставляемые системы, они практически недоступны большинству государств. Подобные атаки известны, они проводились множество раз. Как правило, все такого рода атаки осуществляются в отложенном режиме, без участия человека, так как целевые РСУ, обслуживающие объекты КИИ, тщательно изолированы от внешнего воздействия.

Целями или источниками атак могут быть любые технические средства, компоненты РСУ (разд. [13.1.1](#)). Следовательно, основным методом атаки является эксплуатация уязвимостей приведенных выше компонентов РСУ. Такая эксплуатация может осуществляться в автоматическом режиме или посредством воздействия человека.

13.3.1. Виды атак

Существует множество видов атак, способных отрицательно повлиять на РСУ. Значимой угрозой для РСУ является нарушение целостности и доступности. Нарушение доступности может повлечь за собой нарушение целостности (консистентности, разд. [6.6](#)) данных целевой системы. Атакована может быть часть локальных узлов, а подсистемы РСУ могут не получать данные из атакуемых сегментов.

Любая РСУ может включать огромное число разнообразных объектов, компонентов, подсистем, влияющих на работу РСУ.

Данные об этих компонентах могут быть полезны для проведения разнообразных атак, среди них можно выделить данные о:

- конфигурации конечного оборудования;
- сопряжении подсистем и компонентов;
- источниках измерений и средствах управления;
- технологических сигналах, их состоянии;
- характере изменений состояния;
- технологических границах сигналов;
- диагностических сообщениях и тревогах;
- управляющих командах;
- алгоритмах и сценариях управления;
- конфигурации средств защиты информации;
- конфигурации средств управления ИБ.

Понимая логику работы целевой системы, обладая знаниями о деталях ее конфигурации, злоумышленник может нанести серьезный урон целевой РСУ.

По времени воздействия атаки делят на активные и отложенные (пассивные). Зрелые РСУ обычно содержат средства диагностики и управления информационной безопасностью (СУИБ). Поэтому активные атаки, как правило, так или иначе немедленно становятся заметны сразу же.

С отложенными атаками дело обстоит сложнее. Источники проведения отложенных атак могут быть размещены в РСУ на любом из этапов жизненного цикла целевой системы. Такие атаки могут быть незаметны до тех пор, пока не будут активированы.

Отложенные атаки могут быть активированы по условию. Таковым может быть встроенный таймер, команда злоумышленника или определенное состояние целевой РСУ. Например, атака может быть проведена в процессе маневрирования ядерной энергетической установкой.

Активные атаки могут быть незаметны, их результатом может стать то, что РСУ будет вести себя неправильно длительное

время так, что сама система управления может незаметно навредить объекту автоматизации. Такие атаки могут вызвать преждевременную деградацию (разд. 7.3) оборудования объекта автоматизации и в итоге привести к отказу ее подсистем.

По количеству источников воздействия атаки делят на точечные и распределенные. Очевидно, что точечная атака на РСУ не способна нанести ей существенный вред, если только она не направлена на критически важную подсистему РСУ, такую как система безопасности управления. Распределенные атаки на РСУ являются самыми неприятными.

Деградация РСУ, связанная с проведением активной распределенной атаки, может быть идентифицирована. А отложенные распределенные атаки являются самыми сложными в идентификации и при этом самыми эффективными, хотя их сложнее всего реализовать.

Отложенная распределенная атака на РСУ может в один момент привести к молниеносной активации всех заложенных в подсистемы РСУ компонентов атаки. В момент проведения атаки все такие компоненты становятся активными, они могут мгновенно повредить работу РСУ, обслуживающей целевой объект автоматизации. Отложенная атака, влияющая на изменение поведения РСУ, может привести к непредсказуемому результату управления целевым объектом автоматизации. В такой ситуации управление технологическим процессом может быть действительно утрачено. Отложенные распределенные атаки особенно опасны для РСУ, обслуживающих объекты КИИ, нарушение работы которых может привести к катастрофическим последствиям.

Про большинство видов атак написано достаточно статей и книг. Важно обратить внимание на один из самых сложных в реализации методов атак, заключающийся в изменении логики РСУ целевого объекта автоматизации.

13.3.2. Жизненный цикл атак

Подготовка к атаке на РСУ может быть проведена на любом из этапов жизненного цикла (ЖЦ) целевой системы. Подготовка к атаке, а точнее инкапсуляция компонентов атаки в целевую систему, может быть осуществлена на следующих этапах:

- проектирование;
- пусконаладка;
- ввод в эксплуатацию;
- эксплуатация;
- вывод из эксплуатации для ремонта;
- плановый ремонт;
- ввод в эксплуатацию после планового ремонта;
- вывод из эксплуатации опасных производств.

Самым эффективным способом подготовки к проведению целевой атаки считается подготовка атаки на этапе проектирования системы.

На этапе проектирования злоумышленник может осуществить поставку средств проектирования и разработки, содержащих ошибки в расчетных кодах моделирования систем целевого объекта автоматизации. Неправильные расчеты могут привести к преждевременной деградации важных для эксплуатации узлов целевого объекта автоматизации. Например, «плохо» рассчитанная газовая турбина энергоблока может развалиться. Такого рода последствия могут и не привести к катастрофе, однако объекты генерации могут прекратить выработку электроэнергии, что сильно повлияет на экономику региона.

Злоумышленник может быть осведомлен о деталях устройства алгоритмической логики управления объектом автоматизации. В этом случае злоумышленник может повлиять на проектируемую систему таким образом, чтобы в сочетании с применяемой алгоритмической логикой управления нанести

максимальный урон объекту автоматизации с помощью вроде бы доказанно доверенной РСУ в режиме эксплуатации.

Атаки уровня проектирования сложны. Они требуют длительной подготовки, колоссальных ресурсов, серьезной аналитики и вовлечения сети агентов влияния. Подобные атаки имели место в истории, а их последствия были весьма серьезными. Подготовить такие атаки могут крупные распределенные хакерские группировки — как с целью нарушения работы целых отраслей промышленности, так и дискредитации конкурирующих технологий.

13.3.3. Логические атаки

Опасным видом атак являются атаки, эксплуатирующие внутреннюю логику работы РСУ, такие как атака, эксплуатирующая способность РСУ навредить самой себе. Подобная атака может создать условия, при которых в РСУ будет генерироваться слишком много диагностических сообщений, которые могут лавинообразно «забить» используемые в РСУ важные каналы связи. Лавина может вызвать отказ в обслуживании. Результатом может стать необходимость остановки объекта автоматизации до устранения причин.

Логическая атака может вызвать изменение алгоритмов управления целевым объектом автоматизации и нанести ему существенный ущерб.

13.4. Методы защиты РСУ

Методы защиты РСУ от целевых атак весьма разнообразны. Важным условием успеха защиты РСУ является выработка моделей защиты целевого объекта автоматизации на каждом из этапов ЖЦ. На каждом из этапов ЖЦ объекта автоматизации следует внимательно оценить доверие к применяемым компонентам РСУ. Каждый этап ЖЦ позволяет выработать иной подход к оценке доверия к компонентам РСУ.

Важно определить степени доверия к компонентам РСУ и понять, как его можно повысить на этапах ЖЦ, на которых потенциальная атака может нанести существенный урон. Важно выработать культуру инвентаризации, чтобы было понятно, какие компоненты должны быть оценены с точки зрения доверия.

Особое внимание необходимо уделить блокировке недоверенных компонентов на изменение конфигурации, для чего могут быть эффективно применены методы доверенной загрузки компонентов РСУ.

Не следует забывать, что распределенная система обладает существенными преимуществами перед централизованными системами с точки зрения защиты. Распределенная система может демонстрировать технологическое разнообразие компонентов, которое способно защитить объект автоматизации и существенно снизить вероятность успешной атаки. Эту особенность РСУ рекомендуется использовать при проектировании управляющих систем и методов защиты управления.

13.4.1. Понятие доверия в РСУ

Доверие РСУ, очевидно, определяется совокупностью доверенных узлов. Возникает вопрос, считать ли систему доверенной, если один из узлов подвергся негативному воздействию.

Состояние (степень) доверия к РСУ можно классифицировать как консистентность (разд. 6.6). В контексте такой классификации разумно рассмотреть следующие состояния доверия:

- строгое доверие к системе;
- слабое доверие;
- доверие в конечном счете;
- доверие по времени;
- строгое доверие в конечном счете.

Под строгим доверием к системе следует понимать абсолютное доверие к системе. Применять такую степень доверия к целевой системе весьма опасно.

Слабое доверие к целевой системе является весьма разумным выбором. В этом случае необходимо определить условия, которые должны быть выполнены, чтобы в конечном счете системе можно было доверять, или то, в какие временные интервалы система будет находиться в требуемом состоянии доверия.

13.4.2. Повышение доверия

Базовый метод обеспечения защиты РСУ заключается в повышении доверия к разнообразию компонентов аппаратного и программного обеспечения и к системе в целом. Иногда этот процесс выглядит как создание доверенной системы из недоверенных компонентов.

Самым простым шагом к повышению доверия является применение доверенного железа и ПО. Вопрос в том, на основе каких факторов делают вывод о доверии. Таких факторов может быть много, например, это может быть наличие сертификации регулятора по ИБ, репутация вендора или поставщика, а лучше все вместе.

Современные требования к кибербезопасности объектов КИИ включают в себя самые жесткие меры по повышению доверия. Требования подразумевают использование доверенных процессоров, что само по себе является невыполнимой задачей в масштабе.

На рынке существуют технические решения доверенных БСВВ для серверного и клиентского оборудования. Подобрать контроллер с доверенной прошивкой, который будет выполнять все требования целевой системы, не так просто.

Коммутаторы с доверенными прошивками широко представлены на рынке, но их стоимость существенно превышает стоимость обычных.

Существующее доверенное общее ПО, такое как ОС, СУБД, шины данных, в целом, является относительно доверенным. Какие бы ни осуществлялись исследования пакетов ПО, входящих, например, в распространенные, сертифицированные дистрибутивы на основе Linux, они не гарантируют абсолютной защиты компонентов PCY и всей системы в целом.

13.4.3. Инвентаризация

Современные PCY позволяют управлять подсистемами и отдельными объектами. Такие PCY обеспечивают разнообразные дополнительные второстепенные функции. Например, для проведения диагностики новых объектов их прежде всего необходимо зарегистрировать в системе. Подобные действия уже являются некой формой инвентаризации.

Зрелые системы позволяют вести учет изменений состояния и конфигурации объектов PCY, а также подробный автоматизированный или полуавтоматизированный учет изменений. Если PCY умеет опрашивать конфигурации конечного оборудования, это существенно повышает надежность целевой системы. Можно отслеживать попытки несанкционированного изменения конфигураций оборудования.

Некоторые системы могут опрашивать конфигурации, управлять прошивками конечных устройств, осуществлять их обновление и конфигурирование.

13.4.4. Доверенная загрузка

Одним из методов защиты конфигурации объектов PCY является обеспечение доверенной загрузки аппаратных компонентов (разд. [13.1.1](#)).

Доверенная загрузка подразумевает использование специальных программных или аппаратно-программных средств на объектах, подразумевающих наличие конфигурации, прошивки, ОС. Средства доверенной загрузки позволяют осуществить загрузку устройства, чтение конфигурации из доверенного, защищенного хранилища. Такое хранилище в самом простом варианте реализации может быть неперезаписываемым или криптографически защищенным.

При изменении конфигурации или прошивки средства доверенной загрузки могут идентифицировать несанкционированные изменения конфигурации оборудования и сигнализировать об этом диагностической подсистеме РСУ, таким образом оповестив оператора.

13.4.5. Распределенность как метод защиты

Хорошо спроектированные РСУ, как правило, устойчивы к разделению и могут находиться в таком состоянии длительное регламентное время, достаточное для устранения неприятности, приведшей к нарушению консистентности. Некоторые РСУ могут работать в режиме длительной неконсистентности.

Если разделение системы и последующая утрата консистентности возникли в связи с тем, что система испытывает отрицательное воздействие (атаку), то атаку можно диагностировать. Диагностика подобной атаки вполне тривиальна, если при проектировании учесть возможность возникновения такого рода событий.

Локализация такой атаки может быть осуществлена путем принудительного отключения узла, в котором атака зафиксирована, чтобы остановить распространение последствий атаки на смежные узлы и подсистемы.

В любом случае РСУ должна уметь сохранять функции управления. Управление должно осуществляться в локальных,

отключенных от централизованного управления узлах, подсистемах. В подобных случаях могут оказаться полезными различные локальные моделирующие функции, прогнозирующие поведение централизованной системы.

Решение о принудительном отключении узла или подсистемы, системы должно быть принято на основании заранее определенной логики. Чаще всего применяют метод мажоритарной логики. Предположим, что малая часть узлов системы стала неконсистентной, количество неконсистентных данных аномально выросло. Такие данные претендуют на роль источника синхронизации, однако этот узел никогда раньше не был источником таких изменений. Таким образом, можно смело считать, что данный узел подвергся атаке и должен быть отключен или приведен в состояние разделения с остальными узлами системы. В ВНС подобные непрерывные проверки приведут к еще большей нагрузке на систему. Если безопасность системы является приоритетом функционирования системы, при проектировании такой системы придется запланировать такого рода ресурсы.

13.4.6. Защита управления

Отдельно следует уделить внимание защите РСУ при управляющих воздействиях. Когда система находится в состоянии слабого доверия, в интервалы управления следует отключать все лишнее. Все источники, не участвующие в процессе управления, которые могут вызвать лавину или лишнюю активность, должны быть заблокированы. Данные от таких источников не должны обрабатывать узлы, участвующие в управлении.

При наличии средств защиты информации РСУ может взаимодействовать с ними в режиме управления. В случае выявления атак или аномального состояния РСУ может остановиться, не осуществляя управление до устранения причин тревоги.

13.4.7. Типовые методы защиты

В РСУ могут быть применены самые разнообразные методы защиты информации, выявления и предотвращения атак. Возможно применение наложенных средств защиты, однако встроенные в целевую систему средства защиты всегда будут вести себя лучше.

Средства защиты сетей и конечных узлов рекомендуется интегрировать со средствами диагностики и оповещения целевой системы.

Применение средств криптографической защиты информации (СКЗИ) в РСУ полезно, если используются публичные каналы связи. При необходимости повсеместного применения СКЗИ в целевой системе лучше обеспечить соответствующую интеграцию.

Приложение 1. Принятые сокращения

Сокр.	Расшифровка	Альт.
ACID	atomicity, consistency, isolation, durability	
AMP	asymmetric multiprocessing	
BASE	basically available, soft state, eventually consistent	
CAP	availability, consistency, partition tolerance	
CRDT	Conflict-Free Replicated Data Type	
CRUD	create, read, update, delete	
DBMS	database management system	СУБД
DCS	distributed control system	PCY
DDoS	distributed DoS	
DoS	denial of service	
DRP	disaster recovery plan	
DRS	disaster recovery solution	
DSM	distributed shared memory	РРП
FIFO	first in, first out	
GbE	gigabit ethernet	
HDD	hard disk drive	
HLS	high load system	ВНС
IIoT	industrial IoT	
IoT	internet of things	
ITIL	IT infrastructure library	
LFU	least-frequently used	
LIFO	last in, first out	

Сокр.	Расшифровка	Альт.
LRU	least recently used	
MPS	messages per second	
MQCA	multi queue caching algorithm	
MRU	most recently used	
MSA	microservices architecture	MCA
MTBF	mean time between failures	
NUMA	non-uniform memory access	
NVMe	non-volatile memory express	
OTBF	operating time between failures	
RAID	redundant array of independent disks	
RAIM	redundant array of independent memory	
RAIN	redundant array of independent nodes	
RDMA	remote direct memory access	
RoCE	RDMA over converged ethernet	
RPO	recovery point objective	
RT	real time	PB
RTO	recovery time objective	
RTS	real time system	CPB
SCADA	supervisory control and data acquisition	
SDM	software defined memory	
SDN	software defined network	
SLA	service level agreement	
SLI	service level indicators	
SMP	symmetric multiprocessing	

Приложение 1. Принятые сокращения

Сокр.	Расшифровка	Альт.
SOA	service-oriented architecture	COA
SSD	solid-state storage device	
SQL	structured query language	
QoS	quality of service	
АО	аппаратное обеспечение	HW
АСУ	автоматизированные системы управления	ACS
АПК	аппаратно-программный комплекс	
АЭС	атомная электростанция	
ВНС	высоконагруженная система	HLS
ИБ	информационная безопасность	
ЖЦ	жизненный цикл	
ЗИП	Запасные части, инструменты и принадлежности	
КИИ	критическая информационная инфраструктура	
МСА	микросервисная архитектура	MSA
ОЗУ	оперативно-записывающее устройство	
ОС	операционная система	OS
ПАЗ	противоаварийная защита	SIS
ПАК	программно-аппаратный комплекс	
ПЗУ	постоянно-записывающее устройство	RAM
ПЛК	программно-логический контроллер	PLC
ПТК	программно-технический комплекс	
ПО	программное обеспечение	SW
ППИ	прикладной программный интерфейс	API
РВ	реальное время	RT

Сокр.	Расшифровка	Альт.
РРП	распределенная разделяемая память	DSM
РС	распределенная система	
PCY	распределенная система управления	DCS
PCУБД	распределенная СУБД	
СЗИ	средства защиты информации	
СКЗИ	средства криптографической защиты информации	
CPB	система реального времени	RTS
COA	сервис-ориентированная архитектура	SOA
СУБД	система управления базами данных	DBMS
СУИБ	система управления информационной безопасностью	SIEM
СХД	система хранения данных	
ТП	технологический процесс	
ФГУ	функционально-групповое управление	
ЦД	цифровой двойник	
ЦОД	центр обработки данных	DC

Приложение 2. Перечень шаблонов проектирования

Перечень упомянутых в книге шаблонов проектирования.

Шаблон	Pattern	Раздел
Амбассадор	Ambassador	11.1
Брокер очередей	Queue broker	11.1.4
Брокер сообщений	Message broker	11.1
Ведущий-ведомый	Leader-follower	11.1
Выбор лидера	Leader election	11.1
Выделенное ядро	Dedicated core	11.1.6
Двухфазная фиксация	Two-phase commit	11.2.8
Единая точка входа	Common entry point	11.1
Избыточность	Redundancy	11.1
Издатель-подписчик	Publisher-subscriber	11.1.7
Интерконнект	Interconnect	11.2.1
Каталог данных	Data catalog	10.4
Каналы и фильтры	Pipes & filters	11.1.1
Клиент-сервер	Client-server	11.1.2
Конвейер	Pipeline	11.1.1
Кэш на стороне	Cache-aside	11.1
Микросервисная архитектура	Micro Service Architecture (MSA)	11.1.4
Многоуровневая архитектура	Tiered architecture	11.1.2
Многослойная архитектура	Layered Architecture	11.1.5

Шаблон	Pattern	Раздел
Общие данные	Shared data	11.1
Одноранговая сеть	Peer to peer	11.1.2
Оркестратор	Orchestrator	11.2.5
Очередь сообщений	Message queue	11.1
Поточная очередь	Thread pool	11.1
Прицеп	Sidecar	11.1
Сага	Saga	11.2.9
Сервис-ориентированная архитектура	Service-Oriented Architecture (SOA)	11.2.3
Событийно управляемая архитектура	Event-driven architecture (EDA)	11.2.2
Суперконвейер	Super pipeline	11.1.3
Разделение ответственности команд и запросов	Command Query Responsibility Segregation (CQRS)	11.2.7
Разделяемая база данных	Shared database	11.1
Распределенное ядро	Distributed core	11.2.6
Реплицированные сервисы с распределением нагрузки	Replicated Load-Balanced Services (RLBS)	11.2
Фабрика данных	Data fabric	10.4
Фабрика сервисов	Service fabric	10.4
Хореография	Choreography	11.2.5
Шардинг	Sharding	11.1

Приложение 3. Список литературы

1. Вентцель Е. С., Овчаров Л. А. Теория вероятностей. М.: Наука, 1969. 368 с.
2. Воеводин В. В., Воеводин Вл. В. Параллельные вычисления. СПб.: БХВ-Петербург, 2002. 608 с.
3. Елизаров И. А., Мартемьянов Ю. Ф. Технические средства автоматизации. Программно-технические комплексы и контроллеры: учебное пособие. М.: «Издательство Машиностроение-1», 2004. 180 с.
4. Ивченко Г. И., Каштанов В. А., Коваленко И. Н. Теория массового обслуживания. Учебное пособие для вузов. М.: Высшая школа, 1982. 256 с.
5. Оленев Н. Н. Основы параллельного программирования в системе MPI. М.: ВЦ РАН, 2005. 80 с.
6. Петров О. Внутреннее устройство баз данных — глубокое погружение в то, как работают распределённые системы баз данных. O'Reilly, 2019. 349 с.
7. Подольный В. П. Архитектура высоконагруженных систем. Системы сбора информации, распределенные системы управления, системы реального времени. М.: OneBook, 2022. 160 с.
8. Согомонян Е. С., Слабаков Е. В. Самопроверяемые устройства и отказоустойчивые системы. М.: Радио и связь, 1989. 207 с.
9. Танненбаум Э. Архитектура компьютера. 5-е изд. СПб.: Питер, 2013. 884 с.
10. Танненбаум Э., ван Стеен М., Распределенные системы. Принципы и парадигмы. СПб.: Питер, 2003. 877 с.
11. Фаулер М., Садаладж П. Дж. NoSQL. новая методология разработки нереляционных баз данных. М.: «Вильямс», 2013. 192 с.

12. Sunshine CA, editor. Computer Network Architectures and Protocols. Springer Science & Business Media, 2013. 542 p.
13. Severance C, Dowd K. High Performance Computing (RISC Architectures, Optimization & Benchmarks), 2nd ed. O'Reilly Media, 1998. 466 p.
14. McCreary D, Kelly A. Making Sense of NoSQL: A guide for managers and the rest of us. Manning Publications, 2013. 312 p.
15. Dhamdhere DM. Operating Systems. McGraw-Hill Science/Engineering/Math, 2008. 864 p.
16. Dubrova E. Fault-Tolerant Design. New York: Springer New York, 2013. 185 p.
17. Foster I, Kesselman C. The Grid: Blueprint for a New Computing Infrastructure. San Francisco, CA: Morgan Kaufmann Publishers, 1998. 562 p.
18. Hohpe G, Woolf B. Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions. Pearson Education, 2012. 736 p.
19. Davies H, Bressan B, editors. A History of International Research Networking: The People who Made it Happen. John Wiley & Sons, 2010. 347 p.
20. Huang D, Wu H. Mobile Cloud Computing: Foundations and Service Models. Morgan Kaufmann, 2017. 317 p.
21. Kshemkalyani AD, Singhal M. Distributed Computing: Principles, Algorithms, and Systems. Cambridge University Press, 2011.
22. Molyneaux I. The Art of Application Performance Testing: Help for Programmers and Quality Assurance. O'Reilly Media, 2009.
23. Curé O, Blin G. Chapter 2. Database Management Systems. In: RDF Database Systems: Triples Storage and SPARQL Query Processing. Elsevier Science, 2014. 256 p.

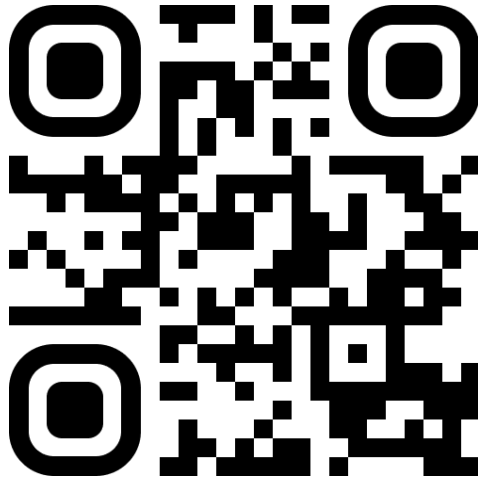
Приложение 3. Список литературы

24. Patterson DA, Hennessy JL. Computer Architecture: A Quantitative Approach. 4th ed. Burlington, Massachusetts: Morgan Kaufmann, 2006. 704 p.
25. Chevance RJ. Server Architectures: Multiprocessors, Clusters, Parallel Systems, Web Servers, Storage Solutions. Digital Press, 2004. 784 p.
26. Sadalage PJ, Fowler M. NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence. Addison-Wesley Professional, 2012. 192 p.
27. Tiwari Sh. Professional NoSQL. John Wiley & Sons Inc, 2011. 384 p.
28. Solihin Y. Fundamentals of Parallel Multicore Architecture. Boca Raton, Florida: Chapman and Hall/CRC, 2015. 468 p.
29. Sorin DJ, Hill MD, Wood DA. A Primer on Memory Consistency and Cache Coherence. Morgan & Claypool, 2011. 210 p.
30. Sosinsky B. Cloud Computing Bible. Wiley, 2010.
31. Stallings W. Operating Systems – Internals and Design Principles. 7th ed. Prentice Hall, 2011. 816 p.

Приложение 4. Ссылки

В книге приведены ссылки на источники (статьи, прочие материалы), начинающиеся с символа «#». Символом «►» отмечены ссылки на рекомендуемые видеоматериалы.

Ссылки на источники можно найти на странице книги по следующей ссылке в QR-коде.



Вадим Подольный

**Архитектура
высоконагруженных систем**

Издание второе

Литературный редактор – Надежда Тихомирова

Корректоры – Максим Кузнецов

Иллюстратор – Анастасия Балякно

Верстка – Любовь Подольная

ISBN 978-5-00227-164-1



Подписано в печать 02.02.2024 г.

Формат 60х90/8. Бумага офсетная. Печать цифровая.

Заказ No 25000. Тираж 100 экз.

Отпечатано в типографии «OneBook.ru», www.onebook.ru

ООО «Сам Полиграфист»

109316, г. Москва, Волгоградский проспект, дом 42, корп. 5,
«Технополис Москва».